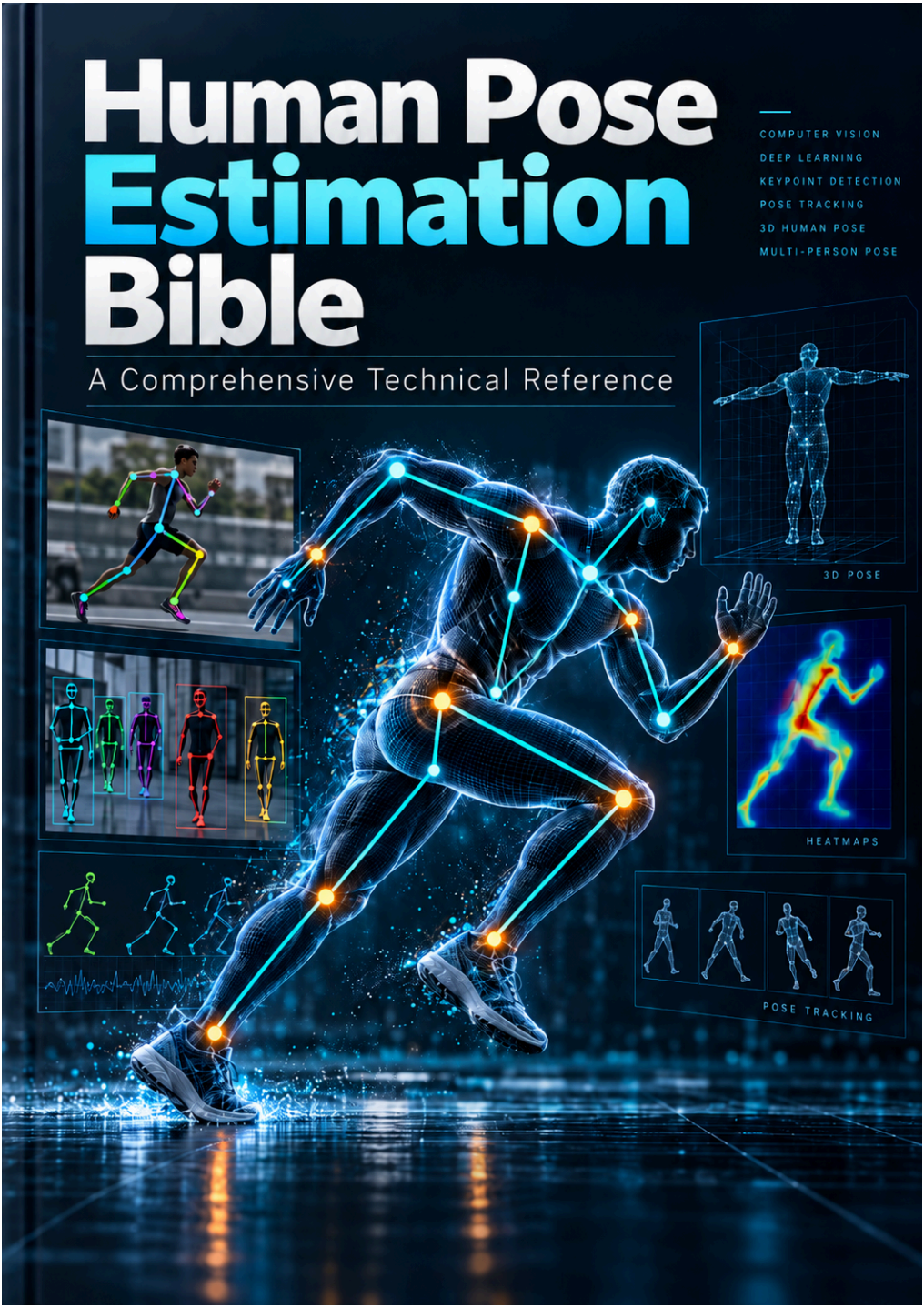
Human Pose Estimation Bible

A Comprehensive Technical Reference

COMPUTER VISION
DEEP LEARNING
KEYPOINT DETECTION
POSE TRACKING
3D HUMAN POSE
MULTI-PERSON POSE



Acknowledgment

First and foremost, we would like to express our deepest gratitude to **Almighty Allah** for His countless blessings, guidance, and mercy. By His grace and support, we have been able to complete our journey and successfully reach this important milestone of graduating from our college. All praise and thanks belong to Him.

We would also like to extend our heartfelt appreciation to our beloved **parents and families** for their unconditional love, continuous encouragement, endless patience, and sincere prayers throughout our academic journey. Their unwavering support has been our greatest source of strength and motivation, and this achievement would not have been possible without them.

Finally, we would like to express our sincere gratitude to **all our professors, teaching assistants, and everyone at the faculty** who supported, guided, and encouraged us throughout our years of study. Their knowledge, dedication, and valuable advice have played a significant role in shaping our academic and personal growth. We are truly thankful for their efforts and generosity.

To everyone who has contributed to our success, whether directly or indirectly, we extend our deepest appreciation and gratitude.

Table of Contents

CHAPTER 1..... 11
Human Pose Estimation Landscape (HPE)..... 11
1.1 What Is Human Pose Estimation (HPE) ?..... 11
1.2 Examples of Applications..... 13
1.2.1 Activity Recognition..... 13
1.2.2 Medical Applications..... 14
1.2.3 Robotics and Human–Robot Interaction..... 14
1.2.4 Animation and Entertainment..... 15
1.2.5 Fitness and Sports Analytics..... 15
1.2.6 Safety and Security..... 16
1.2.7 Additional Applications..... 16
1.3 Types of HPE..... 17
1.3.1 2D Human Pose Estimation..... 17
1.3.2 3D Human Pose Estimation..... 18
1.4 Main Challenges of Human Pose Estimation (HPE)..... 19
1.4.1 Lack of Accuracy..... 19
1.4.2 Occlusion..... 20
1.4.3 Crowd Complexity..... 20
1.4.4 Time Complexity and Real-Time Performance..... 20
1.4.5 Preprocessing Challenges..... 21
1.4.6 Data Security and Privacy..... 21
1.4.7 Depth Ambiguity..... 21
1.4.8 Appearance Variability..... 22
1.4.9 Scale and Viewpoint Variation..... 22
1.4.10 Left–Right Ambiguity..... 22
1.4.11 Rare Poses and Long-Tail Distribution..... 23
1.4.12 Data Scarcity and Annotation Cost..... 23
1.4.13 Adverse Imaging Conditions..... 23
1.4.14 Root-Relative Ambiguity and Global Positioning..... 24

1.4.15 Temporal Inconsistency in Video-Based HPE..... 24
1.5 Evaluation Metrics..... 24
1.5.1 Percentage of Correct Parts (PCP)..... 25
1.5.2 Percentage of Correct Keypoints (PCK)..... 26
1.5.3 Percentage of Detected Joints (PDJ)..... 27
1.5.4 Mean Per Joint Position Error (MPJPE)..... 27
1.5.5 Object Keypoint Similarity (OKS)..... 28
1.5.6 Average Precision (AP) and Average Recall (AR)..... 29
1.5.7 Area Under Curve (AUC)..... 30
1.5.8 Normalized Mean Error (NME)..... 30
1.6 Benchmark Datasets..... 31
1.6.1 MPII Human Pose Dataset..... 31
1.6.2 COCO Keypoints Dataset..... 32
1.6.3 Leeds Sports Pose (LSP) and LSP Extended (LSPE)..... 33
1.6.5 CrowdPose Dataset..... 34
1.6.6 Human3.6M Dataset..... 35
1.6.7 PoseTrack Dataset..... 36
1.6.8 AI Challenger Human Keypoint Dataset..... 37
1.6.9 3DPW (3D Poses in the Wild)..... 37
CHAPTER 2..... 38
Human Pose Estimation Approaches..... 38
2.1 Human Body Models..... 38
2.1.1 Kinematic Models (Skeleton-Based Models)..... 40
2.1.2 Planar Models (2D Part-Based Models)..... 41
2.1.3 Volumetric Models..... 44
2.2 2D Human Pose Estimation..... 46
2.2.1 Classical Approaches for 2D HPE..... 47
2.2.2.1 Regression-Based Methods..... 49
2.2.2.2 Heatmap-Based Methods..... 50
2.2.3 Comparison Between Regression and Heatmap..... 52
2.3 3D Human Pose Estimation..... 56
2.3.1 Classical Approaches for 3D HPE..... 58
2.3.2 Monocular 3D Human Pose Estimation..... 59
2.3.2.1 Direct Regression Methods..... 59

2.3.2.2 2D-to-3D Lifting Methods....60
2.3.2.3 Volumetric Heatmap Methods....61
2.3.3 Single-Person 3D Pose Estimation....65
2.3.4 Multi-Person 3D Pose Estimation....65
CHAPTER 3....67
HPE Using Deep Learning....67
3.1 Convolutional Neural Networks for Human Pose Estimation....67
3.1.1 Fundamental Principles of CNNs for Human Pose Estimation 68
3.1.1.1 Translation Equivariance....68
3.1.1.2 Hierarchical Feature Learning....68
3.1.1.3 Weight Sharing and Parameter Efficiency....69
3.1.1.4 Multi-Scale Representation Learning....69
3.1.2 Output Representations for Human Pose Estimation....70
3.1.2.1 Coordinate Regression....70
3.1.2.2 Heatmap Regression....71
3.1.3 Spatial Resolution Recovery and Decoder Architectures....72
3.1.3.1 Transposed Convolution....72
3.1.3.2 Bilinear Upsampling and Convolution....72
3.1.3.3 Skip Connections and U-Net Structures....72
3.1.4 Dilated Convolutions for Context Modeling....73
3.1.5 Landmark CNN Architectures for Human Pose Estimation.73
3.1.6 CNNs as Universal Visual Feature Extractors....76
3.2 Transformer and Attention-Based Architectures for Human Pose Estimation....77
3.2.1 Self-Attention in HPE Backbones....78
3.2.2 Cross-Attention for Person–Keypoint Association....83
3.2.3 Keypoint-to-Keypoint Attention....85
3.3 Graph Neural Networks (GNNs) for Structural Consistency in Human Pose Estimation....88
3.3.1 Structural Modeling of the Human Skeleton....88
3.3.2 Graph Convolutional Foundations for Pose Estimation....89
3.3.3 GNN Architectures for 3D Human Pose Estimation....90
3.3.4 GNNs for 2D Human Pose Estimation....93

3.3.5 Structural Constraints as Learnable Objectives.....95
3.3.6 Spatio-Temporal Graph Neural Networks.....95
3.3.7 Hybrid GNN-Transformer Architectures.....96
3.4 Diffusion-Based Pose Estimation..... 97
3.4.1 Diffusion for 2D Human Pose Estimation..... 99
3.4.2 Diffusion for 3D Human Pose Estimation..... 100
3.4.3 Temporal Diffusion Models for Video-Based HPE..... 101
3.4.4 Structural Guidance in Diffusion Models.....102
3.4.5 Hypothesis Aggregation Strategies..... 104
3.4.6 Diffusion as a Pose Prior..... 104
3.4.7 Conditioning Strategies in Diffusion-Based HPE..... 105
3.4.8 Computational Challenges and Efficiency Improvements. 106
3.4.9 Diffusion Models and Structural Consistency..... 106
3.5 Vision-Language & LLM Integration for Human Pose Estimation.... 107
3.5.1 From Vision-Centric Estimation to Vision–Language Reasoning.....107
3.5.2 Contrastive Vision–Language Pretraining for Pose Representation Learning..... 108
3.5.2.1 CLIP-Style Alignment for Pose Understanding.....108
3.5.2.2 Language-Assisted Multi-Person Pose Estimation...109
3.5.2.3 LLM-Generated Kinematic Priors.....110
3.5.3 Text-Guided Conditional Pose Estimation.....111
3.5.3.1 Action-Guided 3D Pose Estimation..... 111
3.5.3.2 Text-Guided Multi-View 3D Pose Estimation.....112
3.5.3.3 Prompt-Driven Diffusion Pose Estimation..... 112
3.5.4 Multimodal LLMs for Pose Reasoning and Generation..... 113
3.5.4.1 Conversational Pose Estimation..... 113
3.5.4.2 Unified Pose-Centric Multimodal Models..... 114
3.5.4.3 Motion-Oriented LLMs..... 115
3.5.4.4 Instruction-Tuned Pose Understanding.....116
3.5.5 Architectural Integration Strategies.....116
3.5.5.1 Frozen Language Encoders with Auxiliary Supervision.. 116
3.5.5.2 Learnable Prompt Tuning.....117

3.5.5.3 Cross-Modal Attention Fusion....117
3.5.5.4 Pose Token Generation within LLMs.... 117
3.5.5.5 Diffusion with Language Conditioning....118
3.5.6 Cross-Modal Alignment Mechanisms.... 118
3.5.6.1 Contrastive Alignment....118
3.5.6.2 Feature-Space Projection.... 118
3.5.6.3 Attention-Based Communication.... 119
3.5.6.4 Instruction-Based Pose Reasoning....119
3.5.7 Emerging Trends and Future Directions.... 119
3.5.7.1 Semantic-Aware Pose Understanding.... 119
3.5.7.2 Explainable Pose Estimation.... 120
3.5.7.3 Interactive Pose Editing....120
3.5.7.4 Unified Human-Centric Multimodal Models.... 120
CHAPTER 4....121
4.1Taxonomy of Human Pose Estimation Models....121
4.1.1 OpenPose....122
4.1.2 High-Resolution Representation Learning for Human Pose Estimation (HRNet)....129
4.1.3 YOLO-Pose and YOLOv8-Pose....136
4.1.4 RTMPose....143
4.1.5 ViTPose / ViTPose++.... 153
4.1.6 Human Mesh Recovery (HMR).... 163
4.1.7 PoseFormer....168
4.1.8 MANO (Model of the Articulated Hand)....174
4.2 Practical Implementation: HPE-Bible Project Suite....179
4.2.1 Project Overview.... 180
4.2.2 YOLO Test 1: Tiled Pose Estimation for Far-View Scenes.....180
4.2.2.1 Concept and Motivation....180
4.2.2.2 Architecture and Key Components....181
4.2.2.3 Use Cases.... 185
4.2.3 YOLO Test 2: Two-Stage Pose Estimation....186
4.2.3.1 Concept and Motivation....186
4.2.3.2 Architecture and Key Components....186
4.2.4 RTMPose: Real-Time Multi-Person Pose Estimation with

Tracking..... 194
4.2.4.1 Concept and Motivation..... 194
4.2.4.2 Architecture and Key Components..... 195
4.2.5 Comparative Analysis..... 202
4.2.6 Key Functions Summary..... 203
4.2.7 Implementation Insights..... 205
Appendix A: Multi-Player Pose Estimation in Wide-Field Soccer Broadcasts Using Image Tiling and YOLOv8..... 206
Appendix B: Human Pose Estimation Using Yolov8-pose in two Phases when Detector run first and Pose run second..... 238
Appendix C : Real-Time Football Analytics: Player Detection, Tracking, and Pose Estimation via RTMPose..... 335

Tree of the Book

CHAPTER 1 Human Pose Estimation Landscape (HPE) 8

- 1.1 What Is Human Pose Estimation (HPE) ? 9
- 1.2 Examples of Applications 11
  - 1.2.1 Activity Recognition 11
  - 1.2.2 Medical Applications 12
  - 1.2.3 Robotics and Human–Robot Interaction 12
  - 1.2.4 Animation and Entertainment 13
  - 1.2.5 Fitness and Sports Analytics 13
  - 1.2.6 Safety and Security 14
  - 1.2.7 Additional Applications 14
- 1.3 Types of HPE 14
  - 1.3.1 2D Human Pose Estimation 15
  - 1.3.2 3D Human Pose Estimation 16
- 1.4 Main Challenges of HPE 17
  - 1.4.1 Lack of Accuracy 17
  - 1.4.2 Occlusion 18
  - 1.4.3 Crowd Complexity 18
  - 1.4.4 Time Complexity and Real-Time Performance 18
  - 1.4.5 Preprocessing Challenges 19
  - 1.4.6 Data Security and Privacy 19
  - 1.4.7 Depth Ambiguity 19
  - 1.4.8 Appearance Variability 20
  - 1.4.9 Scale and Viewpoint Variation 20
  - 1.4.10 Left–Right Ambiguity 20
  - 1.4.11 Rare Poses and Long-Tail Distribution 21
  - 1.4.12 Data Scarcity and Annotation Cost 21
  - 1.4.13 Adverse Imaging Conditions 21
  - 1.4.14 Root-Relative Ambiguity and Global Positioning 22
  - 1.4.15 Temporal Inconsistency in Video-Based HPE 22
- 1.5 Evaluation Metrics 22
  - 1.5.1 Percentage of Correct Parts (PCP) 23
  - 1.2.2 Percentage of Correct Keypoints (PCK) 24
  - 1.3.3 Percentage of Detected Joints (PDJ) 25
  - 1.5.4 Mean Per Joint Position Error (MPJPE) 25
  - 1.5.5 Object Keypoint Similarity (OKS) 26
  - 1.5.6 Average Precision (AP) and Average Recall (AR) 27
  - 1.5.7 Area Under Curve (AUC) 28
  - 1.5.8 Normalized Mean Error (NME) 28
- 1.6 Benchmark Datasets 29
  - 1.6.1 MPII Human Pose Dataset 29
  - 1.6.2 COCO Keypoints Dataset 30
  - 1.6.3 Leeds Sports Pose (LSP) and LSP Extended (LSPE) 31
  - 1.6.5 CrowdPose Dataset 32
  - 1.6.6 Human3.6M Dataset 33
  - 1.6.7 PoseTrack Dataset 34
  - 1.6.8 AI Challenger Human Keypoint Dataset 35
  - 1.6.9 3DPW (3D Poses in the Wild) 35

CHAPTER 2 Human Pose Estimation Approaches 36

- 2.1 Human Body Models 36
  - 2.1.1 Kinematic Models (Skeleton-Based Models) 38
  - 2.1.2 Planar Models (2D Part-Based Models) 39
  - 2.1.3 Volumetric Models 42
- 2.2 2D Human Pose Estimation 44
  - 2.2.1 Classical Approaches for 2D HPE 45
    - 2.2.2.1 Regression-Based Methods 47
    - 2.2.2.2 Heatmap-Based Methods 48
  - 2.2.3 Comparison Between Regression and Heatmap 50
- 2.3 3D Human Pose Estimation 54
  - 2.3.1 Classical Approaches for 3D HPE 56
  - 2.3.2 Monocular 3D Human Pose Estimation 57
    - 2.3.2.1 Direct Regression Methods 57
    - 2.3.2.2 2D-to-3D Lifting Methods 58
    - 2.3.2.3 Volumetric Heatmap Methods 59
  - 2.3.3 Single-Person 3D Pose Estimation 63
  - 2.3.4 Multi-Person 3D Pose Estimation 63

CHAPTER 3 HPE Using Deep Learning 65

- 3.1 Convolutional Neural Networks for HPE 65
  - 3.1.1 Fundamental Principles of CNNs for HPE 66
    - 3.1.1.1 Translation Equivariance 66
    - 3.1.1.2 Hierarchical Feature Learning 66
    - 3.1.1.3 Weight Sharing and Parameter Efficiency 67
    - 3.1.1.4 Multi-Scale Representation Learning 67
  - 3.1.2 Output Representations for HPE 68
    - 3.1.2.1 Coordinate Regression 68
    - 3.1.2.2 Heatmap Regression 68
  - 3.1.3 Spatial Resolution Recovery and Decoder Architectures 69
    - 3.1.3.1 Transposed Convolution 69
    - 3.1.3.2 Bilinear Upsampling and Convolution 70
    - 3.1.3.3 Skip Connections and U-Net Structures 70
  - 3.1.4 Dilated Convolutions for Context Modeling 70
  - 3.1.5 Landmark CNN Architectures for HPE 71
  - 3.1.6 CNNs as Universal Visual Feature Extractors 73
- 3.2 Transformer and Attention-Based Architectures for HPE 75
  - 3.2.1 Self-Attention in HPE Backbones 76
  - 3.2.2 Cross-Attention for Person–Keypoint Association 81
  - 3.2.3 Keypoint-to-Keypoint Attention 83
- 3.3 Graph Neural Networks (GNNs) for Structural Consistency in HPE 85
  - 3.3.1 Structural Modeling of the Human Skeleton 85
  - 3.3.2 Graph Convolutional Foundations for Pose Estimation 86
  - 3.3.3 GNN Architectures for 3D Human Pose Estimation 87
  - 3.3.4 GNNs for 2D Human Pose Estimation 91
  - 3.3.5 Structural Constraints as Learnable Objectives 92
  - 3.3.6 Spatio-Temporal Graph Neural Networks 93
  - 3.3.7 Hybrid GNN-Transformer Architectures 93
- 3.4 Diffusion-Based Pose Estimation 95
- 3.5 Vision-Language & LLM Integration for HPE 104

CHAPTER 4 Taxonomy of HPE Models & Practical Implementation 119

- 4.1 Taxonomy of Human Pose Estimation Models 119
  - 4.1.1 OpenPose 119
  - 4.1.2 High-Resolution Representation Learning for HPE (HRNet) 126
  - 4.1.3 YOLO-Pose and YOLOv8-Pose 133
  - 4.1.4 RTMPose 141
  - 4.1.5 ViTPose / ViTPose++ 150
  - 4.1.6 Human Mesh Recovery (HMR) 161
  - 4.1.7 PoseFormer 166
  - 4.1.8 MANO (Model of the Articulated Hand) 171
- 4.2 Practical Implementation: HPE-Bible Project Suite 177
  - 4.2.1 Project Overview 177
  - 4.2.2 YOLO Test 1: Tiled Pose Estimation for Far-View Scenes 178
  - 4.2.3 YOLO Test 2: Two-Stage Pose Estimation 183
  - 4.2.4 RTMPose: Real-Time Multi-Person Pose Estimation with Tracking 191
  - 4.2.5 Comparative Analysis 199
  - 4.2.6 Key Functions Summary 201
  - 4.2.7 Implementation Insights 202

Appendix A: Multi-Player Pose Estimation in Wide-Field Soccer Broadcasts Using Image Tiling and YOLOv 204

Appendix B: Human Pose Estimation Using Yolov8-pose in two Phases when Detector run first and Pose run second 236

Appendix C: Real-Time Football Analytics: Player Detection, Tracking, and Pose Estimation via RTMPose 333

CHAPTER 1

Human Pose Estimation Landscape (HPE)

1.1 What Is Human Pose Estimation (HPE) ?

Human Pose Estimation (HPE) is a fundamental task in computer vision that aims to detect and localize human body joints—commonly referred to as keypoints, such as elbows, wrists, knees, and ankles—from images or video sequences. The primary objective of HPE is to infer the spatial configuration and orientation of the human body, thereby enabling a structured representation of human posture.

Despite its significance, HPE remains a challenging problem due to several inherent complexities. These include inter-person occlusions, variations in clothing, complex human poses, and difficulties in accurately localizing joints under diverse environmental conditions. Additionally, background clutter and foreground occlusion further complicate the estimation process. Such challenges, combined with high computational and spatial complexity, often limit the practical applicability of pose estimation systems.

At its core, HPE seeks to estimate body configuration from a single image—typically monocular—although it can also be extended to video data. The problem can be categorized along multiple dimensions. One key distinction lies between single-person and multi-person pose estimation, depending on the number of individuals present within a frame. Another important factor involves the representation of the human body, where

various modeling approaches have been proposed, including shape-based, skeletal (stick-figure), and mesh-based models.
Feature extraction strategies also play a critical role in the effectiveness of HPE systems. In the context of deep learning, pose estimation approaches can be broadly categorized into several perspectives: (1) holistic methods, which directly regress joint positions; (2) local methods, which focus on detecting individual body parts; and (3) hybrid approaches that integrate both global and local information to improve accuracy. Additional considerations include motion-based features, pose estimation in video sequences, and challenges such as foreshortening effects.

Historically, the development of HPE can be traced back to early efforts in the 1960s and 1970s within computer graphics and animation. During this period, human motion tracking primarily relied on manual techniques or specialized hardware, such as marker-based motion capture systems, rather than automated estimation from visual data. In the 1990s, the field began to gain momentum within the computer vision community through the introduction of model-based approaches. Notably, pictorial structures emerged as a powerful framework, representing the human body as a collection of interconnected parts with spatial constraints. The work of Pedro Felzenszwalb and Daniel Huttenlocher (2000) formalized this concept for object recognition, laying the foundation for subsequent advancements in pose estimation. Later contributions by Greg Mori and Jitendra Malik (2002) further explored exemplar-based matching techniques for human body modeling.

In recent years, the field has experienced significant progress with the adoption of deep learning techniques, which have substantially improved the accuracy and robustness of both 2D and 3D human pose estimation from images and videos. These advancements continue to address longstanding challenges and expand the range of real-world applications, including sports analytics, healthcare, surveillance, and human–computer interaction.

In summary, Human Pose Estimation is concerned with identifying the precise locations of key anatomical landmarks—such as the head, neck, shoulders, elbows, wrists, hips, knees, ankles, and facial features (e.g., eyes, ears, and nose)—in order to reconstruct a meaningful representation of human posture from visual data.

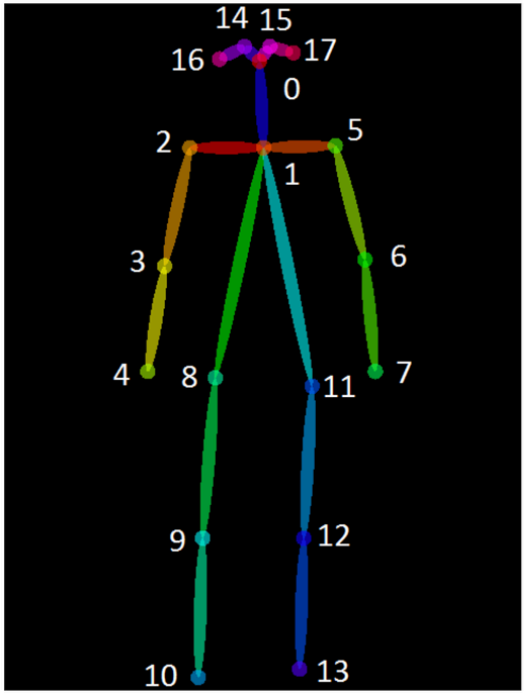


Figure 1-1. Human Pose Estimation

1.2 Examples of Applications

Human Pose Estimation (HPE) has become a critical component in a wide range of real-world applications due to its ability to accurately model human body movements and interactions. Its versatility allows it to be integrated into various domains, including activity recognition, healthcare, robotics, animation, sports analytics, and safety systems.

1.2.1 Activity Recognition

Activity recognition involves identifying and interpreting human actions and behaviors from visual or sensor data. It plays a significant role in applications such as intelligent user interfaces, health monitoring systems, security surveillance, and behavioral analysis. By leveraging pose estimation, systems

can extract meaningful spatial and temporal features that improve the discrimination between different actions.

Recent advancements in deep learning have significantly enhanced activity recognition performance. Modern approaches combine object detection and pose estimation techniques to identify complex human activities, such as industrial assembly tasks. For instance, deep neural networks can simultaneously detect tools and estimate human joint positions, enabling accurate recognition of task-specific actions. These systems have demonstrated high accuracy by effectively modeling both spatial configurations and temporal dynamics of repeated actions.

1.2.2 Medical Applications

In the medical domain, HPE is widely used for posture analysis, rehabilitation, and clinical diagnostics. It enables the detection and classification of postural abnormalities, such as scoliosis, by analyzing deviations in body alignment. Additionally, pose estimation supports physical therapy by providing quantitative feedback on patient movements, allowing clinicians to monitor progress and adjust treatment plans accordingly.

Advanced systems have been developed to operate in challenging clinical environments, where noise, occlusions, and lighting variations are common. These systems often employ deep learning models tailored to individual patients, improving robustness and personalization. Techniques such as probabilistic filtering are also used to refine joint predictions over time, resulting in more stable and accurate motion tracking. Overall, HPE contributes to more objective, data-driven healthcare solutions.

1.2.3 Robotics and Human–Robot Interaction

HPE plays a vital role in robotics, particularly in applications involving human–robot interaction and assistive robotics. Service and care robots increasingly rely on pose estimation to understand human behavior, monitor conditions, and respond appropriately in environments such as nursing homes and assisted living facilities. Basic pose classification—such as

identifying whether a person is standing, sitting, or has fallen—can provide critical information for decision-making.

More advanced frameworks enable robots to estimate 3D human poses and use this information for coordination and interaction. For example, robots can align their positions relative to humans by matching visual keypoints and solving geometric optimization problems. Additionally, pose estimation facilitates demonstration-based learning, where robots learn tasks by observing human actions without requiring markers or specialized equipment. This capability is essential for developing intuitive and adaptive robotic systems.

1.2.4 Animation and Entertainment

In animation and entertainment, HPE has significantly reduced the reliance on manual character animation and marker-based motion capture systems. Modern pose estimation techniques enable markerless motion capture, allowing real-time mapping of human movements onto digital characters.

Recent systems can estimate both 2D and 3D poses from standard RGB video and reconstruct them into animated sequences. These approaches often follow a pipeline that begins with 2D pose detection and subsequently lifts the poses into 3D representations. Such systems are widely used in gaming, virtual reality, and film production, where real-time performance and realism are essential. Additionally, efficient architectures have been developed to model relationships between body parts, enabling accurate and fast pose estimation suitable for live applications.

1.2.5 Fitness and Sports Analytics

In the fitness and sports domain, HPE is used to analyze human movement and improve performance. It enables real-time form correction during exercises such as weightlifting, yoga, and rehabilitation routines, providing immediate feedback without the need for a human trainer.

Athletes and coaches benefit from detailed performance analytics, including measurements of joint angles, stride length, and movement efficiency. These

insights help optimize training strategies and enhance overall performance. Furthermore, pose estimation contributes to injury prevention by identifying incorrect or potentially harmful movement patterns before they lead to physical damage.

1.2.6 Safety and Security

HPE is increasingly applied in safety-critical systems for monitoring and risk detection. In healthcare and assisted living environments, it is used for fall detection, automatically alerting caregivers when an individual experiences a fall. In surveillance systems, pose-based analysis enables the detection of abnormal or suspicious behavior, such as aggression or accidents.

In industrial settings, pose estimation helps ensure worker safety by monitoring adherence to safety protocols. For example, it can detect whether workers are wearing protective equipment or performing tasks in a hazardous manner. These applications contribute to reducing workplace accidents and improving overall safety standards.

1.2.7 Additional Applications

Beyond the domains discussed above, HPE continues to expand into new areas. In augmented reality (AR) and virtual reality (VR), it enables immersive experiences by tracking user movements and integrating them into virtual environments. In retail, it is used for customer behavior analysis and automated checkout systems. Moreover, HPE has applications in human–computer interaction, sign language recognition, education, and biomechanics research.

Overall, the broad applicability of Human Pose Estimation highlights its importance as a foundational technology in modern computer vision, with ongoing advancements continuing to unlock new possibilities across diverse industries.

1.3 Types of HPE

Human Pose Estimation can be broadly categorized into two main types based on the dimensionality of the predicted joint coordinates: 2D pose estimation and 3D pose estimation. Each type differs in complexity, representation, and application scope.

1.3.1 2D Human Pose Estimation

Definition

2D Human Pose Estimation (2D HPE) involves predicting the pixel coordinates (x,y) of anatomical keypoints from a single image or video frame. The output is typically represented as a skeletal structure overlaid on the image plane, indicating the positions of body joints such as the shoulders, elbows, hips, and knees.

In essence, 2D HPE focuses on identifying the spatial locations of body joints within the two-dimensional coordinate system of an image. It is computationally efficient and widely used due to the availability of large annotated datasets and relatively simpler modeling requirements.

Main Approaches

1. Regression-Based Methods

These methods directly map image features to joint coordinate vectors. A neural network processes the input image and outputs the (x,y) coordinates of each keypoint in a single step. While computationally efficient, these approaches often struggle with accuracy, particularly in complex scenes with occlusions or multiple subjects.

2. Heatmap-Based Methods

Heatmap-based approaches have become the dominant paradigm in 2D HPE. Instead of predicting coordinates directly, the model generates a probability distribution (heatmap) for each joint. The peak of each heatmap corresponds to the most likely location of the respective keypoint.

These methods provide higher spatial accuracy and robustness, especially in challenging scenarios, and are widely adopted in modern deep learning architectures.

Key Characteristics

- Operates in image space (pixel coordinates)
- Relatively lower computational complexity
- Highly effective for real-time applications
- Limited in capturing depth information and true spatial structure

1.3.2 3D Human Pose Estimation

Definition

3D Human Pose Estimation (3D HPE) aims to predict the spatial coordinates (x,y,z) of body joints, reconstructing the human skeleton in a three-dimensional space. The output is typically expressed either in real-world metric units or normalized coordinates relative to a reference joint, commonly the pelvis.

The goal of 3D HPE is to infer depth information and recover the full geometric structure of the human body from visual input, such as images or video sequences.

Core Concepts

3D HPE extends 2D pose estimation by incorporating depth, enabling a more comprehensive understanding of human posture and movement. This allows for more accurate modeling of real-world interactions, making it essential for applications such as biomechanics, robotics, and augmented reality.

Common Approaches

1. **Monocular 3D Pose Estimation**
   Estimates 3D pose from a single RGB image. This is highly challenging due to depth ambiguity, as multiple 3D configurations can correspond to the same 2D projection.

2. **Multi-View Pose Estimation**
   Utilizes multiple camera views to reconstruct 3D pose more accurately through triangulation. This approach reduces ambiguity but requires a controlled setup.
3. **2D-to-3D Lifting Methods**
   First estimate 2D keypoints and then map them into 3D space using learned models. This is one of the most widely used strategies in deep learning-based systems.

Key Characteristics

- Captures full spatial structure of the human body
- Enables accurate motion analysis and interaction modeling
- More complex and computationally expensive than 2D HPE

1.4 Main Challenges of Human Pose Estimation (HPE)

Human Pose Estimation (HPE) is a highly challenging task in computer vision due to the complexity of human body structures, environmental variations, and computational constraints. Although significant advancements have been achieved through deep learning techniques, several challenges continue to limit the robustness, accuracy, and generalization capabilities of HPE systems. The following are the major challenges associated with Human Pose Estimation.

1.4.1 Lack of Accuracy

Achieving high accuracy remains one of the primary objectives of any HPE model. Accurate localization of body joints is essential for reliable pose estimation and downstream applications such as activity recognition, healthcare, and robotics. However, many approaches prioritize computational efficiency or architectural complexity at the expense of precision. Small localization errors can significantly affect overall pose reconstruction,

especially in fine-grained tasks involving hands, facial landmarks, or rapid body movements.

1.4.2 Occlusion

Occlusion is one of the most difficult challenges in HPE and occurs when body parts become partially or completely hidden. Occlusion can be categorized into:

- **Self-occlusion:** when one body part blocks another (e.g., crossed arms).
- **External occlusion:** when objects, furniture, or other individuals obstruct the body.

Occlusion affects both single-person and multi-person pose estimation by limiting visible information required for accurate joint detection. In heavily crowded scenes, large portions of the body may become invisible, forcing the model to infer missing joints using contextual reasoning, body structure priors, and temporal motion information.

1.4.3 Crowd Complexity

Crowded environments introduce additional challenges, particularly in multi-person pose estimation. In scenes containing many individuals, body parts overlap and limbs become intertwined, making it difficult to correctly associate detected joints with the corresponding person.

Bottom-up approaches face complex joint-association problems, while top-down methods often struggle when overlapping individuals produce inaccurate or merged bounding boxes. Dense crowd scenarios, such as concerts, sports stadiums, or protests, remain an open research problem in modern HPE systems.

1.4.4 Time Complexity and Real-Time Performance

Efficient processing is essential for real-world HPE applications, especially those requiring real-time inference such as robotics, autonomous systems, gaming, and augmented reality. Many traditional and deep learning-based

approaches achieve high accuracy at the cost of increased computational complexity and inference time.

Modern architectures, including transformer-based networks and multi-stage refinement systems, often require substantial computational resources. Deploying these models on edge devices such as mobile phones, drones, or embedded systems requires lightweight architectures, model pruning, quantization, or optimization techniques, which may reduce overall accuracy.

1.4.5 Preprocessing Challenges

Preprocessing plays a crucial role in the effectiveness of HPE systems. Tasks such as body-part localization, background subtraction, image normalization, calibration, and data augmentation significantly influence model performance.

Poor preprocessing can introduce noise, distortions, or inconsistencies that negatively affect pose estimation accuracy. Variations in camera quality, lighting conditions, and image resolution further complicate preprocessing pipelines.

1.4.6 Data Security and Privacy

Security and privacy are major concerns in machine learning and deep learning applications involving human data. Since HPE systems often process sensitive visual information, ensuring data confidentiality and secure handling is essential.

Applications such as healthcare monitoring, surveillance systems, and smart environments require careful management of personal data and compliance with privacy regulations. Distinguishing between sensitive and non-sensitive information is therefore critical during data collection, storage, and deployment.

1.4.7 Depth Ambiguity

Depth ambiguity is a fundamental problem in monocular 3D Human Pose Estimation. A single 2D image represents a projection of a three-dimensional

scene, meaning multiple 3D body configurations may correspond to the same 2D joint arrangement.

As a result, models must infer depth information without explicit spatial cues. Distinguishing whether a limb is moving toward or away from the camera becomes extremely difficult from a single viewpoint. Although deep learning models utilize statistical pose priors to reduce ambiguity, they often fail in unusual or highly articulated poses.

1.4.8 Appearance Variability

Human appearance varies significantly across clothing styles, body shapes, skin tones, hairstyles, accessories, and lighting conditions. Loose garments, winter clothing, or cultural attire may obscure body contours and reduce joint visibility.

Unlike rigid objects, the human body is highly deformable and covered by variable textures, making feature extraction more challenging. Consequently, models trained under limited conditions may fail to generalize effectively in diverse real-world environments.

1.4.9 Scale and Viewpoint Variation

People may appear at vastly different scales depending on camera distance and perspective. Additionally, pose estimation systems must handle arbitrary viewpoints such as frontal, side, top-down, or bird’s-eye views.

Extreme viewpoint changes alter the visual arrangement of joints and may remove critical depth cues. Similarly, small-scale individuals in low-resolution images often lack sufficient visual detail for precise keypoint localization.

1.4.10 Left–Right Ambiguity

Human body parts are largely symmetrical, which frequently causes confusion between left and right limbs. This issue becomes more severe when limbs overlap, cross each other, or appear partially hidden.

Models often misclassify left and right joints in frontal views or mirrored poses, leading to incorrect skeletal structures. This “flipping” problem remains a persistent challenge even in advanced pose estimation systems.

1.4.11 Rare Poses and Long-Tail Distribution

Most publicly available datasets are dominated by common activities such as standing, walking, and sitting. Rare or extreme poses—including gymnastics, yoga, breakdancing, or falling—are significantly underrepresented.

Deep learning models tend to optimize for frequently occurring patterns, causing poor generalization on uncommon poses. Collecting balanced datasets for rare human motions is inherently difficult, making long-tail pose distributions a major challenge in HPE research.

1.4.12 Data Scarcity and Annotation Cost

Large-scale annotated datasets are essential for training robust HPE models. However, accurately labeling human joints is a labor-intensive and time-consuming process.

The challenge becomes even greater in 3D pose estimation, where obtaining ground-truth annotations often requires expensive motion-capture systems, calibrated multi-camera setups, and controlled laboratory environments. As a result, many 3D datasets lack environmental diversity, limiting model generalization to real-world scenes.

1.4.13 Adverse Imaging Conditions

Real-world environments frequently produce poor image quality due to motion blur, low resolution, camera noise, shadows, extreme lighting, or out-of-focus capture.

These conditions reduce the visibility of body joints and significantly degrade localization accuracy. For example, low-resolution surveillance footage may render small joints indistinguishable, while motion blur can obscure limb boundaries entirely.

1.4.14 Root-Relative Ambiguity and Global Positioning

Most monocular 3D HPE methods estimate joint positions relative to a root joint, typically the pelvis, rather than predicting absolute positions in world coordinates.

Without camera calibration or external spatial references, recovering true global position from a single image is mathematically ambiguous. This limitation affects applications such as autonomous driving, sports analytics, and biomechanics, where accurate real-world positioning is required.

1.4.15 Temporal Inconsistency in Video-Based HPE

In video sequences, pose predictions should remain temporally smooth and stable across consecutive frames. However, frame-by-frame estimation approaches often produce jittering or flickering predictions.

Maintaining temporal consistency requires modeling motion dynamics and preserving contextual information over time. Sudden movements, occlusions, and camera motion make this process more challenging. Furthermore, recursive temporal models may accumulate errors over long sequences, while sliding-window approaches increase computational latency.

1.5 Evaluation Metrics

To ensure fair and reliable comparisons among Human Pose Estimation (HPE) models, standardized evaluation metrics and high-quality benchmark datasets are essential. Evaluation metrics quantify the accuracy of predicted keypoints and provide objective criteria for assessing model performance. Without unified evaluation standards, comparisons between different approaches become inconsistent and potentially misleading.

Unlike image classification, pose estimation is considered a structured regression problem, where the objective is not only to localize individual joints accurately but also to preserve anatomically plausible body configurations. A

model may predict joints close to their ground-truth positions while still generating unrealistic skeletal structures. Therefore, evaluation protocols in HPE measure both localization precision and structural consistency.

Dataset quality also plays a crucial role in model performance and generalization. Factors such as annotation accuracy, pose diversity, scene complexity, viewpoint variation, and occlusion levels significantly influence the effectiveness of training and evaluation. As a result, the development of robust datasets and unified benchmark protocols remains fundamental to the advancement of 2D and 3D Human Pose Estimation.

Performance metrics serve as quantitative indicators of pose estimation accuracy. Different metrics evaluate different aspects of prediction quality, including joint localization accuracy, structural consistency, scale invariance, and detection robustness.

This section presents the most widely used evaluation metrics in HPE, including both traditional and modern approaches for measuring pose estimation accuracy.

1.5.1 Percentage of Correct Parts (PCP)

Percentage of Correct Parts (PCP) is one of the earliest evaluation metrics used in pose estimation. It evaluates whether an entire limb segment is correctly localized rather than evaluating individual joints independently.

A body part is considered correctly estimated if both endpoints of the predicted limb lie within a predefined threshold distance from the corresponding ground-truth joint locations.

The PCP metric is defined as:

$$\text{PCP} = \frac{Number\ of\ Correctly\ Detected\ Parts}{Total\ Number\ of\ Parts}$$

Higher PCP values indicate better pose estimation performance.

Although PCP was widely used in early HPE research, it suffers from several limitations. In particular, it penalizes short limbs more heavily than long limbs

because the acceptable localization threshold depends on limb length. Due to these limitations, PCP has largely been replaced by more robust metrics such as PCK and OKS.

1.5.2 Percentage of Correct Keypoints (PCK)

Percentage of Correct Keypoints (PCK) is one of the most widely used evaluation metrics in both 2D and 3D pose estimation. Unlike PCP, PCK evaluates each joint independently.

A predicted keypoint is considered correct if the Euclidean distance between the predicted joint and the ground-truth joint is smaller than a predefined fraction of a reference body segment, such as torso length or head size.

The condition for correctness is defined as:

where:

- $\hat{J}_i$ represents the predicted joint position,
- j_i represents the ground-truth joint position,
- L is the reference body length,
- α is the threshold factor.

The overall PCK score is computed as:

$$\text{PCK} = \frac{Number\ of\ Correct\ Keypoints}{Total\ Number\ of\ \ Keypoints}$$

Common variants include:

- **PCK@0.2:** threshold equals 20% of torso length.
- **PCKh@0.5:** threshold equals 50% of head segment length.

PCK is extensively used in benchmark datasets such as MPII and LSP due to its simplicity and interpretability.

1.5.3 Percentage of Detected Joints (PDJ)

Percentage of Detected Joints (PDJ) is closely related to PCK but uses the torso diameter or torso diagonal as the normalization reference.

A joint prediction is considered correct if:

$$\| \hat{J}_i - J_i \|_2 < \alpha \times D_{torso}$$

Where:

- D_{torso} is the torso diameter or torso diagonal,
- α is a predefined threshold value.

The PDJ metric is calculated as:

$$\text{PDJ} = \frac{Number\ of\ Correctly\ Detected\ Joints}{Total\ Number\ of\ Joints}$$

PDJ reduces the bias toward shorter limbs that exists in PCP and is commonly applied in 2D pose estimation tasks.

1.5.4 Mean Per Joint Position Error (MPJPE)

Mean Per Joint Position Error (MPJPE) is the most fundamental evaluation metric for 3D Human Pose Estimation. It measures the average Euclidean distance between predicted and ground-truth joint positions.

The metric is computed as:

$$MPJPE = \frac{1}{N} \sum_{i=1}^{N} \| \hat{J}_i - J_i \|_2$$

Where:

$$\| \hat{J}_i - J_i \|_2 < \alpha \times L$$

- N is the number of joints,
- $\hat{J}_i$ represents the predicted 3D joint coordinate,
- J_i represents the ground-truth 3D joint coordinate

In most cases, MPJPE is computed after aligning both predicted and ground-truth poses relative to a root joint, typically the pelvis, to eliminate global translation ambiguity.

A lower MPJPE value indicates better performance.

Despite its popularity, MPJPE penalizes rigid transformations such as rotation or translation errors, even if the predicted body structure itself is accurate.

1.5.5 Object Keypoint Similarity (OKS)

Object Keypoint Similarity (OKS) is the standard evaluation metric used in the COCO benchmark for multi-person 2D pose estimation.

OKS measures the similarity between predicted and ground-truth keypoints while accounting for object scale and keypoint-specific uncertainty.

The metric is defined as:

$$OKS = \frac{\sum_i \exp\left(-\frac{d_i^2}{2s^2\ \sigma_i^2}\right)\delta(v_i > 0)}{\sum_i \delta(v_i > 0)}$$

Where:

- d_i is the Euclidean distance between predicted and ground-truth keypoints,
- s represents the object scale,
- σ_i is a keypoint-specific normalization constant,
- v_i indicates keypoint visibility,
- $\delta\left(v_i > 0\right)$ is an indicator function for visible keypoints.

An OKS value of 1 indicates perfect prediction accuracy.

OKS is considered superior to earlier metrics because it is scale-invariant and accounts for the varying localization difficulty of different body joints.

1.5.6 Average Precision (AP) and Average Recall (AR)

Average Precision (AP) and Average Recall (AR) are derived from OKS and are the primary evaluation metrics used in the COCO benchmark.

A predicted pose is considered a true positive if its OKS exceeds a predefined threshold.

The Average Precision metric is calculated as:

$$AP = \int_0^1 P\left(r\right) dr$$

where:

- $P\left(r\right)$ represents precision as a function of recall.

COCO evaluates AP across multiple OKS thresholds ranging from 0.50 to 0.95.

Important variants include:

- $AP50$: OKS threshold = 0.50
- $AP75$: OKS threshold = 0.75
- AP_M: AP for medium-sized persons
- AP_L: AP for large-sized persons

Average Recall (AR) evaluates the maximum recall achievable across different OKS thresholds and detection counts.

These metrics are especially important in multi-person pose estimation benchmarks.

1.5.7 Area Under Curve (AUC)

Area Under Curve (AUC) summarizes pose estimation performance across multiple thresholds.

It is computed by plotting PCK values over a range of thresholds and calculating the area under the resulting curve:

$$AUC = \int_0^1 PCK\left(\alpha\right) d\alpha$$

where:

- $PCK\left(\alpha\right)$ represents the PCK score at threshold α.

AUC provides a more comprehensive evaluation than single-threshold metrics because it captures performance across varying error tolerances.

1.5.8 Normalized Mean Error (NME)

Normalized Mean Error (NME) measures the average localization error normalized by a reference distance such as inter-ocular distance, torso size, or bounding-box dimensions.

The metric is defined as:

$$NME = \frac{1}{N} \sum_{i=1}^{N} \frac{\parallel \hat{J}_i - J_i \parallel_2}{d}$$

where:

- d is the normalization reference distance,
- N is the number of joints.

NME is commonly used in facial landmark detection and fine-grained pose estimation tasks where precise localization is required.

Lower NME values indicate higher localization accuracy.

1.6 Benchmark Datasets

Benchmark datasets play a fundamental role in the development and evaluation of Human Pose Estimation (HPE) systems. They provide standardized data for training, validation, and performance comparison across different algorithms. High-quality datasets typically include diverse poses, varying viewpoints, occlusions, complex backgrounds, and accurate keypoint annotations, enabling models to generalize effectively to real-world scenarios.

Over the years, several benchmark datasets have become widely adopted in both 2D and 3D pose estimation research. These datasets differ in scale, annotation style, environmental complexity, and application focus.

1.6.1 MPII Human Pose Dataset

The MPII Human Pose Dataset is one of the most widely used benchmarks for single-person 2D human pose estimation. Developed by the Max Planck Institute for Informatics, the dataset contains approximately 25,000 images with more than 40,000 annotated human instances.

The images were collected from YouTube videos and organized according to a taxonomy of everyday human activities covering more than 400 activity categories. Each person instance is annotated with 16 body keypoints representing major human joints.

A major strength of MPII is its diversity in pose configurations, activities, viewpoints, and scene contexts. In addition to standard 2D annotations, the dataset includes occlusion labels and coarse 3D orientation information for the torso and head in the test set.

Key Characteristics

- Approximately 25,000 images
- More than 40,000 annotated persons

- 16 body keypoints
- Diverse real-world daily activities
- Rich contextual and occlusion annotations

Common Evaluation Metric

- PCKh@0.5

MPII remains one of the standard benchmarks for evaluating single-person pose estimation models.

1.6.2 COCO Keypoints Dataset

The COCO Keypoints Dataset, introduced as part of the Microsoft Common Objects in Context (COCO) benchmark, is currently the dominant benchmark for multi-person 2D pose estimation.

The dataset contains more than 200,000 images and over 250,000 annotated person instances. Each individual is labeled with 17 anatomical keypoints, including facial landmarks and major body joints such as shoulders, elbows, wrists, hips, knees, and ankles.

Each keypoint annotation includes:

- 2D coordinates (x, y)
- Visibility labels indicating whether the keypoint is visible or occluded

COCO is highly challenging due to its large diversity in scene complexity, crowd density, viewpoint variation, object scale, and real-world environmental conditions.

Key Characteristics

- Over 200,000 images
- More than 250,000 person annotations
- 17 body keypoints
- Complex real-world scenes
- Multi-person interactions and crowd scenarios

Common Evaluation Metrics

- OKS
- AP and AR

Due to its scale and difficulty, COCO has become the standard benchmark for evaluating state-of-the-art multi-person pose estimation methods.

1.6.3 Leeds Sports Pose (LSP) and LSP Extended (LSPE)

The Leeds Sports Pose (LSP) dataset is one of the earliest benchmark datasets developed for human pose estimation research. It focuses primarily on sports activities involving highly articulated and dynamic human poses.

The original LSP dataset contains 2,000 images divided equally into training and testing sets. The images depict activities such as football, gymnastics, athletics, and skateboarding. Each image is annotated with 14 body keypoints.

An extended version, known as LSP Extended (LSPE), introduced an additional 10,000 training images to improve dataset diversity and model robustness.

Key Characteristics

- LSP: 2,000 images
- LSPE: 10,000 additional training images
- 14 body keypoints
- Sports-oriented dynamic poses
- Challenging body articulations and backgrounds

Although newer datasets such as COCO and MPII have largely superseded LSP, it remains historically important in the development of early pose estimation algorithms.

1.6.4 OCHuman Dataset

The OCHuman Dataset was specifically designed to address the challenge of severe occlusion in multi-person pose estimation.

The dataset contains approximately 6,700 images and more than 11,000 human instances with heavy occlusion. Most human instances exhibit an occlusion ratio exceeding 60%, making OCHuman one of the most challenging benchmarks in human-centric computer vision.

Each person instance includes:

- Bounding box annotations
- Keypoint annotations
- Segmentation masks
- Visibility flags for occluded joints

Key Characteristics

- Severe human occlusion
- Dense crowd interactions
- Fine-grained visibility annotations
- High difficulty level

OCHuman is widely used to evaluate the robustness of pose estimation models under challenging occlusion conditions.

1.6.5 CrowdPose Dataset

The CrowdPose Dataset was developed specifically for evaluating pose estimation performance in crowded multi-person scenes.

The dataset contains approximately 20,000 images with more than 80,000 annotated human instances. Each person is labeled using the same 14-keypoint format as COCO.

A unique contribution of CrowdPose is the introduction of the **Crowd Index**, a metric designed to quantify crowd density and occlusion complexity. Based on this index, the dataset is divided into:

- Easy
- Medium
- Hard

subsets to facilitate balanced evaluation under varying crowd conditions.

Key Characteristics

- Approximately 20,000 images
- More than 80,000 person instances
- High overlap between individuals
- Challenging joint association problems

CrowdPose is commonly used to benchmark algorithms designed for dense multi-person environments.

1.6.6 Human3.6M Dataset

Human3.6M is the largest and most widely used benchmark dataset for 3D Human Pose Estimation.

The dataset contains approximately 3.6 million video frames recorded from 11 professional actors performing 15 everyday activities, including walking, sitting, eating, discussing, and greeting.

Data acquisition was performed using a high-precision motion capture system within a controlled indoor laboratory environment. The dataset provides accurate 3D joint coordinates in both camera and world coordinate systems.

Although the full dataset contains 32 joints, most evaluation protocols reduce the representation to 17 major joints.

Key Characteristics

- 3.6 million video frames

- 11 actors
- 17 activity categories
- Accurate 3D ground-truth annotations
- Controlled laboratory environment

Common Evaluation Metrics

- MPJPE
- P-MPJPE
- 3D PCK

Despite its importance, Human3.6M suffers from limited environmental diversity because all recordings were captured indoors under controlled conditions. Consequently, models trained exclusively on Human3.6M often struggle to generalize to real-world outdoor scenes.

1.6.7 PoseTrack Dataset

PoseTrack is a large-scale benchmark designed for video-based multi-person pose estimation and pose tracking.

Unlike image-based datasets, PoseTrack focuses on temporal consistency across video frames. It includes annotations for body keypoints as well as identity tracking labels across sequences.

Applications

- Video pose estimation
- Multi-person tracking
- Temporal consistency evaluation

PoseTrack is widely used in research involving motion analysis and real-time video understanding.

1.6.8 AI Challenger Human Keypoint Dataset

The AI Challenger dataset is a large-scale benchmark for 2D human keypoint detection developed to support robust large-scale pose estimation research.

It contains hundreds of thousands of annotated human instances with significant diversity in appearance, pose, and environmental conditions.

Key Characteristics

- Large-scale annotations
- Diverse viewpoints and occlusions
- Complex real-world scenarios

This dataset is commonly used for large-scale training of deep learning models.

1.6.9 3DPW (3D Poses in the Wild)

3DPW is one of the most important datasets for evaluating 3D pose estimation in unconstrained real-world environments.

Unlike Human3.6M, which is recorded in controlled indoor laboratories, 3DPW captures outdoor scenes with natural lighting, camera motion, and background complexity.

The dataset provides:

- 3D body pose annotations
- Camera parameters
- Ground-truth body meshes

Importance

- Real-world outdoor evaluation
- Strong benchmark for generalization capability
- Widely used for testing robust monocular 3D HPE systems

CHAPTER 2

Human Pose Estimation Approaches

2.1 Human Body Models

Human Pose Estimation (HPE) relies heavily on human body models to represent the structural configuration of the human body from visual input data. These models provide the underlying representation used to detect, describe, and predict human posture in both 2D and 3D spaces. By modeling the relationships between body parts, pose estimation systems can infer anatomically plausible poses from images or video sequences.

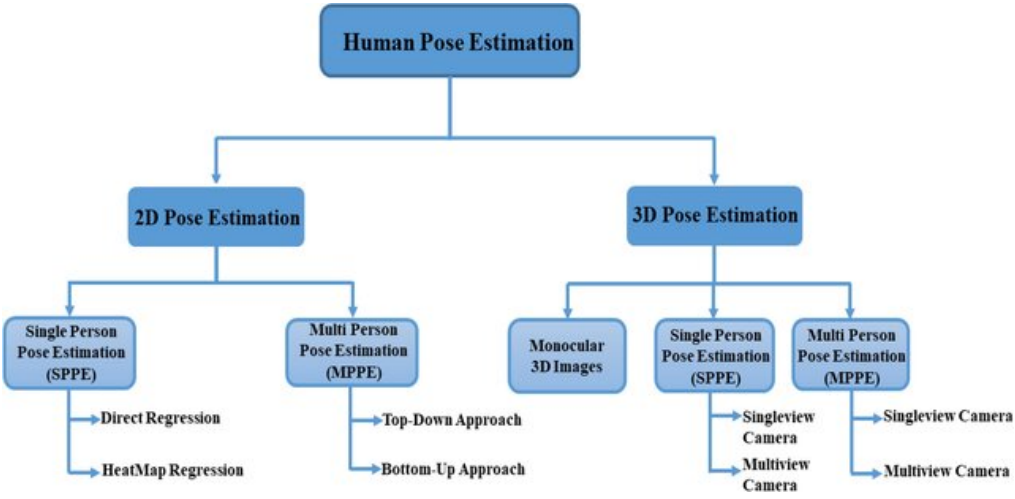


Figure 2-1. the taxonomy in Human Pose Estimation approach

Human body models are fundamental to pose estimation because they define how the body is represented computationally. They also provide constraints that help reduce ambiguity during pose inference. In general, human body models can be categorized into three primary types:

1. Kinematic Models
2. Planar Models
3. Volumetric Models

Each model differs in representation complexity, computational cost, and ability to capture human body geometry and appearance.

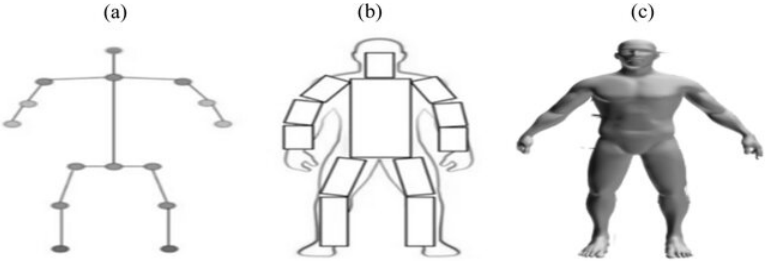


Figure 2-2.Human body models from the left skeleton-based model; contour-based model; volume-based model

2.1.1 Kinematic Models (Skeleton-Based Models)

Definition

Kinematic models, also known as skeleton-based or stick-figure models, represent the human body as an articulated structure composed of rigid bones connected through joints. In this representation, the human body is reduced to a graph-like topology in which:

- Nodes represent body joints (e.g., shoulders, elbows, knees).
- Edges represent bones connecting adjacent joints.

These models focus primarily on the geometric and anatomical structure of the body rather than its appearance or surface details.

Mathematical Representation

A kinematic skeleton with KKK joints can be represented as:

$$J = \{j_1, j_2, \ldots\ldots, j_K\}$$

Where $j_i \in R^3$ for 3D pose estimation or, $j_i \in R^2$ for 2D pose estimation.
Alternatively, the skeleton may be parameterized using:

- Joint angles θ
- Bone lengths L

relative to a root joint, typically the pelvis.

The resulting structure forms a kinematic tree with branches corresponding to the spine, arms, and legs.

Degrees of Freedom (DoF)

Each joint possesses a specific number of rotational degrees of freedom:

- **Ball joints** (shoulders and hips): 3 DoF
- **Hinge joints** (knees and elbows): 1 DoF
- **Ellipsoidal or saddle joints** (wrists and ankles): 2 DoF

A full-body human skeleton may contain between 20 and 60 or more degrees of freedom depending on modeling granularity.

Role in Human Pose Estimation

Kinematic models form the foundation of most modern pose estimation systems. Even when advanced neural architectures are employed internally, the final output is typically represented as a skeleton.

These models are also used to enforce anatomical constraints, including:

- Bone-length consistency
- Joint-angle limitations
- Temporal smoothness across frames

Such constraints help generate anatomically plausible and physically realistic poses.

Advantages

- Lightweight and computationally efficient
- Anatomically interpretable
- Suitable for biomechanics and motion analysis
- Efficient forward kinematics computation

Limitations

- No representation of body surface or volume
- Cannot model clothing or appearance
- Unable to capture body thickness and self-occlusion
- Multiple body configurations may produce similar skeletal structure

2.1.2 Planar Models (2D Part-Based Models)

Definition

Planar models extend skeleton-based representations by attaching 2D geometric primitives—such as rectangles, ellipses, or patches—to body parts. These models represent the human body as a collection of flat components arranged within the image plane according to skeletal constraints.

Unlike purely kinematic representations, planar models incorporate appearance information extracted directly from the image.

Mathematical Representation

Each body part iii is represented by a set of parameters:

$$P_i = (x_i, y_i, \theta_i, w_i, h_i)$$

where:

- (x_i, y_i) denotes the part position,
- θ_i represents orientation,
- w_i and h_i correspond to width and height.

In elliptical representations, body parts may also be modeled using covariance matrices: $\sum_i$

which describe the spatial spread of the ellipse.

The complete human body is represented as a graphical model in which:

- Nodes correspond to body parts,
- Edges define spatial relationships and geometric constraints between connected parts.

Pictorial Structure Framework

One of the most influential planar modeling approaches is the **Pictorial Structures Framework**. In this framework:

- The body is decomposed into multiple parts,
- Each part is associated with local image features,

- Pairwise geometric constraints enforce plausible body configurations.

The objective function typically combines:

- Appearance likelihoods
- Spatial deformation costs

to determine the globally optimal body configuration.

Inference is commonly performed using:

- Dynamic programming
- Belief propagation
- Distance transform optimization

Variants of Planar Models

Several important variants of planar models have been proposed:

- **Cardboard Models:** represent limbs using rectangular patches with fixed aspect ratios.
- **Flexible Mixtures of Parts:** utilize multiple appearance templates to handle viewpoint variations and foreshortening.
- **Constellation Models:** represent body parts as Gaussian distributions within the image plane.

Role in Human Pose Estimation

Before the emergence of deep learning, planar models dominated human pose estimation research due to their ability to jointly model:

- Part appearance
- Spatial relationships
- Occlusion reasoning

These models enabled robust body-part localization by combining local image evidence with structural body constraints.

Advantages

- Explicit modeling of spatial relationships
- Efficient global optimization
- Ability to reason about occluded body parts
- Adaptability to viewpoint variations

Limitations

- Limited to 2D image space
- No depth or volumetric reasoning
- Rigid appearance templates struggle under clothing and lighting variations
- Cannot accurately model self-occlusion in 3D space

2.1.3 Volumetric Models

Definition

Volumetric models provide a complete three-dimensional representation of the human body, including body shape, surface geometry, and physical volume. Unlike skeleton-based or planar representations, volumetric models capture the full body structure and appearance.

These models are commonly represented using:

- 3D meshes
- Cylinders
- Spheres
- Parametric body surfaces

Volumetric representations are widely used in advanced 3D Human Pose Estimation, animation, biomechanics, and virtual reality applications.

Parametric Human Body Models

Modern volumetric approaches often rely on parametric body models such as:

- Skinned Multi-Person Linear (SMPL)

- SCAPE
- STAR

These models represent the human body using:

- Pose parameters
- Shape parameters
- Surface mesh deformations

The body surface is controlled by an embedded kinematic skeleton, allowing realistic articulation and deformation.

Mathematical Representation

A volumetric mesh can be represented as:

$$M(\beta, \theta)$$

where:

- β represents body shape parameters,
- θ denotes pose parameters.

The resulting mesh contains thousands of 3D vertices that define the body surface geometry.

Role in Human Pose Estimation

Volumetric models are essential for:

- 3D body reconstruction
- Motion capture
- Virtual avatar generation
- Human animation
- Biomechanical analysis

They provide significantly richer information than skeletal representations by modeling both pose and body shape simultaneously.

Advantages

- Realistic representation of body shape and surface
- Accurate modeling of self-occlusion and depth
- Suitable for animation and virtual environments
- Enables full-body reconstruction

Limitations

- High computational complexity
- Large memory requirements
- Difficult optimization and training procedures
- Requires large-scale annotated 3D datasets

Integration of Human Body Models in Modern HPE Systems

Modern Human Pose Estimation systems often combine multiple body model representations to leverage their respective strengths.

A common hierarchical pipeline includes:

1. Volumetric Model
   Provides full 3D body shape and surface geometry.
2. Kinematic Skeleton
   Acts as the internal articulation structure controlling body motion.
3. Planar Projection
   Projects the 3D model onto the 2D image plane for comparison with visual evidence.

This integration allows modern systems to:

- Maintain anatomical consistency,
- Align predicted poses with image observations,
- Generate realistic 3D reconstructions,
- Improve robustness under occlusion and viewpoint changes.

2.2 2D Human Pose Estimation

Introduction

2D Human Pose Estimation (2D HPE) is a computer vision task that focuses on detecting and localizing anatomical keypoints of the human body from a single RGB image or video frame. The objective is to estimate the spatial locations of body joints, such as the shoulders, elbows, wrists, hips, knees, and ankles, in the image plane using two-dimensional coordinates (x, y). The final output is commonly represented as a skeletal structure connecting the detected keypoints.

2D pose estimation serves as a fundamental component in numerous applications, including activity recognition, sports analytics, motion tracking, surveillance, animation, healthcare, robotics, and augmented reality. Although the task appears straightforward, accurate localization of body joints is challenging due to variations in body posture, clothing, illumination, viewpoint, occlusion, and background complexity.

2.2.1 Classical Approaches for 2D HPE

Before the emergence of deep learning, traditional computer vision methods dominated the field of 2D human pose estimation. These approaches relied heavily on handcrafted features and geometric body representations.

Early systems often represented the human body using simple geometric primitives such as cylinders, rectangles, or articulated stick figures. The pose was estimated by detecting individual body parts and modeling the spatial relationships between them.

Several classical techniques were widely adopted, including:

- Histogram of Oriented Gradients (HOG)
- Edgelet features
- Pictorial Structure Models (PSM)
- Part-based graphical models

Pictorial Structure Models

One of the most influential classical approaches was the Pictorial Structure Model. In this framework, the human body is represented as a collection of interconnected body parts arranged in a tree-structured graphical model.

Each body part is described by:

- A local appearance model
- Spatial constraints relative to neighboring parts

The model combines two important components:

1. **Appearance term:** Measures how well a body part matches local image evidence.
2. **Deformation term:** Enforces geometric consistency between connected body parts.

The tree-based structure enabled efficient inference using dynamic programming techniques. Pictorial structures became highly influential because they provided a principled framework for modeling articulated human bodies.

Despite their effectiveness, classical methods suffered from several major limitations:

- Dependence on handcrafted features
- Weak robustness to illumination and clothing variations
- Difficulty handling severe occlusion
- Limited global contextual reasoning
- High computational complexity
- Poor generalization to unconstrained real-world environments

These limitations motivated the transition toward deep learning-based pose estimation systems.

2.2.2 Single-Person 2D Human Pose Estimation

Single-Person Pose Estimation (SPPE) assumes that only one person is

present in the image or that the target person has already been isolated using a detector. The primary objective is to localize the anatomical keypoints of that individual accurately.

In multi-person scenes, a preprocessing stage is often applied to crop the image around each detected person before performing pose estimation.

Modern SPPE methods are generally categorized into two major approaches:

- Regression-Based Methods
- Heatmap-Based Methods

2.2.2.1 Regression-Based Methods

Regression-based approaches directly predict joint coordinates from image features. Instead of producing intermediate representations, the network maps the input image to a vector containing the coordinates of all body joints.

For a human body with K keypoints, the predicted pose can be represented as:

$$P = \{(x_1, y_1), (x_1, y_2), \ldots, (x_K, y_K)\}$$

For example, if 17 joints are estimated, the network outputs a vector of size:

$$17 \times 2$$

containing the (x, y) coordinates of each keypoint.

Characteristics of Regression-Based Methods

- Simple end-to-end architecture
- Direct coordinate prediction
- Lower computational overhead
- Faster inference speed

Limitations

Direct coordinate regression is highly nonlinear and difficult to optimize. Small localization errors can significantly affect the training process, leading to unstable convergence.

Additionally, regression-based methods often struggle to preserve spatial relationships between neighboring pixels because the final coordinate vector discards detailed spatial information present in feature maps.

Although these approaches are computationally efficient, they are generally less accurate than heatmap-based methods.

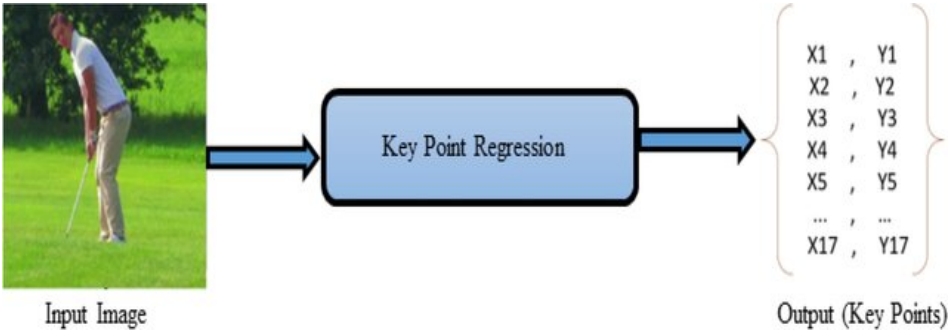


Figure 2-3. KeyPoint regression

2.2.2.2 Heatmap-Based Methods

Heatmap-based methods have become the dominant paradigm in modern 2D human pose estimation.

Instead of directly predicting coordinates, the network predicts a probability heatmap for each keypoint. Each heatmap indicates the likelihood of the corresponding joint appearing at every pixel location in the image.

For a joint j, the network predicts a heatmap: $H_j(x, y)$

where high-intensity regions correspond to high confidence for the joint location.

The final joint position is obtained by selecting the pixel with the maximum response: $(x_j, y_j) = argmax H_j(x, y)$

Advantages of Heatmap-Based Methods

- Preserve spatial information
- Easier optimization during training
- Higher localization accuracy
- More robust to small annotation noise
- Better handling of uncertainty

Heatmap regression naturally transforms pose estimation into a dense prediction problem, making it highly compatible with convolutional neural networks.

Challenges of Heatmap Regression

Despite their effectiveness, heatmap-based approaches introduce two important challenges:

1. Encoding Problem

Ground-truth coordinates must be converted into Gaussian heatmaps during training.

2. Decoding Problem

The predicted heatmaps must be converted back into precise coordinate values during inference.

Nevertheless, heatmap-based approaches consistently outperform direct regression methods and remain the standard choice in state-of-the-art systems.

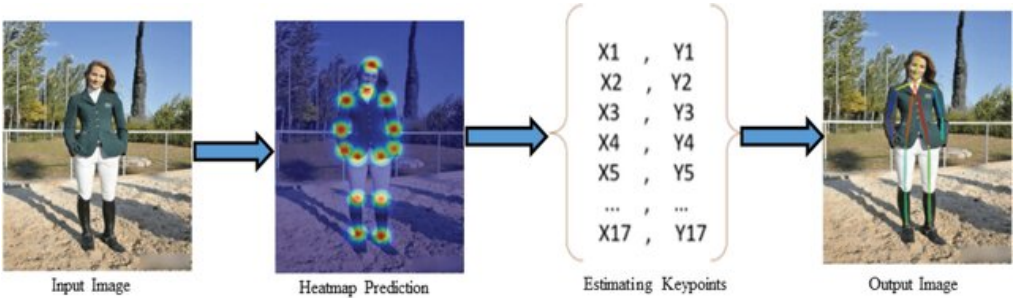


Figure 2-4. Pose estimation using heat map regression

2.2.3 Comparison Between Regression and Heatmap

Aspect	Regression-Based	Heatmap-Based
Output	Direct coordinates	Probability heatmaps
Spatial awareness	Limited	Strong
Optimization difficulty	High	Lower
Accuracy	Moderate	High
Computational cost	Lower	Higher
Robustness	Lower	Higher
Dominant in modern systems	No	Yes

Regression-based methods are generally faster and simpler, whereas heatmap-based methods achieve significantly better localization accuracy due to their ability to preserve spatial structure.

Multi-Person 2D Human Pose Estimation

Multi-Person Pose Estimation aims to detect and estimate the poses of all individuals present in a scene simultaneously. This problem is considerably more challenging than single-person estimation because:

- The number of people is unknown
- Human bodies may overlap
- Occlusions are frequent
- Limb association becomes ambiguous

The central challenge is the **association problem**, which involves determining which detected joints belong to the same person.

Modern multi-person pose estimation approaches are divided into two primary paradigms:

1. Top-Down Approaches
2. Bottom-Up Approaches

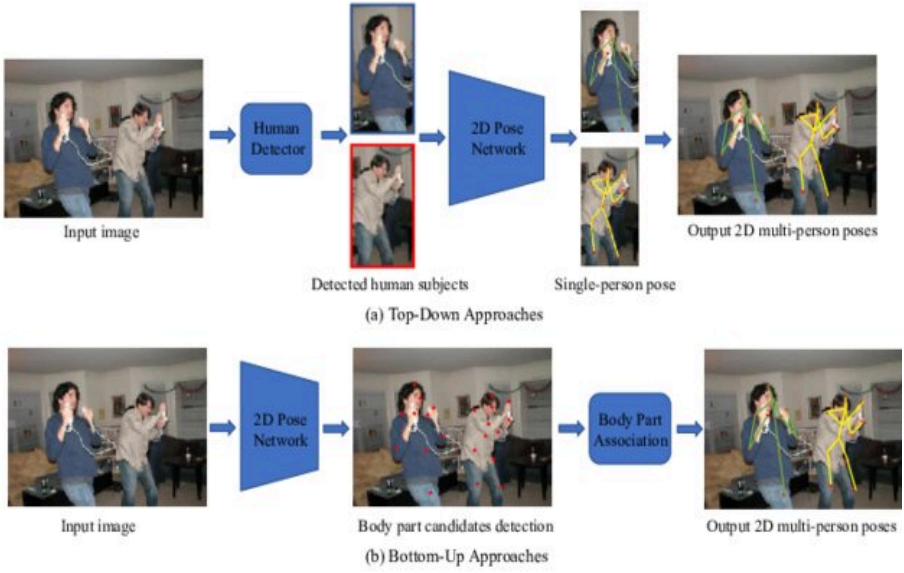


Figure 3-5. (a) Top-Down Approach and (b) Bottom-Up Approach

Top-Down Approaches

The top-down strategy follows a two-stage pipeline:

1. Detect all persons in the image
2. Perform single-person pose estimation for each detected individual

A human detector first generates bounding boxes around each person using object detection networks such as:

- Faster R-CNN
- YOLO
- Mask R-CNN
- RT-DETR

Each cropped region is then processed independently by a single-person pose estimator.

Characteristics

- High localization accuracy
- Strong performance in moderate crowd scenarios
- Better joint association reliability

Limitations

- Computational cost increases linearly with the number of people:

$$O(n)$$

where n is the number of detected individuals.

- Performance degrades in heavily crowded scenes due to overlapping bounding boxes.

Representative Models

- Mask R-CNN
- AlphaPose
- HRNet

- SimpleBaseline

Bottom-Up Approaches

Bottom-up approaches reverse the processing order.

Instead of detecting persons first, the model initially detects all body joints in the image and then groups them into individual skeletons.

The pipeline generally consists of:

1. Joint detection
2. Joint association
3. Skeleton construction

Additional features such as affinity fields or embedding vectors are often used to determine which joints belong together.

Characteristics

- Near-constant inference time regardless of crowd size
- Efficient for dense crowd scenes
- Better scalability

Limitations

- Joint association is highly challenging
- More sensitive to occlusion
- Slightly lower accuracy in complex scenes

Representative Models

- OpenPose
- PifPaf
- HigherHRNet
- DEKR

Comparison Between Top-Down and Bottom-Up Approaches

Aspect	Top-Down	Bottom-Up
Processing order	Detect people first	Detect joints first
Accuracy	Generally higher	Slightly lower
Speed in crowded scenes	Slower	Faster
Scalability	Limited	Better
Joint association difficulty	Lower	Higher
Occlusion handling	Moderate	Challenging

Top-down methods generally provide higher accuracy, whereas bottom-up methods offer superior scalability and efficiency in crowded scenes.

2.3 3D Human Pose Estimation

Introduction

3D Human Pose Estimation (3D HPE) is a computer vision task that focuses on predicting the three-dimensional spatial coordinates of anatomical keypoints of the human body from images or video sequences. Unlike 2D pose estimation, which estimates only image-plane coordinates (x, y), 3D HPE reconstructs the complete spatial configuration of the human body by estimating (x, y, z)coordinates for each joint.

The primary objective of 3D HPE is to recover the articulated structure of the human body in three-dimensional space from visual observations. The output is typically represented as a 3D skeletal model composed of interconnected joints such as the shoulders, elbows, wrists, hips, knees, and ankles.

3D pose estimation plays a critical role in numerous applications, including:

- Motion capture
- Sports biomechanics
- Robotics
- Human-computer interaction
- Autonomous systems
- Healthcare and rehabilitation
- Animation and virtual reality
- Surveillance and activity analysis

Compared to 2D HPE, estimating 3D human pose is substantially more challenging because a three-dimensional scene is projected onto a two-dimensional image plane, resulting in significant depth ambiguity and information loss.

Challenges in 3D Pose Estimation

Recovering accurate 3D body structure from monocular RGB images is an inherently ill-posed problem because multiple 3D poses can produce identical 2D projections. Consequently, the model must infer missing depth information from contextual and anatomical priors.

Several factors further complicate 3D pose estimation:

- Self-occlusion and inter-person occlusion
- Variations in viewpoint and camera perspective
- Complex human articulations
- Clothing and appearance diversity
- Background clutter
- Lighting variations
- Motion blur
- Limited availability of accurate 3D annotations

In addition, collecting large-scale 3D datasets is significantly more difficult than collecting 2D datasets because obtaining accurate ground-truth 3D coordinates typically requires expensive motion capture systems, synchronized cameras, or depth sensors.

2.3.1 Classical Approaches for 3D HPE

Early approaches to 3D human pose estimation extended traditional 2D methods into the three-dimensional domain.

Pictorial Structure Models

One of the earliest approaches involved extending Pictorial Structure Models (PSMs) to 3D pose estimation. In these methods, the human body is modeled as a collection of articulated body parts connected through spatial constraints.

The model attempts to infer the optimal 3D body configuration by combining:

- Local image evidence
- Geometric constraints
- Spatial relationships between joints

Although these methods provided structured representations of the human body, they suffered from several limitations:

- High computational complexity
- Weak robustness to occlusion
- Limited scalability
- Difficulty modeling complex articulations

Feature-Based Methods

Classical methods also relied heavily on handcrafted visual descriptors such as:

- Histogram of Oriented Gradients (HOG)
- Edge-based descriptors
- Silhouette features

HOG descriptors were particularly popular because they captured local shape and appearance information effectively.

In many systems, dimensionality reduction techniques such as Principal Component Analysis (PCA) were applied to reduce the high dimensionality of

feature representations before regression techniques were used to estimate joint coordinates.

However, handcrafted feature-based approaches lacked robustness under unconstrained real-world conditions and were eventually surpassed by deep learning-based methods.

2.3.2 Monocular 3D Human Pose Estimation

Monocular 3D Human Pose Estimation refers to estimating 3D human pose from a single RGB image or video stream captured by a single camera.

This is considered one of the most difficult problems in computer vision because a single image contains no explicit depth information.

Despite its difficulty, monocular 3D HPE has become one of the most widely researched areas due to the accessibility and low cost of RGB cameras.

Modern monocular 3D HPE methods are generally divided into several categories:

1. Direct Regression Methods
2. 2D-to-3D Lifting Methods
3. Volumetric Heatmap Methods
4. Parametric Model-Based Methods

2.3.2.1 Direct Regression Methods

Direct regression approaches attempt to map raw image pixels directly to 3D joint coordinates using deep neural networks.

For a skeleton with K joints, the predicted pose can be represented as:

$$P = (x_1, y_1, z_1), (x_2, y_2, z_2), \ldots, (x_K, y_K, z_K)$$

The network directly outputs a vector of size: $3K$

containing the spatial coordinates of all joints.

Advantages

- End-to-end learning framework
- No intermediate processing stages
- Simpler inference pipeline

Limitations

- Highly nonlinear mapping problem
- Difficult optimization process
- Sensitive to viewpoint and occlusion
- Requires large-scale training data

Because of these limitations, direct regression methods are generally less accurate than modern lifting-based approaches.

2.3.2.2 2D-to-3D Lifting Methods

The most dominant paradigm in modern monocular 3D HPE is the 2D-to-3D lifting approach.

This strategy divides the problem into two stages:

1. Estimate accurate 2D keypoints
2. Lift the 2D skeleton into 3D space

The process can be represented as:

$$I \rightarrow P_2D \rightarrow P_3D$$

where:

- I represents the input image
- P_{2D} denotes the estimated 2D pose
- P_{3D} denotes the reconstructed 3D pose

This approach became highly successful because 2D pose estimation had already achieved high accuracy using mature deep learning architectures.

Advantages

- More stable optimization
- Better generalization
- Reduced complexity
- Ability to leverage pretrained 2D pose estimators

Limitations

- Strong dependence on 2D keypoint quality
- Depth ambiguity remains unresolved
- Difficulty estimating absolute scale and global position

Despite these challenges, 2D-to-3D lifting remains the dominant approach in state-of-the-art monocular 3D HPE systems.

2.3.2.3 Volumetric Heatmap Methods

Volumetric methods extend the concept of 2D heatmaps into three-dimensional space.

Instead of predicting 2D probability maps, the model predicts a 3D voxel grid representing the likelihood of joint locations in space.

For each joint j, the network predicts:

$$H_j(x, y, z)$$

where each voxel represents the probability of the joint appearing at a specific spatial location.

Advantages

- High localization accuracy

- Better spatial reasoning
- Strong geometric consistency

Limitations

- Extremely high memory consumption
- Large computational cost
- Difficult deployment in real-time systems

Although volumetric heatmaps provide accurate predictions, their computational requirements remain a major limitation.

Parametric Body Model Approaches

Another major direction in modern 3D HPE involves predicting parameters of statistical human body models rather than directly estimating joints.

SMPL Model

The most widely used parametric body model is the SMPL (Skinned Multi-Person Linear) model.

SMPL represents the human body as a dense triangulated mesh composed of thousands of vertices controlled by:

- Shape parameters β
- Pose parameters θ

Shape Parameters

The shape vector β\betaβ controls body characteristics such as:

- Height
- Weight
- Body proportions
- Musculature

Pose Parameters

The pose vector θ represents joint rotations and body articulation.

The SMPL function can be expressed as:

$$M(\beta, \theta)$$

where M generates a complete 3D human mesh from the shape and pose parameters.

Advantages

- Realistic full-body surface reconstruction
- Anatomically consistent body shapes
- Suitable for animation and graphics applications

Limitations

- More computationally expensive
- Optimization can be unstable
- Requires accurate body priors

Parametric body models have become highly influential in applications involving realistic human reconstruction and motion synthesis.

Multi-View 3D Pose Estimation

Multi-view 3D HPE utilizes multiple synchronized cameras observing the same subject from different viewpoints.

The general pipeline consists of:

1. Estimating 2D poses in each camera view
2. Triangulating the 3D joint positions geometrically

Because multiple viewpoints provide explicit geometric constraints, multi-view systems significantly reduce depth ambiguity.

Advantages

- Very high accuracy
- Better occlusion handling
- Improved depth perception

Limitations

- Requires camera calibration
- Expensive hardware setup
- Limited scalability in real-world environments

Multi-view systems are widely used in:

- Professional motion capture studios
- Sports biomechanics laboratories
- Research datasets
- Film and animation production

RGB-D Based Pose Estimation

RGB-D systems use both RGB images and depth maps captured by specialized sensors such as:

- Microsoft Kinect
- Intel RealSense
- LiDAR-based systems

Depth information provides explicit spatial measurements, greatly simplifying 3D reconstruction.

Advantages

- Reduced depth ambiguity
- Improved localization accuracy

- Strong indoor performance

Limitations

- Limited outdoor usability
- Infrared interference
- Lower image resolution
- Restricted operating range

RGB-D approaches are commonly used in robotics, gaming, and indoor interaction systems.

2.3.3 Single-Person 3D Pose Estimation

Single-person 3D pose estimation focuses on reconstructing the pose of one individual from an image or video sequence.

Most approaches assume:

- Minimal occlusion
- Clear visibility of body parts
- Controlled or semi-controlled environments

Although monocular depth ambiguity remains challenging, modern deep learning models trained on large-scale datasets can achieve highly accurate predictions for isolated individuals.

Common strategies include:

- Direct regression
- 2D-to-3D lifting
- Volumetric reconstruction
- Parametric mesh recovery

2.3.4 Multi-Person 3D Pose Estimation

Multi-person 3D pose estimation extends the problem to scenes containing multiple interacting individuals.

This is substantially more difficult because it introduces additional challenges:

- Severe inter-person occlusion
- Depth ordering ambiguity
- Complex social interactions
- Joint association problems
- Absolute positioning of multiple skeletons in shared 3D space

Another major limitation is the scarcity of large-scale annotated multi-person 3D datasets captured in unconstrained environments.

Modern systems often combine:

- Multi-person 2D detection
- Joint association mechanisms
- Temporal tracking
- Multi-view geometry
- Transformer-based architectures

to improve robustness in crowded scenes.

CHAPTER 3

HPE Using Deep Learning

3.1 Convolutional Neural Networks for Human Pose Estimation

Introduction

Convolutional Neural Networks (CNNs) represent the first deep learning paradigm that successfully transformed human pose estimation from a hand-crafted feature engineering problem into an end-to-end trainable learning task. Prior to the emergence of CNN-based approaches, pose estimation largely relied on graphical models, pictorial structures, and manually designed appearance descriptors. The introduction of deep convolutional architectures enabled the automatic learning of hierarchical visual representations directly from image data, substantially improving robustness to variations in appearance, scale, viewpoint, illumination, and background clutter.

The success of CNNs in human pose estimation originates from their ability to exploit the spatial structure of images through localized receptive fields, shared parameters, and hierarchical feature extraction. These properties allow CNNs to effectively identify body parts, model spatial relationships among joints, and generate precise localization predictions. Although recent advances increasingly employ transformers, graph neural networks, and diffusion models, CNNs remain the foundation upon which most modern pose estimation systems are built. Even in architectures where pose reasoning is performed by alternative mechanisms, CNNs frequently serve as the primary visual feature extractor.

This chapter examines the fundamental principles underlying CNN-based human pose estimation, the evolution of landmark architectures, and the continuing role of convolutional representations in contemporary pose estimation pipelines.

3.1.1 Fundamental Principles of CNNs for Human Pose Estimation

3.1.1.1 Translation Equivariance

One of the most important properties of convolutional networks is translation equivariance. A convolutional operation satisfies

$$f(T(x)) = T(f(x))$$

where T(.) denotes a spatial translation. In practical terms, if a person shifts position within an image, the corresponding feature responses shift by the same amount. This property is particularly important for human pose estimation because body joints may appear at arbitrary image locations.

Unlike fully connected networks, which must learn separate representations for identical structures appearing at different spatial positions, convolutional networks learn a feature once and apply it throughout the image. Consequently, a detector trained to recognize an elbow can identify elbows regardless of their location within the image.

3.1.1.2 Hierarchical Feature Learning

CNNs construct visual understanding through a hierarchy of increasingly abstract feature representations. Early layers typically capture low-level image characteristics such as edges, corners, and gradients. As depth increases, receptive fields expand and features become progressively more semantic.

Small receptive fields in shallow layers primarily encode local appearance cues, whereas deeper layers capture larger body regions, inter-limb relationships, and eventually entire human configurations. This hierarchical

composition allows CNNs to build pose representations from simple visual primitives to complete body structures.

The progression from local evidence to global context closely resembles human visual perception, where individual body parts are first identified before being integrated into a coherent understanding of overall posture.

3.1.1.3 Weight Sharing and Parameter Efficiency

Convolutional layers employ shared kernels that are applied uniformly across all spatial locations. This parameter sharing dramatically reduces model complexity while improving generalization.

For pose estimation, weight sharing enables a single convolutional filter to detect similar anatomical structures regardless of location. The same filter can respond to elbows, wrists, or knees appearing anywhere within the image, eliminating the need to learn location-specific detectors.

However, this efficiency introduces a limitation. CNNs do not explicitly encode skeletal topology. Anatomical relationships, such as the connection between wrists and elbows, must be learned implicitly from data rather than being enforced through architectural constraints.

3.1.1.4 Multi-Scale Representation Learning

Human bodies appear at a wide range of scales due to variations in camera distance, image resolution, and perspective. CNNs naturally address this challenge through hierarchical feature pyramids generated by successive pooling and downsampling operations.

High-resolution feature maps preserve fine spatial details necessary for accurate joint localization, while low-resolution feature maps capture semantic context and global body structure. Combining information across multiple scales allows pose estimators to localize both small distant individuals and large close-up subjects within the same framework.

Modern architectures frequently employ feature pyramid structures and skip connections to explicitly fuse multi-scale information, enhancing localization accuracy across diverse visual conditions.

3.1.2 Output Representations for Human Pose Estimation

A central challenge in pose estimation is determining how joint locations should be represented and predicted. Two major paradigms have emerged: direct coordinate regression and heatmap regression.

3.1.2.1 Coordinate Regression

Coordinate regression formulates pose estimation as a direct mapping from an input image to a vector of joint coordinates:

$$\hat{y} = f_{CNN}(x;\theta)$$

where $\hat{y}$ contains the predicted two-dimensional or three-dimensional coordinates of all body joints.

Training is typically performed using a mean squared error objective:

$$L_{reg} = \left\| \hat{y} - y_{gt} \right\|^2$$

This formulation was pioneered by DeepPose and represented the first successful application of deep learning to human pose estimation.

Despite its conceptual simplicity, coordinate regression suffers from several important limitations. The coordinate vector lacks explicit spatial structure, making it difficult for the network to represent uncertainty or multimodal predictions. When multiple joint locations are plausible, the network often predicts an average position that may not correspond to a physically valid pose. Furthermore, the model must implicitly learn spatial relationships between joints without direct geometric supervision.

3.1.2.2 Heatmap Regression

Heatmap regression addresses the shortcomings of coordinate prediction by representing each joint as a probability distribution over image space. For each joint K, the network predicts a heatmap:

$$H_k \in R^{H \times W}$$

where high-intensity regions indicate likely joint locations.

Ground-truth heatmaps are typically generated using Gaussian kernels centered at annotated joint positions:

$$H_k(u,v) = \frac{1}{2\pi\sigma^2} exp\left(-\frac{\left(u - xx_k\right)^2 + \left(v - y_k\right)^2}{2\sigma^2}\right)$$

Training is performed through pixel-wise supervision:

$$L_{heatmap} = \sum_{k=1}^{K} \sum_{u,v} \| \widehat{H}^k(u,v) - H_k^{gt}(u,v) \|$$

Heatmaps preserve spatial structure and naturally support multimodal predictions. They enable CNNs to exploit their inherent strengths in dense spatial reasoning and allow uncertainty to be represented explicitly through probability distributions.

The introduction of heatmap regression marked a major turning point in human pose estimation and became the dominant paradigm for nearly a decade.

3.1.3 Spatial Resolution Recovery and Decoder Architectures

Most convolutional backbones progressively reduce feature map resolution through pooling and strided convolutions. Consequently, effective mechanisms are required to restore spatial detail for precise joint localization.

3.1.3.1 Transposed Convolution

Transposed convolutions perform learnable upsampling by reversing the spatial reduction introduced during feature extraction. These layers increase spatial resolution while simultaneously learning how features should be reconstructed.

Although highly effective, transposed convolutions may introduce checkerboard artifacts due to uneven overlap between kernel applications. Such artifacts can create spurious heatmap peaks that negatively affect localization accuracy.

3.1.3.2 Bilinear Upsampling and Convolution

An alternative strategy employs fixed bilinear interpolation followed by standard convolutional refinement. Because interpolation is deterministic, checkerboard artifacts are eliminated.

While the upsampling operation itself is not learnable, subsequent convolutions compensate by refining interpolated features and recovering localization precision.

3.1.3.3 Skip Connections and U-Net Structures

Skip connections directly transfer high-resolution features from encoder stages to corresponding decoder stages. These connections preserve spatial information that would otherwise be lost during downsampling.

Architectures such as the Stacked Hourglass Network extensively exploit skip connections to combine semantic context from deep layers with precise

localization cues from shallow layers. This combination is particularly important for accurately localizing small joints and handling complex articulations.

3.1.4 Dilated Convolutions for Context Modeling

A persistent challenge in pose estimation is balancing contextual understanding with localization precision. Standard pooling operations increase receptive field size but reduce spatial resolution.

Dilated convolutions address this limitation by introducing gaps between kernel elements:

$$y[i] = \sum_{k} x[i + r \cdot k] W[k]$$

where r denotes the dilation rate.

Dilated kernels significantly enlarge the effective receptive field without increasing parameter count or reducing feature map resolution. This enables the network to incorporate broader contextual information while maintaining precise spatial representations.

Such mechanisms are particularly valuable for estimating occluded joints, where information from distant body parts may be required to infer plausible joint locations.

3.1.5 Landmark CNN Architectures for Human Pose Estimation

DeepPose

DeepPose introduced the first successful deep learning framework for human pose estimation by formulating pose prediction as a direct coordinate regression problem. The architecture employed a convolutional backbone followed by fully connected regression layers and a cascaded refinement strategy.

Although later surpassed by heatmap-based methods, DeepPose established the feasibility of end-to-end neural pose estimation and initiated the modern era of deep learning for articulated human analysis.

Convolutional Pose Machines

Convolutional Pose Machines reformulated pose estimation as an iterative refinement process. Multiple sequential stages progressively improved joint confidence maps by incorporating predictions from previous stages.

The architecture demonstrated that convolutional networks could implicitly learn spatial relationships among body joints through repeated context aggregation, providing a differentiable alternative to traditional graphical models.

Stacked Hourglass Networks

The Stacked Hourglass architecture introduced repeated bottom-up and top-down processing through symmetric encoder-decoder modules. Each hourglass captures information across multiple scales while preserving localization accuracy through skip connections.

Intermediate supervision at each stage enables effective optimization of very deep networks and facilitates progressive refinement of pose predictions.

The architecture became one of the most influential designs in human pose estimation and inspired numerous subsequent developments.

Simple Baseline

Simple Baseline demonstrated that strong backbone representations combined with minimal decoder structures can achieve competitive performance.

Using a pretrained ResNet followed by only a few deconvolution layers, the architecture achieved state-of-the-art accuracy while significantly reducing architectural complexity. This work shifted attention toward backbone design

and representation learning rather than increasingly sophisticated decoder engineering.

High-Resolution Network (HRNet)

HRNet fundamentally changed pose estimation architecture design by maintaining high-resolution representations throughout the entire network.

Instead of repeatedly reducing and restoring spatial resolution, HRNet processes multiple resolutions in parallel and continuously exchanges information among them. High-resolution streams retain localization precision, while low-resolution streams contribute semantic context.

The repeated multi-resolution fusion strategy produces highly accurate joint localization and established HRNet as one of the most influential architectures in modern pose estimation.

Cascaded Pyramid Network

The Cascaded Pyramid Network separates pose estimation into coarse global detection and fine local refinement stages.

A global feature pyramid generates initial predictions, while a refinement module focuses specifically on difficult joints through online hard keypoint mining. This targeted refinement strategy improves performance on occluded and challenging body parts.

OpenPose and Part Affinity Fields

OpenPose introduced a fundamentally different bottom-up paradigm for multi-person pose estimation.

Rather than detecting poses person by person, OpenPose first predicts all body joints and then associates them using Part Affinity Fields (PAFs), which encode limb connectivity as vector fields. The resulting framework achieves computational efficiency because runtime remains largely independent of the number of people present in the image.

The concept of Part Affinity Fields remains one of the most influential contributions to multi-person pose estimation.

3.1.6 CNNs as Universal Visual Feature Extractors

Although many state-of-the-art pose estimation systems now employ transformers, graph neural networks, or diffusion models for reasoning, CNNs continue to play a critical role as visual feature extractors.

In transformer-based architectures, convolutional layers often provide patch embeddings and spatially structured visual tokens. In graph neural networks, CNN features serve as node representations associated with detected joints. Diffusion models utilize convolutional features as conditioning signals during denoising, while vision-language systems rely on CNN encoders to align visual information with semantic text representations.

Consequently, CNNs have evolved from complete pose estimation systems into foundational components that support a wide variety of higher-level reasoning architectures.

Design Trade-Offs in CNN-Based Pose Estimation

The design of a CNN-based pose estimator involves several important trade-offs.

Kernel size determines the balance between local detail and contextual awareness. Pooling strategies influence computational efficiency while affecting localization precision. Upsampling mechanisms determine how effectively spatial information can be recovered after downsampling. Normalization methods affect optimization stability and generalization.

A recurring challenge throughout the evolution of CNN architectures has been the tension between contextual understanding and spatial precision. Larger receptive fields improve robustness to occlusion and ambiguity but often reduce localization accuracy. Architectural innovations such as HRNet, skip connections, and dilated convolutions can be viewed as attempts to resolve this fundamental trade-off.

Limitations and Continuing Relevance of CNNs

Despite their success, CNNs possess inherent limitations. Their local connectivity restricts the modeling of long-range dependencies, and anatomical relationships must generally be learned implicitly rather than encoded explicitly. Furthermore, repeated downsampling operations can compromise spatial precision.

These limitations motivated the emergence of transformers, graph neural networks, and diffusion-based approaches that provide stronger mechanisms for global reasoning and structural modeling.

Nevertheless, CNNs remain indispensable. Their computational efficiency, mature optimization techniques, extensive pretrained models, and ability to generate dense spatial feature maps continue to make them essential components of modern human pose estimation systems.

Increasingly, the strongest architectures combine convolutional feature extraction with transformer-based global reasoning, graph-based structural modeling, or diffusion-based generative refinement. As a result, understanding convolutional neural networks remains a prerequisite for understanding virtually all contemporary human pose estimation methodologies.

3.2 Transformer and Attention-Based Architectures for Human Pose Estimation

The rapid advancement of deep learning has significantly transformed the field of Human Pose Estimation (HPE). While convolutional neural networks (CNNs) dominated early deep learning-based approaches, recent years have witnessed the emergence of Transformer and attention-based architectures as a powerful alternative for both 2D and 3D pose estimation tasks. These architectures enable models to capture long-range spatial and temporal dependencies more effectively than traditional convolutional methods,

thereby improving robustness under occlusion, viewpoint variation, and complex body articulation.

Unlike CNNs, which rely on local receptive fields and progressively deeper layers to aggregate contextual information, Transformers employ self-attention mechanisms that directly model relationships between all image regions or body joints. This property is particularly beneficial in HPE, where distant anatomical structures often exhibit strong correlations. For example, the movement of one arm may influence the posture of the opposite leg during walking or athletic motion. Attention mechanisms allow such dependencies to be learned explicitly.

Transformer-based HPE systems can generally be categorized into three major groups:

1. Self-attention-based backbone architectures
2. Cross-attention mechanisms for person–keypoint association
3. Keypoint-to-keypoint attention frameworks for anatomical reasoning

These categories are discussed in detail in the following sections.

3.2.1 Self-Attention in HPE Backbones

Self-attention mechanisms are used in HPE backbones to replace or augment convolutional feature extractors. Instead of relying solely on convolutional kernels, self-attention enables every image patch or joint token to interact directly with all other tokens in the sequence. Consequently, the model can capture global contextual dependencies without requiring extremely deep networks.

The standard self-attention operation is defined as:

$$Attention(Q, K, V) = Softmax(\frac{QK^T}{\sqrt{d_k}})V$$

where:

- Q denotes the query matrix,
- K denotes the key matrix,
- V denotes the value matrix,
- d_k represents the dimensionality of the key vectors.

The attention weights determine how strongly one token attends to another token. In HPE, these tokens may correspond to image patches, feature embeddings, or body joints.

Self-attention allows the network to learn global anatomical relationships directly. For instance, the model may infer the location of an occluded wrist by attending to visible shoulders, elbows, and body orientation cues.

Transformer-Based 2D Human Pose Estimation

TransPose

TransPose was among the earliest Transformer-based architectures proposed for 2D HPE. The model combined a CNN backbone with Transformer encoder layers to capture long-range dependencies between body joints. A convolutional network first extracted spatial feature maps, after which Transformer layers modeled relationships between different image regions.

One of the major contributions of TransPose was demonstrating that self-attention improves robustness to occlusion by enabling the network to utilize contextual information from visible body parts. The architecture also showed that lightweight Transformer modules could outperform several purely convolutional approaches while maintaining computational efficiency.

TokenPose

TokenPose introduced the concept of learnable joint tokens for pose estimation. Instead of relying exclusively on image patches, the architecture incorporated dedicated tokens representing anatomical joints such as elbows, knees, and wrists.

The model processes two types of tokens:

- Visual tokens extracted from image patches
- Joint tokens representing body keypoints

Through self-attention operations, joint tokens attend to visual tokens to gather relevant image evidence while simultaneously interacting with other joint tokens to model skeletal structure.

This design explicitly embeds anatomical prior knowledge into the Transformer architecture and improves the modeling of inter-joint dependencies.

ViTPose

ViTPose demonstrated that generic Vision Transformers (ViTs) without specialized pose-specific modifications could achieve state-of-the-art performance in 2D HPE.

The architecture follows a backbone-heavy and decoder-light design philosophy. Large Vision Transformer backbones are employed to extract global representations, while the decoder remains relatively simple. Unlike earlier CNN-based approaches that required complex multi-stage refinement, ViTPose showed that sufficiently large Transformer backbones alone are capable of accurate keypoint localization.

Another important contribution of ViTPose is scalability. Performance consistently improves as model size increases, indicating that Transformer-based HPE benefits significantly from large-scale training and high-capacity architectures.

Efficient Transformer Variants

Although Vision Transformers provide strong accuracy, their computational complexity grows quadratically with the number of image tokens:

$$O(N^2)$$

where NNN is the number of image patches.

To address this limitation, recent architectures introduced more efficient attention mechanisms.

EViTPose

EViTPose reduces computational cost by selecting only informative image patches associated with body joints. Learnable joint tokens guide the selection of relevant regions, thereby decreasing unnecessary attention computations.

UniTransPose

UniTransPose combines local and global attention operations through three complementary mechanisms:

- Local patch-to-patch attention
- Global patch-to-joint attention
- Local joint-to-patch attention

This design improves multi-scale feature modeling and enhances localization accuracy under large scale variation.

Transformer-Based 3D Human Pose Estimation

Transformer architectures have also achieved remarkable success in 3D HPE, particularly for temporal sequence modeling from videos.

PoseFormer

PoseFormer was the first purely Transformer-based framework for monocular 3D pose estimation from video sequences.

The architecture contains two major components:

Spatial Transformer Encoder

Processes body joints within each frame and models anatomical relationships among joints.

Temporal Transformer Encoder

Processes frame embeddings across time to capture motion dynamics and temporal dependencies.

This cascaded spatial-temporal design enables the model to learn both body structure and motion patterns simultaneously.

PoseFormer demonstrated that self-attention can effectively replace recurrent networks and temporal convolutions in video-based 3D HPE.

MHFormer

Monocular 3D HPE is inherently ambiguous because multiple 3D poses may project to the same 2D configuration. MHFormer addresses this issue through a multi-hypothesis strategy.

The architecture generates several candidate pose hypotheses and refines them using attention-based communication mechanisms:

1. Hypothesis generation
2. Self-hypothesis interaction
3. Cross-hypothesis aggregation

Self-attention allows the model to reason about multiple plausible 3D interpretations before synthesizing the final pose estimate.

MixSTE

MixSTE proposed an alternating spatial-temporal Transformer structure.

The architecture alternates between:

- Temporal attention blocks
- Spatial attention blocks

Temporal blocks model motion trajectories of individual joints over time, whereas spatial blocks model anatomical relationships within each frame.

This alternating strategy improves efficiency while capturing both motion dynamics and skeletal dependencies effectively.

PoseFormerV2

PoseFormerV2 introduced frequency-domain processing to reduce computational cost for long video sequences.

The architecture employs the Discrete Cosine Transform (DCT) to compress temporal information before Transformer processing. This substantially decreases computational complexity while preserving important motion patterns.

The method also improves robustness to noisy 2D detections and unstable temporal sequences.

3.2.2 Cross-Attention for Person–Keypoint Association

Multi-person HPE introduces a challenging association problem: determining which detected joints belong to which individual.

Cross-attention mechanisms address this problem by allowing person-level queries to interact selectively with image features and keypoint representations.

DETR-Based Pose Estimation

Inspired by Detection Transformers (DETR), modern end-to-end pose estimation methods formulate HPE as a set prediction problem.

PETR

PETR introduced a dual-decoder Transformer architecture consisting of:

- Pose decoder

- Joint decoder

The pose decoder predicts coarse person instances, while the joint decoder refines anatomical keypoints.

Cross-attention binds person queries to relevant image regions and subsequently refines keypoint localization.

QueryPose

QueryPose employs sparse learnable queries for pose estimation.

The framework contains:

- A bounding-box detection branch
- A pose estimation branch

Cross-attention enables person queries to attend selectively to person-specific image features before estimating joints.

ED-Pose

ED-Pose models each keypoint as a small bounding box rather than a single coordinate point.

The architecture uses dual-stage decoding:

1. Human instance detection
2. Fine-grained pose estimation

Cross-attention provides spatial alignment between keypoint queries and image features, improving localization precision.

GroupPose

GroupPose introduced structured self-attention mechanisms that separate:

- Within-instance attention
- Across-instance attention

Within-instance attention models anatomical relationships among joints of the same person, whereas across-instance attention aggregates information between identical joint types across multiple individuals.

This separation simplifies optimization and improves performance in crowded scenes.

Cross-Attention in Regression-Based HPE

Poseur

Poseur reformulated HPE as a sequence prediction problem using direct coordinate regression.

The architecture employs deformable cross-attention mechanisms where each keypoint query attends adaptively to relevant image regions.

Instead of predicting single deterministic coordinates, Poseur predicts probability distributions over possible keypoint locations, improving robustness and reducing localization uncertainty.

3.2.3 Keypoint-to-Keypoint Attention

Keypoint-to-keypoint attention explicitly models anatomical dependencies between body joints. In these methods, the human skeleton is treated as a graph where:

- Joints are graph nodes
- Anatomical relationships are graph edges

Attention weights dynamically determine the importance of relationships between joints.

Graph Transformer Architectures

MotionAGFormer

MotionAGFormer combines Graph Convolutional Networks (GCNs) with Transformers.

The architecture contains two parallel streams:

Transformer Stream

Captures global joint relationships through self-attention.

GCN Stream

Captures local anatomical dependencies between adjacent joints.

An adaptive fusion mechanism combines both representations dynamically.

This hybrid design improves efficiency while preserving both local and global structural information.

STGFormer

STGFormer introduces spatio-temporal graph attention mechanisms.

The architecture separates attention into:

- Spatial attention
- Temporal attention

Spatial attention models joint relationships within each frame, while temporal attention models motion evolution across time.

This separation improves temporal consistency and motion understanding.

Hierarchical Attention Mechanisms

HSTFormer

HSTFormer models body relationships hierarchically through four levels:

1. Spatial Correlation

2. Joint-Level Temporal Correlation
3. Body-Part Temporal Correlation
4. Pose-Level Temporal Correlation

This hierarchical framework progressively aggregates information from individual joints to complete body motion representations.

Skeleton-Aware Attention

JointFormer

JointFormer treats 2D-to-3D lifting as a pure Transformer problem.

Each joint is embedded as a token, and self-attention learns anatomical relationships implicitly without requiring explicit graph structures.

The model learns skeletal topology directly from training data.

GCANet

GCANet combines graph convolutions with attention mechanisms at multiple semantic levels:

- Joint-level graph convolution
- Part-based convolution
- Global body attention

The attention mechanism is constrained by the anatomical adjacency matrix:

$$A' = \frac{H_i \, H_j^T}{d} \cdot A$$

where:

- H_i and H_j are joint feature representations,
- A is the skeleton adjacency matrix,
- d is a normalization factor.

This formulation enforces anatomically plausible relationships while still allowing adaptive attention weighting.

3.3 Graph Neural Networks (GNNs) for Structural Consistency in Human Pose Estimation

3.3.1 Structural Modeling of the Human Skeleton

Human pose estimation is inherently a structured prediction problem in which anatomical joints are not independent entities, but rather interconnected components constrained by the biomechanics of the human body. The skeletal structure naturally forms a non-Euclidean graph, where joints correspond to graph nodes and bones define graph edges. This representation motivates the application of Graph Neural Networks (GNNs), which are specifically designed to process relational and topological data.

Unlike convolutional neural networks that operate on regular grid structures or fully connected layers that disregard anatomical topology, GNNs directly model the skeletal graph through message passing mechanisms. This enables the network to preserve structural consistency while learning spatial dependencies between body joints. Consequently, GNN-based pose estimation frameworks inherently enforce anatomical plausibility during inference.

The graph formulation introduces several important structural priors:

- **Kinematic consistency**, ensuring that adjacent joints maintain realistic bone lengths and motion constraints.
- **Symmetry preservation**, enabling correspondence between left and right body parts.
- **Occlusion reasoning**, where invisible joints can be inferred through neighboring visible joints.
- **Long-range dependency modeling**, allowing distant joints to influence one another through graph propagation.

These characteristics make GNNs highly suitable for both 2D and 3D human pose estimation tasks, particularly under challenging scenarios involving occlusion, self-intersection, or ambiguous body configurations.

3.3.2 Graph Convolutional Foundations for Pose Estimation

The fundamental operation of graph neural networks in pose estimation is graph convolution. Given a skeletal graph $G = (V, E)$, where V denotes joints and E represents anatomical connections, graph convolution propagates information between neighboring joints through the adjacency matrix.

The standard graph convolution formulation can be expressed as:

$$H^{(l+1)} = \sigma(\tilde{D}^{-\frac{1}{2}} \tilde{A}\ \tilde{D^{-\frac{1}{2}}} H^{(l)} W^{(l)})$$

where:

- $\tilde{A} = A + I$ represents the adjacency matrix with self-connections,
- $\tilde{D}$ denotes the degree matrix,
- $H^{(l)}$ contains node features at layer l,
- $W^{(l)}$is the learnable transformation matrix,
- and $\sigma(.)$ denotes the nonlinear activation function.

In the context of pose estimation, each node embedding corresponds to a joint representation, while the adjacency matrix encodes skeletal connectivity. Through iterative message passing, each joint aggregates contextual information from neighboring joints, gradually constructing a globally coherent pose representation.

Although early graph convolutional formulations successfully demonstrated the feasibility of skeletal reasoning, they suffered from several limitations:

- identical transformation weights for all joints,
- restricted one-hop receptive fields,
- inability to model action-dependent topological variations,
- and limited capability to capture long-range dependencies.

Subsequent research addressed these challenges through semantic graph convolutions, adaptive graph construction, hierarchical reasoning, and dynamic topology learning.

3.3.3 GNN Architectures for 3D Human Pose Estimation

Semantic Graph Convolution Networks

Semantic Graph Convolution Networks introduced one of the earliest successful applications of GCNs for 3D pose lifting. The primary objective was to map 2D joint coordinates into anatomically valid 3D skeletons while preserving structural consistency.

Instead of applying identical transformations to all joints, semantic graph convolutions learn channel-wise edge importance weights, enabling the network to assign different significance to different anatomical relationships. This design substantially improves the expressive capability of graph reasoning.

To capture global dependencies beyond direct skeletal edges, semantic graph convolution layers are commonly combined with non-local operations that compute pairwise interactions between all joints in the skeleton. As a result, the network simultaneously models:

- local kinematic constraints,
- long-range body correlations,
- and global pose coherence.

This hybrid local-global reasoning significantly improves depth estimation accuracy while maintaining compact parameterization.

Weight Unsharing Strategies

Conventional graph convolutions employ shared transformation weights across all joints, implicitly assuming that all anatomical relationships are equivalent. However, the relationship between hip and knee differs fundamentally from that between wrist and shoulder.

Weight unsharing strategies address this issue by assigning node-specific or edge-specific transformations. Several formulations have been explored:

- pre-aggregation transformations,
- post-aggregation transformations,
- fully unshared edge-specific weights,
- and convolution-style displacement-based operations.

These approaches improve representational flexibility by allowing each joint to learn specialized feature transformations. However, fully unshared formulations substantially increase the number of learnable parameters, motivating more efficient alternatives such as modulation-based methods.

Modulated Graph Convolution Networks

Modulated GCNs introduce a parameter-efficient solution to node-specific reasoning through feature modulation mechanisms. Rather than learning separate transformation matrices for every joint, the model learns modulation vectors that adapt a shared transformation matrix.

This strategy achieves two important objectives:

1. preserving joint-specific expressiveness,
2. while avoiding excessive parameter growth.

In addition, modulated GCNs extend the skeletal adjacency matrix with learnable affinity terms, enabling the network to dynamically strengthen or suppress connections depending on the current pose configuration.

Consequently, the graph topology becomes adaptive rather than fixed, allowing the model to capture non-anatomical but semantically meaningful relationships, such as interactions between hands and feet during complex movements.

High-Order Graph Convolution

Standard GCNs are inherently limited to local one-hop neighborhoods, restricting their ability to model global body dependencies. High-order graph

convolution addresses this limitation by aggregating information from multi-hop neighborhoods.

This design enables hierarchical reasoning across different spatial scales:

- one-hop neighbors capture local bone constraints,
- two-hop neighborhoods encode limb rigidity,
- while higher-order neighbors model full-body coherence.

The integration of multi-order features substantially improves structural reasoning, particularly for complex articulated motions involving coordinated full-body movement.

Dynamic Skeletal Graph Networks

Dynamic skeletal graph models focus on hard poses characterized by self-occlusion, rare articulations, and depth ambiguity. Instead of relying on a fixed skeletal topology, these methods dynamically construct graph connections according to motion patterns and temporal consistency.

Temporal-aware graph construction reinforces edges between joints exhibiting correlated motion across frames. Consequently, even anatomically distant joints may establish strong connections when participating in coordinated movement.

Such adaptive graph construction significantly improves robustness under severe occlusion and challenging motion patterns.

Hierarchical Poselet-Based Graph Networks

Hierarchical poselet-based frameworks decompose the body into semantic substructures such as:

- torso,
- arms,
- legs,
- and head regions.

Each poselet is modeled independently before being integrated into a global skeletal representation. This hierarchical decomposition enables localized feature propagation while preserving global body coherence.

In addition, geometric constraint losses are introduced to explicitly regularize:

- bone lengths,
- bone directions,
- and joint orientations.

These constraints ensure that generated poses remain anatomically plausible during training and inference.

Global-Local Adaptive Graph Architectures

Global-local graph architectures combine sequence-level reasoning with joint-specific refinement. These frameworks first reconstruct a global spatiotemporal representation and subsequently refine individual joints using specialized local estimators.

This design acknowledges that different joints exhibit distinct motion characteristics and temporal dependencies. For example:

- head trajectories are typically smooth and stable,
- whereas hand motions are rapid and highly dynamic.

Adaptive local refinement therefore improves motion fidelity while preserving global structural consistency.

3.3.4 GNNs for 2D Human Pose Estimation

Although GNNs are predominantly associated with 3D pose estimation, they also play a significant role in 2D HPE, particularly for:

- occlusion recovery,
- keypoint refinement,
- and multi-person association.

Dynamic Graph Convolution for Multi-Person Pose Estimation

Dynamic graph convolution frameworks for 2D HPE model relationships between all detected keypoints rather than relying solely on predefined skeletal edges.

Soft adjacency matrices encode probabilistic relationships between joints based on spatial statistics, while stochastic edge sampling improves robustness to missing detections and occlusion.

This dynamic formulation enables effective reasoning under crowded scenes where conventional bottom-up pipelines often struggle.

Image-Guided Graph Reasoning

Image-guided graph reasoning integrates visual appearance features into each graph propagation stage. Rather than relying exclusively on skeletal topology, message passing is conditioned on actual image evidence.

This fusion of structural priors and visual semantics improves occluded joint estimation by allowing the graph to adapt its reasoning according to image context.

Graph-Based Pose Refinement

Graph refinement networks operate on features extracted from coarse keypoint predictions. Initial heatmaps provide approximate joint locations, after which graph message passing iteratively refines the coordinates.

Unlike purely heatmap-based refinement strategies, graph refinement directly models inter-joint dependencies, leading to improved structural consistency and localization accuracy.

3.3.5 Structural Constraints as Learnable Objectives

Beyond architectural modifications, many approaches explicitly incorporate structural priors into the training objective.

Common geometric regularization terms include:

Bone Length Constraint

$$L_{bone} = \sum_{(i,j)} (\parallel p_i - p_j \parallel - l_{ij})^2$$

This prevents unrealistic limb stretching or compression.

Joint Angle Constraint
Joint angle regularization enforces anatomically feasible articulation ranges by penalizing implausible angular configurations.

Symmetry Constraint
Symmetry losses maintain bilateral consistency between left and right body parts.
Temporal Smoothness Constraint
Temporal regularization minimizes abrupt motion variations across consecutive frames, reducing jitter in video-based pose estimation.

3.3.6 Spatio-Temporal Graph Neural Networks

For video-based HPE, the skeletal graph is extended into the temporal domain. The node representation becomes:

$$V = \{v_{i,t}\}$$

where iii denotes the joint index and ttt represents the temporal frame index.

The graph then contains:

- spatial edges within each frame,
- temporal edges connecting identical joints across frames,
- and optionally spatio-temporal edges linking correlated joints across time.

This formulation enables simultaneous modeling of:

- anatomical structure,
- motion continuity,
- and temporal dynamics.

Spatio-temporal GNNs significantly improve video-based 3D pose estimation by enforcing motion coherence across long sequences.

3.3.7 Hybrid GNN-Transformer Architectures

Recent advances increasingly combine graph neural networks with transformer-based attention mechanisms. This fusion seeks to exploit the complementary strengths of both paradigms:

- GNNs provide strong local topological inductive bias,
- while transformers capture global long-range dependencies through self-attention.

Hybrid architectures typically employ:

- graph convolutions for local skeletal reasoning,
- transformer encoders for global spatial-temporal modeling,
- and adaptive fusion modules to integrate both representations.

These models currently represent one of the most effective directions in modern human pose estimation research.

Design Trade-Offs in GNN-Based Human Pose Estimation

Different graph modeling strategies introduce distinct advantages and limitations.

Weight Sharing vs. Weight Unsharing

- Shared weights improve parameter efficiency and generalization.
- Unshared weights increase expressiveness but significantly expand model complexity.

Static vs. Dynamic Graphs

- Static graphs provide stable anatomical priors.
- Dynamic graphs adapt better to pose variations and occlusion but are more difficult to optimize.

Local vs. Multi-Hop Reasoning

- One-hop convolutions effectively model local kinematic constraints.
- Multi-hop propagation captures global body coherence but may introduce noisy long-range interactions.

Graph-Based Architectures vs. Graph-Based Losses

- Graph architectures directly embed structural reasoning into the network.
- Graph-based loss functions provide flexible topology-aware supervision that can be integrated with arbitrary backbones.

3.4 Diffusion-Based Pose Estimation

Diffusion-based Human Pose Estimation (HPE) has recently emerged as a powerful generative paradigm for addressing the inherent ambiguity and uncertainty of pose reconstruction. Unlike conventional discriminative approaches that learn a deterministic mapping from visual input to joint coordinates, diffusion models learn the conditional probability distribution of plausible human poses. This formulation is particularly important for 3D pose estimation, where a single 2D observation may correspond to multiple anatomically valid 3D configurations because of depth ambiguity, occlusion, truncation, and missing visual evidence.

Traditional CNN-, Transformer-, and GNN-based regressors generally collapse this multimodal solution space into a single estimate. Although effective under constrained settings, deterministic regression frequently suffers from overconfidence in ambiguous scenarios, regression-to-the-mean artifacts, and limited uncertainty estimation. Diffusion models overcome these limitations by treating pose estimation as an iterative denoising process in which plausible human structures are progressively reconstructed from noisy representations. Consequently, diffusion-based frameworks naturally support multi-hypothesis generation, uncertainty-aware inference, and anatomically coherent pose reconstruction.

The fundamental idea is inspired by denoising diffusion probabilistic models, where a clean pose representation is gradually perturbed through a forward diffusion process and then reconstructed through a learned reverse process conditioned on visual or structural information.

Mathematical Foundations of Diffusion-Based Pose Estimation

Let y_0 denote the clean pose representation. In 3D HPE, $y_0 \in R^{J\times 3}$ represents the spatial coordinates of J joints, whereas in 2D HPE, the representation may consist of stacked heatmaps corresponding to each keypoint.

The forward diffusion process progressively injects Gaussian noise into the clean pose representation over T timesteps:

$$q(y_t \mid y_0) = \sqrt{\bar{\alpha}_t}\, y_0 \; + \sqrt{1 \, - \, \bar{\alpha}_t \epsilon}$$

where:

$$\bar{\alpha}_t = \prod_{s=1}^{t} (1 = \beta_s)$$

and: $$\epsilon \sim N(0, I)$$

The reverse process learns to reconstruct clean poses from noisy samples. A neural network ϵ_θ predicts the injected noise conditioned on the noisy pose, timestep, and conditioning information:

$$L = E_{y_{0,t,\epsilon}}[\| \epsilon - \epsilon_\theta(y_t, t, c) \|^2]$$

where c may represent image features, 2D keypoints, temporal context, text prompts, or structural priors.

During inference, the model iteratively denoises random samples into anatomically plausible poses. Since conventional diffusion sampling requires hundreds of denoising steps, practical HPE systems employ accelerated sampling techniques such as DDIM-based deterministic sampling, timestep skipping, or coarse initialization strategies to reduce computational overhead.

3.4.1 Diffusion for 2D Human Pose Estimation

Generative Heatmap Reconstruction

Early diffusion-based 2D HPE methods reformulated pose estimation as a generative heatmap synthesis problem. Instead of predicting heatmaps independently for each joint, diffusion models generate all heatmaps jointly, thereby enforcing global structural consistency across the human body.

A representative architecture consists of three main components:

- An image encoder that extracts visual features from the input image.
- A forward diffusion module that injects noise into keypoint coordinates or heatmaps.
- A structure-guided diffusion decoder that progressively reconstructs refined heatmaps.

Structural conditioning plays a critical role in these systems. Keypoint masks and skeleton masks are commonly used to inject anatomical information into the denoising trajectory. Skeleton masks explicitly encode limb connectivity, allowing the network to infer occluded joints using surrounding body structure.

For example, the location of an invisible wrist may be inferred from the visible shoulder–elbow relationship.

Spatial-channel attention mechanisms are frequently integrated into diffusion decoders. Spatial attention improves localization precision by focusing on local neighborhoods around joints, while channel attention enhances semantic consistency between different feature representations.

An important observation from diffusion-based 2D HPE is that diffusion primarily acts as a refinement mechanism rather than a replacement for strong visual backbones. Modern frameworks therefore combine diffusion refinement with high-capacity feature extractors such as HRNet or Vision Transformers.

3.4.2 Diffusion for 3D Human Pose Estimation

Diffusion-based approaches have shown particularly strong performance in 3D HPE, where ambiguity is substantially greater than in 2D estimation.

Multi-Hypothesis 3D Pose Generation

A key advantage of diffusion models is their ability to generate multiple plausible 3D poses from the same 2D observation. Different random initializations produce distinct anatomically valid hypotheses, allowing the model to represent uncertainty explicitly.

Instead of collapsing uncertainty into a single estimate, diffusion models sample multiple candidate poses and subsequently aggregate them using statistical strategies such as averaging, median aggregation, or reprojection-based selection.

This capability is especially important in scenarios involving:

- Severe self-occlusion,
- Frontal-view ambiguity,
- Truncation,
- Missing limbs,
- Rare or complex articulations.

Single-Image Diffusion-Based Lifting

Single-image diffusion frameworks typically condition the denoising process on:

- 2D keypoint coordinates,
- 2D heatmaps,
- Image features,
- Or combinations of these modalities.

Some approaches preserve the entire distribution of candidate 2D joint locations instead of reducing heatmaps to deterministic coordinates. This enables the diffusion model to reason over uncertainty directly during 3D reconstruction.

Visual-context-aware models additionally incorporate multi-resolution image features surrounding the detected skeleton. This contextual information helps resolve ambiguities that purely coordinate-based lifting methods cannot address.

Transformer-based attention modules are commonly employed to fuse:

- Pose features,
- Visual context,
- Joint-to-joint dependencies,
- And timestep embeddings.

These architectures have demonstrated substantial improvements on highly ambiguous subsets of standard 3D pose benchmarks.

3.4.3 Temporal Diffusion Models for Video-Based HPE

Video-based diffusion frameworks exploit temporal consistency as an additional structural constraint. Instead of conditioning on a single frame, the denoising network receives a temporal window of 2D pose sequences or image features.

Temporal diffusion models benefit from:

- Motion continuity,
- Periodic movement patterns,
- Inter-frame correlations,
- And trajectory smoothness.

Transformer-based denoisers are widely adopted in this setting because self-attention effectively captures long-range temporal dependencies.

Some recent architectures introduce hierarchical temporal pruning mechanisms to reduce the computational cost associated with long video sequences. These methods construct sparse temporal graphs that preserve informative frames while discarding redundant motion patterns.

Such pruning strategies significantly improve efficiency while maintaining state-of-the-art accuracy.

3.4.4 Structural Guidance in Diffusion Models

A major research direction involves incorporating explicit anatomical structure into the diffusion process.

Skeleton-Guided Diffusion

Several methods inject skeletal topology directly into the denoising trajectory through:

- Skeleton masks,
- Keypoint connectivity priors,
- Graph neural networks,
- Or kinematic constraints.

These mechanisms ensure that generated poses remain anatomically plausible even under severe noise conditions.

Graph-Based Denoisers

While many diffusion frameworks rely on transformers, some approaches use Graph Convolutional Networks (GCNs) as denoising architectures. In these systems:

- Joints are treated as graph nodes,
- Bones define graph edges,
- And message passing propagates structural information during denoising.

Graph-based denoisers explicitly enforce:

- Bone-length consistency,
- Joint-angle constraints,
- Symmetry,
- And local kinematic relationships.

This structural inductive bias improves stability and anatomical realism throughout the diffusion trajectory.

Prompt-Driven Structural Guidance

More recent frameworks introduce semantic conditioning through textual prompts. These systems encode:

- Action categories,
- Body-part semantics,
- Motion descriptors,
- And kinematic context.

Prompt embeddings are injected into the diffusion process through transformer-based communication modules. This establishes explicit interactions between semantic body concepts and geometric pose representations.

Prompt-guided diffusion enables fine-grained control over pose reconstruction while improving robustness in complex scenarios.

3.4.5 Hypothesis Aggregation Strategies

Because diffusion models generate multiple plausible outputs, aggregation strategies become critically important.

Common aggregation methods include:

Strategy	Description
Average Aggregation	Computes the mean of all generated hypotheses
Median Aggregation	Computes per-joint median estimates for robustness
Random Sampling	Selects a single hypothesis
Oracle Selection	Chooses the best hypothesis relative to ground truth
Joint-Wise Aggregation	Aggregates hypotheses independently for each joint

Empirical studies demonstrate that median aggregation generally outperforms simple averaging because it is more robust to outlier hypotheses.

Joint-wise aggregation methods further improve performance by allowing different body parts to be selected from different hypotheses.

3.4.6 Diffusion as a Pose Prior

Diffusion models are increasingly used not only as primary estimators but also as structural priors for pose refinement and completion.

Pose Refinement
Diffusion-based refiners operate as post-processing modules that improve noisy predictions generated by conventional estimators. These frameworks learn the distribution of realistic human poses and iteratively map corrupted poses toward anatomically plausible configurations.

Pose Completion and Denoising

Diffusion priors can also reconstruct missing joints and repair corrupted pose sequences. This capability is particularly valuable in:

- Occluded environments,
- Low-resolution videos,
- Crowded scenes,
- And motion capture systems with missing markers.

Motion Generation
Some methods extend diffusion modeling to full-body motion synthesis. By learning diffusion priors over temporal pose sequences, these systems can generate realistic human motion conditioned on:

- Images,
- Text prompts,
- Or sparse pose observations.

3.4.7 Conditioning Strategies in Diffusion-Based HPE

The effectiveness of diffusion-based HPE largely depends on the conditioning modality employed during denoising.

Conditioning Strategy	Purpose
Image Features	Provide visual context and occlusion cues
2D Keypoints	Supply lightweight geometric guidance
2D Heatmaps	Preserve uncertainty distributions
Temporal Sequences	Enforce motion consistency
Structural Masks	Inject skeletal topology
Text Prompts	Encode semantic body-part information

Modern diffusion systems often combine multiple conditioning modalities simultaneously to improve robustness and structural consistency.

3.4.8 Computational Challenges and Efficiency Improvements

Despite their strong performance, diffusion models remain computationally expensive because inference requires iterative denoising.

Several strategies have been proposed to reduce complexity:

Challenge	Solution
Large Number of Timesteps	DDIM sampling and timestep skipping
Multiple Hypotheses	Parallelized batch decoding
Long Video Sequences	Hierarchical temporal pruning
Quadratic Attention Complexity	Sparse attention mechanisms
Poor Initialization	Coarse pose initialization

These optimizations have significantly improved the practicality of diffusion-based HPE for real-time and resource-constrained applications.

3.4.9 Diffusion Models and Structural Consistency

A recurring theme in diffusion-based HPE is the integration of anatomical structure into the generative process. Modern frameworks increasingly combine:

- Graph-based structural reasoning,
- Transformer attention,
- Kinematic priors,
- Temporal modeling,
- And probabilistic diffusion sampling.

This convergence represents an important shift in human pose estimation research. Rather than relying solely on deterministic regression, diffusion models treat pose estimation as probabilistic structured generation, enabling anatomically plausible, uncertainty-aware, and multi-hypothesis human pose reconstruction.

3.5 Vision-Language & LLM Integration for Human Pose Estimation

3.5.1 From Vision-Centric Estimation to Vision–Language Reasoning

Conventional Human Pose Estimation (HPE) systems are fundamentally vision-centric. Images or videos are processed by convolutional or transformer-based architectures, and the network directly regresses joint coordinates, heatmaps, or body parameters. Although this paradigm has achieved remarkable numerical accuracy, it neglects two sources of information that humans naturally exploit during pose interpretation: semantic language and commonsense reasoning.

Human observers rarely interpret poses solely through geometric cues. Instead, they rely on semantic descriptions such as “crossing the arms,” “kneeling,” or “raising the right hand,” all of which implicitly encode biomechanical constraints, spatial relations, and contextual intent. Furthermore, humans apply world knowledge when resolving ambiguity. A partially occluded person holding a tennis racket is more likely to exhibit a serving posture than a sitting posture, even if the visual evidence is incomplete.

The integration of Vision–Language Models (VLMs) and Large Language Models (LLMs) into HPE seeks to bridge this gap between geometric perception and semantic understanding. Rather than treating pose estimation as a purely visual regression problem, these approaches formulate pose reasoning as a multimodal inference process in which visual observations, linguistic semantics, and structural priors interact jointly.

Current research directions in this domain can be broadly categorized into three complementary paradigms:

- **Feature enrichment**, where language-supervised representations improve robustness to occlusion and ambiguity.
- **Conditional pose generation**, where textual prompts guide the pose estimation process toward semantically meaningful configurations.
- **Reasoning-oriented multimodal systems**, where LLMs interpret, refine, generate, and explain human poses through dialogue and instruction-following behavior.

This transition represents a broader conceptual shift: human pose is no longer treated solely as a geometric entity but as a semantic and reasoning-aware modality.

3.5.2 Contrastive Vision–Language Pretraining for Pose Representation Learning

3.5.2.1 CLIP-Style Alignment for Pose Understanding

Contrastive Language–Image Pretraining introduces a shared embedding space in which visual and textual representations are aligned through contrastive objectives. Within HPE, this principle is adapted to align body configurations with semantic pose descriptions, action categories, and anatomical concepts.

Given an image representation and a textual representation, the model learns to maximize similarity between corresponding image–text pairs while minimizing similarity between unrelated pairs. As a result, the visual encoder acquires semantically meaningful representations that are highly beneficial under challenging conditions such as occlusion, truncation, and crowding.

In pose estimation, textual prompts may describe:

- Action categories (“walking,” “running,” “sitting”)
- Body parts (“left wrist,” “right ankle”)
- Structural relationships (“arms crossed,” “knees bent”)

- Spatial configurations (“hands above head”)

The learned representation space therefore becomes anatomically and semantically structured rather than purely appearance-driven.

3.5.2.2 Language-Assisted Multi-Person Pose Estimation

One of the earliest applications of language supervision in multi-person HPE demonstrated that semantic prompts can substantially improve robustness in crowded and occluded scenes.

The architecture consists of two complementary components:

Instance-Level Prompting

Instance prompts describe each person globally within the scene. Examples include:

- “person on the left”
- “partially occluded individual”
- “person in the foreground”

These prompts generate semantic embeddings that interact with image features to produce instance-aware representations, improving person separation in crowded environments.

Joint-Level Prompting

Joint prompts provide body-part semantics such as:

- “left elbow”
- “right shoulder”
- “nose”

Alignment occurs at both:

- The keypoint level, through feature similarity
- The pixel level, through dense feature correlation

The combination of instance-level and joint-level prompting enables the network to distinguish both *who* a person is and *which body part* is being observed.

A key consequence is improved robustness under heavy occlusion. Even when joints are invisible, semantic embeddings provide strong anatomical anchors that guide localization.

3.5.2.3 LLM-Generated Kinematic Priors

More advanced approaches move beyond handcrafted prompts and instead use LLMs to generate anatomically meaningful textual priors automatically.

These methods inject language information through two stages:

Joint Semantic Injection

The LLM generates detailed descriptions of individual joints and their biomechanical roles, for example:

- “the elbow is a hinge joint connecting the upper arm and forearm”
- “the knee bends primarily in the sagittal plane”

These semantic descriptions are encoded into language embeddings and aligned with visual features through cross-modal attention.

Spatial Relationship Modeling

The LLM additionally produces descriptions of inter-joint spatial relations such as:

- “hips are below shoulders”
- “ankles are beneath knees”
- “hands are near the torso”

These textual spatial priors participate in contrastive learning with visual joint representations, encouraging anatomically consistent predictions.

This approach effectively transforms language into a high-dimensional structural prior that complements visual evidence during inference.

3.5.3 Text-Guided Conditional Pose Estimation

A second research direction treats language not merely as supervision, but as an explicit conditioning signal that directly influences the pose estimation process.

3.5.3.1 Action-Guided 3D Pose Estimation

Temporal lifting architectures often rely primarily on motion continuity, overlooking the fact that different actions possess distinct pose priors.

Action-guided methods introduce textual action prompts such as:

- “walking”
- “eating”
- “sitting”
- “jumping”

These prompts are encoded into semantic embeddings and fused with temporal pose features through attention mechanisms.

The framework typically includes two modules:

Action-Related Text Prompting

Action labels are embedded using learnable textual context vectors. Temporal motion features are incorporated to enrich semantic representations, enabling distinctions such as:

- “walking slowly”
- “walking rapidly”

The resulting embeddings guide the pose encoder toward action-specific structural patterns.

Action-Specific Pose Prompting

Learnable pose templates corresponding to different action categories are introduced into transformer decoders through cross-attention. These prompts act as semantic priors that refine pose predictions.

Such methods demonstrate that language-guided action awareness improves estimation accuracy particularly for difficult motion categories and long temporal sequences.

3.5.3.2 Text-Guided Multi-View 3D Pose Estimation

In domains such as sports analysis and online coaching, visually similar actions may differ only in subtle biomechanical details. Language guidance becomes particularly valuable in these scenarios.

These systems combine:

- Geometric features (joint distances and angles)
- Textual action descriptions
- Multi-view or pseudo-view temporal representations

Text prompts encode detailed biomechanical constraints such as:

- “knees aligned over toes”
- “torso leaning forward”
- “maintaining straight back posture”

The language embeddings are fused with geometric features to improve action-sensitive pose understanding.

This framework is especially effective for distinguishing semantically similar but biomechanically distinct actions.

3.5.3.3 Prompt-Driven Diffusion Pose Estimation

Diffusion-based HPE methods further extend text conditioning through fine-grained prompt injection.

Part-aware prompts encode:

- Global actions
- Coarse body regions
- Fine-grained joints

These prompts interact with diffusion denoisers through cross-attention modules and timestep-conditioned transformations. As a result, the reverse diffusion trajectory becomes semantically aware.

The denoising process therefore incorporates:

- Structural semantics
- Temporal reasoning
- Action context
- Body-part specificity

This creates a strong convergence between language-guided reasoning and probabilistic generative modeling.

3.5.4 Multimodal LLMs for Pose Reasoning and Generation

The most transformative development in this field is the integration of pose estimation into multimodal LLM frameworks.

In these systems, human pose becomes a native modality alongside text and images.

3.5.4.1 Conversational Pose Estimation

Early multimodal pose-chat systems formulate pose estimation as an instruction-following task.

The architecture typically includes:

- A frozen vision encoder
- A large language model backbone
- A pose projection module predicting SMPL parameters

Special pose tokens are inserted into the language vocabulary. During generation, the hidden representation associated with these tokens is decoded into body parameters.

This design enables:

- Image-to-pose estimation
- Text-to-pose generation
- Pose editing through dialogue
- Speculative reasoning about unseen poses

For example:

- “What pose is the person performing?”
- “Generate a seated version of this pose.”
- “Raise the left arm higher.”

Unlike conventional estimators, these systems reason semantically about pose meaning rather than only predicting coordinates.

3.5.4.2 Unified Pose-Centric Multimodal Models

More advanced multimodal architectures treat pose as both input and output.

These systems include:

- Pose encoders transforming body parameters into latent embeddings
- Pose decoders reconstructing body configurations
- Projection layers aligning pose representations with LLM latent spaces

Training proceeds through multiple stages:

1. Pose–vision alignment
2. Pose-language pretraining
3. Instruction tuning

The resulting models support:

- Pose estimation
- Pose editing
- Text-to-pose synthesis
- Multimodal reasoning

Although these systems currently underperform specialized HPE architectures numerically, they introduce unprecedented flexibility and interpretability.

3.5.4.3 Motion-Oriented LLMs

Motion-focused LLMs extend multimodal reasoning from static pose to temporal motion.

Continuous motion sequences are discretized into motion tokens using vector quantization methods. These tokens are then processed autoregressively by the LLM.

Capabilities include:

- Text-to-motion generation
- Motion captioning
- Multi-person motion synthesis
- Motion-conditioned reasoning

Because motion is represented as language-compatible tokens, the model inherits the autoregressive reasoning capabilities of modern LLMs.

This enables nuanced semantic understanding such as distinguishing:

- “walking normally”
- “walking like a zombie”
- “walking while carrying an object”

Such semantic richness is difficult to capture using purely geometric methods.

3.5.4.4 Instruction-Tuned Pose Understanding

Instruction-tuned pose-language systems enhance interpretability by integrating explicit keypoint reasoning into conversational datasets.

These systems are capable of:

- Describing body posture precisely
- Explaining action intent
- Understanding human–object interaction
- Answering anatomy-aware questions

Unlike regression-oriented HPE methods, these models emphasize explainability and semantic interaction.

3.5.5 Architectural Integration Strategies

Despite their diversity, most vision–language HPE methods follow a small set of recurring architectural patterns.

3.5.5.1 Frozen Language Encoders with Auxiliary Supervision

In this paradigm:

- The language encoder remains frozen
- Visual features are optimized to align with text embeddings

Language acts primarily as a supervisory regularizer.

Advantages:

- Preserves pretrained language knowledge
- Computationally efficient

Limitations:

- Limited adaptation to pose-specific semantics

3.5.5.2 **Learnable Prompt Tuning**

Learnable prompt vectors are prepended to textual inputs while keeping the majority of the language model frozen.

Advantages:

- Parameter efficiency
- Reduced catastrophic forgetting

Limitations:

- Lower expressive capacity compared with full fine-tuning

3.5.5.3 Cross-Modal Attention Fusion

Cross-modal attention enables dense interaction between textual and visual tokens.

Advantages:

- Fine-grained semantic alignment
- Strong localization capability

Limitations:

- Increased computational complexity
- Sensitivity to token alignment strategies

3.5.5.4 Pose Token Generation within LLMs

Pose parameters are represented as discrete tokens processed autoregressively.

Advantages:

- Unified reasoning and generation
- Natural instruction-following behavior

Limitations:

- Quantization may reduce geometric precision
- Sequential decoding increases latency

3.5.5.5 Diffusion with Language Conditioning

Text embeddings guide diffusion denoisers through cross-attention.

Advantages:

- High-quality generative modeling
- Strong ambiguity handling

Limitations:

- Iterative denoising is computationally expensive

3.5.6 Cross-Modal Alignment Mechanisms

The success of multimodal HPE depends critically on how different modalities are aligned.

3.5.6.1 Contrastive Alignment

Visual and textual embeddings are aligned through cosine similarity objectives, producing semantically structured representation spaces.

3.5.6.2 Feature-Space Projection

Projection networks map pose embeddings into language-model latent spaces, enabling compatibility between modalities.

3.5.6.3 Attention-Based Communication

Cross-attention mechanisms establish fine-grained correspondences between:

- Body parts
- Spatial regions
- Semantic descriptions

This allows textual semantics to guide geometric reasoning directly.

3.5.6.4 Instruction-Based Pose Reasoning

Instruction tuning reframes pose estimation as a conversational reasoning task.

Examples include:

- “Describe the posture.”
- “Estimate the pose.”
- “Correct the body alignment.”

This formulation leverages the reasoning capabilities of LLMs for pose understanding.

3.5.7 Emerging Trends and Future Directions

The integration of vision, language, and reasoning into HPE represents one of the most significant conceptual transitions in the field.

Several emerging directions are particularly important:

3.5.7.1 Semantic-Aware Pose Understanding

Future systems are expected to move beyond coordinate prediction toward semantic comprehension of human behavior and intent.

3.5.7.2 Explainable Pose Estimation

LLM-integrated frameworks introduce interpretability by allowing models to explain:

- Why a pose was predicted
- Which cues were important
- How ambiguity was resolved

3.5.7.3 Interactive Pose Editing

Natural language interfaces enable intuitive human–AI interaction for:

- Pose correction
- Motion design
- Animation control
- Sports coaching

3.5.7.4 Unified Human-Centric Multimodal Models

Pose estimation is increasingly converging with:

- Action recognition
- Motion generation
- Scene understanding
- Human–object interaction

This suggests the emergence of unified multimodal systems capable of holistic human-centered reasoning.

CHAPTER 4

Practical Implementation&Taxonomy

4.1Taxonomy of Human Pose Estimation Models

4.1.1 OpenPose

Introduction

OpenPose represents one of the most influential milestones in the evolution of multi-person human pose estimation. Introduced in 2017, it was the first framework to demonstrate that accurate real-time multi-person pose estimation could be achieved through a bottom-up strategy. Unlike conventional approaches that first detect individual persons and subsequently estimate their poses, OpenPose formulates pose estimation as a global body-part detection and association problem. This design enables the system to process crowded scenes efficiently while maintaining real-time performance.

The framework gained widespread recognition after winning the COCO Keypoints Challenge and later evolved into a comprehensive pose estimation system capable of detecting full-body, facial, hand, and foot keypoints. Its open-source release further accelerated research and industrial adoption, making it one of the most widely used pose estimation frameworks.

Motivation

Before the introduction of OpenPose, most multi-person pose estimation systems followed a top-down paradigm. In such approaches, a person detector first identifies individual human instances, after which a single-person pose estimation network is applied separately to each detected region. Although effective in relatively simple scenes, this strategy suffers from several inherent limitations.

The computational complexity of top-down methods increases proportionally with the number of individuals present in an image, making them unsuitable for real-time applications involving large crowds. Furthermore, heavy occlusion, overlapping persons, and truncated body parts frequently lead to inaccurate detections and fragmented pose predictions.

OpenPose was developed to overcome these challenges through a bottom-up formulation. The central idea was to detect all body joints simultaneously and subsequently associate them using a novel

representation known as Part Affinity Fields (PAFs). This representation encodes not only the locations of body parts but also the directional relationships between them, enabling robust assembly of complete human skeletons even in highly crowded scenes.

Architectural Design

Feature Extraction Backbone

OpenPose employs the initial layers of the VGG-19 network as a shared feature extraction backbone. The backbone generates dense feature maps that capture both low-level visual information and higher-level semantic cues. These feature maps serve as the common input for subsequent prediction stages.

The use of a shared backbone reduces computational redundancy and allows multiple prediction branches to operate on a common visual representation.

Dual-Branch Multi-Stage Network

A defining characteristic of OpenPose is its multi-stage architecture composed of two parallel prediction branches.

The first branch estimates Part Affinity Fields. These fields are two-dimensional vector maps that describe the orientation and association strength between pairs of body joints. Each limb type is assigned a dedicated vector field, allowing the network to learn spatial relationships between connected body parts.

The second branch predicts confidence maps corresponding to individual body joints. Each confidence map represents the probability distribution of a specific keypoint location across the image.

Rather than producing final predictions in a single step, OpenPose adopts an iterative refinement strategy. Predictions generated in earlier stages are repeatedly refined through subsequent stages while continuously incorporating information from the original feature maps. This iterative

process allows the network to progressively improve both localization accuracy and limb association quality.

An important architectural refinement introduced in the later version of OpenPose was the prioritization of Part Affinity Field refinement. Experimental analysis revealed that accurate limb association contributes more significantly to overall performance than repeated refinement of confidence maps. Consequently, the architecture was redesigned so that PAFs undergo extensive refinement before being used as priors for joint localization. This modification simultaneously improved both computational efficiency and estimation accuracy.

Part Affinity Fields

The principal innovation of OpenPose lies in the introduction of Part Affinity Fields.

A Part Affinity Field is a vector representation associated with a specific limb. For every pixel belonging to a limb region, the vector points from one joint endpoint toward the other. Pixels outside the limb region are assigned zero vectors.

Unlike traditional midpoint-based association techniques, PAFs encode both spatial location and orientation information. This allows the model to distinguish between multiple overlapping limbs belonging to different individuals.

For example, in scenes where several arms intersect, conventional methods often struggle to determine which wrist belongs to which shoulder. By integrating directional information along candidate limb segments, PAFs provide a robust mechanism for resolving such ambiguities.

This representation became one of the most influential contributions in pose estimation and inspired numerous subsequent association-based frameworks.

Pose Parsing and Skeleton Assembly

After confidence maps and Part Affinity Fields have been predicted, OpenPose reconstructs complete human skeletons through a parsing stage.

Candidate joint locations are first extracted from confidence maps using non-maximum suppression. Potential limb connections are then evaluated by integrating the corresponding Part Affinity Field vectors along the line connecting two candidate joints. Connections with strong directional consistency receive higher scores.

The final pose assembly process is performed using a greedy bipartite matching algorithm. This algorithm associates joints into limb structures and progressively merges limbs into complete human skeletons.

A notable design decision is the use of a minimal tree-based skeleton representation rather than a fully connected graph. This significantly reduces computational complexity while preserving sufficient structural information for accurate pose reconstruction.

Training Strategy

Training is performed using a multi-stage supervision framework. Separate loss functions are applied to both confidence maps and Part Affinity Fields at every refinement stage.

The confidence map loss measures the discrepancy between predicted and ground-truth joint probability distributions, while the PAF loss evaluates the difference between predicted and ground-truth limb vector fields.

Intermediate supervision plays a crucial role in optimization. By applying losses at multiple stages, the network receives stronger gradient signals throughout training, alleviating the vanishing gradient problem commonly encountered in deep multi-stage architectures.

To handle incomplete annotations and unlabeled body parts, a binary masking mechanism is employed during training. This prevents the model from being penalized for predictions in regions where ground-truth annotations are unavailable.

Extensive data augmentation, including random scaling, rotation, flipping, and cropping, is used to improve robustness to variations in viewpoint, body orientation, and human scale.

Evaluation Protocol

OpenPose is commonly evaluated using standard pose estimation metrics. Performance is measured through Average Precision and Average Recall, typically computed using the Object Keypoint Similarity criterion.

Additional evaluations assess localization precision at different similarity thresholds and analyze performance across varying human scales, including small, medium, and large persons.

Beyond accuracy, runtime efficiency is considered a critical evaluation criterion because one of the primary objectives of OpenPose is real-time multi-person pose estimation.

Experimental Performance

OpenPose demonstrated state-of-the-art performance on major benchmarks at the time of its introduction.

On multi-person pose estimation datasets, it significantly outperformed previous bottom-up methods while simultaneously achieving dramatically faster inference speeds. The framework established a new benchmark for balancing accuracy and computational efficiency.

Experimental studies further confirmed the effectiveness of Part Affinity Fields. When ground-truth joints were provided, the parsing algorithm achieved association quality approaching that of ground-truth limb connections, demonstrating that the proposed representation effectively captures inter-joint relationships.

Additional ablation studies revealed that increasing the number of Part Affinity Field refinement stages primarily improves detection recall, whereas refining confidence maps contributes more strongly to localization precision.

Perhaps most importantly, OpenPose maintained nearly constant inference time regardless of the number of people present in the scene, validating one of its central design goals.

Strengths

One of the most significant advantages of OpenPose is its runtime scalability. Because all body parts are detected simultaneously, computational cost remains largely independent of crowd size, making the framework particularly suitable for densely populated scenes.

The introduction of Part Affinity Fields provides a powerful mechanism for resolving ambiguous body-part associations and enables reliable pose reconstruction under moderate occlusion and overlap conditions.

The architecture also exhibits strong generalization capabilities. The same framework has been successfully applied to other keypoint detection tasks without requiring substantial architectural modifications, demonstrating that the underlying association mechanism is not limited to human pose estimation.

The use of intermediate supervision enhances optimization stability and facilitates the training of deep multi-stage networks. Furthermore, the adoption of a minimal skeletal representation reduces computational overhead while preserving structural consistency.

Finally, the open-source ecosystem surrounding OpenPose played a major role in its widespread adoption across both academic research and industrial applications.

Limitations

Despite its many advantages, OpenPose exhibits several limitations.

The framework struggles with highly unusual body configurations, particularly inverted or rarely observed poses. Severe occlusion remains challenging because missing body parts often lead to incomplete skeleton reconstruction.

In highly crowded scenes, Part Affinity Fields generated by neighboring individuals may interfere with one another, increasing the likelihood of incorrect associations and merged skeletons.

Performance also decreases for very small-scale individuals due to the difficulty of accurately localizing tiny body parts within low-resolution image regions.

Another limitation arises from false positive detections on human-like objects such as statues, mannequins, and certain animals, indicating that the model's learned representation is strongly biased toward human-like visual structures.

Finally, the predefined skeletal topology restricts the flexibility of the framework when applied to tasks involving alternative body structures or non-standard keypoint configurations.

Impact on Human Pose Estimation

OpenPose fundamentally changed the direction of multi-person pose estimation research by demonstrating that bottom-up approaches could achieve both high accuracy and real-time performance. The introduction of Part Affinity Fields established a new paradigm for keypoint association and inspired a large body of subsequent research on bottom-up pose estimation.

The framework influenced the development of numerous later systems, including lightweight mobile pose estimators, three-dimensional pose estimation extensions, and motion analysis applications. Its concepts have also been adopted in related domains such as autonomous driving, human-computer interaction, sports analytics, motion capture, and augmented reality.

More broadly, OpenPose demonstrated that pose estimation could be formulated as a global keypoint association problem rather than a sequential detection-and-estimation pipeline. This insight remains one of the most important conceptual contributions in the history of human pose estimation.

4.1.2 High-Resolution Representation Learning for Human Pose Estimation (HRNet)

Introduction

HRNet (High-Resolution Network) represents one of the most influential architectural advances in human pose estimation. Introduced in 2019, HRNet challenged the long-established paradigm of progressively reducing image resolution throughout a convolutional network and subsequently recovering spatial details through decoder structures. Unlike conventional architectures such as ResNet, VGGNet, Hourglass Networks, and U-Net variants, HRNet maintains high-resolution representations throughout the entire forward propagation process.

The architecture was specifically designed for spatially sensitive vision tasks, particularly human pose estimation, where precise localization of anatomical landmarks is essential. By preserving detailed spatial information while simultaneously integrating semantic information from lower-resolution representations, HRNet achieves an exceptional balance between localization accuracy and contextual understanding. This design philosophy established HRNet as a foundational architecture not only for pose estimation but also for semantic segmentation, object detection, facial landmark localization, and numerous dense prediction tasks.

Motivation

Most convolutional neural networks designed for visual recognition follow a hierarchical encoding strategy in which spatial resolution is gradually reduced through pooling operations or strided convolutions. While this approach is highly effective for image classification, it presents significant challenges for pose estimation.

As feature maps become increasingly coarse, fine-grained spatial information associated with joint locations is inevitably lost. Although decoder modules

and upsampling operations attempt to reconstruct these details, the recovered representations often lack the spatial precision available in the original high-resolution features. Furthermore, reconstructed feature maps may exhibit interpolation artifacts and limited semantic consistency.

HRNet was developed based on the hypothesis that maintaining high-resolution representations throughout the network, while continuously exchanging information with parallel lower-resolution branches, would generate feature representations that are both spatially accurate and semantically rich. This strategy eliminates the need for expensive resolution recovery procedures and allows pose estimation to be performed directly on highly detailed feature maps.

Architectural Design

Parallel Multi-Resolution Representation Learning

The fundamental innovation of HRNet lies in its parallel multi-resolution architecture. Instead of processing feature maps through a single stream that progressively decreases in resolution, HRNet maintains several streams operating at different resolutions simultaneously.

The network is organized into multiple stages. At each stage, a new lower-resolution branch is introduced while previously existing branches continue to operate. Consequently, the architecture preserves a high-resolution stream from the beginning of the network to its final layer.

Each branch specializes in a particular level of visual abstraction. High-resolution streams focus on precise spatial localization, whereas lower-resolution streams capture increasingly global semantic information. The coexistence of these complementary representations forms the foundation of HRNet's effectiveness.

Multi-Scale Fusion Through Exchange Units

A critical component of HRNet is the repeated information exchange between resolution streams. This process is implemented through specialized exchange units inserted throughout the network.

For a target resolution branch k, the output representation is computed as

$$Y_k = \sum_{i=1}^{s} a(X_i, k)$$

where X_i denotes the feature representation from branch i, and $a(X_i, k)$ represents a resolution alignment operation.

Three alignment strategies are employed:

- Identity mapping when source and target resolutions are identical.
- Strided convolutions when features must be downsampled.
- Upsampling followed by convolution when features must be transferred from lower to higher resolutions.

This repeated fusion mechanism enables high-resolution representations to receive semantic information from coarse-resolution branches while simultaneously providing detailed spatial information to low-resolution branches. As a result, every branch progressively becomes both spatially precise and semantically informative.

Network Configuration

The canonical HRNet architecture consists of four stages containing progressively increasing numbers of parallel branches. The most commonly used variants are:

- **HRNet-W32**, where the highest-resolution branch contains 32 channels.
- **HRNet-W48**, where the highest-resolution branch contains 48 channels.

The lower-resolution branches employ proportionally larger channel capacities to compensate for their reduced spatial dimensions. Through

repeated exchange operations, all branches collectively contribute to the final feature representation.

For pose estimation tasks, only the highest-resolution representation is typically used to generate keypoint heatmaps. This variant is commonly referred to as **HRNetV1**.

Training Objective

Human pose estimation in HRNet follows the heatmap regression paradigm. Given an input image containing K body joints, the network predicts a set of heatmaps

$$\left\{H_1, H_2, \ldots, H_K\right\}$$

where each heatmap represents the probability distribution of a specific anatomical keypoint.

Ground-truth heatmaps are generated by placing Gaussian kernels centered at annotated joint locations. The objective is to minimize the discrepancy between predicted and target heatmaps using Mean Squared Error (MSE) loss:

$$L = \frac{1}{K} \sum_{k=1}^{K} \left\| H_k - H_k^{\cdot} \right\|^2$$

where:

- H_k denotes the predicted heatmap of joint kkk,
- $H_k^{\cdot}$ denotes the corresponding ground-truth heatmap,
- K represents the total number of keypoints.

The use of heatmap regression allows the network to preserve spatial structure throughout training and enables robust localization even in cases of uncertainty or partial occlusion.

Optimization is typically performed using the Adam optimizer with scheduled learning-rate decay and extensive data augmentation strategies, including random scaling, rotation, horizontal flipping, and color perturbation.

Evaluation Metrics

HRNet is commonly evaluated using standard pose estimation benchmarks and metrics.

For the COCO dataset, performance is measured using Average Precision (AP) based on the Object Keypoint Similarity (OKS) metric. Additional measures include:

- AP50 and AP75 for coarse and fine localization accuracy,
- APM and APL for medium- and large-scale subjects,
- Average Recall (AR).

For the MPII benchmark, performance is reported using Percentage of Correct Keypoints normalized by head size (PCKh).

Computational efficiency is typically quantified through parameter count and floating-point operations (GFLOPs).

Experimental Results

HRNet demonstrated substantial improvements over previous convolutional architectures across multiple benchmarks.

On the COCO validation benchmark, HRNet-W32 achieved an Average Precision of 74.4% using an input resolution of 256×192, surpassing a substantially larger ResNet-50-based baseline while simultaneously reducing both parameter count and computational complexity. When evaluated at a higher input resolution of 384×288, HRNet-W32 further improved performance to 75.8% AP.

The larger HRNet-W48 configuration achieved 76.3% AP, establishing a new state of the art among purely convolutional pose estimation architectures at the time of publication.

A particularly significant finding was that HRNet consistently outperformed deeper and computationally more expensive ResNet-based systems. Compared with ResNet-152, HRNet-W48 achieved approximately two additional AP points while requiring fewer parameters and lower computational cost. These results demonstrated that preserving high-resolution representations is more beneficial for pose estimation than simply increasing network depth.

On the MPII Human Pose benchmark, HRNet-W32 achieved 90.3% PCKh@0.5, surpassing previous state-of-the-art approaches. The most notable improvements were observed on challenging joints such as wrists and ankles, where precise spatial localization is critical. This finding confirmed that maintaining high-resolution feature maps substantially improves the localization of anatomically small and visually ambiguous body parts.

Beyond pose estimation, HRNet also achieved state-of-the-art performance in semantic segmentation and competitive results in object detection. These experiments demonstrated that the architecture's benefits extend beyond keypoint localization and are applicable to a broad class of dense prediction tasks.

Strengths

HRNet introduced several important advantages over traditional encoder-decoder architectures.

Its most significant contribution is the simultaneous preservation of spatial precision and semantic richness. By maintaining high-resolution representations throughout the network, HRNet avoids the irreversible information loss associated with aggressive downsampling.

The repeated multi-scale fusion mechanism enables continuous interaction between local details and global context, resulting in feature representations that are highly informative for precise joint localization.

Another major advantage is the architecture's favorable efficiency-performance trade-off. HRNet consistently delivers higher accuracy

than considerably deeper networks while maintaining competitive computational requirements.

The architecture is also highly versatile. The same backbone design can be applied across pose estimation, segmentation, object detection, and landmark localization without fundamental structural modifications.

Finally, HRNet naturally handles scale variation through its parallel multi-resolution streams, making it particularly robust in real-world scenarios involving subjects of different sizes.

Limitations

Despite its success, HRNet exhibits several limitations.

Maintaining multiple parallel resolution streams throughout the network increases memory consumption compared with conventional single-stream architectures. Consequently, training HRNet requires greater GPU resources, particularly for large input resolutions.

The architecture is also relatively challenging to deploy on mobile and edge devices. Although lightweight variants such as Lite-HRNet were later proposed, the original model remains computationally demanding for real-time embedded applications.

Another limitation is the absence of explicit long-range dependency modeling. As a purely convolutional architecture, HRNet relies on local receptive fields and repeated fusion operations to propagate information. This can be less effective than self-attention mechanisms when modeling distant relationships between body parts.

Furthermore, HRNet was originally designed for single-image processing and does not inherently capture temporal information. Video-based pose estimation therefore requires additional temporal modeling modules.

Impact and Legacy

HRNet fundamentally altered the design philosophy of pose estimation backbones. It demonstrated that maintaining high-resolution representations throughout the network is a viable and highly effective alternative to the traditional encoder-decoder paradigm that had dominated computer vision for decades.

The architecture rapidly became the standard backbone for human pose estimation and inspired numerous subsequent developments, including HigherHRNet for bottom-up multi-person pose estimation, Lite-HRNet for mobile deployment, and HRFormer, which combines HRNet's multi-resolution design with transformer-based self-attention mechanisms.

Beyond pose estimation, HRNet has influenced research in semantic segmentation, object detection, medical image analysis, 3D reconstruction, and multimodal learning. Its principle of parallel multi-resolution representation learning has become a widely adopted architectural concept across computer vision.

As a result, HRNet is widely regarded as one of the most influential convolutional architectures in modern human pose estimation and remains a foundational model in the evolution of dense prediction networks.

4.1.3 YOLO-Pose and YOLOv8-Pose

Overview

YOLO-Pose represents one of the earliest successful attempts to unify object detection and human pose estimation within a single end-to-end trainable framework. Proposed by Maji et al. in 2022, the method extended the YOLO object detection paradigm to directly estimate human keypoints without relying on intermediate heatmap representations. Unlike conventional pose estimation approaches that either detect persons before estimating poses or require complex keypoint grouping procedures, YOLO-Pose formulates pose estimation as a direct regression problem in which each detected person is associated with a complete set of body keypoints.

Building upon this concept, YOLOv8-Pose integrated pose estimation into the broader YOLOv8 framework released by Ultralytics in 2023. The model introduced several architectural refinements, including anchor-free detection, decoupled prediction heads, improved feature extraction modules, and enhanced training strategies. These developments transformed YOLO-based pose estimation into one of the most widely adopted solutions for real-time human pose analysis across industrial and research applications.

The emergence of YOLO-Pose marked a significant departure from the heatmap-dominated paradigm that had governed human pose estimation for nearly a decade. By eliminating heatmap generation and post-processing pipelines, the framework demonstrated that competitive pose estimation performance could be achieved through direct coordinate regression while maintaining the computational efficiency characteristic of the YOLO family.

Motivation

Historically, multi-person pose estimation methods were divided into two dominant categories: top-down and bottom-up approaches.

Top-down methods first detect individual persons and subsequently estimate keypoints for each detected instance. Although highly accurate, their computational complexity increases proportionally with the number of people present in the scene, limiting their applicability in real-time environments and crowded scenarios.

Bottom-up methods address this limitation by detecting all body joints simultaneously and then associating them into complete human skeletons. While computationally efficient, these methods rely heavily on heatmap representations and sophisticated grouping algorithms, such as associative embedding, bipartite matching, or affinity field parsing. Such procedures are typically non-differentiable and cannot be optimized jointly with the neural network during training.

YOLO-Pose was developed to overcome these limitations by integrating person detection and pose estimation into a unified architecture. The central hypothesis was that the visual challenges encountered in object

detection—such as scale variation, occlusion, and deformation—are fundamentally similar to those encountered in pose estimation. Consequently, a detector capable of localizing a person could also directly regress the spatial coordinates of that person's body joints.

This formulation offers several advantages. Runtime becomes independent of the number of detected individuals, keypoint grouping is inherently solved because all joints belong to the same detection instance, and the entire pipeline can be optimized end-to-end using a single objective function.

Architectural Design

YOLO-Pose

The original YOLO-Pose architecture extends the YOLOv5 framework with minimal structural modifications. The network employs a CSP-Darknet53 backbone to extract multi-scale visual representations from the input image. These features are subsequently aggregated through a Path Aggregation Network (PANet), which enhances information flow across different spatial resolutions.

At the prediction stage, each detection head simultaneously estimates bounding box parameters and human keypoints. Instead of producing heatmaps, the network directly regresses the coordinates and confidence values of all body joints. For the COCO benchmark, each detected person is represented by seventeen keypoints, with each keypoint described by its horizontal coordinate, vertical coordinate, and visibility confidence.

A notable design characteristic is that keypoints are predicted relative to the anchor center rather than being constrained within the predicted bounding box. This design choice allows the model to localize occluded or partially visible joints that may extend beyond the visible body region.

YOLOv8-Pose

YOLOv8-Pose introduces several improvements over the original formulation. The architecture adopts the CSPDarknet backbone enhanced with C2f

modules, which provide more efficient gradient propagation and richer feature representations. The neck continues to employ PANet-based feature fusion but benefits from the improved feature extraction capabilities of the new backbone.

One of the most important innovations is the transition from anchor-based detection to an anchor-free formulation. This eliminates the need for predefined anchor boxes and anchor matching procedures during training, thereby simplifying optimization and improving robustness to scale variations.

The prediction head is also redesigned using a decoupled structure, separating classification and regression tasks into independent branches. This reduces task interference and allows the network to learn more discriminative representations for both object localization and keypoint estimation.

Additional enhancements include the adoption of TaskAlignedAssigner for sample assignment and the Spatial Pyramid Pooling Fast (SPPF) module for efficient multi-scale context aggregation.

Loss Function

A major contribution of YOLO-Pose is the introduction of an Object Keypoint Similarity (OKS)-based loss function for direct keypoint optimization.

The overall training objective combines four complementary loss terms:

$$L_{total} = \lambda_{box} L_{box} + \lambda_{obj} L_{obj} + \lambda_{kpts} L_{kpts} + \lambda_{conf} L_{conf}$$

where the individual components correspond to bounding box regression, objectness prediction, keypoint localization, and keypoint confidence estimation.

Bounding box localization is optimized using Complete Intersection over Union (CIoU) loss:

$$L_{box} = 1 - CIoU(B_{pred}, B_{gt})$$

which simultaneously considers overlap area, center distance, and aspect ratio consistency.

The most distinctive component is the OKS-based keypoint loss:

$$L_{kpts} = 1 - \frac{\sum_k exp\left(\frac{-d_k^2}{2s^2\sigma_k^2}\right) 1\left(v_k > 0\right)}{\sum_k 1(v_k > 0)}$$

where d_k denotes the Euclidean distance between predicted and ground-truth keypoints, s represents the object scale, σ_k is a keypoint-specific normalization factor, and v_k indicates visibility.

Unlike traditional L1 or L2 regression losses, the OKS formulation directly mirrors the evaluation metric used by the COCO benchmark. Consequently, optimization becomes aligned with the final performance objective rather than relying on surrogate coordinate-based losses.

For visibility prediction, a binary cross-entropy objective is employed:

$$L_{conf} = BCE(v_{pred}, v_{gt})$$

allowing the network to distinguish visible, occluded, and absent keypoints.

YOLOv8-Pose retains the multi-task optimization strategy while incorporating additional improvements such as Distribution Focal Loss (DFL) for localization and updated optimization schemes based on AdamW and cosine learning-rate scheduling.

Experimental Evaluation and Findings

Evaluation on the COCO benchmark demonstrated that YOLO-Pose successfully challenged the conventional assumption that heatmaps are necessary for high-quality pose estimation.

Among the original YOLO-Pose variants, the larger models achieved performance levels comparable to state-of-the-art bottom-up approaches while requiring substantially fewer computational resources. In particular, YOLOv5l6-Pose surpassed many contemporary bottom-up methods in AP50 performance while maintaining significantly lower computational complexity.

The experiments further revealed that direct OKS optimization produces more accurate keypoint predictions than conventional coordinate regression objectives. This finding provided strong evidence that metric-aligned learning objectives can compensate for the absence of heatmap supervision.

YOLOv8-Pose further improved this performance-efficiency trade-off. The lightweight YOLOv8n-Pose variant achieved real-time inference on edge devices while maintaining competitive accuracy. Larger variants such as YOLOv8x-Pose demonstrated substantial gains in localization precision and overall Average Precision, approaching the performance levels of considerably more expensive top-down methods.

An important observation from these experiments is that the YOLO family achieves its results without test-time augmentation, multi-scale evaluation, or flip testing. Consequently, reported performance reflects actual deployment conditions rather than artificially enhanced benchmark settings.

The experimental results collectively demonstrate that direct keypoint regression can provide a viable alternative to heatmap-based pose estimation when supported by an appropriate loss formulation and a strong detection backbone.

Strengths

The primary strength of YOLO-Pose lies in its fully end-to-end trainable design. Unlike traditional bottom-up methods, the framework does not require non-differentiable grouping procedures, coordinate refinement algorithms, or affinity-based post-processing.

Another important advantage is its inherent keypoint association mechanism. Because all joints are predicted within a single detection instance, the model naturally avoids the cross-person ambiguity commonly observed in bottom-up approaches.

The direct optimization of the OKS metric further aligns training objectives with evaluation criteria, improving convergence behavior and practical performance.

From a deployment perspective, YOLO-Pose offers exceptional efficiency. Runtime remains effectively constant regardless of the number of people present in the scene, making the framework highly suitable for surveillance systems, robotics, sports analytics, and human-computer interaction applications.

The availability of multiple model scales, ranging from lightweight edge-oriented variants to high-capacity server-grade models, further enhances the flexibility and applicability of the framework.

Limitations

Despite its advantages, YOLO-Pose exhibits several limitations.

The most significant limitation is the remaining accuracy gap relative to state-of-the-art top-down approaches, particularly in terms of fine-grained localization accuracy. Heatmap-based methods retain an advantage in sub-pixel precision because they explicitly model spatial probability distributions rather than directly regressing coordinates.

Performance degradation is also observed when detecting very small persons, a challenge shared by many single-stage object detectors. The reduced spatial resolution available for small-scale instances limits the accuracy of keypoint localization.

Although YOLOv8's anchor-free design addresses several weaknesses of the original YOLO-Pose formulation, highly crowded scenes can still create ambiguities during sample assignment and pose prediction.

Another limitation is the dependence on a predefined skeletal structure. Extending the framework to whole-body pose estimation, animal pose estimation, or domain-specific keypoint configurations typically requires architectural modifications and retraining.

Finally, as a purely convolutional framework, YOLO-Pose lacks explicit mechanisms for modeling long-range dependencies and global body relationships, which can become important in highly articulated or unusual poses.

Impact and Legacy

YOLO-Pose established a new research direction by demonstrating that human pose estimation can be formulated as a direct extension of object detection. This challenged the long-standing dominance of heatmap-based methods and encouraged the exploration of regression-based pose estimation frameworks.

Its influence is particularly evident in modern real-time pose estimation systems, where efficiency and deployment practicality are often as important as raw benchmark performance. The integration of pose estimation into the broader YOLO ecosystem significantly lowered the barrier to entry for practitioners and accelerated industrial adoption.

YOLOv8-Pose has become one of the most widely deployed real-time pose estimation frameworks, serving as the foundation for numerous applications in healthcare, sports analytics, autonomous systems, surveillance, and augmented reality.

More broadly, the success of YOLO-Pose demonstrated that advances in object detection architectures can be directly transferred to pose estimation, thereby unifying two previously distinct research domains. This conceptual shift continues to influence contemporary research on efficient, end-to-end human pose estimation systems.

4.1.4 RTMPose

Introduction

RTMPose (Real-Time Models for Pose Estimation) was introduced by Tao Jiang and colleagues at Shanghai AI Laboratory as a real-time human pose estimation framework designed to bridge the gap between state-of-the-art research performance and practical industrial deployment. Developed within the MMPose 1.0 ecosystem, RTMPose represents a comprehensive optimization of the entire pose estimation pipeline rather than a single architectural innovation. The framework systematically revisits five key aspects of pose estimation systems: estimation paradigm, backbone architecture, localization strategy, training methodology, deployment efficiency, and inference speed.

Unlike many previous approaches that prioritized benchmark accuracy at the expense of computational efficiency, RTMPose was explicitly designed to achieve an optimal balance between accuracy, latency, model size, and deployment flexibility. Consequently, it emerged as one of the most influential real-time pose estimation frameworks within the OpenMMLab ecosystem and became widely adopted in both academic and industrial applications.

Motivation

Although human pose estimation performance had improved substantially over the preceding years, many state-of-the-art methods remained difficult to deploy in real-world systems. Architectures such as HRNet and transformer-based models achieved excellent benchmark results but required considerable computational resources and often exhibited high inference latency.

Another limitation stemmed from the widespread reliance on heatmap-based localization. These methods typically required high-resolution output maps, deconvolution layers, coordinate refinement procedures, and additional post-processing operations, all of which increased computational overhead and deployment complexity.

RTMPose was motivated by the observation that advances in modern object detectors had significantly reduced the computational burden of the detection stage. Consequently, a carefully optimized top-down framework could

potentially achieve real-time performance while preserving the superior localization accuracy traditionally associated with top-down approaches.

The central hypothesis underlying RTMPose is that efficient detection combined with lightweight coordinate classification can eliminate many inefficiencies of heatmap-based pipelines while maintaining competitive localization performance. Rather than treating keypoint estimation as a dense heatmap regression problem, RTMPose reformulates localization as a coordinate classification task, thereby reducing computational complexity and improving deployment efficiency.

Architecture

Overall Framework

RTMPose follows the conventional top-down paradigm consisting of two sequential stages. First, a person detector identifies human instances within the image. Second, each detected bounding box is processed independently by a dedicated pose estimation network that predicts the keypoint locations.

The framework supports several detectors, including RTMDet, YOLO-based detectors, PicoDet, and Faster R-CNN. However, RTMDet is commonly employed due to its favorable balance between speed and detection accuracy.

CSPNeXt Backbone

The backbone of RTMPose is CSPNeXt, a network architecture originally developed for object detection tasks. CSPNeXt incorporates Cross-Stage Partial (CSP) connections to improve gradient propagation while reducing redundant computation. In addition, the architecture utilizes large-kernel depthwise convolutions to expand the receptive field at a relatively low computational cost.

Unlike traditional classification backbones such as ResNet, CSPNeXt was specifically designed for dense prediction tasks and therefore offers a more favorable accuracy-efficiency trade-off for pose estimation.

SimCC-Based Localization

The most distinctive component of RTMPose is its coordinate localization strategy based on SimCC (Simple Coordinate Classification).

Instead of generating two-dimensional heatmaps for each keypoint, SimCC decomposes localization into two independent one-dimensional classification problems corresponding to the horizontal and vertical image axes. The coordinate space is discretized into a set of bins, and the network predicts the probability distribution over these bins for each axis.

If the input image has width W and height H , and a splitting factor k is applied, the coordinate spaces become:

$$N_x = W \times k$$
$$N_y = H \times k$$

where N_x and N_y denote the numbers of classification bins along the horizontal and vertical dimensions, respectively.

This formulation substantially reduces computational requirements compared with heatmap regression while preserving localization precision.

Architectural Enhancements

RTMPose introduces several improvements over the original SimCC framework.

A lightweight Gated Attention Unit (GAU) is inserted between the backbone and the coordinate classifiers to model relationships among body joints. Unlike transformer self-attention mechanisms, GAU employs a compressed latent representation that significantly reduces computational cost while retaining the ability to capture long-range dependencies between keypoints.

The architecture also replaces the final 1×1 convolution with a larger 7×7 convolution, enabling richer local feature aggregation before coordinate prediction. Additionally, separate Gaussian smoothing parameters

are used for the horizontal and vertical coordinate distributions, improving localization accuracy by accounting for directional differences in spatial resolution.

Datasets

RTMPose was evaluated across a diverse collection of pose estimation benchmarks.

The primary benchmark was the COCO Keypoints dataset, which contains 17 annotated body joints and represents one of the most widely used evaluation datasets in pose estimation research. Additional experiments were conducted on MPII Human Pose, CrowdPose, COCO-WholeBody, AI Challenger, and AP-10K.

COCO-WholeBody extends conventional body pose estimation by annotating 133 keypoints, including body joints, facial landmarks, hand keypoints, and foot landmarks. CrowdPose focuses on heavily occluded and crowded scenes, whereas AP-10K evaluates the framework's ability to generalize to animal pose estimation.

The authors also demonstrated the effectiveness of multi-dataset training. Models jointly trained on AI Challenger and COCO exhibited improved generalization and achieved higher performance than models trained solely on COCO.

Loss Function

SimCC Coordinate Classification Loss

RTMPose formulates keypoint localization as a coordinate classification problem optimized through Kullback–Leibler Divergence (KLD).

For a ground-truth coordinate μ_x , the target distribution along the horizontal axis is represented by a Gaussian distribution:

$$p_k(x_i \mid \mu_x) = \frac{1}{\sqrt{2\pi\sigma}} exp\left(-\frac{\left(x_i - \mu_x\right)^2}{2\sigma^2}\right)$$

where:

- x_i denotes the coordinate bin,
- μ_x represents the ground-truth coordinate,
- σ controls the Gaussian spread.

Rather than assigning all probability mass to a single coordinate bin, neighboring bins receive gradually decreasing probabilities. This Gaussian label smoothing introduces spatial continuity into the classification process and improves localization robustness.

The training objective minimizes the Kullback–Leibler divergence between the predicted coordinate distribution and the target distribution:

$$L_{KLD} = KL(\hat{p}_k \parallel p_k^{\cdot})$$

where:

- $\hat{p}_k$ is the predicted coordinate distribution,
- $p_k^{\cdot}$ is the ground-truth Gaussian distribution.

Visibility-Aware Optimization

Visibility annotations are incorporated directly into the training process. Visible and occluded keypoints contribute fully to the loss, whereas absent keypoints are excluded from optimization. This prevents the network from learning incorrect coordinate distributions for unannotated joints.

Training Strategy

RTMPose employs a highly optimized training pipeline that includes:

- Exponential Moving Average (EMA) parameter updates,

- Flat cosine learning-rate scheduling,
- Strong-to-weak augmentation transition,
- UDP pretraining,
- Removal of weight decay for normalization layers and biases.

Collectively, these strategies contribute significantly to the final performance gains observed by the framework.

Evaluation Metrics

Performance is primarily evaluated using the standard COCO Object Keypoint Similarity (OKS) protocol.

The principal metric is Average Precision (AP), computed across multiple OKS thresholds ranging from 0.50 to 0.95. Additional metrics include AP^{50}, AP^{75}, AP^{M}, AP^{L}, and Average Recall (AR).

Because RTMPose was designed for real-time deployment, inference latency was also measured across multiple hardware platforms, including CPUs, GPUs, TensorRT deployments, ONNXRuntime, and mobile processors such as the Snapdragon 865.

Results and Findings

COCO Human Pose Estimation

RTMPose demonstrated that carefully optimized top-down architectures can achieve both high accuracy and real-time performance.

RTMPose-M achieved approximately **75.3% AP** on the COCO validation set while requiring only **1.93 GFLOPs**, surpassing the widely used HRNet-W32 baseline despite using substantially lower computational resources. RTMPose-L further improved performance to **76.3% AP**, establishing a new efficiency benchmark among real-time top-down approaches.

The results indicate that coordinate classification can match and occasionally exceed traditional heatmap-based methods while significantly reducing computational complexity.

Whole-Body Pose Estimation

On the COCO-WholeBody benchmark, RTMPose achieved state-of-the-art performance among real-time models. The large variant obtained **66.1% AP** while requiring substantially fewer floating-point operations and lower latency than comparable HRNet-based approaches.

These results demonstrate that the coordinate classification framework remains effective even when the number of keypoints increases from 17 body joints to 133 whole-body landmarks.

Cross-Dataset Generalization

RTMPose exhibited strong generalization capabilities across diverse benchmarks.

On CrowdPose, the framework substantially outperformed HRNet-based baselines, indicating superior robustness in crowded scenes with severe occlusion. On MPII, RTMPose achieved approximately **90.7% PCKh@0.5**, exceeding HRNet-W32 despite requiring considerably less computation.

Furthermore, experiments on AP-10K showed that the framework generalizes effectively beyond human pose estimation and can be successfully applied to animal pose estimation tasks.

Real-Time Deployment Performance

One of the most significant findings of RTMPose is its exceptional deployment efficiency.

The smallest variant, RTMPose-T, achieved inference speeds exceeding **100 FPS** on mobile hardware while maintaining competitive accuracy. RTMPose-S delivered more than **70 FPS** on Snapdragon-class devices while achieving over **72% AP**, making it suitable for real-time mobile applications.

On GPU platforms, RTMPose-M achieved several hundred frames per second while preserving accuracy comparable to significantly larger architectures.

Key Findings

The experimental results lead to several important conclusions:

1. Coordinate classification is a practical and effective alternative to heatmap regression for human pose estimation.
2. Eliminating heatmap generation and deconvolution layers dramatically reduces computational cost without sacrificing localization quality.
3. CSPNeXt provides a more favorable speed-accuracy trade-off than traditional classification backbones when used for pose estimation.
4. The integration of GAU attention modules improves anatomical consistency and enhances localization accuracy at a negligible computational cost.
5. A carefully optimized training pipeline contributes as much to performance improvements as architectural innovations themselves.
6. RTMPose establishes that real-time pose estimation systems can achieve state-of-the-art accuracy while remaining deployable across CPUs, GPUs, edge devices, and mobile platforms.

Strengths

RTMPose offers several notable advantages. The coordinate classification paradigm substantially reduces computational overhead compared with heatmap-based approaches while maintaining competitive accuracy. The CSPNeXt backbone provides an excellent balance between efficiency and representational capacity, and the lightweight GAU module effectively models keypoint relationships without introducing significant latency.

The framework also demonstrates strong cross-domain generalization, performing effectively across human pose estimation, whole-body estimation, crowded scenes, and animal pose estimation. Furthermore, its compatibility with deployment frameworks such as TensorRT, ONNXRuntime, OpenVINO, ncnn, and MMDeploy makes it particularly attractive for industrial applications.

Limitations

Despite its advantages, RTMPose retains the inherent limitations of top-down approaches. Since each detected individual must be processed independently, inference time increases with the number of persons present in the scene. Consequently, performance may degrade in extremely crowded environments compared with one-stage or bottom-up methods.

The framework is also highly dependent on detector quality, as missed detections directly translate into missed poses. Additionally, coordinate classification remains constrained by discretization granularity, which may limit localization precision in applications requiring extremely fine measurements.

Although the GAU module improves contextual reasoning, severe occlusions continue to present challenges, particularly when multiple body parts are entirely invisible.

Impact

RTMPose has become one of the most influential real-time pose estimation frameworks within the modern pose estimation landscape. As the flagship real-time solution of the MMPose ecosystem, it has established new standards for balancing accuracy, efficiency, and deployability.

The framework directly inspired subsequent models such as RTMO and RTMW, both of which extend the coordinate classification paradigm to more advanced settings. Its success also accelerated the adoption of SimCC-based localization techniques throughout the pose estimation community.

Beyond academia, RTMPose has been widely adopted in sports analytics, rehabilitation monitoring, fitness tracking, augmented reality, behavioral neuroscience, and mobile vision systems. Its combination of strong performance, deployment flexibility, and open-source availability has established it as one of the foundational frameworks for modern real-time human pose estimation.

4.1.5 ViTPose / ViTPose++

Introduction

ViTPose was introduced by Yufei Xu and colleagues in 2022 as a transformer-based human pose estimation framework that challenged many long-standing assumptions in pose estimation research. Unlike previous transformer-based methods that relied on specialized architectures, hierarchical designs, or CNN-assisted feature extraction, ViTPose demonstrated that a standard Vision Transformer (ViT), originally designed for image classification, could serve as an exceptionally effective backbone for pose estimation with minimal architectural modifications.

The central contribution of ViTPose was the demonstration that powerful transformer representations, particularly when combined with Masked Autoencoder (MAE) pre-training, substantially reduce the need for sophisticated pose-specific architectures. This finding established a new design philosophy in which backbone quality becomes the dominant factor, while decoder complexity plays a relatively minor role.

Building upon this foundation, ViTPose++ extended the framework toward generic body pose estimation. Through a Mixture-of-Experts (MoE) knowledge factorization mechanism, ViTPose++ enabled a single model to learn human body pose estimation, whole-body pose estimation, and animal pose estimation simultaneously while avoiding the task conflicts commonly observed in multi-task learning. The framework subsequently achieved state-of-the-art performance across numerous benchmarks, including COCO, MPII, OCHuman, COCO-WholeBody, AP-10K, and APT-36K.

Motivation

Prior to ViTPose, most state-of-the-art pose estimation systems were built upon increasingly sophisticated convolutional architectures such as HRNet, HigherHRNet, and various attention-enhanced CNN frameworks. Although highly successful, these architectures often required carefully engineered multi-resolution branches, feature fusion modules, and task-specific architectural components.

Transformer-based alternatives had emerged, but most retained significant architectural complexity by incorporating hierarchical structures or CNN feature extractors. Consequently, it remained unclear whether the advantages of transformers originated from the transformer mechanism itself or from the surrounding architectural design.

ViTPose was motivated by a simple but fundamental research question:

> Can a plain Vision Transformer, originally developed for image classification, serve as a highly effective pose estimation backbone without task-specific modifications?

The underlying hypothesis was that self-attention naturally models long-range dependencies between body joints and that large-scale self-supervised pre-training can produce sufficiently rich representations to eliminate the need for complex pose estimation architectures.

ViTPose++ further addressed a limitation of the original framework. While ViTPose excelled at human pose estimation, practical applications often require a unified model capable of handling multiple body structures, including humans, animals, and whole-body keypoint configurations. Traditional multi-task learning frequently suffers from conflicting optimization objectives due to differing anatomical structures and keypoint definitions. ViTPose++ hypothesized that knowledge factorization could separate shared anatomical representations from task-specific knowledge, thereby enabling effective cross-domain learning.

Architecture

Plain Vision Transformer Backbone

The core architecture of ViTPose is based entirely on the original Vision Transformer design.

An input image of dimensions $H \times W \times 3$ is divided into non-overlapping image patches of size $P \times P$. Each patch is flattened and projected into a token embedding space. Learnable positional embeddings are subsequently added to preserve spatial information.

The resulting token sequence is processed by a stack of standard transformer encoder layers consisting of Multi-Head Self-Attention (MHSA), feed-forward networks (FFNs), residual connections, and layer normalization.

The self-attention operation is defined as:

$$Attention(Q, K, V) = Softmax\left(\frac{Q^{KT}}{\sqrt{d_k}}\right)V$$

where:

- Q denotes the query matrix,
- K denotes the key matrix,
- V denotes the value matrix,
- d_k represents the dimensionality of the key vectors.

Unlike previous pose estimation architectures, no hierarchical stages, feature pyramids, or multi-resolution branches are introduced. The transformer encoder remains architecturally identical to its image classification counterpart.

Decoder Design

After feature extraction, ViTPose employs lightweight decoders to generate keypoint heatmaps.

The standard decoder consists of two deconvolution layers followed by a final convolutional projection to the heatmap space. However, one of the most significant findings of the study is that decoder complexity contributes very little to overall performance when a transformer backbone is employed.

The authors demonstrated that replacing the conventional decoder with a simple bilinear upsampling operation followed by a single convolution results in only a marginal performance reduction. This observation suggests that transformer representations already contain highly informative spatial and semantic information, making extensive decoder processing largely unnecessary.

Bottom-Up Extension

ViTPose also supports bottom-up pose estimation through the integration of associative embedding techniques. In this formulation, the network predicts both keypoint heatmaps and embedding vectors. The embedding vectors allow keypoints belonging to the same individual to be grouped together during inference.

To control the computational complexity of self-attention at high resolutions, window-based attention mechanisms are incorporated into the bottom-up architecture.

ViTPose++: Knowledge Factorization

The principal innovation of ViTPose++ is the introduction of a Mixture-of-Experts decomposition within the transformer feed-forward networks.

Instead of treating all tasks equally, each FFN is divided into shared and task-specific channels.

The initial projection layer remains fully shared across all tasks, allowing the model to learn common anatomical representations. Following activation, the hidden features are partitioned into two components:

$$(1-\alpha)\,\gamma C$$

channels assigned to a shared expert responsible for universal anatomical knowledge, and $\alpha\gamma C$

channels assigned to task-specific experts that capture specialized representations for humans, animals, or whole-body configurations.

The outputs are subsequently recombined and projected back to the original feature dimension.

A particularly important characteristic of this design is that the shared and task-specific branches are merged during inference. Consequently,

ViTPose++ introduces virtually no additional computational cost compared with the original ViTPose architecture.

Datasets

ViTPose was trained and evaluated on a diverse collection of pose estimation benchmarks.

For human pose estimation, the primary datasets included COCO Keypoints, MPII Human Pose, AI Challenger, CrowdPose, and OCHuman. These datasets collectively provide a wide range of body configurations, scales, and occlusion scenarios.

ViTPose++ expanded training to include COCO-WholeBody, which contains 133 annotated keypoints encompassing body joints, facial landmarks, hand joints, and foot keypoints. Additionally, the framework incorporated AP-10K and APT-36K for animal pose estimation, covering dozens of animal species and anatomical configurations.

A notable finding from the study was that pre-training on human pose datasets transfers effectively to animal pose estimation. Models pre-trained on human pose data achieved superior performance compared with models initialized solely from generic image classification datasets, suggesting that pose-specific representations capture transferable anatomical concepts.

Loss Function

Heatmap Regression Loss

For top-down pose estimation, ViTPose adopts the conventional heatmap regression paradigm. The network predicts a set of heatmaps corresponding to the target keypoints, and optimization is performed using Mean Squared Error (MSE):

$$L_{heatmap} = \frac{1}{K} \sum_{k=1}^{K} \| H_k - H_k^{\cdot} \|^2$$

where:

- K represents the number of keypoints,
- H_k denotes the predicted heatmap,
- $H_k^{\cdot}$ denotes the ground-truth heatmap.

The target heatmaps are generated by placing Gaussian kernels centered at the annotated keypoint locations. This formulation encourages the network to assign high confidence to the correct keypoint positions while maintaining spatial smoothness.

Associative Embedding Loss

For bottom-up pose estimation, ViTPose jointly optimizes heatmap detection and associative embedding objectives.

The embedding loss consists of two complementary components. The first term pulls keypoints belonging to the same person closer together in the embedding space, while the second term pushes embeddings associated with different individuals apart.

The loss can be expressed as:

$$L_{embed} = L_{pull} + L_{push}$$

where:

- L_{pull} minimizes intra-person embedding variance,
- L_{push} maximizes inter-person embedding separation.

This formulation enables accurate grouping of keypoints without requiring person-level bounding boxes.

Knowledge Distillation in ViTPose++

To transfer knowledge from large transformer models to smaller variants, ViTPose++ employs both output-based and token-based distillation.

The output distillation loss is defined as:

$$L_{od} = MSE(K_s, K_t)$$

where:

- K_s represents student predictions,
- K_t represents teacher predictions.

Additionally, a learnable knowledge token is introduced during training and optimized to capture task-relevant information. The final objective combines both distillation components:

$$L = L_{td} + L_{od}$$

This strategy enables efficient compression while preserving the performance advantages of large-scale transformer models.

Evaluation Metrics

ViTPose and ViTPose++ are evaluated using standard human pose estimation metrics.

The primary evaluation metric is Average Precision (AP) computed under the Object Keypoint Similarity (OKS) protocol. Additional metrics include AP^{50}, AP^{75}, AP^{M}, AP^{L}, and Average Recall (AR).

MPII performance is measured using PCKh@0.5, while computational efficiency is assessed through throughput, memory consumption, and inference speed measurements on modern GPU hardware.

Results and Findings

Human Pose Estimation Performance

ViTPose established a new benchmark for transformer-based pose estimation. The ViTPose-B model achieved approximately **75.8% AP** on COCO validation, while the larger ViTPose-L and ViTPose-H variants increased performance to **78.3% AP** and **79.1% AP**, respectively.

These results surpassed several specialized transformer architectures while relying on a significantly simpler design. The findings demonstrated that architectural simplicity does not necessarily imply reduced performance when paired with powerful transformer representations.

Decoder Analysis

One of the most important experimental findings concerns decoder design.

For ViTPose-B, replacing the conventional deconvolution decoder with a simple upsampling-based decoder reduced performance by less than **0.3 AP**. In contrast, analogous experiments using ResNet-based backbones resulted in performance drops approaching **18 AP**.

This result provides strong evidence that Vision Transformers produce semantically rich feature representations that require minimal downstream processing.

Large-Scale Performance

The largest model, ViTPose-G, achieved approximately **80.9% AP** on the COCO test-dev benchmark using a single model. This performance exceeded previous state-of-the-art systems, including several ensemble-based approaches.

The result demonstrated that scaling transformer architectures continues to provide measurable gains for pose estimation tasks, mirroring trends previously observed in image classification and language modeling.

ViTPose++ Multi-Task Learning

ViTPose++ successfully unified human, animal, and whole-body pose estimation within a single framework.

Across multiple benchmarks, the model achieved state-of-the-art performance while avoiding the degradation commonly observed in conventional multi-task learning systems. The MoE-based knowledge factorization mechanism effectively separated shared anatomical knowledge from task-specific representations.

Particularly notable improvements were observed on OCHuman, AP-10K, and APT-36K, where ViTPose++ established new benchmark results and demonstrated strong cross-domain generalization.

Bottom-Up Pose Estimation

The transformer backbone also proved effective in bottom-up pose estimation settings.

ViTPose-L achieved approximately **70.1% AP**, surpassing previous bottom-up state-of-the-art approaches such as HigherHRNet. This finding indicates that the benefits of transformer representations are not restricted to top-down pipelines and extend to bottom-up formulations as well.

Efficiency Analysis

Despite containing significantly more parameters than conventional CNN backbones, ViTPose exhibited superior hardware efficiency.

Experiments revealed that ViTPose-B achieved higher accuracy, lower memory consumption, and greater throughput than HRNet-based alternatives. Throughput exceeded **700 FPS** on modern GPUs while maintaining state-of-the-art accuracy, highlighting the computational advantages of transformer architectures on contemporary hardware.

Key Findings

The experimental results lead to several important conclusions:

1. Plain Vision Transformers are highly effective pose estimation backbones and can outperform carefully engineered CNN architectures.

2. Transformer feature representations reduce the importance of decoder complexity, enabling extremely simple decoding pipelines.
3. MAE pre-training plays a critical role in achieving strong pose estimation performance.
4. Transformer scaling continues to improve accuracy across a wide range of model sizes.
5. Knowledge factorization through Mixture-of-Experts effectively resolves task conflicts in multi-domain pose estimation.
6. A single transformer backbone can successfully support human pose estimation, whole-body estimation, and animal pose estimation within a unified framework.

Strengths

ViTPose offers several significant advantages. Its architecture is remarkably simple, highly scalable, and capable of leveraging advances in large-scale transformer pre-training. The framework demonstrates exceptional flexibility, supporting both top-down and bottom-up paradigms while maintaining competitive performance.

ViTPose++ further extends these advantages by enabling efficient multi-task learning across diverse anatomical domains. The Mixture-of-Experts design successfully combines shared representation learning with task specialization without increasing inference cost.

Additionally, the framework exhibits excellent hardware efficiency, strong transfer learning capabilities, and foundation-model characteristics that facilitate adaptation to a wide variety of downstream vision tasks.

Limitations

Despite its success, ViTPose inherits several limitations associated with transformer architectures.

Large model variants require substantial computational resources and memory, making deployment on mobile and embedded devices challenging.

Furthermore, the quadratic complexity of global self-attention increases computational cost significantly as image resolution grows.

The framework also retains the inherent limitations of top-down pose estimation, including dependence on person detection quality and inference latency that scales with the number of individuals present in a scene.

Although ViTPose++ mitigates task interference through knowledge factorization, effective training still requires large-scale, diverse datasets. Consequently, adaptation to highly specialized domains may necessitate additional pre-training and domain-specific data collection.

Impact

ViTPose represents one of the most influential milestones in transformer-based pose estimation research. The framework fundamentally altered the field's understanding of backbone design by demonstrating that powerful general-purpose transformers can outperform highly specialized pose estimation architectures.

Its success accelerated the adoption of transformer backbones across numerous dense prediction tasks and contributed to the broader shift toward foundation-model-based computer vision systems. ViTPose++ further extended this paradigm by establishing a practical framework for unified multi-domain pose estimation.

Beyond pose estimation, the architectural principles introduced by ViTPose have influenced research in object detection, visual tracking, 3D pose estimation, mesh recovery, and multimodal vision systems. Today, ViTPose and ViTPose++ are widely regarded as foundational references for transformer-based pose estimation and remain among the most influential architectures in the evolution of modern human pose estimation research.

4.1.6 Human Mesh Recovery (HMR)

Introduction

Human Mesh Recovery (HMR) was introduced by Kanazawa et al. in their seminal CVPR 2018 paper titled “End-to-End Recovery of Human Shape and Pose.” It represented the first deep learning framework capable of directly regressing 3D human mesh parameters from a single RGB image in a fully end-to-end manner, bypassing intermediate 2D keypoint detection. The model outputs a complete 3D mesh parameterized by the SMPL (Skinned Multi-Person Linear) body model, thereby enabling a wide range of downstream applications, including part segmentation, dense correspondences, and animation retargeting, that extend well beyond traditional skeleton-based pose estimation.

HMR 2.0 was subsequently proposed by Goel et al. in their ICCV 2023 paper “Humans in 4D: Reconstructing and Tracking Humans with Transformers.” This iteration modernized the original architecture by replacing the convolutional neural network (CNN) encoder with a Vision Transformer (ViT-H/16) and introducing a transformer decoder centered around a learnable SMPL query token. These modifications yielded substantial gains in accuracy while preserving the end-to-end design philosophy of the original framework.

Motivation

Prior to HMR, 3D human pose and shape estimation predominantly relied on optimization-based approaches such as SMPLify (2016), which iteratively fitted parametric body models to 2D keypoint detections. These methods suffered from several critical limitations: slow inference times (often requiring seconds per image), high sensitivity to initialization and noisy 2D detections, and poor generalization to in-the-wild imagery due to training on controlled laboratory data.

Although early deep learning methods existed, they were largely restricted to regressing 3D joint locations or required paired 2D-to-3D supervision, which was scarce and typically available only in constrained environments (e.g., Human3.6M and MPI-INF-3DHP).

The core innovation of HMR was to enable end-to-end learning of 3D mesh recovery by leveraging abundant in-the-wild 2D keypoint annotations from

datasets such as LSP, MPII, and COCO, in combination with a data-driven adversarial prior. The central hypothesis was that a discriminator trained on real 3D human meshes could provide weak supervision, guiding the network toward plausible 3D geometries even in the absence of explicit 3D ground truth for every training image.

Architecture

HMR (2018) Architecture

Image Encoder: A ResNet-50 network pretrained on ImageNet extracts a 2048-dimensional global feature vector ϕ_g through global average pooling.

Iterative Error Feedback (IEF) Module: The central architectural contribution is the Iterative Error Feedback mechanism, which progressively refines the SMPL parameters over T iterations (typically T=3):

$$\Theta_{t+1} = \Theta_t + \Delta\Theta_t, \quad \text{where} \quad \Delta\Theta_t = f(\phi_g, \Theta_t)$$

At each iteration, the current parameter estimate Θ_t is concatenated with the image features ϕ_g, and two fully connected layers (each with 1024 neurons and dropout) regress the parameter update $\Delta\Theta_t$. This iterative refinement alleviates the difficulty of directly regressing high-dimensional SMPL parameters in a single forward pass.

SMPL Parameterization: The network predicts:

- $\theta \in \mathbb{R}^{72}$: 3D joint rotations (24 joints × 3 axis-angle parameters),
- : Body shape coefficients,
- Weak-perspective camera parameters (scale s s s and translation t t t).

SMPL Mesh Generation: The predicted parameters (θ, β) are passed through the SMPL function $M(\theta, \beta)$ to produce a triangulated mesh with 6,890 vertices and 23 joints. A linear regressor $J = W M$ subsequently extracts 3D joint locations $\beta \in \mathbb{R}^{10}$ from the mesh vertices.

HMR 2.0 (2022) Architecture

ViT Encoder: A ViT-H/16 model comprising 50 transformer layers, pretrained on 2D keypoint localization (ViTPose-H), processes 256×192 input images and produces 16×12 image tokens of dimension 1280.

Transformer Decoder: A 6-layer transformer decoder employs multi-head self-attention (8 heads, 64 dimensions), multi-head cross-attention (8 heads, 64 dimensions), and feed-forward networks (hidden dimension 1024). A single learnable 2048-dimensional SMPL query token is fed into the decoder, which attends to the image tokens. A final linear layer maps the output token to the pose θ, shape β, and camera parameters.

Training Losses (HMR 2.0):

SMPL parameter loss (MSE), 2D keypoint reprojection loss $(\lambda_{2D} = 5)$, 3D keypoint loss $\left(\lambda_{3D} = 5\right)$, and mesh vertex loss $\left(\lambda_{\text{mesh}} = 60\right)$.

Training involved 1 million iterations on 8 A100 GPUs with a batch size of 384 using the AdamW optimizer (learning rate 1×10^{-5}).

Dataset

HMR (2018) Training Data: The model was trained on a combination of 2D keypoint datasets (LSP, LSP-extended, MPII, MS COCO) for in-the-wild diversity and 3D-annotated datasets (Human3.6M, MPI-INF-3DHP) for supervision, supplemented by CMU MoShed meshes for adversarial training. Images were resized to 224×224 while preserving aspect ratio, with data augmentation including random scaling, translation, and flipping.

HMR 2.0 Training Data: The dataset was significantly expanded to include video frames from AVA (1 fps) and InstaVariety (5 fps), as well as AI Challenger. Pseudo-ground-truth SMPL parameters were generated using ProHMR with ViTDet detections and ViTPose keypoints. Sampling probabilities were assigned across datasets to balance representation.

Loss Function

HMR (2018) Loss

The total loss is a weighted sum:

$$\mathcal{L} = \lambda_{\text{reproj}} \mathcal{L}_{\text{reproj}} + \lambda_{3D} \mathcal{L}_{3D} + \lambda_{\text{SMPL}} \mathcal{L}_{\text{SMPL}} + \lambda_{\text{adv}} \mathcal{L}_{\text{adv}}$$

- **2D Reprojection Loss:** Measures the discrepancy between projected SMPL joints and ground-truth 2D keypoints under weak-perspective projection.
- **3D Joint Loss:** Applied when 3D supervision is available.
- **SMPL Parameter Loss:** ℓ_2 distance on pose and shape parameters when ground truth is present.
- **Adversarial Loss:** Trained to distinguish real SMPL parameters (from CMU MoShed) from network predictions.

Factorized Adversarial Prior

The discriminator is factorized into shape, per-joint pose, and full-pose components. This design proved essential for stable training and effective regularization; without it, the model frequently produced implausible (“monstrous”) meshes.

Results

On Human3.6M (Protocol 1), HMR achieved competitive MPJPE while operating in real time. Under Protocol 2, it reported a PA-MPJPE of 56.8 mm. On the more challenging in-the-wild 3DPW dataset, HMR 2.0 variants achieved PA-MPJPE values of approximately 48–54 mm, demonstrating strong performance relative to contemporary methods such as SPIN and PARE. Notably, optimized training strategies enabled the HMR framework to reach 47.31 mm PA-MPJPE on 3DPW without fine-tuning. HMR 2.0 additionally serves as the foundation for 4DHumans, extending single-image recovery to video-based tracking and reconstruction.

Strengths

HMR introduced several enduring advantages: fully end-to-end trainability enabling real-time inference (~30 FPS), weakly supervised learning through adversarial priors that leverage abundant 2D annotations, rich SMPL-based output representations supporting diverse applications, and the effective Iterative Error Feedback mechanism. HMR 2.0 further enhanced scalability via transformer architectures and pretraining strategies, while maintaining holistic inference even under occlusion and truncation.

Limitations

Key limitations include the global feature bottleneck of the original CNN encoder, the restrictive weak-perspective camera model, the limited expressiveness of the SMPL shape space, challenges with extreme poses and axis-angle discontinuities, hallucination of occluded parts, training instability in adversarial settings, and dataset bias leading to degraded performance on out-of-distribution data. HMR 2.0, while more accurate, incurs higher computational costs due to its large ViT backbone.

Impact

HMR is widely regarded as a foundational contribution to 3D human understanding, shifting the field from optimization-based to regression-based paradigms. It established core principles—SMPL parameter regression, adversarial/optimization priors for weak supervision, and end-to-end differentiability—that continue to influence subsequent works such as SPIN, PyMAF, PARE, CLIFF, and TokenHMR. The framework has been extensively adopted in both academia (e.g., OpenMMLab's MMHuman3D) and industry, powering applications in virtual try-on, sports analytics, neural rendering, and video understanding through 4DHumans. Its CVPR 2018 paper remains one of the most highly cited works in the domain.

4.1.7 PoseFormer

Introduction

PoseFormer was introduced by Zheng et al. in their ICCV 2021 paper titled "3D Human Pose Estimation with Spatial and Temporal Transformers." It was

the first purely transformer-based model for 2D-to-3D pose lifting from video sequences, entirely eliminating convolutional architectures from the pipeline. Inspired by the success of Vision Transformers in image classification, the work extended transformer capabilities to the spatio-temporal domain of human motion.

PoseFormerV2, presented by Zhao et al. at CVPR 2023, addressed two major limitations of the original model: computational inefficiency with long sequences and fragility to noisy 2D joint detections. It introduced a frequency-domain representation based on Discrete Cosine Transform (DCT) coefficients, enabling efficient scaling of the receptive field while enhancing robustness to detection noise.

Motivation

Prior to PoseFormer, 3D human pose estimation from video relied primarily on temporal convolutional networks, recurrent networks such as LSTMs or GRUs, and graph convolutional networks. Temporal convolutional approaches, exemplified by VideoPose3D, employed dilated convolutions that suffered from limited receptive fields and fixed connectivity patterns. Recurrent models struggled with long-range dependencies and lacked efficient parallelization. Graph-based methods modeled the human skeleton using predefined adjacency matrices that could not adapt to input-dependent joint relationships.

These architectures exhibited inherent constraints: convolutional networks required dilation to capture global context, recurrent networks were limited by sequential processing, and graph networks relied on static structures. The core motivation behind PoseFormer was to harness the self-attention mechanism of transformers to simultaneously model spatial correlations between joints within each frame (such as kinematic chains and bilateral symmetry) and temporal dependencies across frames (including motion continuity and periodicity). The central hypothesis was that a factorized spatial-temporal transformer could achieve this while avoiding the quadratic complexity of full joint spatio-temporal attention.

PoseFormerV2 was further motivated by the practical challenges of sequence length bottlenecks, where increasing temporal context dramatically raised computational costs due to the quadratic scaling of self-attention, and by the sensitivity of existing models to noisy 2D detections from detectors such as CPN and HRNet, particularly for occluded joints.

Architecture

PoseFormer: Factorized Spatial-Temporal Transformer

The input consists of a sequence of f 2D poses, each with J joints, represented as $X \in \mathbb{R}^{f \times J \times 2}$. The architecture comprises three main modules. In the spatial transformer module, each joint's 2D coordinates are projected into a c -dimensional embedding via a spatial patch embedding, and spatial positional embeddings encode joint identities. For each frame i , the input is processed through L layers of standard transformer blocks consisting of multi-head self-attention, MLP, layer normalization, and residual connections. The temporal transformer module then flattens and concatenates the per-frame spatial features, adds temporal positional embeddings, and applies M transformer layers to model frame-to-frame dependencies. Finally, a regression head performs a weighted mean aggregation across temporal features followed by an MLP with layer normalization to output the 3D pose $y \in \mathbb{R}^{1 \times (J \cdot 3)}$.

This factorized design yields a computational complexity of $O(f \cdot J^2 \cdot c + f^2 \cdot J \cdot c)$, which is significantly more efficient than the naive joint spatio-temporal approach of $O((f \cdot J)^2 \cdot c)$.

PoseFormerV2: Frequency-Domain Enhancement

The key innovation in PoseFormerV2 is the replacement of direct processing of all frames with a compact frequency-domain representation. For each joint trajectory across F frames, Discrete Cosine Transform coefficients are computed, and only the first n low-frequency coefficients (where $n \ll F$) are retained. These capture global motion trends while filtering high-frequency noise. The architecture employs a dual-stream temporal encoder with a time-domain stream processing f central frames for short-term dynamics and

a frequency-domain stream handling the n DCT coefficients for long-term trends. A specialized FreqMLP module fuses features from both streams within each transformer layer.

This design expands the effective receptive field from f to F frames with minimal computational overhead and allows flexible trade-offs between speed and accuracy by independently specifying f and n . When f=n , the complexity remains constant regardless of the full sequence length F.

Dataset

The models were primarily evaluated and trained on the Human3.6M dataset, which serves as the main benchmark with 11 actors performing 17 actions captured indoors using marker-based motion capture systems. Cross-domain validation was conducted on MPI-INF-3DHP, featuring 8 actors, 8 actions, and diverse indoor and outdoor scenes across 14 camera views. HumanEva provided additional validation with 3 subjects and 6 actions using synchronized video and motion capture.

Two-dimensional poses were obtained using the Cascaded Pyramid Network (CPN) detector for Human3.6M experiments, while ground-truth 2D poses were employed for MPI-INF-3DHP and ablation studies. Data augmentation included horizontal flipping (during both training and testing), random scaling and rotation (training only), and stochastic depth with a rate of 0.1 for regularization.

Loss Function

Both PoseFormer and PoseFormerV2 are trained using the standard mean per joint position error (MPJPE) as the primary loss:

$$\mathscr{L} = \frac{1}{J}\sum_{k=1}^{J}\left\| p_k - \widehat{p}_k \right\|_2^2,$$

where p_k and $\widehat{p}_k$ denote the ground-truth and predicted 3D coordinates of the k-th joint, respectively. In PoseFormerV2, the DCT representation

provides additional implicit regularization by naturally smoothing trajectories through low-frequency components, acting as a temporal regularizer.

Training utilized the Adam optimizer with an initial learning rate of 2×10^{-4}, exponential decay (factor 0.98 per epoch), 130 epochs, a batch size of 1024, weight decay of 0.1, and stochastic depth of 0.1.

Metrics

Evaluation relied on several standard metrics. MPJPE (Protocol 1) measures the mean per joint position error in millimeters using direct Euclidean distance. P-MPJPE (Protocol 2), or Procrustes-aligned MPJPE, applies rigid alignment (translation, rotation, and scale) before computing the error. Additional metrics include the Percentage of Correct Keypoints (PCK) within a 150 mm threshold, the Area Under the PCK Curve (AUC), model computational complexity in MFLOPs, and the effective receptive field, defined as the number of frames influencing the prediction of the center frame.

Results

On Human3.6M under Protocol 1 with CPN-detected 2D inputs, PoseFormer achieved a state-of-the-art MPJPE of 44.3 mm using 9 frames, outperforming previous methods such as VideoPose3D (46.8 mm with 81 frames) and other temporal convolutional and attention-based baselines. With ground-truth 2D inputs, performance improved further to 31.3 mm. PoseFormerV2 variants demonstrated strong efficiency-accuracy trade-offs; for instance, processing 81 frames with appropriate DCT coefficients yielded MPJPE values around 46.0–47.6 mm while significantly reducing computational cost compared to the original PoseFormer.

Under Protocol 2, PoseFormerV2 reached 36.1 mm P-MPJPE, surpassing several contemporary approaches. On MPI-INF-3DHP, PoseFormerV2 achieved superior cross-domain performance with a PCK of 97.9, AUC of 78.8, and MPJPE of 27.8 mm. Efficiency comparisons highlighted that PoseFormerV2 attained approximately seven times lower FLOPs than

PoseFormerV1 while handling three times longer sequences, and up to 44 times lower FLOPs than MixSTE, all while maintaining competitive accuracy.

Strengths

PoseFormer established the viability of pure transformer architectures for 2D-to-3D pose lifting, demonstrating that convolutions are not essential and inspiring subsequent transformer-based methods. Its factorized spatial-temporal design efficiently captures both intra-frame joint relationships and long-range temporal dependencies without prohibitive quadratic costs. Attention visualizations revealed that the model autonomously learns anatomically meaningful patterns, such as focusing on specific limbs or bilateral symmetries. PoseFormerV2 further advanced the approach through frequency-domain processing, providing a compact, noise-robust representation that expands the effective receptive field without increasing inference overhead and offers improved robustness to 2D detection noise. The frequency-domain insight has proven generalizable to other transformer architectures, enhancing their performance as well.

Limitations

As a 2D-to-3D lifting method, PoseFormer remains fundamentally dependent on the quality of upstream 2D detectors, with substantial error propagation observed when moving from ground-truth to detected inputs. Although factorized, the temporal transformer still exhibits quadratic complexity with respect to the number of processed frames, which can become limiting for extremely long sequences without the DCT enhancement. Both variants are designed for single-person scenarios and require additional components for multi-person settings. Training predominantly on controlled indoor datasets such as Human3.6M leads to reduced generalization on in-the-wild data. The model operates solely on 2D joint coordinates, discarding rich image texture cues that could help resolve depth ambiguities. Additionally, aggressive DCT truncation in PoseFormerV2 may smooth out fine-grained rapid motions, and adaptation to different skeleton topologies or non-human subjects requires architectural modifications. The approach also relies on large-scale 3D-annotated datasets, which are costly to obtain.

Impact

PoseFormer and its successor have exerted considerable influence on 3D human pose estimation by validating pure transformers as a powerful alternative for temporal 2D-to-3D lifting. This work catalyzed numerous follow-up transformer-based architectures, including MHFormer, MixSTE, P-STMO, MotionBERT, and STCFormer. The introduction of frequency-domain representations via DCT in PoseFormerV2 opened a new direction for efficient long-sequence modeling, impacting not only pose estimation but also action recognition, motion prediction, and real-time motion capture systems. Attention map visualizations provided a valuable template for interpretability, showing how self-attention naturally discovers human kinematic structures. The models have found applications in sports analytics, virtual production, healthcare gait analysis, and animation. With over one thousand citations for the original ICCV 2021 paper, PoseFormer is recognized as a foundational reference in transformer-based 3D pose estimation.

4.1.8 MANO (Model of the Articulated Hand).

Introduction

MANO (a Model of the Articulated and Non-rigid defOrmations of the hand) was introduced by Romero, Tzionas, and Black in their SIGGRAPH Asia 2017 paper titled “Embodied Hands: Modeling and Capturing Hands and Bodies Together,” published in ACM Transactions on Graphics. MANO is a low-dimensional parametric model of the human hand derived from high-resolution 3D scans. Developed as a natural extension of the SMPL body model, it specifically addresses the challenges unique to hand modeling, including extreme articulation with over 27 degrees of freedom, small scale relative to the body, frequent self-occlusion, and complex non-rigid deformations during grasping and gesturing. The model is fully differentiable, compatible with standard graphics pipelines, and capable of fitting any human hand. Released with open-source code, pre-trained models, and fitting tools, MANO has become the de facto standard parametric hand model in both computer vision and computer graphics.

Motivation

Prior to MANO, hand modeling and tracking primarily relied on skeletal models using simple geometric primitives such as cylinders, spheres, and capsules, which were computationally efficient but visually unrealistic and unable to capture soft-tissue deformations. High-resolution mesh sequences from dense 3D scans offered visual accuracy but were high-dimensional, non-parametric, and difficult to animate or fit to new observations. Existing body models like SMPL treated hands as rigid appendages with minimal articulation, proving inadequate for the expressive range of hand motions.

The core motivation was to create a model that combines the low dimensionality and efficiency of skeletal representations with the realism of dense scans, while ensuring compatibility with existing body models for full-body capture. The key insight was that hands, like full bodies, exhibit strong statistical correlations in shape and pose that can be compactly represented through Principal Component Analysis. By learning these priors from 3D scan data, the model effectively constrains the solution space to anthropometrically plausible hand configurations.

Architecture

Parametric Formulation

MANO defines a differentiable function $M(\beta, \theta)$ that maps shape parameters β and pose parameters θ to a posed hand mesh:

$$M(\beta, \theta) = W\left(T_P(\beta, \theta), J(\beta), \theta, \mathcal{W}\right),$$

where the template mesh deformation is given by $T_P(\beta, \theta) = \overline{T} + B_S(\beta) + B_P(\theta)$. Here, $\overline{T} \in \mathbb{R}^{778 \times 3}$ represents the mean hand template mesh with 778 vertices and 1538 triangular faces;

$$B_S(\beta) = \sum_{n=1}^{10} \beta_n S_n$$

denotes the shape blend shapes derived from PCA; and $B_P(\theta) = \sum_{m=1}^{K} (\theta_m - \bar{\theta}_m) P_m$ captures pose-dependent corrective displacements for non-rigid soft-tissue deformations such as muscle bulging and skin stretching. A sparse linear joint regressor $J(\beta) = J \cdot T_P(\beta, \theta = 0)$,with $J \in \mathbb{R}^{16 \times 778}$, maps vertices to 16 joint locations. Linear Blend Skinning (LBS) with learned blend weights $\mathscr{W} \in \mathbb{R}^{778 \times 16}$ is then applied to generate the final mesh.

Parameter Space and Kinematic Structure

The model uses 10 shape parameters β \beta β corresponding to PCA coefficients learned from approximately 1000 scans of 31 subjects, and up to 48 pose parameters θ \theta θ (16 joints × 3 axis-angle parameters), which can be further reduced via PCA to 6–45 components for efficiency. Global rotation and translation parameters complete the parameterization. The kinematic structure includes 21 keypoints (with 16 articulating joints: wrist plus three per finger for the index to pinky and a specialized thumb chain) and five non-articulating fingertip landmarks.

Pose Synergy Manifold

A significant innovation is the learned pose PCA manifold, which projects the full 48-dimensional pose space onto a lower-dimensional linear subspace derived from real hand poses:

$$\theta = \bar{\theta} + \sum_{k=1}^{K_{\text{pca}}} c_k \cdot v_k.$$

This approach prevents anatomically implausible configurations while preserving natural pose variation.

SMPL+H Integration

MANO integrates seamlessly with the SMPL body model to form SMPL+H, replacing rigid hand capsules with fully articulated MANO hands. It features a

shared shape space derived from full-body scans and a unified parameterization combining body and hand shape and pose parameters.

Dataset

MANO was learned from a proprietary high-resolution 3D scan dataset collected at the Max Planck Institute for Intelligent Systems, comprising approximately 1000 scans of 31 individuals exhibiting diverse hand shapes and a wide variety of poses, including flat hands, fists, precision and power grips, individual finger movements, and complex grasping configurations. Scans were acquired using structured light or multi-view stereo techniques capable of capturing fine surface details such as wrinkles, tendons, and veins. Shape space training utilized scans in a canonical flat-hand pose, while pose and blend shape training employed diverse posed scans with and without soft-tissue analysis.

Loss Function

Although MANO itself is a statistical model learned offline via PCA rather than gradient descent, it serves as a differentiable layer in optimization and deep learning pipelines. In optimization-based fitting, the total objective typically combines 2D reprojection loss, 3D joint loss (when available), shape and pose priors (Gaussian or Mahalanobis distance in PCA space), and collision penalties to prevent interpenetration. In learning-based regression approaches, networks directly regressing MANO parameters are trained with vertex and joint losses, silhouette consistency terms, and adversarial losses to promote realistic outputs.

Metrics

Standard evaluation metrics for MANO-based hand reconstruction include Mean Per Joint Position Error (MPJPE) for 3D joint accuracy, Procrustes-aligned MPJPE (PA-MPJPE) for shape-independent pose evaluation, Mean Per Vertex Position Error (MPVPE) for mesh surface accuracy, F-score at various distance thresholds for reconstruction quality, bidirectional Chamfer distance for point cloud comparison, Percentage of

Correct Keypoints (PCK), Area Under the PCK Curve (AUC), and contact accuracy measures for hand-object interactions.

Results

On the FreiHAND benchmark, methods leveraging MANO have demonstrated progressive improvements, with recent approaches achieving MPJPE and MPVPE below 8 mm and high F-scores at 5 mm and 15 mm thresholds. In full-body evaluations on the EHF dataset, SMPL+H significantly reduces vertex-to-vertex and joint errors compared to SMPL with naive hand modeling, confirming the value of expressive hand articulation. These results underscore MANO's effectiveness in producing accurate and realistic hand reconstructions across diverse scenarios.

Strengths

MANO offers statistical realism derived from real 3D scans, constraining outputs to plausible anthropometric configurations and avoiding unrealistic artifacts. Its low dimensionality (10 shape and 6–45 pose parameters) enables efficient optimization and learning, while full differentiability supports end-to-end training in modern frameworks. Pose blend shapes effectively capture soft-tissue dynamics beyond standard linear blend skinning. The model integrates seamlessly with SMPL for unified body-hand modeling and has become the standardized representation across datasets and benchmarks. It is computationally efficient for real-time use and highly extensible, as evidenced by subsequent extensions incorporating texture, biomechanics, and accessories.

Limitations

Despite its strengths, MANO's relatively low mesh resolution (778 vertices) limits the capture of fine surface details such as fingerprints, wrinkles, or nails. The fixed topology precludes topological changes or adaptive refinement, and the PCA pose space may underrepresent extreme or novel configurations outside the training distribution. Axis-angle representations introduce discontinuities, while linear blend skinning can still produce artifacts around joints despite pose corrections. The model addresses single hands in

isolation, complicating bimanual interactions, and exhibits shape-pose coupling limitations along with dataset biases toward adult hands from limited demographics. It also lacks intrinsic modeling of hand-object contact deformations.

Impact

MANO has profoundly influenced hand and body modeling across computer vision, graphics, robotics, and human-computer interaction. It enabled the creation of SMPL+H and SMPL-X for unified full-body and face modeling and inspired extensions such as HTML for appearance, NIMBLE for musculoskeletal simulation, and others. The model standardized output representations in major benchmarks including FreiHAND, HO-3D, DexYCB, InterHand2.6M, and GRAB, driving progress in hand-object interaction and dexterous manipulation. Virtually all contemporary hand mesh recovery methods—whether optimization-based, regression-based, or hybrid—adopt MANO parameters. Its applications span AR/VR gesture tracking, robotic grasp planning, virtual try-on, healthcare rehabilitation, and animation. With thousands of citations, the original work is recognized as the foundational contribution that established parametric hand modeling as a mature subfield of 3D human understanding.

4.2 Practical Implementation: HPE-Bible Project Suite

This section presents the practical implementation component of this thesis—a comprehensive Human Pose Estimation toolkit named HPE-Bible, developed to demonstrate and compare three distinct pose estimation strategies. The project suite consists of three specialized implementations, each optimized for different scenarios and built upon the theoretical foundations discussed in the preceding chapters.

4.2.1 Project Overview

Approach	Best For	Strategy	Framework
YOLO Test 1	Far-view scenes (sports stadiums, surveillance)	Image tiling with YOLOv8-Pose	Ultralytics YOLOv8
YOLO Test 2	High-accuracy multi-person analysis	Two-stage detection + pose pipeline	Ultralytics YOLOv8
RTMPose	Real-time applications with tracking	Top-down with ByteTrack	OpenMMLab (MMDet + MMPose)

The project was developed by Hadi Shaheen, Mostafa Khaled, Marawan Sitten, Mohamed Salah, and Mosa Mohamed. The complete source code is available at: https://github.com/HadiShaheen249/HPE-Bible.

4.2.2 YOLO Test 1: Tiled Pose Estimation for Far-View Scenes

4.2.2.1 Concept and Motivation

The first implementation addresses a critical challenge in pose estimation: detecting poses in distant subjects where individuals occupy only a small fraction of the image pixels. In wide-angle scenes such as football matches, basketball games, or surveillance footage, standard pose estimation models

often fail because the subject's resolution is too low for reliable keypoint detection.

The core idea is intelligent image tiling: the input image or video frame is automatically divided into a configurable grid (e.g., 2×2, 3×3). Each tile is processed independently at full resolution, effectively magnifying distant subjects within their respective tiles. The detected keypoints are then mapped back to the original image coordinates and merged into a unified result.

Why Tiling Works:

- When subjects are far from the camera, they may occupy only 50–100 pixels in a 1920×1080 frame
- By splitting into a 3×3 grid, each tile is approximately 640×360 pixels
- The same subject now occupies a larger relative area within its tile, dramatically improving detection accuracy
- The approach is particularly effective for sports analytics and crowd monitoring applications

4.2.2.2 Architecture and Key Components

The project structure for YOLO Test 1 is organized as follows:

```
yolo_test1/
├── main.py              # CLI entry point and interactive menu
├── pose_estimator.py    # Core YOLOv8PoseEstimator class
├── config.py            # Project configuration
├── app.py               # Interactive application interface
└── requirements.txt     # Dependencies
```

Core Class: YOLOv8PoseEstimator

The YOLOv8PoseEstimator class is the heart of this implementation. It encapsulates all pose estimation functionality:

```
class YOLOv8PoseEstimator:
    def __init__(self, model_name: str = 'yolov8n-pose.pt',
                 conf_threshold: float = 0.5):
        self.model_name = model_name
        self.conf_threshold = conf_threshold
```

Key Methods:

Method	Purpose
`_load_model()`	Loads YOLOv8-Pose model from local cache or downloads from Ultralytics hub
`_split_image_to_tiles()`	Divides input image into grid tiles with configurable size
`_process_tiled_frame()`	Processes each tile independently and merges results
`predict_image()`	Full image processing pipeline with tiling support
`predict_video()`	Video/camera processing with real-time FPS display

Tiling Implementation:

The tiling logic is implemented in _split_image_to_tiles():

```
def _split_image_to_tiles(self, image, grid_size=(3, 3)):
    h, w, _ = image.shape
    rows, cols = grid_size
    tile_h, tile_w = h // rows, w // cols
    tiles = []
    for i in range(rows):
        for j in range(cols):
            x1, y1 = j * tile_w, i * tile_h
            x2, y2 = x1 + tile_w, y1 + tile_h
            crop = image[y1:y2, x1:x2]
            tiles.append((crop, (x1, y1)))
    return tiles
```

The key challenge in tiling is coordinate remapping. When a pose is detected within a tile, the keypoint coordinates are relative to that tile's local coordinate system. The system adjusts these back to the original image coordinates:

```
# Adjust keypoints to original image coordinates
adjusted_kpts = person_kpts.clone()
adjusted_kpts[:, 0] += x1  # x offset
adjusted_kpts[:, 1] += y1  # y offset
```

Skeleton Drawing:

The implementation draws the COCO-format skeleton (17 keypoints, 19 connections) with distinct colors for each keypoint:

```
skeleton = [

    [16, 14], [14, 12], [17, 15], [15, 13], [12, 13],   # legs & hips

    [6, 12], [7, 13], [6, 7], [6, 8], [7, 9],            # torso & shoulders

    [8, 10], [9, 11], [2, 3], [1, 2], [1, 3],            # arms

    [2, 4], [3, 5], [4, 6], [5, 7]                       # head

]
```

Configuration (config.py):

```
class Config:

    BASE_DIR = Path(__file__).parent.absolute()

    MODEL_NAME = 'yolov8n-pose.pt'

    CONFIDENCE_THRESHOLD = 0.5

    TILE_GRID = (3, 3)  # Default 3×3 tiling grid


    # Directory structure

    MODELS_DIR = BASE_DIR / 'models'

    INPUT_IMAGES_DIR = BASE_DIR / 'input' / 'images'

    OUTPUT_IMAGES_DIR = BASE_DIR / 'output' / 'images'
```

The TILE_GRID parameter is configurable at runtime, allowing users to balance between detection accuracy (larger grid = smaller tiles = higher effective resolution) and computational cost.

Command-Line Interface (main.py):

The CLI supports multiple operation modes:

```
# Process image with tiling

python main.py --image path/to/image.jpg --model
yolov8m-pose.pt

# Process video

python main.py --video path/to/video.mp4 --conf 0.7

# Use camera

python main.py --camera

# Interactive mode

python main.py --interactive
```

4.2.2.3 Use Cases

- Sports field analysis: Football, basketball, and athletics where players are distant
- Crowd monitoring: Surveillance systems requiring pose analysis in wide areas
- Marathon tracking: Long-distance runner pose and gait analysis

4.2.3 YOLO Test 2: Two-Stage Pose Estimation

4.2.3.1 Concept and Motivation

The second implementation employs a two-stage pipeline that separates person detection from pose estimation. This architecture offers several advantages over single-stage approaches:

- Specialized models: The detection model focuses solely on finding people (faster and more efficient), while the pose model works on cropped, centered images (better keypoint localization)
- Flexible resource allocation: Different model sizes can be used for detection (fast, small model) versus pose estimation (accurate, large model)
- Reduced false positives: The detection stage filters out non-person regions before pose estimation
- Better handling of overlapping persons: Each detected person is processed independently

This approach is particularly suited for group photos, action recognition, fitness tracking, and sports pose analysis where high precision is required.

4.2.3.2 Architecture and Key Components

The project structure is more modular than Test 1:

```
yolo_test2/
├── main.py               # Application entry point and CLI
├── models.py             # Model loading and management
├── detector.py           # Person detection (Stage 1)
├── pose_estimator.py     # Pose estimation (Stage 2)
├── video_processor.py    # Frame-by-frame processing
pipeline
├── config.py             # Comprehensive configuration
├── utils.py              # Helper utilities
└── requirements.txt
```

Stage 1: Person Detection (detector.py)

The PlayerDetector class wraps a YOLOv8 detection model (e.g., yolov8m.pt) to detect persons in the input frame:

```
class PlayerDetector:
    def __init__(self, model):
        self.model = model  # DetectionModel wrapper
        self.config = DETECTION_CONFIG
        self.filter_config = FILTER_CONFIG

    def detect_players(self, frame: np.ndarray) ->
List[Dict]:
        results = self.model.predict(frame)
        detections = []
        for i in range(len(results.boxes)):
            detection = {
                'bbox': [x1, y1, x2, y2],
                'confidence': float,
                'class_id': int,
                'center': (cx, cy),
                'area': float
            }
            detections.append(detection)

        return self._filter_detections(detections)
```

Key features of the detector:

- Area-based filtering: Removes detections that are too small or too large (min_bbox_area=1000, max_bbox_area=500000)
- Smart cropping: Extracts tight bounding boxes around each person with configurable padding
- Non-Maximum Suppression (NMS): Removes overlapping detections using IoU thresholding
- Multi-player tracking: The MultiPlayerDetector extension adds simple centroid-based tracking across frames

Stage 2: Pose Estimation (pose_estimator.py)

The PoseEstimator class processes each cropped person region independently:

```
class PoseEstimator:

    def __init__(self, model):

        self.model = model  # PoseModel wrapper

        self.config = POSE_CONFIG

        self.keypoints_map = YOLO_KEYPOINTS  # 17 COCO
keypoints

        self.skeleton_connections = SKELETON_CONNECTIONS

    def estimate_pose(self, image: np.ndarray) ->
Optional[Dict]:

        results = self.model.predict(image)

        kpts = results.keypoints.data[0].cpu().numpy()[0]

        visible_count = np.sum(kpts[:, 2] >
self.config['confidence_threshold'])

        if visible_count <
self.filter_config['min_keypoints_visible']:

            return None  # Reject low-quality poses

        return {

            'keypoints': kpts.tolist(),

            'bbox': bbox,

            'confidence': confidence,

            'visible_keypoints': int(visible_count),

            'total_keypoints': len(kpts)

        }
```

Pose Analysis Extensions:

The implementation includes a PoseAnalyzer class that adds action recognition capabilities:

```
class PoseAnalyzer:

    def detect_action(self, pose_data: Dict) -> str:

        angles = self.estimator.get_joint_angles(pose_data)



        # Kicking detection

        if 'left_knee' in angles and angles['left_knee'] >
160:

            return "kicking_left"

        elif 'right_knee' in angles and angles['right_knee']
> 160:

            return "kicking_right"

        # Running detection

        elif 90 < angles['left_knee'] < 140 and 90 <
angles['right_knee'] < 140:

            return "running"

        # Jumping detection

        elif angles['left_knee'] > 160 and
angles['right_knee'] > 160:

            return "jumping"

        return "standing"
```

Joint Angle Calculation:

```
def calculate_angle(self, p1, p2, p3):
    """Calculate angle between three points (in degrees)"""
    v1 = np.array(p1[:2]) - np.array(p2[:2])
    v2 = np.array(p3[:2]) - np.array(p2[:2])
    cos_angle = np.dot(v1, v2) / (np.linalg.norm(v1) *
np.linalg.norm(v2) + 1e-6)
    return np.degrees(np.arccos(np.clip(cos_angle, -1.0,
1.0)))
```

Model Management (models.py):

The ModelManager class handles automatic device selection and model downloading:

```
class ModelManager:
    def _get_device(self):
        if torch.cuda.is_available():
            return "cuda"
        elif torch.backends.mps.is_available():
            return "mps"  # Apple Silicon
        else:
            return "cpu"
```

```
    def load_detection_model(self):

        model_path = DETECTION_CONFIG["model_path"]

        if not model_path.exists():

            self.detection_model = YOLO(model_name)   #
Auto-download

            # Cache to local models directory

        return DetectionModel(self.detection_model,
DETECTION_CONFIG)
```

Video Processing Pipeline (video_processor.py):

The VideoProcessor class orchestrates the full two-stage pipeline:

```
class VideoProcessor:

    def process_frame(self, frame, frame_idx):

        # Stage 1: Detect players

        detections = self.detector.detect_players(frame)



        # Update tracking (if enabled)

        if self.tracking_enabled:

            detections =
self.detector.update_tracking(detections)

        # Stage 2: Estimate pose for each detection

        poses = []

        for det in detections:
```

```
            cropped = self.detector.crop_player(frame,
det['bbox'])

            pose =
self.pose_estimator.estimate_pose(cropped)

            # Adjust keypoints back to original frame
coordinates

            pose = self._adjust_keypoints_to_frame(pose,
det['bbox'])

            # Detect action

            pose['action'] =
self.pose_analyzer.detect_action(pose)

            poses.append(pose)

        # Draw annotations

        annotated_frame =
self._draw_detections_with_info(frame, detections, poses)

        return annotated_frame, detections, poses
```

Video Processing Pipeline (video_processor.py):

The VideoProcessor class orchestrates the full two-stage pipeline:

```
# Model Selection (interactive or auto)

SELECTED_MODEL = 'm'  # n, s, m, l, x

DETECTION_CONFIG = {

    "model_name": f"yolov8{SELECTED_MODEL}.pt",

    "confidence_threshold": 0.5,
```

```
    "iou_threshold": 0.45,
    "target_class": 0,  # person in COCO
    "device": "cuda",
    "imgsz": 640
}
POSE_CONFIG = {
    "model_name": f"yolov8{SELECTED_MODEL}-pose.pt",
    "confidence_threshold": 0.5,
    "device": "cuda",
    "imgsz": 640
}
# Keypoint definitions (COCO format)
YOLO_KEYPOINTS = {
    0: "nose", 1: "left_eye", 2: "right_eye", ..., 16:
"right_ankle"
}
SKELETON_CONNECTIONS = [
    (0, 1), (0, 2), (1, 3), (2, 4),  # head
    (5, 6), (5, 7), (7, 9), (6, 8), (8, 10),  # arms
    (5, 11), (6, 12), (11, 12),  # torso
    (11, 13), (13, 15), (12, 14), (14, 16)  # legs
```

```
]
```

Data Export:

The system supports multiple output formats:

JSON: Structured pose data with coordinates and confidence scores

CSV: Tabular format for analysis in Excel/Python

Annotated video: Visual results with drawn poses and bounding boxes

4.2.4 RTMPose: Real-Time Multi-Person Pose Estimation with Tracking

4.2.4.1 Concept and Motivation

The third implementation represents the state-of-the-art approach, leveraging the OpenMMLab ecosystem (MMDetection + MMPose) with integrated ByteTrack for multi-object tracking. This architecture follows the top-down paradigm:

1. Detection: RTMDet (Medium) detects all persons in the frame
2. Pose Estimation: RTMPose (Medium) estimates 17 COCO keypoints for each detected person
3. Tracking: ByteTrack maintains consistent person IDs across video frames

Why RTMPose?

- SimCC Architecture: Uses Simple Coordinate Classification instead of heatmap regression, achieving more precise keypoint localization

- Speed-Accuracy Balance: ~3× faster than traditional top-down methods while maintaining higher AP on COCO
- Production-Ready: Optimized for real-time deployment with minimal latency
- Robust Tracking: ByteTrack handles occlusion and crowded scenes effectively

4.2.4.2 Architecture and Key Components

```
RTMPose/
├── main.py                # CLI and interactive menu
├── pose_estimator.py      # RTMPoseEstimator core class
├── byte_tracker.py        # ByteTrack implementation
├── config.py              # Model and tracking configuration
├── utils.py               # Visualization and utilities
├── app.py                 # Interactive application
└── requirements.txt
```

Core Class: `RTMPoseEstimator`

```
class RTMPoseEstimator:

    def __init__(self, det_model_name=None,
pose_model_name=None,

                 det_conf=None, pose_conf=None,
device=None):

        self.det_model_name = det_model_name or
Config.DET_MODEL_NAME
```

```
        self.pose_model_name = pose_model_name or
Config.POSE_MODEL_NAME

        self.det_conf = det_conf or Config.DET_CONFIDENCE

        self.pose_conf = pose_conf or Config.POSE_CONFIDENCE

        self.device = device or Config.validate_device()

        self._load_models()

        self._init_tracker()
```

Model Loading with Registry Management:

A critical challenge when combining MMDetection and MMPose is registry scope conflicts. The implementation resolves this by explicitly managing registry scopes:

```
def _load_models(self):

    from mmdet.apis import init_detector

    from mmpose.apis import init_model as init_pose_model

    # Force MMDET registry scope

    register_mmdet_modules(init_default_scope=True)

    self.det_model = init_detector(det_config,
det_checkpoint, device=self.device)

    # Force MMPOSE registry scope

    register_mmpose_modules(init_default_scope=True)

    self.pose_model = init_pose_model(pose_config,
pose_checkpoint, device=self.device)
```

Detection Pipeline:

```
def _detect_persons(self, frame: np.ndarray) -> np.ndarray:

    from mmdet.apis import inference_detector

    register_mmdet_modules(init_default_scope=True)

        result = inference_detector(self.det_model, frame)

    pred_instances = result.pred_instances

        # Filter for person class (label 0) with confidence
threshold

    person_mask = (pred_instances.labels == 0) & \

                  (pred_instances.scores >= self.det_conf)

    bboxes =
pred_instances.bboxes[person_mask].cpu().numpy()

    scores =
pred_instances.scores[person_mask].cpu().numpy()

    return np.concatenate([bboxes, scores[:, None]], axis=1)
# (N, 5)
```

Pose Estimation Pipeline:

```
def _estimate_pose(self, frame: np.ndarray, bboxes:
np.ndarray) -> List[Dict]:

    from mmpose.apis import inference_topdown

    register_mmpose_modules(init_default_scope=True)

        results = inference_topdown(self.pose_model, frame,
bboxes[:, :4])
```

```
    return results
```

ByteTrack Integration (`byte_tracker.py`):

The implementation includes a complete ByteTrack tracker with:

- Kalman Filter: Predicts object motion between frames
- Two-Stage Association:
  1. High-confidence detections matched with existing tracks
  2. Low-confidence detections used to recover lost tracks
- Track State Management: `new` → `tracked` → `lost` → `removed`

```
class BYTETracker:

    def update(self, output_results, img_info, img_size):

        # Step 1: Split detections by confidence

        remain_inds = scores > self.track_thresh  # High
confidence

        inds_second = (scores > 0.1) & (scores <
self.track_thresh)  # Low confidence

        # Step 2: First association (high score)

        dists = matching.iou_distance(strack_pool,
detections)

        matches, u_track, u_detection =
matching.linear_assignment(dists, thresh=self.match_thresh)

       # Step 3: Second association (low score for lost
tracks)

        r_tracks = [strack_pool[i] for i in u_track if
strack_pool[i].state == 'tracked']
```

```
        dists = matching.iou_distance(r_tracks,
detections_second)

        matches, u_track, u_detection_second =
matching.linear_assignment(dists, thresh=0.5)

         # Step 4: Initialize new tracks

        for inew in u_detection:

            track = detections[inew]

            track.activate(self.kalman_filter,
self.frame_id)
```

Visualization:

```
def _visualize_results(self, frame, pose_results,
tracked_boxes):

    vis_frame = frame.copy()

        # Draw tracked bounding boxes with IDs

    for box in tracked_boxes:

        x1, y1, x2, y2, score, track_id = box

        vis_frame = VisualizationHelper.draw_bbox(

            vis_frame, (x1, y1, x2, y2),

            track_id=int(track_id), color=Config.BBOX_COLOR

        )

        # Draw skeleton and keypoints

    for result in pose_results:
```

```
        keypoints = result.pred_instances.keypoints[0]

        scores = result.pred_instances.keypoint_scores[0]



        vis_frame = VisualizationHelper.draw_skeleton(

            vis_frame, keypoints, scores,
Config.SKELETON_LINKS

        )

        vis_frame = VisualizationHelper.draw_keypoints(

            vis_frame, keypoints, scores,
Config.KEYPOINT_COLORS

        )

        return vis_frame
```

Configuration (`config.py`):

```
class Config:

    # RTMPose Model

    POSE_MODEL_NAME = 'rtmpose-m'

    POSE_CONFIG = 'rtmpose-m_8xb256-420e_coco-256x192.py'

    POSE_CHECKPOINT =
'rtmpose-m_simcc-aic-coco_pt-aic-coco_420e-256x192-63eb25f7_
20230126.pth'

    # RTMDet Detection Model

    DET_MODEL_NAME = 'rtmdet-m'
```

```
    DET_CONFIG = 'rtmdet_m_8xb32-300e_coco.py'

    DET_CHECKPOINT =
'rtmdet_m_8xb32-300e_coco_20220719_112220-229f527c.pth'

    # ByteTrack Settings

    TRACK_THRESH = 0.5       # High threshold for first
association

    TRACK_BUFFER = 30        # Frames to keep lost tracks

    MATCH_THRESH = 0.8       # IoU threshold for matching

    MIN_BOX_AREA = 10        # Minimum box area

    FRAME_RATE = 30          # Assumed frame rate

    # COCO Keypoints (17 points)

    KEYPOINT_NAMES = ['nose', 'left_eye', 'right_eye', ...,
'right_ankle']

    SKELETON_LINKS = [[15, 13], [13, 11], [16, 14], [14,
12], ...]
```

Performance Monitoring:

The `PerformanceMonitor` class tracks real-time FPS:

```
class PerformanceMonitor:

    def add_frame(self):

        self.frame_times.append(time.time())

        self.frame_count += 1
```

```
    def get_current_fps(self):

        n_frames = min(30, len(self.frame_times))

        time_diff = self.frame_times[-1] -
self.frame_times[-n_frames]

        return n_frames / time_diff if time_diff > 0 else 0
```

4.2.5 Comparative Analysis

Feature	YOLO Test 1 (Tiled)	YOLO Test 2 (Two-Stage)	RTMPose + ByteTrack
Paradigm	Single-stage with tiling	Two-stage (detect → pose)	Top-down with tracking
Best For	Far-view, distant subjects	High precision, group analysis	Real-time, multi-person tracking
Speed	Moderate (depends on grid size)	Slower (two inference passes)	Fast (~30+ FPS on GPU)

Accuracy	Good for distant subjects	Excellent (cropped inputs)	State-of-the-art on COCO
Tracking	None	Simple centroid-based	Full ByteTrack integration
Keypoints	17 (COCO)	17 (COCO)	17 (COCO) / 133 (WholeBody)
Framework	Ultralytics YOLOv8	Ultralytics YOLOv8	OpenMMLab (MMDet + MMPose)
Model Size	Configurable (n to x)	Configurable (n to x)	Fixed (rtmpose-m, rtmdet-m)
Output Formats	Image/Video	JSON, CSV, Video	Image/Video, JSON, CSV

4.2.6 Key Functions Summary

Project	Critical Function	Purpose
YOLO Test 1	`_split_image_to_tiles()`	Divides image into grid for independent processing
	`_process_tiled_frame()`	Processes each tile and remaps coordinates
	`predict_image()`	Full image pipeline with tiling
YOLO Test 2	`detect_players()`	Stage 1: Person detection with NMS
	`estimate_pose()`	Stage 2: Keypoint extraction per person
	`detect_action()`	Action recognition from joint angles
	`process_frame()`	Orchestrates two-stage pipeline
RTMPose	`_detect_persons()`	RTMDet person detection

	`_estimate_pose()`	RTMPose keypoint estimation
	`_track_objects()`	ByteTrack ID assignment
	`_visualize_results()`	Unified rendering with tracking IDs

4.2.7 Implementation Insights

1. Tiling vs. Two-Stage Trade-off: Tiling (Test 1) excels at distance but increases computation linearly with grid size. Two-stage (Test 2) offers better accuracy per person but requires careful bounding box padding to avoid cutting off limbs.
2. Registry Scope Management: The RTMPose implementation revealed that combining MMDetection and MMPose requires explicit registry scope resets (`register_mmdet_modules(init_default_scope=True)`) to prevent configuration conflicts.
3. Tracking Consistency: ByteTrack's two-stage association (high + low confidence) significantly improves track persistence during occlusion compared to simple IoU-based tracking.
4. Model Caching: All three implementations implement automatic model downloading and local caching, ensuring first-run usability without manual weight downloads.

5. Hardware Flexibility: Each project auto-detects CUDA, MPS (Apple Silicon), or CPU fallback, making the suite deployable across diverse hardware configurations.

Appendix A: Multi-Player Pose Estimation in Wide-Field Soccer Broadcasts Using Image Tiling and YOLOv8

This framework leverages a custom `YOLOv8PoseEstimator` class to implement automated model weight management and high-resolution player tracking for soccer analytics.

To preserve the fine-grained visual details of distant players, the code segments wide-field video frames into a discrete, uniform grid using a coordinate-offset tiling technique.

Localized keypoint coordinates extracted from each sub-tile are dynamically mapped back to the global coordinate field to maintain holistic tracking across the pitch.

The system utilizes the 17-point COCO human topology to filter and render skeletal skeletons on static images, video files, or real-time camera streams.

While structurally optimized for precision, the sequential tile-by-tile inference introduces a computational throughput trade-off and risks boundary truncation errors.

```
from ultralytics import YOLO
import cv2
import numpy as np
from pathlib import Path
from typing import Union, Optional, List, Tuple
import time
import os
from yolo_test1.config import Config


class YOLOv8PoseEstimator:
    """
    Professional class for pose estimation using YOLOv8
    """

    def __init__(self, model_name: str = 'yolov8n-pose.pt', conf_threshold: float
= 0.5):
        """
        Initialize the model
        """
        self.model_name = model_name
        self.conf_threshold = conf_threshold
        self.model = None
        self.model_path = None
        self._load_model()
```

```
def _load_model(self):
    """Load model from models folder"""
    try:
        print(f" Loading model: {self.model_name}")
        self.model_path = Config.get_model_path(self.model_name)

        if self.model_path.exists():
            print(f" Model found in: {self.model_path}")
            self.model = YOLO(str(self.model_path))
            print(f" Model loaded successfully from models folder!")
        else:
            print(f" Downloading model...")
            temp_model = YOLO(self.model_name)
            downloaded_path = self._find_downloaded_model()

            if downloaded_path and downloaded_path.exists():
                import shutil
                shutil.copy2(downloaded_path, self.model_path)
                print(f" Model copied to: {self.model_path}")
                self.model = YOLO(str(self.model_path))
                print(f" Model loaded successfully!")
            else:
                print(f"  Using model from default location")
                self.model = temp_model

    except Exception as e:
        print(f" Error loading model: {e}")
        raise

def _find_downloaded_model(self) -> Optional[Path]:
    """Find where ultralytics downloaded the model"""
    possible_locations = [
        Path.home() / '.ultralytics' / 'models' / self.model_name,
        Path(self.model_name),
        Path.cwd() / self.model_name,
    ]
    for location in possible_locations:
```

```
            if location.exists():
                return location
        return None

    def _draw_keypoints(self, frame: np.ndarray, keypoints: np.ndarray,
confidence: np.ndarray) -> np.ndarray:
        """Draw keypoints and skeleton lines"""
        colors = [
            (255, 0, 0), (255, 85, 0), (255, 170, 0), (255, 255, 0),
            (170, 255, 0), (85, 255, 0), (0, 255, 0), (0, 255, 85),
            (0, 255, 170), (0, 255, 255), (0, 170, 255), (0, 85, 255),
            (0, 0, 255), (85, 0, 255), (170, 0, 255), (255, 0, 255),
            (255, 0, 170)
        ]
        skeleton = [
            [16, 14], [14, 12], [17, 15], [15, 13], [12, 13],
            [6, 12], [7, 13], [6, 7], [6, 8], [7, 9],
            [8, 10], [9, 11], [2, 3], [1, 2], [1, 3],
            [2, 4], [3, 5], [4, 6], [5, 7]
        ]
        for connection in skeleton:
            start_idx = connection[0] - 1
            end_idx = connection[1] - 1
            if (start_idx < len(keypoints) and end_idx < len(keypoints)
                    and confidence[start_idx] > 0.5 and confidence[end_idx] > 0.5):
                start_point = tuple(map(int, keypoints[start_idx]))
                end_point = tuple(map(int, keypoints[end_idx]))
                cv2.line(frame, start_point, end_point, (0, 255, 0), 2)
        for idx, (keypoint, conf) in enumerate(zip(keypoints, confidence)):
            if conf > 0.5:
                x, y = int(keypoint[0]), int(keypoint[1])
                color = colors[idx % len(colors)]
                cv2.circle(frame, (x, y), 5, color, -1)
                cv2.circle(frame, (x, y), 5, (255, 255, 255), 2)
        return frame
```

```
    def _draw_pose_on_image(self, image: np.ndarray, keypoints) ->
np.ndarray:
        """
        Draw keypoints and skeleton on image
        Args:
            image: Image array
            keypoints: Tensor or numpy array of shape (17, 3) - [x, y, confidence]
        """
        # Convert to numpy if tensor
        if hasattr(keypoints, 'cpu'):
            kpts = keypoints.cpu().numpy()
        else:
            kpts = np.array(keypoints)

        # Skeleton connections (COCO format)
        skeleton = [
            [16, 14], [14, 12], [17, 15], [15, 13], [12, 13],
            [6, 12], [7, 13], [6, 7], [6, 8], [7, 9],
            [8, 10], [9, 11], [2, 3], [1, 2], [1, 3],
            [2, 4], [3, 5], [4, 6], [5, 7]
        ]

        # Draw skeleton connections
        for connection in skeleton:
            idx1 = connection[0] - 1
            idx2 = connection[1] - 1

            if idx1 < len(kpts) and idx2 < len(kpts):
                if kpts[idx1, 2] > 0.5 and kpts[idx2, 2] > 0.5:
                    pt1 = (int(kpts[idx1, 0]), int(kpts[idx1, 1]))
                    pt2 = (int(kpts[idx2, 0]), int(kpts[idx2, 1]))
                    cv2.line(image, pt1, pt2, (0, 255, 0), 2)

        # Draw keypoints
        for i in range(len(kpts)):
            if kpts[i, 2] > 0.5:
                center = (int(kpts[i, 0]), int(kpts[i, 1]))
```

```
            cv2.circle(image, center, 4, (0, 0, 255), -1)
            cv2.circle(image, center, 5, (255, 255, 255), 1)

        return image

    def _split_image_to_tiles(self, image, grid_size=(3, 3)):
        """
        (tiles)
        Args:
            image:(numpy array)
            grid_size: (row numbers, column numbers)
        Returns:
             tuples (crop, (x_offset, y_offset))
        """
        h, w, _ = image.shape
        rows, cols = grid_size
        tile_h, tile_w = h // rows, w // cols

        tiles = []
        for i in range(rows):
            for j in range(cols):
                x1, y1 = j * tile_w, i * tile_h
                x2, y2 = x1 + tile_w, y1 + tile_h
                crop = image[y1:y2, x1:x2]
                tiles.append((crop, (x1, y1)))
        return tiles

    def _process_tiled_frame(self, frame: np.ndarray, grid_size=None) ->
np.ndarray:
        """
        Process image or frame using tiling approach
        Returns only the processed frame (for video processing)
        """
        if grid_size is None:
            grid_size = Config.TILE_GRID

        tiles = self._split_image_to_tiles(frame, grid_size=grid_size)
```

```
    composed = frame.copy()

    for tile, (x_offset, y_offset) in tiles:
        results = self.model(tile, conf=self.conf_threshold, verbose=False)
        for result in results:
            if result.keypoints is not None:
                keypoints = result.keypoints.xy.cpu().numpy()
                confidences = result.keypoints.conf.cpu().numpy()
                for kpts, confs in zip(keypoints, confidences):

                    kpts[:, 0] += x_offset
                    kpts[:, 1] += y_offset
                    composed = self._draw_keypoints(composed, kpts, confs)
    return composed

def predict_image(self, image_path: Union[str, Path],
                  save_result: bool = True,
                  output_path: Optional[str] = None) -> Tuple[np.ndarray, int, int]:
    """
    Analyze a single image with tiling support for distant scenes

    Returns:
        Tuple of (result_frame, num_persons, total_visible_keypoints)
    """
    print(f" Processing image: {image_path}")
    frame = cv2.imread(str(image_path))
    if frame is None:
        raise ValueError(f"Cannot read image: {image_path}")

    # Get grid size from config
    grid_size = Config.TILE_GRID
    h, w = frame.shape[:2]
    rows, cols = grid_size
    tile_h = h // rows
    tile_w = w // cols

    result_img = frame.copy()
```

```
        total_persons = 0
        total_visible_keypoints = 0

        # Process each tile
        for i in range(rows):
            for j in range(cols):
                y1 = i * tile_h
                y2 = (i + 1) * tile_h if i < rows - 1 else h
                x1 = j * tile_w
                x2 = (j + 1) * tile_w if j < cols - 1 else w

                tile = frame[y1:y2, x1:x2]

                # Run pose estimation on tile
                results = self.model(tile, conf=self.conf_threshold, verbose=False)

                if results and len(results) > 0:
                    result = results[0]

                    if hasattr(result, 'keypoints') and result.keypoints is not None:
                        keypoints_data = result.keypoints.data

                        if keypoints_data is not None and len(keypoints_data.shape) >
1:
                            num_persons_in_tile = keypoints_data.shape[0]
                            total_persons += num_persons_in_tile

                            # Draw poses on result image
                            for person_idx in range(num_persons_in_tile):
                                person_kpts = keypoints_data[person_idx]

                                # Adjust keypoints to original image coordinates
                                adjusted_kpts = person_kpts.clone()
                                adjusted_kpts[:, 0] += x1  # x offset
                                adjusted_kpts[:, 1] += y1  # y offset

                                # Count visible keypoints
```

```
                    visible = (adjusted_kpts[:, 2] > 0.5).sum().item()
                    total_visible_keypoints += visible

                    # Draw keypoints and skeleton
                    result_img = self._draw_pose_on_image(result_img,
adjusted_kpts)

        # Save result
        if save_result:
            if output_path is None:
                input_filename = Path(image_path).name
                output_path = Config.OUTPUT_IMAGES_DIR /
f"output_{input_filename}"
            else:
                output_path = Path(output_path)
            output_path.parent.mkdir(parents=True, exist_ok=True)
            cv2.imwrite(str(output_path), result_img)
            print(f" Result saved to: {output_path}")

        print(f"    Total: {total_persons} persons, {total_visible_keypoints} visible
keypoints")

        return result_img, total_persons, total_visible_keypoints

    def predict_video(self, video_path: Union[str, Path, int],
                      save_result: bool = True,
                      output_path: Optional[str] = None,
                      show_live: bool = True):
        """
        Analyze a video using tiling grid from Config
        """
        # Open video or camera
        is_camera = False
        if video_path == 0 or str(video_path).lower() == 'camera':
            cap = cv2.VideoCapture(0)
            is_camera = True
            print(" Opening camera...")
```

```
else:
    cap = cv2.VideoCapture(str(video_path))
    print(f" Processing video: {video_path}")

if not cap.isOpened():
    raise ValueError("Cannot open video/camera")

width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = int(cap.get(cv2.CAP_PROP_FPS)) or 30

writer = None
if save_result:
    if output_path is None:
        if is_camera:
            timestamp = time.strftime("%Y%m%d_%H%M%S")
            output_filename = f"camera_output_{timestamp}.mp4"
        else:
            input_filename = Path(video_path).name
            output_filename = f"output_{input_filename}"
        output_path = Config.OUTPUT_VIDEOS_DIR / output_filename
    else:
        output_path = Path(output_path)
    output_path.parent.mkdir(parents=True, exist_ok=True)
    fourcc = cv2.VideoWriter_fourcc(*'mp4v')
    writer = cv2.VideoWriter(str(output_path), fourcc, fps, (width, height))
    print(f" Saving video to: {output_path}")

frame_count = 0
start_time = time.time()

try:
    while cap.isOpened():
        ret, frame = cap.read()
        if not ret:
            break
```

```
            frame_count += 1
            # use grid from Config
            processed_frame = self._process_tiled_frame(frame,
grid_size=Config.TILE_GRID)

            elapsed_time = time.time() - start_time
            current_fps = frame_count / elapsed_time if elapsed_time > 0 else
0
            cv2.putText(processed_frame, f'FPS: {current_fps:.2f}', (10, 30),
                        cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)
            cv2.putText(processed_frame, f'Frames: {frame_count}', (10, 70),
                        cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)

            if writer is not None:
                writer.write(processed_frame)

            if show_live:
                cv2.imshow(f'YOLOv8 Pose Estimation Grid {Config.TILE_GRID}
- Press Q to Exit', processed_frame)
                if cv2.waitKey(1) & 0xFF == ord('q'):
                    print("  Processing stopped by user")
                    break
        finally:
            cap.release()
            if writer is not None:
                writer.release()
                print(" Video saved successfully!")
            cv2.destroyAllWindows()
            elapsed = time.time() - start_time
            print(f" Processed {frame_count} frames in {elapsed:.2f} seconds")
            print(f" Average FPS: {frame_count / elapsed:.2f}")


def cleanup_default_models():
    """Optional: Clean up models from default ultralytics folder"""
    default_models_path = Path.home() / '.ultralytics' / 'models'
    if default_models_path.exists():
```

```
        print(f"\n Cleaning up default models folder...")
        for model_file in default_models_path.glob('*.pt'):
            try:
                model_file.unlink()
                print(f"   Deleted: {model_file.name}")
            except Exception as e:
                print(f"   Could not delete {model_file.name}: {e}")
        print(" Cleanup complete!")


# Quick test entry point
if __name__ == "__main__":
    Config.print_paths()
    estimator = YOLOv8PoseEstimator()
    print(" Ready for testing!")
```

This configuration class establishes a deterministic directory hierarchy (models, input, output) relative to the script's root using Python's robust pathlib library.
It defines critical hyperparameters and runtime flags for the project, such as a default 3 × 3 slicing matrix (TILE_GRID) and a 50% model confidence threshold.
The module acts as an automated setup utility by invoking create_directories() immediately upon import to generate any missing folders across the project tree.
It abstracts path resolution via class-level methods like get_model_path(), ensuring the main inference engine can consistently locate .pt weight files.
A built-in diagnostic tool (print_paths) provides clear, structured console logging of the entire runtime environment for seamless debugging.

```
from pathlib import Path
import os


class Config:
```

```
"""Project configuration"""

# Base directory - automatically detect project root
BASE_DIR = Path(__file__).parent.absolute()

# Model settings
MODEL_NAME = 'yolov8n-pose.pt'  # n, s, m, l, x
CONFIDENCE_THRESHOLD = 0.5

# Main directories
MODELS_DIR = BASE_DIR / 'models'
INPUT_DIR = BASE_DIR / 'input'
OUTPUT_DIR = BASE_DIR / 'output'

# Input subdirectories
INPUT_IMAGES_DIR = INPUT_DIR / 'images'
INPUT_VIDEOS_DIR = INPUT_DIR / 'videos'

# Output subdirectories
OUTPUT_IMAGES_DIR = OUTPUT_DIR / 'images'
OUTPUT_VIDEOS_DIR = OUTPUT_DIR / 'videos'

# Display settings
SHOW_LIVE = True
SAVE_RESULTS = True

# Default tiling grid for far-view analysis (you can adjust it)
TILE_GRID = (3, 3)

@classmethod
def get_model_path(cls, model_name: str) -> Path:
    """
    Get full path for model file

    Args:
        model_name: Name of the model
```

```
    Returns:
        Full path to model file
    """
    return cls.MODELS_DIR / model_name

@classmethod
def create_directories(cls):
    """Create all required directories"""
    directories = [
        cls.MODELS_DIR,
        cls.INPUT_IMAGES_DIR,
        cls.INPUT_VIDEOS_DIR,
        cls.OUTPUT_IMAGES_DIR,
        cls.OUTPUT_VIDEOS_DIR
    ]

    for directory in directories:
        directory.mkdir(parents=True, exist_ok=True)

    print(" All directories created successfully")
    print(f" Models directory: {cls.MODELS_DIR}")
    print(f" Output images: {cls.OUTPUT_IMAGES_DIR}")
    print(f" Output videos: {cls.OUTPUT_VIDEOS_DIR}")

@classmethod
def print_paths(cls):
    """Print all important paths"""
    print("\n" + "="*60)
    print(" Project Paths:")
    print("="*60)
    print(f"Base Directory: {cls.BASE_DIR}")
    print(f"Models: {cls.MODELS_DIR}")
    print(f"Input Images: {cls.INPUT_IMAGES_DIR}")
    print(f"Input Videos: {cls.INPUT_VIDEOS_DIR}")
    print(f"Output Images: {cls.OUTPUT_IMAGES_DIR}")
    print(f"Output Videos: {cls.OUTPUT_VIDEOS_DIR}")
    print("="*60 + "\n")
```

```
# Create directories when importing the file
Config.create_directories()
```

```
import sys
from pathlib import Path
import argparse

# Import classes
from pose_estimator import YOLOv8PoseEstimator
from config import Config
from app import PoseEstimatorApp


def quick_test():
    """Quick system test"""
    print("\n Running quick test...")

    try:
        # Print paths
        Config.print_paths()

        # Create Estimator object
        estimator = YOLOv8PoseEstimator()
        print(" System is ready!")
        return True
    except Exception as e:
        print(f" Test failed: {e}")
        return False


def run_cli():
    """Run from command line"""
    parser = argparse.ArgumentParser(
```

```
        description='YOLOv8 Pose Estimation Tool',
        formatter_class=argparse.RawDescriptionHelpFormatter,
        epilog="""
Usage examples:
  python main.py --image path/to/image.jpg
  python main.py --video path/to/video.mp4
  python main.py --camera
  python main.py --model yolov8m-pose.pt --conf 0.7 --image test.jpg
        """
    )

    # Arguments
    parser.add_argument('--image', '-i', type=str, help='Image path')
    parser.add_argument('--video', '-v', type=str, help='Video path')
    parser.add_argument('--camera', '-c', action='store_true', help='Use
camera')
    parser.add_argument('--model', '-m', type=str, default='yolov8n-pose.pt',
                        help='Model type (n/s/m/l/x)')
    parser.add_argument('--conf', type=float, default=0.5,
                        help='Confidence threshold (0.0-1.0)')
    parser.add_argument('--output', '-o', type=str, help='Output path (optional)')
    parser.add_argument('--no-save', action='store_true', help='Do not save
results')
    parser.add_argument('--no-show', action='store_true', help='Do not show
results')
    parser.add_argument('--interactive', action='store_true', help='Interactive
mode')
    parser.add_argument('--show-paths', action='store_true', help='Show all
paths and exit')

    args = parser.parse_args()

    # Show paths and exit
    if args.show_paths:
        Config.print_paths()
        return
```

```
# Create Estimator
estimator = YOLOv8PoseEstimator(
    model_name=args.model,
    conf_threshold=args.conf
)

# Process image
if args.image:
    if not Path(args.image).exists():
        print(f" Image not found: {args.image}")
        sys.exit(1)

    estimator.predict_image(
        image_path=args.image,
        save_result=not args.no_save,
        output_path=args.output
    )

# Process video
elif args.video:
    if not Path(args.video).exists():
        print(f" Video not found: {args.video}")
        sys.exit(1)

    estimator.predict_video(
        video_path=args.video,
        save_result=not args.no_save,
        output_path=args.output,
        show_live=not args.no_show
    )

# Use camera
elif args.camera:
    estimator.predict_video(
        video_path=0,
        save_result=not args.no_save,
        output_path=args.output,
```

```
            show_live=True
        )

    # Interactive mode
    elif args.interactive:
        app = PoseEstimatorApp()
        app.run_interactive()

    else:
        parser.print_help()


def print_banner():
    """Print application banner"""
    banner = """

╔═══════════════════════════════════════════════════════════════════════
═══════════════╗
    ║                                                    ║
    ║            YOLOv8 Pose Estimation Tool             ║
    ║                                                    ║
    ║          Professional Pose Estimation Tool         ║
    ║                                                    ║

╚═══════════════════════════════════════════════════════════════════════
═══════════════╝

    """
    print(banner)
    Config.print_paths()


def show_menu():
    """Display main menu"""
    print("\n" + "="*60)
    print(" Choose operation mode:")
    print("="*60)
    print("1. Interactive Mode")
```

```
    print("2. Quick Image Processing")
    print("3. Quick Video Processing")
    print("4. Run Camera")
    print("5. Test System")
    print("6. Show Help")
    print("7. Show Paths")
    print("0. Exit")
    print("="*60)


def main():
    """Main function"""
    print_banner()

    # CLI mode
    if len(sys.argv) > 1:
        run_cli()
        return

    # Interactive menu
    while True:
        show_menu()
        choice = input("\n Your choice: ").strip()

        if choice == '1':
            app = PoseEstimatorApp()
            app.run_interactive()

        elif choice == '2':
            path = input(" Enter image path: ").strip()
            if Path(path).exists():
                estimator = YOLOv8PoseEstimator()
                estimator.predict_image(path)
            else:
                print(" File not found!")

        elif choice == '3':
```

```
            path = input(" Enter video path: ").strip()
            if Path(path).exists():
                estimator = YOLOv8PoseEstimator()
                estimator.predict_video(path)
            else:
                print(" File not found!")

        elif choice == '4':
            print(" Opening camera... (Press Q to exit)")
            estimator = YOLOv8PoseEstimator()
            estimator.predict_video(0)

        elif choice == '5':
            quick_test()

        elif choice == '6':
            print_help()

        elif choice == '7':
            Config.print_paths()

        elif choice == '0':
            print("\n Thank you for using the application!")
            break

        else:
            print(" Invalid choice!")

        input("\n Press Enter to continue...")


def print_help():
    """Print help documentation"""
    help_text = """
```



Command Line Usage:

```
# Process image
python main.py --image path/to/image.jpg

# Process video
python main.py --video path/to/video.mp4

# Use camera
python main.py --camera

# Specify model
python main.py --model yolov8m-pose.pt --image test.jpg

# Set confidence threshold
python main.py --conf 0.7 --video test.mp4
# Custom output path
python main.py --image test.jpg --output custom/path/result.jpg

# Do not save results
python main.py --video test.mp4 --no-save

# Do not show display
python main.py --video test.mp4 --no-show

# Show project paths
python main.py --show-paths

# Run in interactive mode
```

python main.py --interactive

Available Models:

---

yolov8n-pose.pt → Very Fast (Nano)
yolov8s-pose.pt → Fast (Small)
yolov8m-pose.pt → Medium
yolov8l-pose.pt → Large
yolov8x-pose.pt → Extra Large (Best Accuracy)

Models are automatically downloaded to: models/

Output Locations:

Images: output/images/output_[filename]
Videos: output/videos/output_[filename]
Camera: output/videos/camera_output_[timestamp].mp4

Keyboard Controls:

Q → Exit video processing

Folder Structure:

```
yolo/
├── models/            ← Models auto-download here
├── input/
│   ├── images/        ← Place input images here
│   └── videos/        ← Place input videos here
└── output/
    ├── images/        ← Processed images saved here
    └── videos/        ← Processed videos saved here
```

Example Commands:

```
# Run without parameters (interactive menu)
python main.py

# Process all images in a folder
python batch_process.py --input-dir input/images/

# Process distant match footage (soccer)
python main.py --video matches/game1.mp4 --model yolov8m-pose.pt
--conf 0.6

# Quick camera capture
python main.py --camera
```

Get Help

```
    python main.py --help

╚═══════════════════════════════════════════════════════════════════════════
═══════════════════╝
    """
    print(help_text)


if __name__ == "__main__":
    try:
        main()
    except KeyboardInterrupt:
        print("\n\n  Program stopped by user")
        sys.exit(0)
    except Exception as e:
        print(f"\n Unexpected error occurred: {e}")
        import traceback
        traceback.print_exc()
        sys.exit(1)
```

```
# YOLOv8 Pose Estimation Requirements

# pip install -r requirements.txt


ultralytics>=8.0.0


opencv-python>=4.8.0
opencv-contrib-python>=4.8.0
```

```
numpy>=1.23.0
pillow>=9.0.0

# PyTorch
torch>=2.0.0
torchvision>=0.15.0



matplotlib>=3.5.0
pandas>=1.5.0
seaborn>=0.12.0

tqdm>=4.64.0

PyYAML>=6.0

pathlib2>=2.3.7

jupyter>=1.0.0
ipython>=8.0.0

psutil>=5.9.0
```

```
import cv2
import numpy as np
from pathlib import Path
from typing import List, Union
import json


class VideoInfo:
    """Video information class"""

    @staticmethod
    def get_video_info(video_path: Union[str, Path]) -> dict:
```

```
        """
        Get video information

        Args:
            video_path: Path to video file

        Returns:
            Dictionary containing video information
        """
        cap = cv2.VideoCapture(str(video_path))

        if not cap.isOpened():
            return None

        info = {
            'width': int(cap.get(cv2.CAP_PROP_FRAME_WIDTH)),
            'height': int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT)),
            'fps': int(cap.get(cv2.CAP_PROP_FPS)),
            'frame_count': int(cap.get(cv2.CAP_PROP_FRAME_COUNT)),
            'duration': int(cap.get(cv2.CAP_PROP_FRAME_COUNT)) /
int(cap.get(cv2.CAP_PROP_FPS))
        }

        cap.release()
        return info

    @staticmethod
    def print_video_info(video_path: Union[str, Path]):
        """
        Print video information

        Args:
            video_path: Path to video file
        """
        info = VideoInfo.get_video_info(video_path)

        if info:
```

```
        print("\n Video Information:")
        print("="*40)
        print(f" Dimensions: {info['width']}x{info['height']}")
        print(f" FPS: {info['fps']}")
        print(f" Frame Count: {info['frame_count']}")
        print(f"  Duration: {info['duration']:.2f} seconds")
        print("="*40)


class ImageInfo:
    """Image information class"""

    @staticmethod
    def get_image_info(image_path: Union[str, Path]) -> dict:
        """
        Get image information

        Args:
            image_path: Path to image file

        Returns:
            Dictionary containing image information
        """
        img = cv2.imread(str(image_path))

        if img is None:
            return None

        info = {
            'width': img.shape[1],
            'height': img.shape[0],
            'channels': img.shape[2] if len(img.shape) > 2 else 1,
            'size': Path(image_path).stat().st_size / 1024  # KB
        }

        return info
```

```
    @staticmethod
    def print_image_info(image_path: Union[str, Path]):
        """
        Print image information

        Args:
            image_path: Path to image file
        """
        info = ImageInfo.get_image_info(image_path)

        if info:
            print("\n Image Information:")
            print("="*40)
            print(f" Dimensions: {info['width']}x{info['height']}")
            print(f" Channels: {info['channels']}")
            print(f" Size: {info['size']:.2f} KB")
            print("="*40)


class FileManager:
    """File management class"""

    @staticmethod
    def get_all_images(directory: Union[str, Path]) -> List[Path]:
        """
        Get all images in directory

        Args:
            directory: Directory path to search

        Returns:
            List of image file paths
        """
        directory = Path(directory)
        extensions = ['.jpg', '.jpeg', '.png', '.bmp', '.webp']

        images = []
```

```
    for ext in extensions:
        images.extend(directory.glob(f'*{ext}'))
        images.extend(directory.glob(f'*{ext.upper()}'))

    return sorted(images)

@staticmethod
def get_all_videos(directory: Union[str, Path]) -> List[Path]:
    """
    Get all videos in directory

    Args:
        directory: Directory path to search

    Returns:
        List of video file paths
    """
    directory = Path(directory)
    extensions = ['.mp4', '.avi', '.mov', '.mkv', '.flv']

    videos = []
    for ext in extensions:
        videos.extend(directory.glob(f'*{ext}'))
        videos.extend(directory.glob(f'*{ext.upper()}'))

    return sorted(videos)

@staticmethod
def create_output_path(input_path: Union[str, Path],
                       output_dir: Union[str, Path],
                       prefix: str = 'output_') -> Path:
    """
    Create output path for processed file

    Args:
        input_path: Input file path
        output_dir: Output directory
```

```
        prefix: Prefix for output filename

    Returns:
        Output file path
    """
    input_path = Path(input_path)
    output_dir = Path(output_dir)
    output_dir.mkdir(parents=True, exist_ok=True)

    return output_dir / f"{prefix}{input_path.name}"


class ResultSaver:
    """Results saving class"""

    @staticmethod
    def save_results_json(results: dict, output_path: Union[str, Path]):
        """
        Save results to JSON file

        Args:
            results: Results dictionary to save
            output_path: Output file path
        """
        with open(output_path, 'w', encoding='utf-8') as f:
            json.dump(results, f, ensure_ascii=False, indent=2)

        print(f" JSON results saved to: {output_path}")

    @staticmethod
    def save_keypoints_csv(keypoints: np.ndarray, output_path: Union[str,
Path]):
        """
        Save keypoints to CSV file

        Args:
            keypoints: Keypoints array
```

```
        output_path: Output CSV file path
    """
    import pandas as pd

    # Create DataFrame
    data = []
    for frame_idx, frame_kpts in enumerate(keypoints):
        for person_idx, person_kpts in enumerate(frame_kpts):
            for kpt_idx, (x, y, conf) in enumerate(person_kpts):
                data.append({
                    'frame': frame_idx,
                    'person': person_idx,
                    'keypoint': kpt_idx,
                    'x': x,
                    'y': y,
                    'confidence': conf
                })

    df = pd.DataFrame(data)
    df.to_csv(output_path, index=False)

    print(f" Keypoints CSV saved to: {output_path}")


class PerformanceMonitor:
    """Performance monitoring class"""

    def __init__(self):
        """Initialize performance monitor"""
        self.start_time = None
        self.frame_times = []

    def start(self):
        """Start timing"""
        import time
        self.start_time = time.time()
        self.frame_times = []
```

```
def add_frame(self):
    """Add frame timing"""
    import time
    if self.start_time:
        self.frame_times.append(time.time())

def get_stats(self) -> dict:
    """
    Get performance statistics

    Returns:
        Dictionary containing performance metrics
    """
    import time
    if not self.start_time or not self.frame_times:
        return None

    total_time = time.time() - self.start_time
    frame_count = len(self.frame_times)
    avg_fps = frame_count / total_time if total_time > 0 else 0

    return {
        'total_time': total_time,
        'frame_count': frame_count,
        'avg_fps': avg_fps
    }

def print_stats(self):
    """Print performance statistics"""
    stats = self.get_stats()
    if stats:
        print("\n Performance Statistics:")
        print("="*40)
        print(f"  Total Time: {stats['total_time']:.2f} seconds")
        print(f" Frame Count: {stats['frame_count']}")
        print(f" Average FPS: {stats['avg_fps']:.2f}")
```

```
        print("="*40)


def test_utils():
    """Test utility functions"""
    print(" Testing utility functions...")

    # Test FileManager
    print("\n Found files:")
    images = FileManager.get_all_images('input/images')
    print(f"Number of images: {len(images)}")

    videos = FileManager.get_all_videos('input/videos')
    print(f"Number of videos: {len(videos)}")

    print(" Test completed successfully!")


if __name__ == "__main__":
    test_utils()
```

Appendix B: Human Pose Estimation Using Yolov8-pose in two Phases when Detector run first and Pose run second

```
"""
Object Detector class for detecting players in football matches
Uses YOLO model to detect persons (players) in images and videos
"""

import cv2
import numpy as np
import logging
```

```
from typing import List, Tuple, Dict, Optional
import sys

# Import configuration
try:
    from config import (
        DETECTION_CONFIG,
        FILTER_CONFIG,
        VISUALIZATION_CONFIG,
        LOGGING_CONFIG
    )
except ImportError:
    print(" Error: config.py file not found")
    sys.exit(1)

# Setup logging
logger = logging.getLogger(__name__)


class PlayerDetector:
    """
    Class for detecting players (persons) in football match images/videos
    """

    def __init__(self, model):
        """
        Initialize Player Detector

        Args:
            model: DetectionModel instance from models.py
        """
        self.model = model
        self.config = DETECTION_CONFIG
        self.filter_config = FILTER_CONFIG
        self.viz_config = VISUALIZATION_CONFIG

        logger.info(" Player Detector initialized")
```

```
def detect_players(self, frame: np.ndarray) -> List[Dict]:
    """
    Detect all players in a frame

    Args:
        frame: Input image/frame (numpy array)

    Returns:
        List of dictionaries containing detection info:
        [
            {
                'bbox': [x1, y1, x2, y2],
                'confidence': float,
                'class_id': int,
                'class_name': str,
                'center': (cx, cy),
                'area': float
            },
            ...
        ]
    """
    results = self.model.predict(frame)

    if results is None or results.boxes is None:
        return []

    detections = []
    boxes = results.boxes

    for i in range(len(boxes)):
        box = boxes.xyxy[i].cpu().numpy()
        x1, y1, x2, y2 = map(int, box)

        confidence = float(boxes.conf[i].cpu().numpy())

        class_id = int(boxes.cls[i].cpu().numpy())
```

```
            class_name = results.names[class_id]

            cx = int((x1 + x2) / 2)
            cy = int((y1 + y2) / 2)

            area = (x2 - x1) * (y2 - y1)

            detection = {
                'bbox': [x1, y1, x2, y2],
                'confidence': confidence,
                'class_id': class_id,
                'class_name': class_name,
                'center': (cx, cy),
                'area': area
            }

            detections.append(detection)

        filtered_detections = self._filter_detections(detections)

        logger.debug(f"Detected {len(filtered_detections)} players in frame")

        return filtered_detections

    def _filter_detections(self, detections: List[Dict]) -> List[Dict]:
        """
        Filter detections based on area and other criteria

        Args:
            detections: List of detection dictionaries

        Returns:
            Filtered list of detections
        """
        filtered = []

        min_area = self.filter_config['min_bbox_area']
```

```
    max_area = self.filter_config['max_bbox_area']

    for det in detections:
        area = det['area']

        if min_area <= area <= max_area:
            filtered.append(det)
        else:
            logger.debug(f"Filtered detection with area {area} (out of range)")

    return filtered

def crop_player(self, frame: np.ndarray, bbox: List[int],
                padding: int = 10) -> Optional[np.ndarray]:
    """
    Crop a player region from frame with padding

    Args:
        frame: Input frame
        bbox: Bounding box [x1, y1, x2, y2]
        padding: Padding around bbox in pixels

    Returns:
        Cropped player image or None if invalid
    """
    x1, y1, x2, y2 = bbox
    h, w = frame.shape[:2]

    x1 = max(0, x1 - padding)
    y1 = max(0, y1 - padding)
    x2 = min(w, x2 + padding)
    y2 = min(h, y2 + padding)

    if x2 <= x1 or y2 <= y1:
        logger.warning("Invalid cropping coordinates")
        return None
```

```
    cropped = frame[y1:y2, x1:x2]

    return cropped

def draw_detections(self, frame: np.ndarray,
                    detections: List[Dict],
                    draw_labels: bool = True) -> np.ndarray:
    """
    Draw bounding boxes and labels on frame

    Args:
        frame: Input frame
        detections: List of detection dictionaries
        draw_labels: Whether to draw labels

    Returns:
        Frame with drawn detections
    """
    output = frame.copy()

    bbox_color = self.viz_config['bbox_color']
    bbox_thickness = self.viz_config['bbox_thickness']
    text_color = self.viz_config['text_color']
    text_scale = self.viz_config['text_scale']
    text_thickness = self.viz_config['text_thickness']
    text_font = self.viz_config['text_font']

    for i, det in enumerate(detections):
        x1, y1, x2, y2 = det['bbox']
        confidence = det['confidence']

        cv2.rectangle(output, (x1, y1), (x2, y2),
                      bbox_color, bbox_thickness)

        if draw_labels and self.viz_config['draw_labels']:
            label = f"Player {i+1}: {confidence:.2f}"
```

```
            (text_w, text_h), baseline = cv2.getTextSize(
                label, text_font, text_scale, text_thickness
            )

            cv2.rectangle(output,
                          (x1, y1 - text_h - baseline - 5),
                          (x1 + text_w, y1),
                          bbox_color, -1)

            cv2.putText(output, label,
                        (x1, y1 - baseline - 5),
                        text_font, text_scale, text_color,
                        text_thickness, cv2.LINE_AA)

        cx, cy = det['center']
        cv2.circle(output, (cx, cy), 5, (0, 255, 255), -1)

    return output

def get_detection_stats(self, detections: List[Dict]) -> Dict:
    """
    Get statistics about detections

    Args:
        detections: List of detection dictionaries

    Returns:
        Dictionary with statistics
    """
    if not detections:
        return {
            'count': 0,
            'avg_confidence': 0.0,
            'min_confidence': 0.0,
            'max_confidence': 0.0,
            'avg_area': 0.0,
        }
```

```
    confidences = [d['confidence'] for d in detections]
    areas = [d['area'] for d in detections]

    stats = {
        'count': len(detections),
        'avg_confidence': np.mean(confidences),
        'min_confidence': np.min(confidences),
        'max_confidence': np.max(confidences),
        'avg_area': np.mean(areas),
        'min_area': np.min(areas),
        'max_area': np.max(areas),
    }

    return stats

def detect_and_crop_players(self, frame: np.ndarray,
                            padding: int = 10) -> Tuple[List[Dict], List[np.ndarray]]:
    """
    Detect players and crop their regions

    Args:
        frame: Input frame
        padding: Padding for cropping

    Returns:
        Tuple of (detections, cropped_images)
    """
    detections = self.detect_players(frame)

    cropped_images = []
    for det in detections:
        cropped = self.crop_player(frame, det['bbox'], padding)
        if cropped is not None:
            cropped_images.append(cropped)
        else:
            cropped_images.append(None)
```

```
    return detections, cropped_images

def non_max_suppression(self, detections: List[Dict],
                        iou_threshold: float = 0.5) -> List[Dict]:
    """
    Apply Non-Maximum Suppression to remove overlapping detections

    Args:
        detections: List of detection dictionaries
        iou_threshold: IoU threshold for suppression

    Returns:
        Filtered list of detections
    """
    if not detections:
        return []

    boxes = np.array([d['bbox'] for d in detections])
    scores = np.array([d['confidence'] for d in detections])

    x1 = boxes[:, 0]
    y1 = boxes[:, 1]
    x2 = boxes[:, 2]
    y2 = boxes[:, 3]
    areas = (x2 - x1) * (y2 - y1)

    order = scores.argsort()[::-1]

    keep = []
    while order.size > 0:
        i = order[0]
        keep.append(i)

        xx1 = np.maximum(x1[i], x1[order[1:]])
        yy1 = np.maximum(y1[i], y1[order[1:]])
        xx2 = np.minimum(x2[i], x2[order[1:]])
```

```
            yy2 = np.minimum(y2[i], y2[order[1:]])

            w = np.maximum(0.0, xx2 - xx1)
            h = np.maximum(0.0, yy2 - yy1)
            intersection = w * h

            iou = intersection / (areas[i] + areas[order[1:]] - intersection)

            inds = np.where(iou <= iou_threshold)[0]
            order = order[inds + 1]

        return [detections[i] for i in keep]


class MultiPlayerDetector:
    """
    Extended detector with tracking capabilities for multiple players
    """

    def __init__(self, model):
        """
        Initialize Multi-Player Detector

        Args:
            model: DetectionModel instance
        """
        self.detector = PlayerDetector(model)
        self.tracked_players = {}
        self.next_player_id = 1

        logger.info(" Multi-Player Detector with tracking initialized")

    def detect_players(self, frame):
        """Detect players using underlying detector"""
        return self.detector.detect_players(frame)

    def crop_player(self, frame, bbox, padding=10):
```

```
    """Crop player region"""
    return self.detector.crop_player(frame, bbox, padding)

def update_tracking(self, detections: List[Dict],
                    max_distance: float = 50) -> List[Dict]:
    """
    Update player tracking based on new detections

    Args:
        detections: List of detection dictionaries
        max_distance: Maximum distance for matching

    Returns:
        List of detections with player IDs
    """
    updated_detections = []

    for det in detections:
        center = det['center']

        min_dist = float('inf')
        matched_id = None

        for player_id, tracked_center in self.tracked_players.items():
            dist = np.sqrt((center[0] - tracked_center[0])**2 +
                           (center[1] - tracked_center[1])**2)

            if dist < min_dist and dist < max_distance:
                min_dist = dist
                matched_id = player_id

        if matched_id is not None:
            det['player_id'] = matched_id
            self.tracked_players[matched_id] = center
        else:
            det['player_id'] = self.next_player_id
            self.tracked_players[self.next_player_id] = center
```

```
                self.next_player_id += 1

            updated_detections.append(det)

        return updated_detections

    def reset_tracking(self):
        """Reset tracking state"""
        self.tracked_players = {}
        self.next_player_id = 1
        logger.info("Tracking reset")


# Test function
if __name__ == "__main__":
    print(" Testing Player Detector...\n")

    try:
        from models import initialize_models

        detector_model, _, _ = initialize_models()

        player_detector = PlayerDetector(detector_model)

        print("\n Player Detector test successful!")

    except Exception as e:
        print(f"\n Test failed: {str(e)}")
        sys.exit(1)
```

"""
Pose Estimator class for estimating player poses using keypoints
Processes cropped player images and extracts pose keypoints
"""

```
import cv2
import numpy as np
import logging
from typing import List, Dict, Optional, Tuple
import sys
from yolo_test2.config import Config

# Import configuration
try:
    from yolo_test2.config import (
        POSE_CONFIG,
        YOLO_KEYPOINTS,
        SKELETON_CONNECTIONS,
        VISUALIZATION_CONFIG,
        FILTER_CONFIG,
        LOGGING_CONFIG
    )
except ImportError:
    print(" Error: config.py not found")
    sys.exit(1)

# Setup logging
logger = logging.getLogger(__name__)


class PoseEstimator:
    """
    Class for estimating player poses and extracting keypoints
    """

    def __init__(self, model):
        """
        Initialize Pose Estimator

        Args:
            model: PoseModel instance from models.py
        """
```

```
    self.model = model
    self.config = POSE_CONFIG
    self.keypoints_map = YOLO_KEYPOINTS
    self.skeleton_connections = SKELETON_CONNECTIONS
    self.viz_config = VISUALIZATION_CONFIG
    self.filter_config = FILTER_CONFIG

    logger.info(" Pose Estimator initialized")

def estimate_pose(self, image: np.ndarray) -> Optional[Dict]:
    """
    Estimate pose for a single player image

    Args:
        image: Input player image (cropped)

    Returns:
        Dictionary containing pose information:
        {
            'keypoints': [[x, y, confidence], ...],
            'bbox': [x1, y1, x2, y2],
            'confidence': float,
            'visible_keypoints': int,
            'keypoint_names': dict
        }
    """
    results = self.model.predict(image)

    if results is None or results.keypoints is None:
        logger.debug("No pose detected in image")
        return None

    keypoints_data = results.keypoints

    if len(keypoints_data) == 0:
        return None
```

```
        kpts = keypoints_data[0].data.cpu().numpy()[0]

        visible_count = np.sum(kpts[:, 2] > self.config['confidence_threshold'])

        if visible_count < self.filter_config['min_keypoints_visible']:
            logger.debug(f"Visible keypoints ({visible_count}) below minimum")
            return None

        bbox = None
        if results.boxes is not None and len(results.boxes) > 0:
            box = results.boxes[0].xyxy[0].cpu().numpy()
            bbox = [int(x) for x in box]

        confidence = float(results.boxes[0].conf[0].cpu().numpy()) if
results.boxes is not None else 0.0

        pose_data = {
            'keypoints': kpts.tolist(),
            'bbox': bbox,
            'confidence': confidence,
            'visible_keypoints': int(visible_count),
            'keypoint_names': self.keypoints_map,
            'total_keypoints': len(kpts)
        }

        return pose_data

    def estimate_poses_batch(self, images: List[np.ndarray]) ->
List[Optional[Dict]]:
        """
        Estimate poses for multiple player images
        """
        poses = []

        for i, img in enumerate(images):
            if img is None:
                poses.append(None)
```

```
            continue

        pose = self.estimate_pose(img)
        poses.append(pose)

        if pose:
            logger.debug(f"Pose {i+1}: {pose['visible_keypoints']} visible
keypoints")

    return poses

def draw_pose(self, image: np.ndarray, pose_data: Dict,
              draw_keypoints: bool = True,
              draw_skeleton: bool = True) -> np.ndarray:
    """
    Draw pose keypoints and skeleton on image
    """
    output = image.copy()

    if pose_data is None:
        return output

    keypoints = np.array(pose_data['keypoints'])
    conf_threshold = self.config['confidence_threshold']

    kpt_color = self.viz_config['keypoint_color']
    skel_color = self.viz_config['skeleton_color']
    kpt_radius = self.viz_config['keypoint_radius']
    skel_thickness = self.viz_config['skeleton_thickness']

    if draw_skeleton and self.viz_config['draw_skeleton']:
        for connection in self.skeleton_connections:
            idx1, idx2 = connection

            kpt1 = keypoints[idx1]
            kpt2 = keypoints[idx2]
```

```
            if kpt1[2] > conf_threshold and kpt2[2] > conf_threshold:
                pt1 = (int(kpt1[0]), int(kpt1[1]))
                pt2 = (int(kpt2[0]), int(kpt2[1]))

                cv2.line(output, pt1, pt2, skel_color, skel_thickness,
cv2.LINE_AA)

        if draw_keypoints and self.viz_config['draw_keypoints']:
            for i, kpt in enumerate(keypoints):
                x, y, conf = kpt

                if conf > conf_threshold:
                    center = (int(x), int(y))

                    cv2.circle(output, center, kpt_radius, kpt_color, -1, cv2.LINE_AA)
                    cv2.circle(output, center, kpt_radius + 1, (255, 255, 255), 1,
cv2.LINE_AA)

        return output

    def get_keypoint_by_name(self, pose_data: Dict, keypoint_name: str) ->
Optional[Tuple[float, float, float]]:
        """
        Get specific keypoint by name
        """
        if pose_data is None:
            return None

        keypoint_idx = None
        for idx, name in self.keypoints_map.items():
            if name == keypoint_name:
                keypoint_idx = idx
                break

        if keypoint_idx is None:
            logger.warning(f"Keypoint '{keypoint_name}' not found")
            return None
```

```
        keypoints = pose_data['keypoints']
        if keypoint_idx >= len(keypoints):
            return None

        kpt = keypoints[keypoint_idx]
        return tuple(kpt)

    def calculate_angle(self, p1, p2, p3):
        """
        Calculate angle between three points
        """
        p1 = np.array(p1[:2])
        p2 = np.array(p2[:2])
        p3 = np.array(p3[:2])

        v1 = p1 - p2
        v2 = p3 - p2

        cos_angle = np.dot(v1, v2) / (np.linalg.norm(v1) * np.linalg.norm(v2) +
1e-6)
        cos_angle = np.clip(cos_angle, -1.0, 1.0)
        angle = np.arccos(cos_angle)

        return np.degrees(angle)

    def get_joint_angles(self, pose_data: Dict) -> Dict[str, float]:
        """
        Calculate important joint angles
        """
        if pose_data is None:
            return {}

        angles = {}

        try:
            l_shoulder = self.get_keypoint_by_name(pose_data, 'left_shoulder')
```

```
            l_elbow = self.get_keypoint_by_name(pose_data, 'left_elbow')
            l_wrist = self.get_keypoint_by_name(pose_data, 'left_wrist')

            if all(kpt and kpt[2] > 0.5 for kpt in [l_shoulder, l_elbow, l_wrist]):
                angles['left_elbow'] = self.calculate_angle(l_shoulder, l_elbow,
l_wrist)

            r_shoulder = self.get_keypoint_by_name(pose_data, 'right_shoulder')
            r_elbow = self.get_keypoint_by_name(pose_data, 'right_elbow')
            r_wrist = self.get_keypoint_by_name(pose_data, 'right_wrist')

            if all(kpt and kpt[2] > 0.5 for kpt in [r_shoulder, r_elbow, r_wrist]):
                angles['right_elbow'] = self.calculate_angle(r_shoulder, r_elbow,
r_wrist)

            l_hip = self.get_keypoint_by_name(pose_data, 'left_hip')
            l_knee = self.get_keypoint_by_name(pose_data, 'left_knee')
            l_ankle = self.get_keypoint_by_name(pose_data, 'left_ankle')

            if all(kpt and kpt[2] > 0.5 for kpt in [l_hip, l_knee, l_ankle]):
                angles['left_knee'] = self.calculate_angle(l_hip, l_knee, l_ankle)

            r_hip = self.get_keypoint_by_name(pose_data, 'right_hip')
            r_knee = self.get_keypoint_by_name(pose_data, 'right_knee')
            r_ankle = self.get_keypoint_by_name(pose_data, 'right_ankle')

            if all(kpt and kpt[2] > 0.5 for kpt in [r_hip, r_knee, r_ankle]):
                angles['right_knee'] = self.calculate_angle(r_hip, r_knee, r_ankle)

        except Exception as e:
            logger.warning(f"Error calculating angles: {str(e)}")

        return angles

    def get_pose_stats(self, pose_data: Dict) -> Dict:
        """
        Get pose statistics
```

```
        """
        if pose_data is None:
            return {
                'total_keypoints': 0,
                'visible_keypoints': 0,
                'visibility_ratio': 0.0,
                'avg_confidence': 0.0,
                'joint_angles': {}
            }

        keypoints = np.array(pose_data['keypoints'])
        confidences = keypoints[:, 2]

        visible = np.sum(confidences > self.config['confidence_threshold'])

        stats = {
            'total_keypoints': pose_data['total_keypoints'],
            'visible_keypoints': int(visible),
            'visibility_ratio': visible / pose_data['total_keypoints'],
            'avg_confidence': float(np.mean(confidences)),
            'min_confidence': float(np.min(confidences)),
            'max_confidence': float(np.max(confidences)),
            'joint_angles': self.get_joint_angles(pose_data)
        }

        return stats

    def normalize_keypoints(self, pose_data: Dict, reference_size=(256, 256))
-> Dict:
        """
        Normalize keypoints to reference size
        """
        if pose_data is None or pose_data['bbox'] is None:
            return pose_data

        normalized_pose = pose_data.copy()
        keypoints = np.array(pose_data['keypoints'])
```

```
        bbox = pose_data['bbox']

        x1, y1, x2, y2 = bbox
        bbox_w = x2 - x1
        bbox_h = y2 - y1

        normalized_kpts = keypoints.copy()
        normalized_kpts[:, 0] = (keypoints[:, 0] - x1) / bbox_w * reference_size[0]
        normalized_kpts[:, 1] = (keypoints[:, 1] - y1) / bbox_h * reference_size[1]

        normalized_pose['keypoints'] = normalized_kpts.tolist()
        normalized_pose['normalized'] = True
        normalized_pose['reference_size'] = reference_size

        return normalized_pose


class PoseAnalyzer:
    """
    Extended pose analyzer with action recognition capabilities
    """

    def __init__(self, pose_estimator: PoseEstimator):
        """
        Initialize Pose Analyzer
        """
        self.estimator = pose_estimator
        logger.info(" Pose Analyzer initialized")

    def detect_action(self, pose_data: Dict) -> str:
        """
        Detect action based on pose
        """
        if pose_data is None:
            return "unknown"

        angles = self.estimator.get_joint_angles(pose_data)
```

```
    action = "standing"

    try:
        if 'left_knee' in angles and angles['left_knee'] > 160:
            action = "kicking_left"
        elif 'right_knee' in angles and angles['right_knee'] > 160:
            action = "kicking_right"

        elif 'left_knee' in angles and 'right_knee' in angles:
            if 90 < angles['left_knee'] < 140 and 90 < angles['right_knee'] < 140:
                action = "running"

        elif 'left_knee' in angles and 'right_knee' in angles:
            if angles['left_knee'] > 160 and angles['right_knee'] > 160:
                action = "jumping"

    except Exception as e:
        logger.debug(f"Error detecting action: {str(e)}")

    return action

def compare_poses(self, pose1: Dict, pose2: Dict) -> float:
    """
    Compare two poses and calculate similarity
    """
    if pose1 is None or pose2 is None:
        return 0.0

    kpts1 = np.array(pose1['keypoints'])
    kpts2 = np.array(pose2['keypoints'])

    distances = []
    conf_threshold = self.estimator.config['confidence_threshold']

    for i in range(len(kpts1)):
        if kpts1[i][2] > conf_threshold and kpts2[i][2] > conf_threshold:
```

```
            dist = np.linalg.norm(kpts1[i][:2] - kpts2[i][:2])
            distances.append(dist)

        if not distances:
            return 0.0

        avg_distance = np.mean(distances)
        similarity = 1.0 / (1.0 + avg_distance)

        return similarity


if __name__ == "__main__":
    print(" Testing Pose Estimator...\n")

    try:
        from models import initialize_models

        _, pose_model, _ = initialize_models()

        pose_estimator = PoseEstimator(pose_model)

        print("\n Pose Estimator test successful!")

    except Exception as e:
        print(f"\n Test failed: {str(e)}")
        sys.exit(1)
```

```
"""
Models loader and manager for Football Pose Estimation Project
Handles loading and initialization of YOLO models for detection and pose
estimation
"""

import torch
import logging
```

```
from pathlib import Path
from ultralytics import YOLO
import sys

# Import configuration
try:
    from config import (
        DETECTION_CONFIG,
        POSE_CONFIG,
        MODELS_DIR,
        SELECTED_MODEL,
        LOGGING_CONFIG
    )
except ImportError:
    print(" Error: config.py file not found")
    print("Make sure config.py exists in the same directory")
    sys.exit(1)

# Setup logging
logging.basicConfig(
    level=getattr(logging, LOGGING_CONFIG["log_level"]),
    format=LOGGING_CONFIG["log_format"],
    handlers=[
        logging.FileHandler(LOGGING_CONFIG["log_file"]),
        logging.StreamHandler()
    ]
)
logger = logging.getLogger(__name__)


class ModelManager:
    """
    Manages YOLO models for object detection and pose estimation
    """

    def __init__(self):
        """Initialize Model Manager"""
```

```
    self.detection_model = None
    self.pose_model = None
    self.device = self._get_device()

    logger.info(f" Device used: {self.device}")
    logger.info(f" Selected model: YOLOv8{SELECTED_MODEL}")

def _get_device(self):
    """
    Determine the best available device (CUDA, MPS, or CPU)

    Returns:
        str: Device name ('cuda', 'mps', or 'cpu')
    """
    device_config = DETECTION_CONFIG.get("device", "cuda").lower()

    # Check CUDA availability
    if device_config == "cuda" and torch.cuda.is_available():
        device = "cuda"
        gpu_name = torch.cuda.get_device_name(0)
        logger.info(f" GPU detected: {gpu_name}")

    # Check MPS (Apple Silicon) availability
    elif device_config == "mps" and torch.backends.mps.is_available():
        device = "mps"
        logger.info(" Apple Silicon (MPS) detected")

    # Fallback to CPU
    else:
        device = "cpu"
        if device_config != "cpu":
            logger.warning(f" {device_config.upper()} not available, using CPU")
        else:
            logger.info(" CPU will be used")

    return device
```

```
    def load_detection_model(self):
        """
        Load YOLO object detection model

        Returns:
            YOLO: Loaded detection model
        """
        try:
            model_path = DETECTION_CONFIG["model_path"]
            model_name = DETECTION_CONFIG["model_name"]

            logger.info(f" Loading Object Detection model: {model_name}")

            # Check if model exists locally
            if not model_path.exists():
                logger.info(f" Model not found locally, downloading from the
internet...")
                logger.info(f" Model will be saved to: {model_path}")
                self.detection_model = YOLO(model_name)

                import shutil
                from pathlib import Path
                cache_path = Path.home() / '.cache' / 'ultralytics' / model_name
                if cache_path.exists():
                    shutil.copy(cache_path, model_path)
            else:
                logger.info(f" Model found locally: {model_path}")
                self.detection_model = YOLO(str(model_path))

            self.detection_model.to(self.device)

            logger.info(f" Object Detection model loaded successfully!")
            logger.info(f" Number of classes: {len(self.detection_model.names)}")

            return self.detection_model

        except Exception as e:
```

```
            logger.error(f" Error loading Object Detection model: {str(e)}")
            raise

    def load_pose_model(self):
        """
        Load YOLO pose estimation model

        Returns:
            YOLO: Loaded pose model
        """
        try:
            model_path = POSE_CONFIG["model_path"]
            model_name = POSE_CONFIG["model_name"]

            logger.info(f" Loading Pose Estimation model: {model_name}")

            # Check if model exists locally
            if not model_path.exists():
                logger.info(f" Model not found locally, downloading from the
internet...")
                logger.info(f" Model will be saved to: {model_path}")
                self.pose_model = YOLO(model_name)

                import shutil
                from pathlib import Path
                cache_path = Path.home() / '.cache' / 'ultralytics' / model_name
                if cache_path.exists():
                    shutil.copy(cache_path, model_path)
            else:
                logger.info(f" Model found locally: {model_path}")
                self.pose_model = YOLO(str(model_path))

            self.pose_model.to(self.device)

            logger.info(f" Pose Estimation model loaded successfully!")
            logger.info(f" Number of Keypoints: 17 (COCO format)")
```

```
            return self.pose_model

        except Exception as e:
            logger.error(f" Error loading Pose Estimation model: {str(e)}")
            raise

    def load_all_models(self):
        """
        Load both detection and pose models

        Returns:
            tuple: (detection_model, pose_model)
        """
        logger.info("="*60)
        logger.info(" Starting model loading")
        logger.info("="*60)

        detection_model = self.load_detection_model()
        pose_model = self.load_pose_model()

        logger.info("="*60)
        logger.info(" All models loaded successfully!")
        logger.info("="*60)

        return detection_model, pose_model

    def get_model_info(self):
        """
        Get information about loaded models

        Returns:
            dict: Model information
        """
        info = {
            "detection_model": {
                "name": DETECTION_CONFIG["model_name"],
                "path": str(DETECTION_CONFIG["model_path"]),
```

```
            "loaded": self.detection_model is not None,
            "device": self.device,
            "confidence_threshold":
DETECTION_CONFIG["confidence_threshold"],
            "iou_threshold": DETECTION_CONFIG["iou_threshold"],
        },
        "pose_model": {
            "name": POSE_CONFIG["model_name"],
            "path": str(POSE_CONFIG["model_path"]),
            "loaded": self.pose_model is not None,
            "device": self.device,
            "confidence_threshold": POSE_CONFIG["confidence_threshold"],
            "num_keypoints": 17,
        },
        "selected_model_size": SELECTED_MODEL,
    }

    return info

  def print_model_info(self):
    """Print model information in a formatted way"""
    info = self.get_model_info()

    print("\n" + "="*60)
    print(" Model Information")
    print("="*60)

    print("\n Object Detection Model:")
    print(f"  - Name: {info['detection_model']['name']}")
    print(f"  - Path: {info['detection_model']['path']}")
    print(f"  - Loaded: {' Yes' if info['detection_model']['loaded'] else ' No'}")
    print(f"  - Device: {info['detection_model']['device']}")
    print(f"  - Confidence: {info['detection_model']['confidence_threshold']}")
    print(f"  - IoU: {info['detection_model']['iou_threshold']}")

    print("\n Pose Estimation Model:")
    print(f"  - Name: {info['pose_model']['name']}")
```

```
        print(f"  - Path: {info['pose_model']['path']}")
        print(f"  - Loaded: {' Yes' if info['pose_model']['loaded'] else ' No'}")
        print(f"  - Device: {info['pose_model']['device']}")
        print(f"  - Confidence: {info['pose_model']['confidence_threshold']}")
        print(f"  - Number of Keypoints: {info['pose_model']['num_keypoints']}")

        print(f"\n Selected Model Size:
YOLOv8{info['selected_model_size'].upper()}")
        print("="*60 + "\n")


class DetectionModel:
    """Wrapper class for object detection"""

    def __init__(self, model, config):
        """
        Initialize detection model wrapper

        Args:
            model: YOLO model instance
            config: Detection configuration dict
        """
        self.model = model
        self.config = config

    def predict(self, image, conf=None, iou=None):
        """
        Run object detection on image

        Args:
            image: Input image (numpy array or path)
            conf: Confidence threshold (optional)
            iou: IoU threshold (optional)

        Returns:
            Results object from YOLO
        """
```

```
        conf = conf or self.config["confidence_threshold"]
        iou = iou or self.config["iou_threshold"]
        imgsz = self.config["imgsz"]

        results = self.model.predict(
            source=image,
            conf=conf,
            iou=iou,
            imgsz=imgsz,
            classes=[self.config["target_class"]],
            verbose=False
        )

        return results[0] if results else None


class PoseModel:
    """Wrapper class for pose estimation"""

    def __init__(self, model, config):
        """
        Initialize pose model wrapper

        Args:
            model: YOLO pose model instance
            config: Pose configuration dict
        """
        self.model = model
        self.config = config

    def predict(self, image, conf=None):
        """
        Run pose estimation on image

        Args:
            image: Input image (numpy array or path)
            conf: Confidence threshold (optional)
```

```
        Returns:
            Results object from YOLO
        """
        conf = conf or self.config["confidence_threshold"]
        imgsz = self.config["imgsz"]

        results = self.model.predict(
            source=image,
            conf=conf,
            imgsz=imgsz,
            verbose=False
        )

        return results[0] if results else None


def initialize_models():
    """
    Initialize and load all models

    Returns:
        tuple: (DetectionModel, PoseModel, ModelManager)
    """
    try:
        manager = ModelManager()

        detection_model, pose_model = manager.load_all_models()

        detector = DetectionModel(detection_model, DETECTION_CONFIG)
        pose_estimator = PoseModel(pose_model, POSE_CONFIG)

        manager.print_model_info()

        return detector, pose_estimator, manager

    except Exception as e:
```

```
        logger.error(f" Failed to initialize models: {str(e)}")
        raise


# Test function
if __name__ == "__main__":
    print(" Testing model loading...\n")

    try:
        detector, pose_estimator, manager = initialize_models()
        print("\n Models tested successfully!")

    except Exception as e:
        print(f"\n Test failed: {str(e)}")
        sys.exit(1)
```

```
"""
Configuration file for Football Pose Estimation Project
"""

import os
from pathlib import Path

# ==================== MODEL SELECTION ====================
def select_model():
    """
    Interactive model selection function
    """
    print("\n" + "="*60)
```

```
    print(" Choosing the YOLOv8 model for Object Detection and Pose
Estimation")
    print("="*60)

    print("\n📊 Model Comparison:\n")

    models_info = {
        'n': {
            'name': 'YOLOv8n (nano)',
            'speed': 'Very fast',
            'accuracy': ' Good',
            'size': '~6 MB',
            'best_for': 'CPU,Weak devices'
        },
        's': {
            'name': 'YOLOv8s (small)',
            'speed': ' fast ',
            'accuracy': '  Very good',
            'size': '~22 MB',
            'best_for': 'GPU Average, Real-time'
        },
        'm': {
            'name': 'YOLOv8m (medium)  Recommended',
            'speed': ' Medium',
            'accuracy': ' Excellent',
            'size': '~52 MB',
            'best_for': 'Good GPU, high accuracy'
        },
        'l': {
            'name': 'YOLOv8l (large)',
            'speed': ' Relatively slow',
            'accuracy': ' Very excellent',
            'size': '~87 MB',
            'best_for': 'Strong GPU, maximum accuracy'
        },
        'x': {
            'name': 'YOLOv8x (xlarge)',
```

```
            'speed': ' Slow',
            'accuracy': ' The best',
            'size': '~136 MB',
            'best_for': 'Professional GPU, research'
        }
    }

    for key, info in models_info.items():
        print(f"[{key.upper()}] {info['name']}")
        print(f"    Speed: {info['speed']}")
        print(f"    Accuracy: {info['accuracy']}")
        print(f"    Size: {info['size']}")
        print(f"    Best for: {info['best_for']}")
        print()

    print("="*60)

    while True:
        choice = input("\n Choose model (n/s/m/l/x) or press Enter for default
[m]: ").strip().lower()

        if choice == '':
            choice = 'm'
            print(f" Default model selected: YOLOv8m (medium)")
            break

        if choice in ['n', 's', 'm', 'l', 'x']:
            print(f" Selected: {models_info[choice]['name']}")
            break
        else:
            print(" Invalid choice! Choose from (n/s/m/l/x)")

    print("="*60 + "\n")
    return choice
```

```
# ==================== AUTO-SELECT FOR API MODE
====================
# Check if running in API mode (don't show interactive prompt)
if os.environ.get('YOLO_TEST2_AUTO_SELECT') == 'true':
    SELECTED_MODEL = 'm'  # Default for API
    print(" Auto-selected: YOLOv8m (API mode)")
else:
    # Get user's model selection (interactive)
    SELECTED_MODEL = select_model()

# ==================== PROJECT PATHS ====================
# Base directory
BASE_DIR = Path(__file__).resolve().parent

# Input/Output directories
INPUT_DIR = BASE_DIR / "input"
OUTPUT_DIR = BASE_DIR / "output"
MODELS_DIR = BASE_DIR / "models"
LOGS_DIR = BASE_DIR / "logs"

# Create directories if they don't exist
for directory in [INPUT_DIR, OUTPUT_DIR, MODELS_DIR, LOGS_DIR]:
    directory.mkdir(exist_ok=True, parents=True)

# ==================== MODEL CONFIGURATION
====================

# Object Detection Model (YOLO)
DETECTION_CONFIG = {
    "model_name": f"yolov8{SELECTED_MODEL}.pt",
    "model_path": MODELS_DIR / f"yolov8{SELECTED_MODEL}.pt",
    "confidence_threshold": 0.5,
    "iou_threshold": 0.45,
    "target_class": 0,  # 0 = person in COCO dataset
    "device": "cuda",  # Options: "cuda", "cpu", "mps" (for Mac M1/M2)
    "imgsz": 640,  # Image size for inference
}
```

```
# Pose Estimation Model
POSE_CONFIG = {
    "model_type": "yolov8",  # Options: "yolov8", "mediapipe"
    "model_name": f"yolov8{SELECTED_MODEL}-pose.pt",
    "model_path": MODELS_DIR / f"yolov8{SELECTED_MODEL}-pose.pt",
    "confidence_threshold": 0.5,
    "device": "cuda",
    "imgsz": 640,
}

# MediaPipe Configuration (if using MediaPipe)
MEDIAPIPE_CONFIG = {
    "min_detection_confidence": 0.5,
    "min_tracking_confidence": 0.5,
    "model_complexity": 1,  # 0, 1, or 2 (higher = more accurate but slower)
    "enable_segmentation": False,
    "smooth_landmarks": True,
}

# ==================== VIDEO PROCESSING
====================
VIDEO_CONFIG = {
    "fps": 30,  # Output video FPS (None = use original)
    "frame_skip": 1,  # Process every Nth frame (1 = process all frames)
    "resize_width": None,  # Resize width (None = keep original)
    "resize_height": None,  # Resize height (None = keep original)
    "codec": "mp4v",  # Video codec: 'mp4v', 'XVID', 'H264'
    "save_frames": False,  # Save individual frames
}

# ==================== VISUALIZATION ====================
VISUALIZATION_CONFIG = {
    "draw_bboxes": True,  # Draw bounding boxes
    "draw_keypoints": True,  # Draw pose keypoints
    "draw_skeleton": True,  # Draw skeleton connections
    "draw_labels": True,  # Draw player IDs/labels
```

```
    # Colors (BGR format)
    "bbox_color": (0, 255, 0),  # Green
    "keypoint_color": (0, 0, 255),  # Red
    "skeleton_color": (255, 0, 0),  # Blue
    "text_color": (255, 255, 255),  # White

    # Sizes
    "bbox_thickness": 2,
    "keypoint_radius": 4,
    "skeleton_thickness": 2,
    "text_font": 0,  # cv2.FONT_HERSHEY_SIMPLEX
    "text_scale": 0.6,
    "text_thickness": 2,
}

# ==================== YOLO POSE KEYPOINTS
====================
# YOLO Pose keypoints indices (COCO format - 17 keypoints)
YOLO_KEYPOINTS = {
    0: "nose",
    1: "left_eye",
    2: "right_eye",
    3: "left_ear",
    4: "right_ear",
    5: "left_shoulder",
    6: "right_shoulder",
    7: "left_elbow",
    8: "right_elbow",
    9: "left_wrist",
    10: "right_wrist",
    11: "left_hip",
    12: "right_hip",
    13: "left_knee",
    14: "right_knee",
    15: "left_ankle",
    16: "right_ankle",
```

```
}

# Skeleton connections (pairs of keypoint indices)
SKELETON_CONNECTIONS = [
    (0, 1), (0, 2),  # nose to eyes
    (1, 3), (2, 4),  # eyes to ears
    (0, 5), (0, 6),  # nose to shoulders
    (5, 6),  # shoulders
    (5, 7), (7, 9),  # left arm
    (6, 8), (8, 10),  # right arm
    (5, 11), (6, 12),  # shoulders to hips
    (11, 12),  # hips
    (11, 13), (13, 15),  # left leg
    (12, 14), (14, 16),  # right leg
]

# ==================== MEDIAPIPE POSE KEYPOINTS
====================
MEDIAPIPE_KEYPOINTS = {
    0: "nose",
    1: "left_eye_inner",
    2: "left_eye",
    3: "left_eye_outer",
    4: "right_eye_inner",
    5: "right_eye",
    6: "right_eye_outer",
    7: "left_ear",
    8: "right_ear",
    9: "mouth_left",
    10: "mouth_right",
    11: "left_shoulder",
    12: "right_shoulder",
    13: "left_elbow",
    14: "right_elbow",
    15: "left_wrist",
    16: "right_wrist",
    17: "left_pinky",
```

```
    18: "right_pinky",
    19: "left_index",
    20: "right_index",
    21: "left_thumb",
    22: "right_thumb",
    23: "left_hip",
    24: "right_hip",
    25: "left_knee",
    26: "right_knee",
    27: "left_ankle",
    28: "right_ankle",
    29: "left_heel",
    30: "right_heel",
    31: "left_foot_index",
    32: "right_foot_index",
}

# ==================== DATA EXPORT ====================
EXPORT_CONFIG = {
    "save_json": True,  # Save keypoints as JSON
    "save_csv": True,  # Save keypoints as CSV
    "save_video": True,  # Save annotated video
    "save_images": False,  # Save annotated images
    "json_indent": 2,  # JSON formatting
}

# ==================== TRACKING CONFIGURATION
====================
TRACKING_CONFIG = {
    "enable_tracking": True,  # Enable player tracking across frames
    "max_disappeared": 30,  # Max frames before removing tracked object
    "max_distance": 50,  # Max pixel distance for tracking
}

# ==================== FILTERING ====================
FILTER_CONFIG = {
    "min_bbox_area": 1000,  # Minimum bounding box area (pixels²)
```

```
    "max_bbox_area": 500000,  # Maximum bounding box area (pixels²)
    "min_keypoints_visible": 5,  # Minimum visible keypoints for valid pose
}

# ==================== PERFORMANCE ====================
PERFORMANCE_CONFIG = {
    "use_half_precision": False,  # Use FP16 for faster inference (GPU only)
    "batch_size": 1,  # Batch processing (for multiple images)
    "num_workers": 4,  # DataLoader workers
}

# ==================== LOGGING ====================
LOGGING_CONFIG = {
    "log_level": "INFO",  # DEBUG, INFO, WARNING, ERROR, CRITICAL
    "log_file": LOGS_DIR / "pose_estimation.log",
    "log_format": "%(asctime)s - %(name)s - %(levelname)s - %(message)s",
}

# ==================== DEBUG MODE ====================
DEBUG = False  # Enable debug mode for verbose output

# ==================== PRINT CONFIGURATION SUMMARY
====================
def print_config_summary():
    """Print configuration summary"""
    print("\n" + "="*60)
    print(" Configuration Summary")
    print("="*60)
    print(f" Object Detection Model: yolov8{SELECTED_MODEL}.pt")
    print(f" Pose Estimation Model: yolov8{SELECTED_MODEL}-pose.pt")
    print(f" Input Directory: {INPUT_DIR}")
    print(f" Output Directory: {OUTPUT_DIR}")
    print(f" Device: {DETECTION_CONFIG['device']}")
    print(f" Confidence Threshold:
{DETECTION_CONFIG['confidence_threshold']}")
    print("="*60 + "\n")
```

```
# Print configuration when module is loaded
if __name__ != "__main__":
    print_config_summary()


# ==================== CONFIG CLASS FOR IMPORTS
====================
class Config:
    """
    Config class to make all configurations accessible as class attributes
    This allows imports like: from config import Config
    """

    # Model Selection
    SELECTED_MODEL = SELECTED_MODEL

    # Paths
    BASE_DIR = BASE_DIR
    INPUT_DIR = INPUT_DIR
    OUTPUT_DIR = OUTPUT_DIR
    MODELS_DIR = MODELS_DIR
    LOGS_DIR = LOGS_DIR

    # Detection Config
    DETECTION_CONFIG = DETECTION_CONFIG
    POSE_CONFIG = POSE_CONFIG
    MEDIAPIPE_CONFIG = MEDIAPIPE_CONFIG

    # Video Config
    VIDEO_CONFIG = VIDEO_CONFIG

    # Visualization
    VISUALIZATION_CONFIG = VISUALIZATION_CONFIG

    # Keypoints
    YOLO_KEYPOINTS = YOLO_KEYPOINTS
    SKELETON_CONNECTIONS = SKELETON_CONNECTIONS
```

```
    MEDIAPIPE_KEYPOINTS = MEDIAPIPE_KEYPOINTS

    # Export, Tracking, Filtering
    EXPORT_CONFIG = EXPORT_CONFIG
    TRACKING_CONFIG = TRACKING_CONFIG
    FILTER_CONFIG = FILTER_CONFIG

    # Performance & Logging
    PERFORMANCE_CONFIG = PERFORMANCE_CONFIG
    LOGGING_CONFIG = LOGGING_CONFIG

    # Debug
    DEBUG = DEBUG

    @classmethod
    def get_model_path(cls, model_name: str):
        """Get full path for model file"""
        return cls.MODELS_DIR / model_name

    @classmethod
    def update_model_size(cls, size: str):
        """Update model size configuration"""
        cls.SELECTED_MODEL = size
        cls.DETECTION_CONFIG['model_name'] = f"yolov8{size}.pt"
        cls.DETECTION_CONFIG['model_path'] = cls.MODELS_DIR /
f"yolov8{size}.pt"
        cls.POSE_CONFIG['model_name'] = f"yolov8{size}-pose.pt"
        cls.POSE_CONFIG['model_path'] = cls.MODELS_DIR /
f"yolov8{size}-pose.pt"
```

```
"""
Utility functions for Football Pose Estimation Project
Includes helper functions for image/video processing, data export, and
visualization
"""
```

```
import cv2
import numpy as np
import json
import csv
import logging
from pathlib import Path
from typing import List, Dict, Tuple, Optional, Union
from datetime import datetime
import sys

# Import configuration
try:
    from config import (
        OUTPUT_DIR,
        EXPORT_CONFIG,
        VIDEO_CONFIG,
        VISUALIZATION_CONFIG,
        YOLO_KEYPOINTS,
        LOGGING_CONFIG
    )
except ImportError:
    print(" Error: config.py not found")
    sys.exit(1)

# Setup logging
logger = logging.getLogger(__name__)


# ==================== VIDEO PROCESSING
====================

class VideoReader:
    """
    Video reader with frame extraction capabilities
    """
```

```
def __init__(self, video_path: Union[str, Path]):
    """
    Initialize video reader

    Args:
        video_path: Path to video file
    """
    self.video_path = str(video_path)
    self.cap = cv2.VideoCapture(self.video_path)

    if not self.cap.isOpened():
        raise ValueError(f"Cannot open video: {video_path}")

    # Get video properties
    self.fps = int(self.cap.get(cv2.CAP_PROP_FPS))
    self.frame_count = int(self.cap.get(cv2.CAP_PROP_FRAME_COUNT))
    self.width = int(self.cap.get(cv2.CAP_PROP_FRAME_WIDTH))
    self.height = int(self.cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
    self.duration = self.frame_count / self.fps if self.fps > 0 else 0

    logger.info(f" Video opened: {Path(video_path).name}")
    logger.info(f"   Dimensions: {self.width}x{self.height}")
    logger.info(f"   FPS: {self.fps}")
    logger.info(f"   Frame count: {self.frame_count}")
    logger.info(f"   Duration: {self.duration:.2f} seconds")

def read_frame(self) -> Tuple[bool, Optional[np.ndarray]]:
    """Read next frame"""
    ret, frame = self.cap.read()
    return ret, frame

def get_frame_at_index(self, frame_idx: int) -> Optional[np.ndarray]:
    """Get frame at specific index"""
    self.cap.set(cv2.CAP_PROP_POS_FRAMES, frame_idx)
    ret, frame = self.cap.read()
    return frame if ret else None
```

```
    def get_frame_at_time(self, time_sec: float) -> Optional[np.ndarray]:
        """Get frame at specific timestamp"""
        frame_idx = int(time_sec * self.fps)
        return self.get_frame_at_index(frame_idx)

    def reset(self):
        """Reset video to beginning"""
        self.cap.set(cv2.CAP_PROP_POS_FRAMES, 0)

    def release(self):
        """Release video capture"""
        self.cap.release()

    def __enter__(self):
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        self.release()

    def __len__(self):
        return self.frame_count


class VideoWriter:
    """
    Video writer with customizable output settings
    """

    def __init__(self, output_path: Union[str, Path],
                 fps: int, width: int, height: int,
                 codec: str = 'mp4v'):
        """
        Initialize video writer

        Args:
            output_path: Output video path
            fps: Frames per second
```

```
        width: Frame width
        height: Frame height
        codec: Video codec
    """
    self.output_path = str(output_path)
    self.fps = fps
    self.width = width
    self.height = height

    Path(output_path).parent.mkdir(parents=True, exist_ok=True)

    fourcc = cv2.VideoWriter_fourcc(*codec)
    self.writer = cv2.VideoWriter(
        self.output_path, fourcc, fps, (width, height)
    )

    if not self.writer.isOpened():
        raise ValueError(f"Cannot create video file: {output_path}")

    self.frame_count = 0
    logger.info(f" Video writer created: {Path(output_path).name}")

def write_frame(self, frame: np.ndarray):
    """Write frame to video"""
    if frame.shape[1] != self.width or frame.shape[0] != self.height:
        frame = cv2.resize(frame, (self.width, self.height))

    self.writer.write(frame)
    self.frame_count += 1

def release(self):
    """Release writer"""
    self.writer.release()
    logger.info(f" Video saved: {self.frame_count} frames")

def __enter__(self):
    return self
```

```
    def __exit__(self, exc_type, exc_val, exc_tb):
        self.release()


# ==================== IMAGE PROCESSING
====================

def resize_image(image: np.ndarray,
                 width: Optional[int] = None,
                 height: Optional[int] = None,
                 keep_aspect_ratio: bool = True) -> np.ndarray:
    """Resize image with optional aspect ratio"""
    h, w = image.shape[:2]

    if width is None and height is None:
        return image

    if keep_aspect_ratio:
        if width and not height:
            ratio = width / w
            height = int(h * ratio)
        elif height and not width:
            ratio = height / h
            width = int(w * ratio)
    else:
        width = width or w
        height = height or h

    return cv2.resize(image, (width, height), interpolation=cv2.INTER_LINEAR)


def crop_image(image: np.ndarray, bbox: List[int], padding: int = 0) ->
np.ndarray:
    """Crop image using bounding box"""
    h, w = image.shape[:2]
    x1, y1, x2, y2 = bbox
```

```
    x1 = max(0, x1 - padding)
    y1 = max(0, y1 - padding)
    x2 = min(w, x2 + padding)
    y2 = min(h, y2 + padding)

    return image[y1:y2, x1:x2]


def add_text_with_background(image: np.ndarray,
                             text: str,
                             position: Tuple[int, int],
                             font_scale: float = 0.6,
                             thickness: int = 2,
                             text_color: Tuple[int, int, int] = (255, 255, 255),
                             bg_color: Tuple[int, int, int] = (0, 0, 0),
                             padding: int = 5) -> np.ndarray:
    """Add text with a solid background to image"""
    output = image.copy()
    font = cv2.FONT_HERSHEY_SIMPLEX

    (text_w, text_h), baseline = cv2.getTextSize(text, font, font_scale,
thickness)

    x, y = position

    cv2.rectangle(output,
                  (x - padding, y - text_h - padding),
                  (x + text_w + padding, y + baseline + padding),
                  bg_color, -1)

    cv2.putText(output, text, (x, y),
                font, font_scale, text_color, thickness, cv2.LINE_AA)

    return output
```

```
def create_grid_layout(images: List[np.ndarray],
                       grid_size: Optional[Tuple[int, int]] = None,
                       cell_size: Tuple[int, int] = (300, 300),
                       padding: int = 10,
                       bg_color: Tuple[int, int, int] = (50, 50, 50)) -> np.ndarray:
    """Create a grid of images"""
    if not images:
        return np.zeros((100, 100, 3), dtype=np.uint8)

    num_images = len(images)

    if grid_size is None:
        cols = int(np.ceil(np.sqrt(num_images)))
        rows = int(np.ceil(num_images / cols))
    else:
        rows, cols = grid_size

    cell_w, cell_h = cell_size

    grid_h = rows * cell_h + (rows + 1) * padding
    grid_w = cols * cell_w + (cols + 1) * padding
    grid = np.full((grid_h, grid_w, 3), bg_color, dtype=np.uint8)

    for idx, img in enumerate(images):
        if idx >= rows * cols:
            break

        row = idx // cols
        col = idx % cols

        img_resized = cv2.resize(img, (cell_w, cell_h))

        y = row * cell_h + (row + 1) * padding
        x = col * cell_w + (col + 1) * padding

        grid[y:y+cell_h, x:x+cell_w] = img_resized
```

```
    return grid


# ==================== DATA EXPORT ====================

def save_detections_json(detections: List[Dict],
                         frame_idx: int,
                         output_path: Union[str, Path],
                         metadata: Optional[Dict] = None):
    """Save detections JSON"""
    output_path = Path(output_path)
    output_path.parent.mkdir(parents=True, exist_ok=True)

    data = {
        'frame_index': frame_idx,
        'timestamp': datetime.now().isoformat(),
        'num_detections': len(detections),
        'detections': detections
    }

    if metadata:
        data['metadata'] = metadata

    with open(output_path, 'w', encoding='utf-8') as f:
        json.dump(data, f, ensure_ascii=False,
indent=EXPORT_CONFIG['json_indent'])

    logger.debug(f" JSON saved: {output_path}")


def save_poses_json(poses: List[Dict],
                    frame_idx: int,
                    output_path: Union[str, Path],
                    metadata: Optional[Dict] = None):
    """Save poses JSON"""
    output_path = Path(output_path)
    output_path.parent.mkdir(parents=True, exist_ok=True)
```

```
    data = {
        'frame_index': frame_idx,
        'timestamp': datetime.now().isoformat(),
        'num_poses': len([p for p in poses if p is not None]),
        'poses': poses,
        'keypoint_names': YOLO_KEYPOINTS
    }

    if metadata:
        data['metadata'] = metadata

    with open(output_path, 'w', encoding='utf-8') as f:
        json.dump(data, f, ensure_ascii=False,
indent=EXPORT_CONFIG['json_indent'])

    logger.debug(f" JSON saved: {output_path}")


def save_poses_csv(poses_data: List[Dict],
                   output_path: Union[str, Path]):
    """Save poses CSV"""
    output_path = Path(output_path)
    output_path.parent.mkdir(parents=True, exist_ok=True)

    headers = ['frame_idx', 'player_id']

    for kpt_idx, kpt_name in YOLO_KEYPOINTS.items():
        headers.extend([f'{kpt_name}_x', f'{kpt_name}_y', f'{kpt_name}_conf'])

    headers.extend(['visible_keypoints', 'avg_confidence'])

    with open(output_path, 'w', newline='', encoding='utf-8') as f:
        writer = csv.DictWriter(f, fieldnames=headers)
        writer.writeheader()

        for data in poses_data:
```

```
            frame_idx = data['frame_idx']
            poses = data['poses']

            for player_id, pose in enumerate(poses):
                if pose is None:
                    continue

                row = {
                    'frame_idx': frame_idx,
                    'player_id': player_id
                }

                keypoints = pose['keypoints']
                for kpt_idx, kpt_name in YOLO_KEYPOINTS.items():
                    if kpt_idx < len(keypoints):
                        x, y, conf = keypoints[kpt_idx]
                        row[f'{kpt_name}_x'] = x
                        row[f'{kpt_name}_y'] = y
                        row[f'{kpt_name}_conf'] = conf
                    else:
                        row[f'{kpt_name}_x'] = 0
                        row[f'{kpt_name}_y'] = 0
                        row[f'{kpt_name}_conf'] = 0

                row['visible_keypoints'] = pose['visible_keypoints']
                row['avg_confidence'] = pose.get('confidence', 0)

                writer.writerow(row)

    logger.info(f" CSV saved: {output_path}")


def load_poses_json(json_path: Union[str, Path]) -> Dict:
    """Load poses JSON"""
    with open(json_path, 'r', encoding='utf-8') as f:
        return json.load(f)
```

```
# ==================== PROGRESS TRACKING
====================

class ProgressTracker:
    """
    Simple progress tracker for processing
    """

    def __init__(self, total: int, desc: str = "Processing"):
        self.total = total
        self.desc = desc
        self.current = 0
        self.start_time = datetime.now()

    def update(self, n: int = 1):
        self.current += n
        self._print_progress()

    def _print_progress(self):
        percent = (self.current / self.total) * 100
        filled = int(percent / 2)
        bar = '' * filled + '' * (50 - filled)

        elapsed = (datetime.now() - self.start_time).total_seconds()
        eta = (elapsed / self.current) * (self.total - self.current) if self.current > 0
else 0

        print(f'\r{self.desc}: |{bar}| {percent:.1f}% '
              f'[{self.current}/{self.total}] '
              f'ETA: {eta:.1f}s', end='', flush=True)

        if self.current >= self.total:
            print()

    def close(self):
        print()
```

```
# ==================== FILE UTILITIES ====================

def get_output_filename(input_path: Union[str, Path],
                        suffix: str = "_output",
                        extension: Optional[str] = None) -> Path:
    """Generate output filename"""
    input_path = Path(input_path)

    if extension is None:
        extension = input_path.suffix

    output_name = f"{input_path.stem}{suffix}{extension}"
    return OUTPUT_DIR / output_name


def ensure_output_dir(subdir: Optional[str] = None) -> Path:
    """Ensure output directory exists"""
    if subdir:
        output_dir = OUTPUT_DIR / subdir
    else:
        output_dir = OUTPUT_DIR

    output_dir.mkdir(parents=True, exist_ok=True)
    return output_dir


def get_supported_image_extensions() -> List[str]:
    return ['.jpg', '.jpeg', '.png', '.bmp', '.tiff', '.webp']


def get_supported_video_extensions() -> List[str]:
    return ['.mp4', '.avi', '.mov', '.mkv', '.flv', '.wmv']


def is_image_file(file_path: Union[str, Path]) -> bool:
```

```
    return Path(file_path).suffix.lower() in get_supported_image_extensions()


def is_video_file(file_path: Union[str, Path]) -> bool:
    return Path(file_path).suffix.lower() in get_supported_video_extensions()


# Test function
if __name__ == "__main__":
    print(" Utils Test...\n")

    print(" All utility functions loaded successfully!")
    print(f" Output directory: {OUTPUT_DIR}")
    print(f" Supported image formats: {get_supported_image_extensions()}")
    print(f" Supported video formats: {get_supported_video_extensions()}")
```

```
"""
Video Processor for Football Pose Estimation
Processes videos frame by frame with detection and pose estimation
"""

import cv2
import numpy as np
import logging
from pathlib import Path
from typing import Dict, List, Optional, Tuple, Union
from datetime import datetime
import sys

import sys
import os

# Fix encoding for Windows
if sys.platform == 'win32':
    os.environ['PYTHONIOENCODING'] = 'utf-8'
    import locale
```

```
    locale.setlocale(locale.LC_ALL, '')

# Import configuration and modules
try:
    from config import (
        VIDEO_CONFIG,
        EXPORT_CONFIG,
        OUTPUT_DIR,
        TRACKING_CONFIG,
        VISUALIZATION_CONFIG
    )
    from utils import (
        VideoReader,
        VideoWriter,
        ProgressTracker,
        save_poses_json,
        save_poses_csv,
        get_output_filename,
        ensure_output_dir,
        is_video_file,
        add_text_with_background
    )
    from detector import PlayerDetector, MultiPlayerDetector
    from pose_estimator import PoseEstimator, PoseAnalyzer
except ImportError as e:
    print(f" Import error: {str(e)}")
    sys.exit(1)

# Setup logging
logger = logging.getLogger(__name__)


class VideoProcessor:
    """
    Main video processor for football match pose estimation
    """
```

```
    def __init__(self, detector: PlayerDetector,
                 pose_estimator: PoseEstimator,
                 enable_tracking: bool = True):
        """
        Initialize Video Processor

        Args:
            detector: PlayerDetector instance
            pose_estimator: PoseEstimator instance
            enable_tracking: Whether to enable player tracking
        """
        # Use MultiPlayerDetector if tracking is enabled
        if enable_tracking and TRACKING_CONFIG['enable_tracking']:
            self.detector = MultiPlayerDetector(detector.model)
            self.tracking_enabled = True
        else:
            self.detector = detector if isinstance(detector, PlayerDetector) else
PlayerDetector(detector)
            self.tracking_enabled = False

        self.pose_estimator = pose_estimator
        self.pose_analyzer = PoseAnalyzer(pose_estimator)

        # Statistics
        self.stats = {
            'total_frames': 0,
            'processed_frames': 0,
            'total_detections': 0,
            'total_poses': 0,
            'processing_time': 0.0
        }

        logger.info(" Video Processor initialized")
        if self.tracking_enabled:
            logger.info("    Player tracking enabled")

    def process_frame(self, frame: np.ndarray,
```

```
            frame_idx: int) -> Tuple[np.ndarray, List[Dict], List[Dict]]:
    """
    Process single frame

    Args:
        frame: Input frame
        frame_idx: Frame index

    Returns:
        Tuple of (annotated_frame, detections, poses)
    """
    # Detect players
    detections = self.detector.detect_players(frame)

    # Update tracking if enabled
    if self.tracking_enabled:
        detections = self.detector.update_tracking(
            detections,
            TRACKING_CONFIG['max_distance']
        )

    # Process each detection
    poses = []
    annotated_frame = frame.copy()

    for i, det in enumerate(detections):
        # Crop player region
        cropped = self.detector.crop_player(frame, det['bbox'], padding=10)

        if cropped is None or cropped.size == 0:
            poses.append(None)
            continue

        # Estimate pose
        pose = self.pose_estimator.estimate_pose(cropped)

        if pose is not None:
```

```
            # Adjust keypoints to original frame coordinates
            pose = self._adjust_keypoints_to_frame(pose, det['bbox'])

            # Detect action
            action = self.pose_analyzer.detect_action(pose)
            pose['action'] = action

            # Add player ID if tracking
            if self.tracking_enabled and 'player_id' in det:
                pose['player_id'] = det['player_id']

            # Draw pose on frame
            annotated_frame = self.pose_estimator.draw_pose(
                annotated_frame, pose,
                draw_keypoints=VISUALIZATION_CONFIG['draw_keypoints'],
                draw_skeleton=VISUALIZATION_CONFIG['draw_skeleton']
            )

        poses.append(pose)

    # Draw bounding boxes
    if VISUALIZATION_CONFIG['draw_bboxes']:
        annotated_frame = self._draw_detections_with_info(
            annotated_frame, detections, poses
        )

    # Add frame info
    annotated_frame = self._add_frame_info(
        annotated_frame, frame_idx, len(detections), len([p for p in poses if
p])
    )

    return annotated_frame, detections, poses

def _adjust_keypoints_to_frame(self, pose: Dict, bbox: List[int]) -> Dict:
    """
    Adjust keypoints coordinates from cropped image to original frame
```

```
    Args:
        pose: Pose dictionary
        bbox: Bounding box [x1, y1, x2, y2]

    Returns:
        Adjusted pose dictionary
    """
    if pose is None:
        return None

    adjusted_pose = pose.copy()
    keypoints = np.array(pose['keypoints'])

    x1, y1, x2, y2 = bbox

    # Adjust coordinates
    keypoints[:, 0] += x1
    keypoints[:, 1] += y1

    adjusted_pose['keypoints'] = keypoints.tolist()

    return adjusted_pose

def _draw_detections_with_info(self, frame: np.ndarray,
                               detections: List[Dict],
                               poses: List[Dict]) -> np.ndarray:
    """
    Draw bounding boxes with additional information

    Args:
        frame: Input frame
        detections: List of detections
        poses: List of poses

    Returns:
        Annotated frame
```

```
    """
    output = frame.copy()

    bbox_color = VISUALIZATION_CONFIG['bbox_color']
    bbox_thickness = VISUALIZATION_CONFIG['bbox_thickness']

    for i, det in enumerate(detections):
        x1, y1, x2, y2 = det['bbox']

        # Draw bounding box
        cv2.rectangle(output, (x1, y1), (x2, y2), bbox_color, bbox_thickness)

        # Prepare label
        if self.tracking_enabled and 'player_id' in det:
            label = f"P{det['player_id']}"
        else:
            label = f"P{i+1}"

        # Add pose info if available
        if i < len(poses) and poses[i] is not None:
            pose = poses[i]
            visible = pose['visible_keypoints']
            action = pose.get('action', 'unknown')
            label += f" | {visible}kp | {action}"

        # Draw label with background
        output = add_text_with_background(
            output, label, (x1, y1 - 10),
            font_scale=0.5,
            thickness=1,
            text_color=(255, 255, 255),
            bg_color=bbox_color,
            padding=3
        )

    return output
```

```
def _add_frame_info(self, frame: np.ndarray,
                    frame_idx: int,
                    num_detections: int,
                    num_poses: int) -> np.ndarray:
    """
    Add frame information overlay

    Args:
        frame: Input frame
        frame_idx: Frame index
        num_detections: Number of detections
        num_poses: Number of valid poses

    Returns:
        Frame with info overlay
    """
    output = frame.copy()
    h, w = frame.shape[:2]

    # Create info text
    info_lines = [
        f"Frame: {frame_idx}",
        f"Players: {num_detections}",
        f"Poses: {num_poses}"
    ]

    # Draw semi-transparent background
    overlay = output.copy()
    cv2.rectangle(overlay, (10, 10), (250, 100), (0, 0, 0), -1)
    output = cv2.addWeighted(output, 0.7, overlay, 0.3, 0)

    # Draw text
    y_offset = 35
    for line in info_lines:
        cv2.putText(output, line, (20, y_offset),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.6,
                    (255, 255, 255), 2, cv2.LINE_AA)
```

```
            y_offset += 25

        return output

    def process_video(self, video_path: Union[str, Path],
                      output_path: Optional[Union[str, Path]] = None,
                      save_data: bool = True,
                      show_preview: bool = False) -> Dict:
        """
        Process entire video

        Args:
            video_path: Input video path
            output_path: Output video path (None for auto-generate)
            save_data: Whether to save pose data
            show_preview: Whether to show live preview

        Returns:
            Dictionary with processing results and statistics
        """
        video_path = Path(video_path)

        if not video_path.exists():
            raise FileNotFoundError(f"Video not found: {video_path}")

        if not is_video_file(video_path):
            raise ValueError(f"File is not a valid video: {video_path}")

        logger.info("="*60)
        logger.info(f" Starting video processing: {video_path.name}")
        logger.info("="*60)

        # Generate output path if not provided
        if output_path is None:
            output_path = get_output_filename(video_path, "_pose_estimation",
".mp4")
        else:
```

```
        output_path = Path(output_path)

    # Ensure output directory exists
    output_path.parent.mkdir(parents=True, exist_ok=True)

    start_time = datetime.now()

    # Open video
    with VideoReader(video_path) as reader:
        # Get video properties
        fps = VIDEO_CONFIG['fps'] if VIDEO_CONFIG['fps'] else reader.fps
        width = VIDEO_CONFIG['resize_width'] or reader.width
        height = VIDEO_CONFIG['resize_height'] or reader.height

        # Initialize video writer
        if EXPORT_CONFIG['save_video']:
            writer = VideoWriter(
                output_path, fps, width, height,
                codec=VIDEO_CONFIG['codec']
            )
        else:
            writer = None

        # Initialize progress tracker
        progress = ProgressTracker(reader.frame_count, desc="Processing
frames")

        # Storage for data export
        all_poses_data = []

        # Process frames
        frame_idx = 0
        frame_skip = VIDEO_CONFIG['frame_skip']

        while True:
            ret, frame = reader.read_frame()
```

```
            if not ret:
                break

            # Skip frames if configured
            if frame_idx % frame_skip != 0:
                frame_idx += 1
                continue

            # Resize if needed
            if frame.shape[1] != width or frame.shape[0] != height:
                frame = cv2.resize(frame, (width, height))

            # Process frame
            annotated_frame, detections, poses = self.process_frame(frame,
frame_idx)

            # Update statistics
            self.stats['total_detections'] += len(detections)
            self.stats['total_poses'] += len([p for p in poses if p])
            self.stats['processed_frames'] += 1

            # Save frame data
            if save_data:
                all_poses_data.append({
                    'frame_idx': frame_idx,
                    'timestamp': frame_idx / fps,
                    'detections': detections,
                    'poses': poses
                })

            # Write to output video
            if writer:
                writer.write_frame(annotated_frame)

            # Show preview
            if show_preview:
                cv2.imshow('Pose Estimation', annotated_frame)
```

```
                if cv2.waitKey(1) & 0xFF == ord('q'):
                    logger.info("Processing stopped by user")
                    break

                # Update progress
                progress.update()
                frame_idx += 1

            # Clean up
            progress.close()
            if writer:
                writer.release()
            if show_preview:
                cv2.destroyAllWindows()

        # Calculate processing time
        end_time = datetime.now()
        processing_time = (end_time - start_time).total_seconds()
        self.stats['processing_time'] = processing_time
        self.stats['total_frames'] = frame_idx

        # Save data files
        if save_data and all_poses_data:
            self._save_processing_data(all_poses_data, video_path, output_path)

        # Print summary
        self._print_summary(video_path, output_path)

        # Return results
        results = {
            'input_path': str(video_path),
            'output_path': str(output_path),
            'statistics': self.stats.copy(),
            'processing_time': processing_time,
            'fps': self.stats['processed_frames'] / processing_time if
processing_time > 0 else 0
        }
```

```
    return results

def _save_processing_data(self, poses_data: List[Dict],
                          video_path: Path,
                          output_path: Path):
    """
    Save processing data to JSON and CSV

    Args:
        poses_data: List of frame data
        video_path: Input video path
        output_path: Output video path
    """
    logger.info(" Saving processing data...")

    # Create data directory
    data_dir = ensure_output_dir("data")
    base_name = video_path.stem

    # Save JSON
    if EXPORT_CONFIG['save_json']:
        json_path = data_dir / f"{base_name}_poses.json"

        # Prepare full data
        full_data = {
            'video_info': {
                'input_path': str(video_path),
                'output_path': str(output_path),
                'total_frames': self.stats['total_frames'],
                'processed_frames': self.stats['processed_frames']
            },
            'statistics': self.stats,
            'frames': poses_data
        }

        import json
```

```
            with open(json_path, 'w', encoding='utf-8') as f:
                json.dump(full_data, f, ensure_ascii=False, indent=2)

            logger.info(f"    JSON: {json_path.name}")

        # Save CSV
        if EXPORT_CONFIG['save_csv']:
            csv_path = data_dir / f"{base_name}_poses.csv"
            save_poses_csv(poses_data, csv_path)
            logger.info(f"    CSV: {csv_path.name}")

    def _print_summary(self, video_path: Path, output_path: Path):
        """Print processing summary"""
        logger.info("\n" + "="*60)
        logger.info(" Processing Summary")
        logger.info("="*60)
        logger.info(f" Input video: {video_path.name}")
        logger.info(f"Output video: {output_path.name}")
        logger.info(f"  Total frames: {self.stats['total_frames']}")
        logger.info(f" Processed frames: {self.stats['processed_frames']}")
        logger.info(f" Total detections: {self.stats['total_detections']}")
        logger.info(f" Total poses: {self.stats['total_poses']}")
        logger.info(f"  Processing time: {self.stats['processing_time']:.2f}
seconds")

        if self.stats['processing_time'] > 0:
            fps = self.stats['processed_frames'] / self.stats['processing_time']
            logger.info(f" Speed: {fps:.2f} FPS")

        logger.info("="*60 + "\n")

    def reset_stats(self):
        """Reset processing statistics"""
        self.stats = {
            'total_frames': 0,
            'processed_frames': 0,
            'total_detections': 0,
```

```
            'total_poses': 0,
            'processing_time': 0.0
        }

        if self.tracking_enabled:
            self.detector.reset_tracking()


class ImageProcessor:
    """
    Processor for single images or batch of images
    """

    def __init__(self, detector: PlayerDetector,
                 pose_estimator: PoseEstimator):
        """
        Initialize Image Processor

        Args:
            detector: PlayerDetector instance
            pose_estimator: PoseEstimator instance
        """
        self.detector = detector
        self.pose_estimator = pose_estimator
        self.pose_analyzer = PoseAnalyzer(pose_estimator)

        logger.info("✅ Image Processor initialized")

    def process_image(self, image_path: Union[str, Path],
                      output_path: Optional[Union[str, Path]] = None,
                      save_data: bool = True) -> Dict:
        """
        Process single image

        Args:
            image_path: Input image path
            output_path: Output image path (None for auto-generate)
```

```
        save_data: Whether to save pose data

    Returns:
        Dictionary with processing results
    """
    image_path = Path(image_path)

    if not image_path.exists():
        raise FileNotFoundError(f"Image not found: {image_path}")

    logger.info(f"  Processing image: {image_path.name}")

    # Read image
    image = cv2.imread(str(image_path))

    if image is None:
        raise ValueError(f"Cannot read image: {image_path}")

    # Detect and crop players
    detections, cropped_images =
self.detector.detect_and_crop_players(image)

    # Estimate poses
    poses = self.pose_estimator.estimate_poses_batch(cropped_images)

    # Create annotated image
    annotated = image.copy()

    # Draw detections and poses
    if VISUALIZATION_CONFIG['draw_bboxes']:
        annotated = self.detector.draw_detections(annotated, detections)

    for i, pose in enumerate(poses):
        if pose is not None:
            # Adjust keypoints to frame
            det = detections[i]
            pose = self._adjust_keypoints(pose, det['bbox'])
```

```
            # Draw pose
            annotated = self.pose_estimator.draw_pose(annotated, pose)

    # Generate output path
    if output_path is None:
        output_path = get_output_filename(image_path, "_pose_estimation")
    else:
        output_path = Path(output_path)

    # Save annotated image
    cv2.imwrite(str(output_path), annotated)
    logger.info(f" Saved image: {output_path.name}")

    # Save data
    if save_data:
        json_path = output_path.with_suffix('.json')
        save_poses_json(poses, 0, json_path, {
            'image_path': str(image_path),
            'num_detections': len(detections)
        })

    results = {
        'input_path': str(image_path),
        'output_path': str(output_path),
        'num_detections': len(detections),
        'num_poses': len([p for p in poses if p]),
        'detections': detections,
        'poses': poses
    }

    return results

def _adjust_keypoints(self, pose: Dict, bbox: List[int]) -> Dict:
    """
    Adjust keypoints coordinates from cropped image to original frame
```

```
    Args:
        pose: Pose dictionary
        bbox: Bounding box [x1, y1, x2, y2]

    Returns:
        Adjusted pose dictionary
    """
    if pose is None:
        return None

    adjusted_pose = pose.copy()
    keypoints = np.array(pose['keypoints'])

    x1, y1, x2, y2 = bbox

    # Adjust coordinates
    keypoints[:, 0] += x1
    keypoints[:, 1] += y1

    adjusted_pose['keypoints'] = keypoints.tolist()

    return adjusted_pose

def process_batch(self, image_dir: Union[str, Path],
                  output_dir: Optional[Union[str, Path]] = None,
                  save_data: bool = True) -> List[Dict]:
    """
    Process batch of images from directory

    Args:
        image_dir: Directory containing images
        output_dir: Output directory (None for default)
        save_data: Whether to save pose data

    Returns:
        List of processing results
    """
```

```
    image_dir = Path(image_dir)

    if not image_dir.exists() or not image_dir.is_dir():
        raise ValueError(f"Directory does not exist or is invalid: {image_dir}")

    # Get all image files
    from utils import get_supported_image_extensions
    image_extensions = get_supported_image_extensions()
    image_files = []

    for ext in image_extensions:
        image_files.extend(image_dir.glob(f"*{ext}"))

    if not image_files:
        logger.warning(f"No images found in: {image_dir}")
        return []

    logger.info(f" Processing {len(image_files)} images from:
{image_dir.name}")

    # Setup output directory
    if output_dir is None:
        output_dir = ensure_output_dir("images")
    else:
        output_dir = Path(output_dir)
        output_dir.mkdir(parents=True, exist_ok=True)

    # Process each image
    results = []
    progress = ProgressTracker(len(image_files), desc="Processing
images")

    for img_path in image_files:
        try:
            output_path = output_dir /
f"{img_path.stem}_output{img_path.suffix}"
            result = self.process_image(img_path, output_path, save_data)
```

```
                results.append(result)
            except Exception as e:
                logger.error(f"Error processing {img_path.name}: {str(e)}")
                results.append({
                    'input_path': str(img_path),
                    'error': str(e)
                })

            progress.update()

        progress.close()

        # Print summary
        successful = len([r for r in results if 'error' not in r])
        logger.info(f"\n Successfully processed {successful}/{len(image_files)}
images")

        return results


class BatchProcessor:
    """
    Advanced batch processor for multiple videos/images
    """

    def __init__(self, detector: PlayerDetector,
                 pose_estimator: PoseEstimator,
                 enable_tracking: bool = True):
        """
        Initialize Batch Processor

        Args:
            detector: PlayerDetector instance
            pose_estimator: PoseEstimator instance
            enable_tracking: Whether to enable tracking for videos
        """
```

```
        self.video_processor = VideoProcessor(detector, pose_estimator,
enable_tracking)
        self.image_processor = ImageProcessor(detector, pose_estimator)

        logger.info("✅ Batch Processor initialized")

    def process_directory(self, input_dir: Union[str, Path],
                          output_dir: Optional[Union[str, Path]] = None,
                          process_videos: bool = True,
                          process_images: bool = True) -> Dict:
        """
        Process all videos and images in a directory

        Args:
            input_dir: Input directory path
            output_dir: Output directory path
            process_videos: Whether to process videos
            process_images: Whether to process images

        Returns:
            Dictionary with processing results
        """
        input_dir = Path(input_dir)

        if not input_dir.exists() or not input_dir.is_dir():
            raise ValueError(f"Directory does not exist or is invalid: {input_dir}")

        logger.info("="*60)
        logger.info(f" Processing directory: {input_dir.name}")
        logger.info("="*60)

        # Setup output directory
        if output_dir is None:
            output_dir = OUTPUT_DIR / "batch_output"
        else:
            output_dir = Path(output_dir)
```

```
    output_dir.mkdir(parents=True, exist_ok=True)

    results = {
        'input_dir': str(input_dir),
        'output_dir': str(output_dir),
        'videos': [],
        'images': []
    }

    # Process videos
    if process_videos:
        from utils import get_supported_video_extensions
        video_extensions = get_supported_video_extensions()
        video_files = []

        for ext in video_extensions:
            video_files.extend(input_dir.glob(f"*{ext}"))

        if video_files:
            logger.info(f"\n Found {len(video_files)} videos")

            video_output_dir = output_dir / "videos"
            video_output_dir.mkdir(exist_ok=True)

            for video_path in video_files:
                try:
                    output_path = video_output_dir /
f"{video_path.stem}_output.mp4"
                    result = self.video_processor.process_video(
                        video_path, output_path, save_data=True
                    )
                    results['videos'].append(result)

                    # Reset stats for next video
                    self.video_processor.reset_stats()

                except Exception as e:
```

```
                logger.error(f"Error processing video {video_path.name}:
{str(e)}")
                results['videos'].append({
                    'input_path': str(video_path),
                    'error': str(e)
                })

        # Process images
        if process_images:
            from utils import get_supported_image_extensions
            image_extensions = get_supported_image_extensions()
            image_files = []

            for ext in image_extensions:
                image_files.extend(input_dir.glob(f"*{ext}"))

            if image_files:
                logger.info(f"\n  Found {len(image_files)} images")

                image_output_dir = output_dir / "images"
                image_output_dir.mkdir(exist_ok=True)

                batch_results = self.image_processor.process_batch(
                    input_dir, image_output_dir, save_data=True
                )
                results['images'] = batch_results

        # Print final summary
        self._print_batch_summary(results)

        return results

    def _print_batch_summary(self, results: Dict):
        """Print batch processing summary"""
        logger.info("\n" + "="*60)
        logger.info(" Overall processing summary")
        logger.info("="*60)
```

```
    # Videos summary
    videos = results['videos']
    if videos:
        successful_videos = len([v for v in videos if 'error' not in v])
        total_frames = sum(v.get('statistics', {}).get('processed_frames', 0)
                           for v in videos if 'error' not in v)
        total_detections = sum(v.get('statistics', {}).get('total_detections', 0)
                               for v in videos if 'error' not in v)
        total_poses = sum(v.get('statistics', {}).get('total_poses', 0)
                          for v in videos if 'error' not in v)

        logger.info(f" Videos:")
        logger.info(f"   - Processed: {successful_videos}/{len(videos)}")
        logger.info(f"   - Total frames: {total_frames}")
        logger.info(f"   - Total detections: {total_detections}")
        logger.info(f"   - Total poses: {total_poses}")

    # Images summary
    images = results['images']
    if images:
        successful_images = len([i for i in images if 'error' not in i])
        total_detections = sum(i.get('num_detections', 0)
                               for i in images if 'error' not in i)
        total_poses = sum(i.get('num_poses', 0)
                          for i in images if 'error' not in i)

        logger.info(f"\n  Images:")
        logger.info(f"   - Processed: {successful_images}/{len(images)}")
        logger.info(f"   - Total detections: {total_detections}")
        logger.info(f"   - Total poses: {total_poses}")

    logger.info(f"\n Output directory: {results['output_dir']}")
    logger.info("="*60 + "\n")


# Test function
```

```
if __name__ == "__main__":
    print(" Testing Video Processor...\n")

    try:
        from models import initialize_models

        # Initialize models
        detector_model, pose_model, _ = initialize_models()

        # Create detector and pose estimator
        from detector import PlayerDetector
        from pose_estimator import PoseEstimator

        player_detector = PlayerDetector(detector_model)
        pose_estimator = PoseEstimator(pose_model)

        # Create processors
        video_processor = VideoProcessor(player_detector, pose_estimator)
        image_processor = ImageProcessor(player_detector, pose_estimator)
        batch_processor = BatchProcessor(player_detector, pose_estimator)

        print("\n Video Processor tested successfully!")
        print("\nAvailable processors:")
        print("  - VideoProcessor: for processing videos")
        print("  - ImageProcessor: for processing images")
        print("  - BatchProcessor: for processing entire directories")

    except Exception as e:
        print(f"\n Test failed: {str(e)}")
        sys.exit(1)
```

```
"""
Main entry point for Football Pose Estimation Project
Provides command-line interface and easy-to-use functions
```

```
"""

import argparse
import sys
import logging
from pathlib import Path
from typing import Optional, Union

# Import all modules
try:
    from config import (
        INPUT_DIR,
        OUTPUT_DIR,
        LOGGING_CONFIG,
        print_config_summary
    )
    from models import initialize_models
    from detector import PlayerDetector
    from pose_estimator import PoseEstimator
    from video_processor import VideoProcessor, ImageProcessor,
BatchProcessor
    from utils import is_video_file, is_image_file
except ImportError as e:
    print(f" Import error: {str(e)}")
    print("Make sure all required files are present")
    sys.exit(1)

# Setup logging
logging.basicConfig(
    level=getattr(logging, LOGGING_CONFIG["log_level"]),
    format=LOGGING_CONFIG["log_format"],
    handlers=[
        logging.FileHandler(LOGGING_CONFIG["log_file"]),
        logging.StreamHandler()
    ]
)
logger = logging.getLogger(__name__)
```

```
class PoseEstimationApp:
    """
    Main application class for pose estimation
    """

    def __init__(self):
        """Initialize the application"""
        self.detector_model = None
        self.pose_model = None
        self.model_manager = None
        self.player_detector = None
        self.pose_estimator = None
        self.video_processor = None
        self.image_processor = None
        self.batch_processor = None

        self.initialized = False

    def initialize(self):
        """Initialize all models and processors"""
        if self.initialized:
            logger.info("  Application is already initialized")
            return

        try:
            logger.info("\n" + "="*60)
            logger.info(" Initializing Pose Estimation Application")
            logger.info("="*60)

            # Initialize models
            logger.info("\n Loading models...")
            self.detector_model, self.pose_model, self.model_manager =
initialize_models()

            # Create detector and pose estimator
```

```
        logger.info("\n Initializing processors...")
        self.player_detector = PlayerDetector(self.detector_model)
        self.pose_estimator = PoseEstimator(self.pose_model)

        # Create processors
        self.video_processor = VideoProcessor(
            self.player_detector,
            self.pose_estimator,
            enable_tracking=True
        )
        self.image_processor = ImageProcessor(
            self.player_detector,
            self.pose_estimator
        )
        self.batch_processor = BatchProcessor(
            self.player_detector,
            self.pose_estimator,
            enable_tracking=True
        )

        self.initialized = True

        logger.info("\n" + "="*60)
        logger.info(" Application initialized successfully!")
        logger.info("="*60 + "\n")

    except Exception as e:
        logger.error(f"\n Initialization failed: {str(e)}")
        raise

def process_video(self, video_path: Union[str, Path],
                  output_path: Optional[Union[str, Path]] = None,
                  save_data: bool = True,
                  show_preview: bool = False):
    """
    Process a video file
```

```
    Args:
        video_path: Path to input video
        output_path: Path to output video (optional)
        save_data: Whether to save pose data
        show_preview: Whether to show live preview
    """
    if not self.initialized:
        self.initialize()

    video_path = Path(video_path)

    if not video_path.exists():
        raise FileNotFoundError(f"Video not found: {video_path}")

    if not is_video_file(video_path):
        raise ValueError(f"The file is not a valid video: {video_path}")

    return self.video_processor.process_video(
        video_path, output_path, save_data, show_preview
    )

def process_image(self, image_path: Union[str, Path],
                  output_path: Optional[Union[str, Path]] = None,
                  save_data: bool = True):
    """
    Process a single image

    Args:
        image_path: Path to input image
        output_path: Path to output image (optional)
        save_data: Whether to save pose data
    """
    if not self.initialized:
        self.initialize()

    image_path = Path(image_path)
```

```
    if not image_path.exists():
        raise FileNotFoundError(f"Image not found: {image_path}")

    if not is_image_file(image_path):
        raise ValueError(f"The file is not a valid image: {image_path}")

    return self.image_processor.process_image(
        image_path, output_path, save_data
    )

def process_directory(self, input_dir: Union[str, Path],
                      output_dir: Optional[Union[str, Path]] = None,
                      process_videos: bool = True,
                      process_images: bool = True):
    """
    Process all files in a directory

    Args:
        input_dir: Input directory path
        output_dir: Output directory path (optional)
        process_videos: Whether to process videos
        process_images: Whether to process images
    """
    if not self.initialized:
        self.initialize()

    input_dir = Path(input_dir)

    if not input_dir.exists() or not input_dir.is_dir():
        raise ValueError(f"Directory not found: {input_dir}")

    return self.batch_processor.process_directory(
        input_dir, output_dir, process_videos, process_images
    )

def process_file(self, file_path: Union[str, Path],
                 output_path: Optional[Union[str, Path]] = None,
```

```
                save_data: bool = True,
                show_preview: bool = False):
        """
        Automatically detect and process file (video or image)

        Args:
            file_path: Path to input file
            output_path: Path to output file (optional)
            save_data: Whether to save data
            show_preview: Whether to show preview (for videos)
        """
        file_path = Path(file_path)

        if not file_path.exists():
            raise FileNotFoundError(f"File not found: {file_path}")

        if is_video_file(file_path):
            logger.info(" Video detected")
            return self.process_video(file_path, output_path, save_data,
show_preview)
        elif is_image_file(file_path):
            logger.info("  Image detected")
            return self.process_image(file_path, output_path, save_data)
        else:
            raise ValueError(f"Unsupported file type: {file_path}")


def main():
    """Main function with command-line interface"""

    # Print banner
    print_banner()

    # Parse arguments
    parser = argparse.ArgumentParser(
        description=" Football Pose Estimation - Analyze player poses in football
matches",
```

```
        formatter_class=argparse.RawDescriptionHelpFormatter,
        epilog="""
Usage examples:
  # Process a video
  python main.py --video input/match.mp4

  # Process an image
  python main.py --image input/player.jpg

  # Auto process a file
  python main.py --file input/match.mp4

  # Process an entire directory
  python main.py --directory input/

  # Process with live preview
  python main.py --video input/match.mp4 --preview

  # Process without saving data
  python main.py --video input/match.mp4 --no-save
        """
    )

    # Input options
    input_group = parser.add_mutually_exclusive_group(required=False)
    input_group.add_argument(
        '--video', '-v',
        type=str,
        help='Path to video file for processing'
    )
    input_group.add_argument(
        '--image', '-i',
        type=str,
        help='Path to image file for processing'
    )
    input_group.add_argument(
        '--file', '-f',
```

```
    type=str,
    help='Path to file (type will be auto-detected)'
)
input_group.add_argument(
    '--directory', '-d',
    type=str,
    help='Path to directory to process all files'
)

# Output options
parser.add_argument(
    '--output', '-o',
    type=str,
    help='Output file/directory path (optional)',
    default=None
)

# Processing options
parser.add_argument(
    '--no-save',
    action='store_true',
    help='Do not save pose data (JSON/CSV)'
)
parser.add_argument(
    '--preview', '-p',
    action='store_true',
    help='Show live preview for videos during processing'
)
parser.add_argument(
    '--no-videos',
    action='store_true',
    help='Skip video processing when processing a directory'
)
parser.add_argument(
    '--no-images',
    action='store_true',
    help='Skip image processing when processing a directory'
```

```
    )

    # Interactive mode
    parser.add_argument(
        '--interactive',
        action='store_true',
        help='Run in interactive mode'
    )

    args = parser.parse_args()

    # Interactive mode
    if args.interactive or (not any([args.video, args.image, args.file,
args.directory])):
        run_interactive_mode()
        return

    try:
        # Create app instance
        app = PoseEstimationApp()

        # Process based on arguments
        if args.video:
            app.process_video(
                args.video,
                args.output,
                save_data=not args.no_save,
                show_preview=args.preview
            )

        elif args.image:
            app.process_image(
                args.image,
                args.output,
                save_data=not args.no_save
            )
```

```
        elif args.file:
            app.process_file(
                args.file,
                args.output,
                save_data=not args.no_save,
                show_preview=args.preview
            )

        elif args.directory:
            app.process_directory(
                args.directory,
                args.output,
                process_videos=not args.no_videos,
                process_images=not args.no_images
            )

        logger.info("\n Processing completed successfully!")

    except KeyboardInterrupt:
            logger.info("\n\n Processing was stopped by the user")
            sys.exit(0)

    except Exception as e:
            logger.error(f"\n An error occurred: {str(e)}")
            sys.exit(1)


def run_interactive_mode():
    """Run interactive mode with menu"""

    print("\n" + "="*60)
    print(" Interactive Mode")
    print("="*60)

    # Create app instance
    app = PoseEstimationApp()
```

```
while True:
    print("\n Main Menu:")
    print("  [1] Process Video")
    print("  [2] Process Image")
    print("  [3] Auto Process File")
    print("  [4] Process Directory")
    print("  [5] Show Settings")
    print("  [0] Exit")
    print()

    choice = input("Select an option by number: ").strip()

    try:
        if choice == '1':
            process_video_interactive(app)

        elif choice == '2':
            process_image_interactive(app)

        elif choice == '3':
            process_file_interactive(app)

        elif choice == '4':
            process_directory_interactive(app)

        elif choice == '5':
            print_config_summary()
            if app.initialized:
                app.model_manager.print_model_info()

        elif choice == '0':
            print("\n Goodbye!")
            break

        else:
            print(" Invalid choice, please try again")
```

```
        except KeyboardInterrupt:
            print("\n\n  Operation cancelled")
            continue

        except Exception as e:
            logger.error(f"\n An error occurred: {str(e)}")
            continue


def process_video_interactive(app: PoseEstimationApp):
    """Interactive video processing"""
    print("\n" + "="*60)
    print(" Process Video")
    print("="*60)

    # Get video path
    video_path = input("\nEnter the video path: ").strip()

    if not video_path:
        print(" No path entered")
        return

    # Get output path
    output_path = input("Enter the output path (leave empty for default):
").strip()
    if not output_path:
        output_path = None

    # Ask for preview
    preview = input("Show live preview? (y/n) [n]: ").strip().lower() == 'y'

    # Ask for data saving
    save_data = input("Save pose data? (y/n) [y]: ").strip().lower() != 'n'

    # Process
    print("\n Processing...")
    result = app.process_video(video_path, output_path, save_data, preview)
```

```
    print(f"\n Processing completed!")
    print(f" Output file: {result['output_path']}")


def process_image_interactive(app: PoseEstimationApp):
    """Interactive image processing"""
    print("\n" + "="*60)
    print("  Process Image")
    print("="*60)

    # Get image path
    image_path = input("\nEnter the image path: ").strip()

    if not image_path:
        print(" No path entered")
        return

    # Get output path
    output_path = input("Enter the output path (leave empty for default):
").strip()
    if not output_path:
        output_path = None

    # Ask for data saving
    save_data = input("Save pose data? (y/n) [y]: ").strip().lower() != 'n'

    # Process
    print("\n Processing...")
    result = app.process_image(image_path, output_path, save_data)

    print(f"\n Processing completed!")
    print(f" Output file: {result['output_path']}")
    print(f" Number of players: {result['num_detections']}")
    print(f" Number of poses: {result['num_poses']}")
```

```
def process_file_interactive(app: PoseEstimationApp):
    """Interactive file processing (auto-detect)"""
    print("\n" + "="*60)
    print(" Auto Process File")
    print("="*60)

    # Get file path
    file_path = input("\nEnter the file path: ").strip()

    if not file_path:
        print(" No path entered")
        return

    # Get output path
    output_path = input("Enter the output path (leave empty for default):
").strip()
    if not output_path:
        output_path = None

    # Ask for preview (for videos)
    preview = input("Show live preview? (videos only) (y/n) [n]: ").strip().lower()
== 'y'

    # Ask for data saving
    save_data = input("Save pose data? (y/n) [y]: ").strip().lower() != 'n'

    # Process
    print("\n Processing...")
    result = app.process_file(file_path, output_path, save_data, preview)

    print(f"\n Processing completed!")
    print(f" Output file: {result['output_path']}")


def process_directory_interactive(app: PoseEstimationApp):
    """Interactive directory processing"""
    print("\n" + "="*60)
```

```
    print(" Process Directory")
    print("="*60)

    # Get directory path
    dir_path = input("\nEnter the directory path: ").strip()

    if not dir_path:
        print(" No path entered")
        return

    # Get output path
    output_path = input("Enter the output directory path (leave empty for
default): ").strip()
    if not output_path:
        output_path = None

    # Ask what to process
    process_videos = input("Process videos? (y/n) [y]: ").strip().lower() != 'n'
    process_images = input("Process images? (y/n) [y]: ").strip().lower() != 'n'

    # Process
    print("\n Processing...")
    result = app.process_directory(dir_path, output_path, process_videos,
process_images)

    print(f"\n Processing completed!")
    print(f" Output directory: {result['output_dir']}")


def print_banner():
    """Print application banner"""
    banner = """

╔═══════════════════════════════════════════════════════════════════════════
═══════════╗
    ║                                                                       ║
    ║          Football Pose Estimation System                              ║
```

```
    ╠                                                    ╣
    ╠        Analyze player poses in football matches    ╣
    ╠                                                    ╣
╚═══════════════════════════════════════════════════════════════
═══════════╝
    """
    print(banner)
    print(f"    Input directory: {INPUT_DIR}")
    print(f"    Output directory: {OUTPUT_DIR}")
    print()


# Simple usage functions for direct import
def process_video(video_path: str, output_path: str = None):
    """
    Simple function to process a video

    Args:
        video_path: Path to video file
        output_path: Output path (optional)
    """
    app = PoseEstimationApp()
    return app.process_video(video_path, output_path)


def process_image(image_path: str, output_path: str = None):
    """
    Simple function to process an image

    Args:
        image_path: Path to image file
        output_path: Output path (optional)
    """
    app = PoseEstimationApp()
    return app.process_image(image_path, output_path)
```

```
if __name__ == "__main__":
    main()
```

```
# ==================== Requirements for Python 3.11.9
# Tested and verified for full compatibility
# Install order: PyTorch first, then these requirements

# Core Dependencies
numpy==1.26.4
opencv-python==4.9.0.80
opencv-contrib-python==4.9.0.80
Pillow==10.2.0

# Deep Learning & Computer Vision
# NOTE: Install torch separately first (see instructions below)
# torch>=2.1.0  # Will be installed separately
# torchvision>=0.16.0  # Will be installed separately
ultralytics==8.1.20
mediapipe==0.10.11

# Data Processing
pandas==2.2.1
scipy==1.12.0

# Visualization
matplotlib==3.8.3
seaborn==0.13.2

# Progress Bar
tqdm==4.66.2

# Video Processing
imageio==2.34.0
imageio-ffmpeg==0.4.9
```

```
# File I/O
PyYAML==6.0.1

# Utilities
requests==2.31.0
certifi>=2023.7.22

# Optional (for Jupyter notebooks)
# jupyter==1.0.0
# notebook==7.1.0
# ipywidgets==8.1.2
```

Appendix C : Real-Time Football Analytics: Player Detection, Tracking, and Pose Estimation via RTMPose

```
"""
Main entry point for RTMPose + ByteTrack Pose Estimator
"""

import sys
```

```
from pathlib import Path
import argparse

# Import classes
from pose_estimator import RTMPoseEstimator
from config import Config
from app import PoseEstimatorApp


def quick_test():
    """Quick system test"""
    print("\n🧪 Running quick test...")

    try:
        # Print paths
        Config.print_paths()
        Config.print_model_info()

        # Create Estimator object
        estimator = RTMPoseEstimator()
        print("✅ System is ready!")
        return True
    except Exception as e:
        print(f"❌ Test failed: {e}")
        import traceback
        traceback.print_exc()
        return False


def run_cli():
    """Run from command line"""
    parser = argparse.ArgumentParser(
        description='RTMPose + ByteTrack Pose Estimation Tool',
        formatter_class=argparse.RawDescriptionHelpFormatter,
        epilog="""
Usage examples:
  python main.py --image path/to/image.jpg
```

```
  python main.py --video path/to/video.mp4
  python main.py --camera
  python main.py --det-conf 0.7 --pose-conf 0.6 --image test.jpg
  python main.py --output custom/path/result.jpg --image test.jpg
      """
    )

    # Arguments
    parser.add_argument('--image', '-i', type=str, help='Image path')
    parser.add_argument('--video', '-v', type=str, help='Video path')
    parser.add_argument('--camera', '-c', action='store_true', help='Use
camera')

    parser.add_argument('--det-conf', type=float, default=None,
                        help='Detection confidence threshold (0.0-1.0)')
    parser.add_argument('--pose-conf', type=float, default=None,
                        help='Pose confidence threshold (0.0-1.0)')

    parser.add_argument('--output', '-o', type=str, help='Output path (optional)')
    parser.add_argument('--no-save', action='store_true', help='Do not save
results')
    parser.add_argument('--no-show', action='store_true', help='Do not show
results')
    parser.add_argument('--no-track', action='store_true', help='Disable
tracking')

    parser.add_argument('--device', type=str, default=None,
                        help='Device to use (cuda:0 or cpu)')

    parser.add_argument('--interactive', action='store_true', help='Interactive
mode')
    parser.add_argument('--show-paths', action='store_true', help='Show all
paths and exit')
    parser.add_argument('--test', action='store_true', help='Run system test')

    args = parser.parse_args()
```

```
# Show paths and exit
if args.show_paths:
    Config.print_paths()
    Config.print_model_info()
    return

# Run test
if args.test:
    quick_test()
    return

# Create Estimator object
estimator = RTMPoseEstimator(
    det_conf=args.det_conf,
    pose_conf=args.pose_conf,
    device=args.device
)

# Disable tracking if requested
if args.no_track:
    estimator.tracker = None
    print("⚠️  Tracking disabled")

# Process image
if args.image:
    if not Path(args.image).exists():
        print(f"❌ Image not found: {args.image}")
        sys.exit(1)

    estimator.predict_image(
        image_path=args.image,
        save_result=not args.no_save,
        output_path=args.output
    )

# Process video
elif args.video:
```

```
        if not Path(args.video).exists():
            print(f"❌ Video not found: {args.video}")
            sys.exit(1)

        estimator.predict_video(
            video_path=args.video,
            save_result=not args.no_save,
            output_path=args.output,
            show_live=not args.no_show
        )

    # Use camera
    elif args.camera:
        estimator.predict_video(
            video_path=0,
            save_result=not args.no_save,
            output_path=args.output,
            show_live=True
        )

    # Interactive mode
    elif args.interactive:
        app = PoseEstimatorApp()
        app.run_interactive()

    else:
        parser.print_help()


def print_banner():
    """Print application banner"""
    banner = """
╔═══════════════════════════════════════════════════════════════════════════════╗
║                                                                               ║
║      🤸 RTMPose + ByteTrack Pose Estimation 🤸                                ║
║                                                                               ║
```

```
    ╔═══════════════════════════════════════════════════════════════╗
    ║     Professional Pose Estimation with Tracking                ║
    ║        Powered by OpenMMLab & ByteTrack                       ║
    ╚═══════════════════════════════════════════════════════════════╝
    """
    print(banner)
    Config.print_paths()


def show_menu():
    """Display main menu"""
    print("\n" + "="*60)
    print("📋 Choose operation mode:")
    print("="*60)
    print("1. Interactive Mode")
    print("2. Quick Image Processing")
    print("3. Quick Video Processing")
    print("4. Run Camera")
    print("5. Test System")
    print("6. Show Help")
    print("7. Show Paths & Model Info")
    print("0. Exit")
    print("="*60)


def main():
    """Main function"""
    # Print banner
    print_banner()

    # If command line arguments exist
    if len(sys.argv) > 1:
        run_cli()
        return

    # Show main menu
```

```
while True:
    show_menu()
    choice = input("\n👉 Your choice: ").strip()

    if choice == '1':
        # Interactive mode
        app = PoseEstimatorApp()
        app.run_interactive()

    elif choice == '2':
        # Process image
        path = input("📁 Enter image path: ").strip()
        if Path(path).exists():
            estimator = RTMPoseEstimator()
            estimator.predict_image(path)
        else:
            print("❌ File not found!")

    elif choice == '3':
        # Process video
        path = input("📁 Enter video path: ").strip()
        if Path(path).exists():
            estimator = RTMPoseEstimator()
            estimator.predict_video(path)
        else:
            print("❌ File not found!")

    elif choice == '4':
        # Camera
        print("📹 Opening camera... (Press Q to exit)")
        estimator = RTMPoseEstimator()
        estimator.predict_video(0)

    elif choice == '5':
        # Test system
        quick_test()
```

```
        elif choice == '6':
            # Help
            print_help()

        elif choice == '7':
            # Show paths and model info
            Config.print_paths()
            Config.print_model_info()

        elif choice == '0':
            print("\n👋 Thank you for using the application!")
            break

        else:
            print("❌ Invalid choice!")

        input("\n⏸️  Press Enter to continue...")


def print_help():
    """Print help documentation"""
    help_text = """
╔═══════════════════════════════════════════════════════════════════════════════╗
║                              📖 User Guide                                    ║
╚═══════════════════════════════════════════════════════════════════════════════╝

🔹 Command Line Usage:
─────────────────────────────────────────────────────────────────────────────────

# Process image (auto-saves to output/images/)
python main.py --image path/to/image.jpg

# Process video (auto-saves to output/videos/)
```

```
python main.py --video path/to/video.mp4

# Use camera (auto-saves with timestamp)
python main.py --camera

# Set confidence thresholds
python main.py --det-conf 0.7 --pose-conf 0.6 --video test.mp4

# Custom output path
python main.py --image test.jpg --output custom/path/result.jpg

# Do not save results
python main.py --video test.mp4 --no-save

# Do not show results
python main.py --video test.mp4 --no-show

# Disable tracking (pose only)
python main.py --video test.mp4 --no-track

# Use CPU instead of GPU
python main.py --device cpu --image test.jpg

# Show all paths
python main.py --show-paths

# Test system
python main.py --test

# Interactive mode
python main.py --interactive
```

🔹 Available Models:

——

Detection (RTMDet):
  - rtmdet-m → Medium (Default)

Pose Estimation (RTMPose):
  - rtmpose-t → Tiny (Fast)
  - rtmpose-s → Small
  - rtmpose-m → Medium (Default) ✓
  - rtmpose-l → Large

Tracking:
  - ByteTrack v2 (Always enabled)

ℹ️ Models are automatically downloaded on first use

🔹 Output Locations:
──

Images: output/images/output_[filename]
Videos: output/videos/output_[filename]
Camera: output/videos/camera_output_[timestamp].mp4
JSON: output/json/
CSV: output/csv/

🔹 Keyboard Controls:
──

Q → Exit video processing

🔹 Directory Structure:
──

```
project/
├── models/
│   ├── detection/      ← RTMDet models
│   └── pose/           ← RTMPose models
├── configs/
│   ├── detection/      ← Detection configs
│   └── pose/           ← Pose configs
├── input/
│   ├── images/         ← Place input images here
│   └── videos/         ← Place input videos here
└── output/
    ├── images/         ← Processed images
    ├── videos/         ← Processed videos
    ├── json/           ← JSON results
    └── csv/            ← CSV results
```

🔹 Installation:

```
# Install dependencies
pip install -r requirements.txt

# Install OpenMMLab packages using mim
pip install openmim
mim install mmdet mmpose
```

🔹 Practical Examples:

```
# Simple run without parameters
python main.py

# Process with high confidence
```

```
python main.py --det-conf 0.8 --pose-conf 0.7 --video dance.mp4

# Quick camera recording with tracking
python main.py --camera

# Process without saving (preview only)
python main.py --video test.mp4 --no-save

# Process on CPU
python main.py --device cpu --image photo.jpg
```

🔹 Features:
━━

✓ Multi-person pose estimation
✓ Real-time person tracking (ByteTrack)
✓ 17 COCO keypoints detection
✓ GPU acceleration support
✓ Video and image processing
✓ Live camera support
✓ Tracking ID persistence
✓ High accuracy with RTMPose

🔹 For More Help:
━━

```
python main.py --help

╚═══════════════════════════════════════════════════════════════════════
═══════════════════╝
    """
    print(help_text)
```

```
if __name__ == "__main__":
    try:
        main()
    except KeyboardInterrupt:
        print("\n\n⚠️  Program stopped by user")
        sys.exit(0)
    except Exception as e:
        print(f"\n❌ Unexpected error occurred: {e}")
        import traceback
        traceback.print_exc()
        sys.exit(1)
```

```
"""
Configuration file for RTMPose + ByteTrack Pose Estimator
"""

from pathlib import Path
import os

class Config:
    """Project configuration"""

    # Base directory
    BASE_DIR = Path(__file__).parent.absolute()

    # ============================================
    # RTMPose Model Settings
    # ============================================
    # Available models:
    # - rtmpose-t (tiny)
    # - rtmpose-s (small)
    # - rtmpose-m (medium) ← نستخدم ده
    # - rtmpose-l (large)

    POSE_MODEL_NAME = 'rtmpose-m'
```

```
    POSE_CONFIG = 'rtmpose-m_8xb256-420e_coco-256x192.py'
    POSE_CHECKPOINT =
'rtmpose-m_simcc-aic-coco_pt-aic-coco_420e-256x192-63eb25f7_20230126.
pth'

    # ============================================
    # Detection Model Settings (for person detection)
    # ============================================
    # نستخدم RTMDet لكشف الأشخاص
    DET_MODEL_NAME = 'rtmdet-m'
    DET_CONFIG = 'rtmdet_m_8xb32-300e_coco.py'
    DET_CHECKPOINT =
'rtmdet_m_8xb32-300e_coco_20220719_112220-229f527c.pth'

    # ============================================
    # ByteTrack Settings
    # ============================================
    TRACK_THRESH = 0.5         # High threshold for first association
    TRACK_BUFFER = 30          # Number of frames to keep lost tracks
    MATCH_THRESH = 0.8         # IOU threshold for matching
    MIN_BOX_AREA = 10          # Minimum box area
    MOT20 = False              # Whether to use MOT20 settings
    FRAME_RATE = 30            # Assumed frame rate for tracking

    # ============================================
    # Confidence Thresholds
    # ============================================
    DET_CONFIDENCE = 0.5       # Detection confidence
    POSE_CONFIDENCE = 0.5      # Keypoint confidence

    # ============================================
    # Main Directories
    # ============================================
    MODELS_DIR = BASE_DIR / 'models'
    CONFIGS_DIR = BASE_DIR / 'configs'
    INPUT_DIR = BASE_DIR / 'input'
    OUTPUT_DIR = BASE_DIR / 'output'
```

```
# Model subdirectories
DET_MODELS_DIR = MODELS_DIR / 'detection'
POSE_MODELS_DIR = MODELS_DIR / 'pose'

# Config subdirectories
DET_CONFIGS_DIR = CONFIGS_DIR / 'detection'
POSE_CONFIGS_DIR = CONFIGS_DIR / 'pose'

# Input subdirectories
INPUT_IMAGES_DIR = INPUT_DIR / 'images'
INPUT_VIDEOS_DIR = INPUT_DIR / 'videos'

# Output subdirectories
OUTPUT_IMAGES_DIR = OUTPUT_DIR / 'images'
OUTPUT_VIDEOS_DIR = OUTPUT_DIR / 'videos'
OUTPUT_JSON_DIR = OUTPUT_DIR / 'json'
OUTPUT_CSV_DIR = OUTPUT_DIR / 'csv'

# ==========================================
# Display Settings
# ==========================================
SHOW_LIVE = True
SAVE_RESULTS = True

# Visualization settings
SKELETON_COLOR = (0, 255, 0)       # Green
KEYPOINT_COLOR = (0, 0, 255)       # Red
BBOX_COLOR = (255, 0, 0)           # Blue
TRACK_ID_COLOR = (255, 255, 0)     # Yellow

KEYPOINT_RADIUS = 5
SKELETON_THICKNESS = 2
BBOX_THICKNESS = 2
TEXT_THICKNESS = 2
TEXT_SCALE = 0.8
```

```
# ============================================
# COCO Keypoint Definitions (17 keypoints)
# ============================================
KEYPOINT_NAMES = [
    'nose',          # 0
    'left_eye',      # 1
    'right_eye',     # 2
    'left_ear',      # 3
    'right_ear',     # 4
    'left_shoulder',  # 5
    'right_shoulder', # 6
    'left_elbow',    # 7
    'right_elbow',   # 8
    'left_wrist',    # 9
    'right_wrist',   # 10
    'left_hip',      # 11
    'right_hip',     # 12
    'left_knee',     # 13
    'right_knee',    # 14
    'left_ankle',    # 15
    'right_ankle'    # 16
]

# ============================================
# ✅ FIXED: Skeleton connections (COCO format - 0-indexed)
# ============================================
SKELETON_LINKS = [
    # Legs
    [15, 13], [13, 11],  # left leg: ankle->knee->hip
    [16, 14], [14, 12],  # right leg: ankle->knee->hip
    [11, 12],            # hips connection

    # Torso
    [5, 11],   # left shoulder -> left hip
    [6, 12],   # right shoulder -> right hip
    [5, 6],    # shoulders connection
```

```
    # Arms
    [5, 7],    # left shoulder -> left elbow
    [7, 9],    # left elbow -> left wrist
    [6, 8],    # right shoulder -> right elbow
    [8, 10],   # right elbow -> right wrist

    # Head
    [0, 1],    # nose -> left eye
    [0, 2],    # nose -> right eye
    [1, 3],    # left eye -> left ear
    [2, 4],    # right eye -> right ear
    [3, 5],    # left ear -> left shoulder
    [4, 6]     # right ear -> right shoulder
]

# Colors for each keypoint (RGB)
KEYPOINT_COLORS = [
    (255, 0, 0),     # nose - red
    (255, 85, 0),    # left_eye - orange
    (255, 170, 0),   # right_eye - yellow-orange
    (255, 255, 0),   # left_ear - yellow
    (170, 255, 0),   # right_ear - yellow-green
    (85, 255, 0),    # left_shoulder - green
    (0, 255, 0),     # right_shoulder - green
    (0, 255, 85),    # left_elbow - cyan-green
    (0, 255, 170),   # right_elbow - cyan
    (0, 255, 255),   # left_wrist - cyan
    (0, 170, 255),   # right_wrist - light blue
    (0, 85, 255),    # left_hip - blue
    (0, 0, 255),     # right_hip - blue
    (85, 0, 255),    # left_knee - purple
    (170, 0, 255),   # right_knee - purple
    (255, 0, 255),   # left_ankle - magenta
    (255, 0, 170)    # right_ankle - pink
]

# ============================================
```

```
    # Device Settings
    # ============================================
    DEVICE = 'cuda:0'  # or 'cpu' if no GPU

    # ============================================
    # Performance Settings
    # ============================================
    BATCH_SIZE = 1
    NUM_WORKERS = 4

    # ============================================
    # Model URLs (للتحميل التلقائي)
    # ============================================
    MODEL_URLS = {
        'rtmpose-m': {
            'config':
'https://github.com/open-mmlab/mmpose/tree/main/projects/rtmpose/rtmpose/
body_2d_keypoint/rtmpose-m_8xb256-420e_coco-256x192.py',
            'checkpoint':
'https://download.openmmlab.com/mmpose/v1/projects/rtmposev1/rtmpose-m
_simcc-aic-coco_pt-aic-coco_420e-256x192-63eb25f7_20230126.pth'
        },
        'rtmdet-m': {
            'config':
'https://github.com/open-mmlab/mmdetection/tree/main/configs/rtmdet/rtmdet
_m_8xb32-300e_coco.py',
            'checkpoint':
'https://download.openmmlab.com/mmdetection/v3.0/rtmdet/rtmdet_m_8xb32
-300e_coco/rtmdet_m_8xb32-300e_coco_20220719_112220-229f527c.pth'
        }
    }

    # ============================================
    # Methods
    # ============================================

    @classmethod
```

```
def get_det_model_path(cls) -> tuple:
    """
    Get detection model config and checkpoint paths

    Returns:
        tuple: (config_path, checkpoint_path)
    """
    config_path = cls.DET_CONFIGS_DIR / cls.DET_CONFIG
    checkpoint_path = cls.DET_MODELS_DIR / cls.DET_CHECKPOINT
    return config_path, checkpoint_path

@classmethod
def get_pose_model_path(cls) -> tuple:
    """
    Get pose model config and checkpoint paths

    Returns:
        tuple: (config_path, checkpoint_path)
    """
    config_path = cls.POSE_CONFIGS_DIR / cls.POSE_CONFIG
    checkpoint_path = cls.POSE_MODELS_DIR / cls.POSE_CHECKPOINT
    return config_path, checkpoint_path

@classmethod
def create_directories(cls):
    """Create all required directories"""
    directories = [
        cls.MODELS_DIR,
        cls.DET_MODELS_DIR,
        cls.POSE_MODELS_DIR,
        cls.CONFIGS_DIR,
        cls.DET_CONFIGS_DIR,
        cls.POSE_CONFIGS_DIR,
        cls.INPUT_IMAGES_DIR,
        cls.INPUT_VIDEOS_DIR,
        cls.OUTPUT_IMAGES_DIR,
        cls.OUTPUT_VIDEOS_DIR,
```

```
        cls.OUTPUT_JSON_DIR,
        cls.OUTPUT_CSV_DIR
    ]

    for directory in directories:
        directory.mkdir(parents=True, exist_ok=True)

    print("✅ All directories created successfully")
    print(f"📁 Detection models: {cls.DET_MODELS_DIR}")
    print(f"📁 Pose models: {cls.POSE_MODELS_DIR}")
    print(f"📁 Output images: {cls.OUTPUT_IMAGES_DIR}")
    print(f"📁 Output videos: {cls.OUTPUT_VIDEOS_DIR}")

@classmethod
def print_paths(cls):
    """Print all important paths"""
    print("\n" + "="*60)
    print("📂 Project Paths:")
    print("="*60)
    print(f"Base Directory: {cls.BASE_DIR}")
    print(f"\n🔍 Detection Model:")
    print(f"  Config: {cls.DET_CONFIGS_DIR}")
    print(f"  Checkpoint: {cls.DET_MODELS_DIR}")
    print(f"\n🤸 Pose Model:")
    print(f"  Config: {cls.POSE_CONFIGS_DIR}")
    print(f"  Checkpoint: {cls.POSE_MODELS_DIR}")
    print(f"\n📥 Input:")
    print(f"  Images: {cls.INPUT_IMAGES_DIR}")
    print(f"  Videos: {cls.INPUT_VIDEOS_DIR}")
    print(f"\n📤 Output:")
    print(f"  Images: {cls.OUTPUT_IMAGES_DIR}")
    print(f"  Videos: {cls.OUTPUT_VIDEOS_DIR}")
    print(f"  JSON: {cls.OUTPUT_JSON_DIR}")
    print(f"  CSV: {cls.OUTPUT_CSV_DIR}")
    print("="*60 + "\n")

@classmethod
```

```
    def print_model_info(cls):
        """Print model information"""
        print("\n" + "="*60)
        print("🤖 Model Configuration:")
        print("="*60)
        print(f"Detection Model: {cls.DET_MODEL_NAME}")
        print(f"Pose Model: {cls.POSE_MODEL_NAME}")
        print(f"Detection Confidence: {cls.DET_CONFIDENCE}")
        print(f"Pose Confidence: {cls.POSE_CONFIDENCE}")
        print(f"\n🎯 ByteTrack Settings:")
        print(f"  Track Threshold: {cls.TRACK_THRESH}")
        print(f"  Track Buffer: {cls.TRACK_BUFFER}")
        print(f"  Match Threshold: {cls.MATCH_THRESH}")
        print(f"  Min Box Area: {cls.MIN_BOX_AREA}")
        print(f"\n💻 Device: {cls.DEVICE}")
        print("="*60 + "\n")

    @classmethod
    def validate_device(cls):
        """Validate and set device"""
        import torch
        if cls.DEVICE.startswith('cuda') and not torch.cuda.is_available():
            print("⚠️  CUDA not available, switching to CPU")
            cls.DEVICE = 'cpu'
        return cls.DEVICE

# Create directories when importing
Config.create_directories()


---

"""
RTMPose + ByteTrack Pose Estimator Class
Professional class for pose estimation with tracking
"""

import cv2
import numpy as np
```

```
from pathlib import Path
from typing import Union, Optional, List, Dict, Tuple
import time
import torch

# --- Essential Imports ---
from mmengine.registry import init_default_scope
import mmdet.datasets
from mmdet.utils import register_all_modules as register_mmdet_modules
from mmpose.utils import register_all_modules as register_mmpose_modules

# Ensure PackDetInputs is imported
try:
    from mmdet.datasets.transforms import PackDetInputs
except ImportError:
    pass
# -------------------------

from config import Config
from utils import PerformanceMonitor, VisualizationHelper, FileManager

class RTMPoseEstimator:
    """
    Professional class for pose estimation using RTMPose + ByteTrack
    """

    def __init__(self,
                 det_model_name: str = None,
                 pose_model_name: str = None,
                 det_conf: float = None,
                 pose_conf: float = None,
                 device: str = None):
        """
        Initialize the models
        """
        # Set configurations
```

```
        self.det_model_name = det_model_name or
Config.DET_MODEL_NAME
        self.pose_model_name = pose_model_name or
Config.POSE_MODEL_NAME
        self.det_conf = det_conf or Config.DET_CONFIDENCE
        self.pose_conf = pose_conf or Config.POSE_CONFIDENCE
        self.device = device or Config.validate_device()

        # Initialize models
        self.det_model = None
        self.pose_model = None
        self.tracker = None

        # Load models
        self._load_models()

        print(f"✅ RTMPose Estimator initialized successfully!")
        print(f"📱 Device: {self.device}")

    def _load_models(self):
        """Load detection and pose estimation models"""
        try:
            from mmdet.apis import init_detector
            from mmpose.apis import init_model as init_pose_model

            print(f"🔄 Loading models...")

            # --- Load Detection (RTMDet) ---
            det_config, det_checkpoint = Config.get_det_model_path()

            if not det_checkpoint.exists():
                print(f"📥 Detection checkpoint not found, downloading...")
                self._download_det_model()

            if not det_config.exists():
                print(f"📥 Using detection config from mim...")
```

```
            det_config =
f'rtmdet_{self.det_model_name.split("-")[1]}_8xb32-300e_coco.py'

        print(f"📦 Loading detection model: {self.det_model_name}")

        # FORCE MMDET SCOPE
        register_mmdet_modules(init_default_scope=True)

        self.det_model = init_detector(
            str(det_config),
            str(det_checkpoint),
            device=self.device
        )
        print(f"✅ Detection model loaded!")

        # --- Load Pose (RTMPose) ---
        pose_config, pose_checkpoint = Config.get_pose_model_path()

        if not pose_checkpoint.exists():
            print(f"📥 Pose checkpoint not found, downloading...")
            self._download_pose_model()

        if not pose_config.exists():
            print(f"📥 Using pose config from mim...")
            pose_config =
f'rtmpose-{self.pose_model_name.split("-")[1]}_8xb256-420e_coco-256x192.p
y'

        print(f"📦 Loading pose model: {self.pose_model_name}")

        # FORCE MMPOSE SCOPE
        register_mmpose_modules(init_default_scope=True)

        self.pose_model = init_pose_model(
            str(pose_config),
            str(pose_checkpoint),
            device=self.device
```

```
        )
        print(f"✅ Pose model loaded!")

        # Initialize ByteTrack
        self._init_tracker()

    except Exception as e:
        print(f"❌ Error loading models: {e}")
        raise

def _init_tracker(self):
    """Initialize ByteTrack tracker"""
    try:
        from byte_tracker import BYTETracker

        tracker_config = {
            'track_thresh': Config.TRACK_THRESH,
            'track_buffer': Config.TRACK_BUFFER,
            'match_thresh': Config.MATCH_THRESH,
            'min_box_area': Config.MIN_BOX_AREA,
            'mot20': Config.MOT20,
            'frame_rate': Config.FRAME_RATE
        }

        self.tracker = BYTETracker(tracker_config)
        print(f"✅ ByteTrack initialized!")

    except ImportError as e:
        print(f"⚠️  ByteTrack not available: {e}")
        self.tracker = None
    except Exception as e:
        print(f"⚠️  Error initializing tracker: {e}")
        self.tracker = None

def _download_det_model(self):
    from utils import ModelDownloader
    det_config, det_checkpoint = Config.get_det_model_path()
```

```
    if 'rtmdet-m' in self.det_model_name:
        checkpoint_url = Config.MODEL_URLS['rtmdet-m']['checkpoint']
        ModelDownloader.download_file(checkpoint_url, det_checkpoint)
    print(f"✅ Detection model downloaded!")

def _download_pose_model(self):
    from utils import ModelDownloader
    pose_config, pose_checkpoint = Config.get_pose_model_path()
    if 'rtmpose-m' in self.pose_model_name:
        checkpoint_url = Config.MODEL_URLS['rtmpose-m']['checkpoint']
        ModelDownloader.download_file(checkpoint_url, pose_checkpoint)
    print(f"✅ Pose model downloaded!")

def _detect_persons(self, frame: np.ndarray) -> np.ndarray:
    """
    Detect persons in frame
    """
    from mmdet.apis import inference_detector

    # 🔥 CRITICAL FIX: Force Reset MMDET Registry Scope 🔥
    register_mmdet_modules(init_default_scope=True)

    result = inference_detector(self.det_model, frame)

    pred_instances = result.pred_instances
    person_mask = (pred_instances.labels == 0) & \
                  (pred_instances.scores >= self.det_conf)

    bboxes = pred_instances.bboxes[person_mask].cpu().numpy()
    scores = pred_instances.scores[person_mask].cpu().numpy()

    if len(bboxes) > 0:
        detections = np.concatenate([bboxes, scores[:, None]], axis=1)
    else:
        detections = np.empty((0, 5))

    return detections
```

```
def _estimate_pose(self, frame: np.ndarray,
                   bboxes: np.ndarray) -> List[Dict]:
    """
    Estimate pose for detected persons
    """
    from mmpose.apis import inference_topdown

    if len(bboxes) == 0:
        return []

    # 🔥 CRITICAL FIX: Force Reset MMPOSE Registry Scope 🔥
    register_mmpose_modules(init_default_scope=True)

    bboxes_xyxy = bboxes[:, :4]

    results = inference_topdown(
        self.pose_model,
        frame,
        bboxes_xyxy
    )

    return results

def _track_objects(self, detections: np.ndarray,
                   frame_id: int,
                   img_shape: Tuple[int, int]) -> Optional[np.ndarray]:
    if self.tracker is None or len(detections) == 0:
        return None

    online_targets = self.tracker.update(
        detections,
        img_info=img_shape,
        img_size=img_shape
    )

    tracked = []
```

```
    for track in online_targets:
        tlwh = track.tlwh
        track_id = track.track_id
        score = track.score

        x1, y1, w, h = tlwh
        x2 = x1 + w
        y2 = y1 + h

        tracked.append([x1, y1, x2, y2, score, track_id])

    return np.array(tracked) if tracked else None

def _visualize_results(self, frame: np.ndarray,
                       pose_results: List[Dict],
                       tracked_boxes: Optional[np.ndarray] = None) -> np.ndarray:
    vis_frame = frame.copy()

    if tracked_boxes is not None and len(tracked_boxes) > 0:
        for box in tracked_boxes:
            x1, y1, x2, y2, score, track_id = box
            vis_frame = VisualizationHelper.draw_bbox(
                vis_frame,
                (x1, y1, x2, y2),
                track_id=int(track_id),
                color=Config.BBOX_COLOR,
                thickness=Config.BBOX_THICKNESS
            )

    for result in pose_results:
        keypoints = result.pred_instances.keypoints[0]
        scores = result.pred_instances.keypoint_scores[0]

        vis_frame = VisualizationHelper.draw_skeleton(
            vis_frame,
            keypoints,
            scores,
```

```
            Config.SKELETON_LINKS,
            color=Config.SKELETON_COLOR,
            thickness=Config.SKELETON_THICKNESS,
            conf_threshold=self.pose_conf
        )

        vis_frame = VisualizationHelper.draw_keypoints(
            vis_frame,
            keypoints,
            scores,
            Config.KEYPOINT_COLORS,
            radius=Config.KEYPOINT_RADIUS,
            conf_threshold=self.pose_conf
        )

    return vis_frame

def predict_image(self,
                  image_path: Union[str, Path],
                  save_result: bool = True,
                  output_path: Optional[str] = None) -> np.ndarray:
    print(f"📸 Processing image: {image_path}")

    frame = cv2.imread(str(image_path))
    if frame is None:
        raise ValueError(f"Cannot read image: {image_path}")

    detections = self._detect_persons(frame)
    print(f"👥 Detected {len(detections)} person(s)")

    if len(detections) == 0:
        print("⚠️  No persons detected")
        return frame

    pose_results = self._estimate_pose(frame, detections)
    result_frame = self._visualize_results(frame, pose_results)
```

```
        if save_result:
            if output_path is None:
                input_filename = Path(image_path).name
                output_path = Config.OUTPUT_IMAGES_DIR /
f"output_{input_filename}"
            else:
                output_path = Path(output_path)

            output_path.parent.mkdir(parents=True, exist_ok=True)
            cv2.imwrite(str(output_path), result_frame)
            print(f"💾 Result saved to: {output_path}")

        return result_frame

    def predict_video(self,
                      video_path: Union[str, Path, int],
                      save_result: bool = True,
                      output_path: Optional[str] = None,
                      show_live: bool = True) -> None:

        is_camera = False
        if video_path == 0 or str(video_path).lower() == 'camera':
            cap = cv2.VideoCapture(0)
            is_camera = True
            print("📹 Opening camera...")
        else:
            cap = cv2.VideoCapture(str(video_path))
            print(f"📹 Processing video: {video_path}")

        if not cap.isOpened():
            raise ValueError("Cannot open video/camera")

        width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
        height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
        fps = int(cap.get(cv2.CAP_PROP_FPS)) or 30
        img_shape = (height, width)
```

```
    writer = None
    if save_result:
        if output_path is None:
            if is_camera:
                timestamp = FileManager.get_timestamp()
                output_filename = f"camera_output_{timestamp}.mp4"
            else:
                input_filename = Path(video_path).name
                output_filename = f"output_{input_filename}"
            output_path = Config.OUTPUT_VIDEOS_DIR / output_filename
        else:
            output_path = Path(output_path)

        output_path.parent.mkdir(parents=True, exist_ok=True)
        fourcc = cv2.VideoWriter_fourcc(*'mp4v')
        writer = cv2.VideoWriter(str(output_path), fourcc, fps, (width, height))
        print(f"💾 Saving video to: {output_path}")

    monitor = PerformanceMonitor()
    monitor.start()
    frame_id = 0

    try:
        while cap.isOpened():
            ret, frame = cap.read()
            if not ret:
                break

            frame_id += 1
            monitor.add_frame()

            detections = self._detect_persons(frame)

            tracked_boxes = None
            if self.tracker is not None and len(detections) > 0:
                tracked_boxes = self._track_objects(detections, frame_id,
img_shape)
```

```
                if tracked_boxes is not None:
                    detections = tracked_boxes[:, :5]

            pose_results = []
            if len(detections) > 0:
                pose_results = self._estimate_pose(frame, detections)

            result_frame = self._visualize_results(frame, pose_results,
tracked_boxes)

            current_fps = monitor.get_current_fps()
            result_frame = VisualizationHelper.put_fps_text(result_frame,
current_fps)

            info_text = f'Frame: {frame_id} | Persons: {len(pose_results)}'
            result_frame = VisualizationHelper.put_info_text(result_frame,
info_text, position=(10, 70))

            if writer is not None:
                writer.write(result_frame)

            if show_live:
                cv2.imshow('RTMPose + ByteTrack - Press Q to Exit',
result_frame)
                if cv2.waitKey(1) & 0xFF == ord('q'):
                    print("⏹️ Processing stopped by user")
                    break

    finally:
        cap.release()
        if writer is not None:
            writer.release()
            print(f"💾 Video saved successfully!")
        cv2.destroyAllWindows()
        monitor.print_stats()
```

```
def test_estimator():
    print("🧪 Testing RTMPose Estimator...")
    Config.print_paths()
    try:
        estimator = RTMPoseEstimator()
        print("✅ Estimator ready!")
    except Exception as e:
        print(f"❌ Error: {e}")
        import traceback
        traceback.print_exc()

if __name__ == "__main__":
    test_estimator()
```

```
"""
ByteTrack Implementation
Multi-Object Tracking with ByteTrack
"""

import numpy as np
from collections import deque
import cv2

# ============================================
# ✅ FIXED: Import required libraries with proper error handling
# ============================================
try:
    import scipy
    import scipy.linalg
    SCIPY_AVAILABLE = True
except ImportError:
    print("⚠️  Warning: scipy not installed, Kalman filter may not work
properly")
    print("Install with: pip install scipy")
    SCIPY_AVAILABLE = False
```

```
try:
    import lap
    LAP_AVAILABLE = True
except ImportError:
    print("⚠️  Warning: lap not installed, using scipy as fallback")
    print("Install with: pip install lap")
    LAP_AVAILABLE = False

    # Fallback to scipy if lap not available
    if SCIPY_AVAILABLE:
        from scipy.optimize import linear_sum_assignment


class STrack:
    """Single target track"""

    shared_kalman = None  # Shared Kalman filter for all tracks
    track_id_count = 0

    def __init__(self, tlwh, score):
        """
        Initialize track

        Args:
            tlwh: Bounding box in tlwh format [top, left, width, height]
            score: Detection score
        """
        # Wait activate
        self._tlwh = np.asarray(tlwh, dtype=np.float32)
        self.kalman_filter = None
        self.mean, self.covariance = None, None
        self.is_activated = False

        self.score = score
        self.tracklet_len = 0
        self.track_id = 0
        self.state = 'new'
```

```
    self.start_frame = 0
    self.frame_id = 0

def predict(self):
    """Predict next state using Kalman filter"""
    mean_state = self.mean.copy()
    if self.state != 'tracked':
        mean_state[7] = 0
    self.mean, self.covariance = self.kalman_filter.predict(
        mean_state, self.covariance
    )

@staticmethod
def multi_predict(stracks):
    """Predict multiple tracks"""
    if len(stracks) > 0:
        multi_mean = np.asarray([st.mean.copy() for st in stracks])
        multi_covariance = np.asarray([st.covariance for st in stracks])

        for i, st in enumerate(stracks):
            if st.state != 'tracked':
                multi_mean[i][7] = 0

        multi_mean, multi_covariance = STrack.shared_kalman.multi_predict(
            multi_mean, multi_covariance
        )

        for i, (mean, cov) in enumerate(zip(multi_mean, multi_covariance)):
            stracks[i].mean = mean
            stracks[i].covariance = cov

def activate(self, kalman_filter, frame_id):
    """Activate a new tracklet"""
    self.kalman_filter = kalman_filter
    self.track_id = self.next_id()
```

```
        self.mean, self.covariance = self.kalman_filter.initiate(
            self.tlwh_to_xyah(self._tlwh)
        )

        self.tracklet_len = 0
        self.state = 'tracked'

        if frame_id == 1:
            self.is_activated = True

        self.frame_id = frame_id
        self.start_frame = frame_id

    def re_activate(self, new_track, frame_id, new_id=False):
        """Reactivate a lost track"""
        self.mean, self.covariance = self.kalman_filter.update(
            self.mean, self.covariance, self.tlwh_to_xyah(new_track.tlwh)
        )

        self.tracklet_len = 0
        self.state = 'tracked'
        self.is_activated = True
        self.frame_id = frame_id

        if new_id:
            self.track_id = self.next_id()

        self.score = new_track.score

    def update(self, new_track, frame_id):
        """Update track with new detection"""
        self.frame_id = frame_id
        self.tracklet_len += 1

        new_tlwh = new_track.tlwh
        self.mean, self.covariance = self.kalman_filter.update(
            self.mean, self.covariance, self.tlwh_to_xyah(new_tlwh)
```

```
        )

        self.state = 'tracked'
        self.is_activated = True
        self.score = new_track.score

    @property
    def tlwh(self):
        """Get current position in tlwh format"""
        if self.mean is None:
            return self._tlwh.copy()

        ret = self.mean[:4].copy()
        ret[2] *= ret[3]
        ret[:2] -= ret[2:] / 2
        return ret

    @property
    def tlbr(self):
        """Convert to tlbr format [top, left, bottom, right]"""
        ret = self.tlwh.copy()
        ret[2:] += ret[:2]
        return ret

    @staticmethod
    def tlwh_to_xyah(tlwh):
        """Convert tlwh to xyah format [center_x, center_y, aspect_ratio,
height]"""
        ret = np.asarray(tlwh).copy()
        ret[:2] += ret[2:] / 2
        ret[2] /= ret[3]
        return ret

    def to_xyah(self):
        """Convert current position to xyah format"""
        return self.tlwh_to_xyah(self.tlwh)
```

```
    @staticmethod
    def tlbr_to_tlwh(tlbr):
        """Convert tlbr to tlwh format"""
        ret = np.asarray(tlbr).copy()
        ret[2:] -= ret[:2]
        return ret

    @staticmethod
    def tlwh_to_tlbr(tlwh):
        """Convert tlwh to tlbr format"""
        ret = np.asarray(tlwh).copy()
        ret[2:] += ret[:2]
        return ret

    def __repr__(self):
        return f'OT_{self.track_id}_({self.start_frame}-{self.frame_id})'

    @staticmethod
    def next_id():
        """Get next track ID"""
        STrack.track_id_count += 1
        return STrack.track_id_count

    def mark_lost(self):
        """Mark track as lost"""
        self.state = 'lost'

    def mark_removed(self):
        """Mark track as removed"""
        self.state = 'removed'


class BYTETracker:
    """
    ByteTrack: Multi-Object Tracking by Associating Every Detection Box
    """
```

```
def __init__(self, args, frame_rate=30):
    """
    Initialize ByteTrack

    Args:
        args: Configuration dictionary
        frame_rate: Video frame rate
    """
    self.tracked_stracks = []  # type: list[STrack]
    self.lost_stracks = []     # type: list[STrack]
    self.removed_stracks = []  # type: list[STrack]

    self.frame_id = 0

    # Parse args
    if isinstance(args, dict):
        self.track_thresh = args.get('track_thresh', 0.5)
        self.track_buffer = args.get('track_buffer', 30)
        self.match_thresh = args.get('match_thresh', 0.8)
        self.min_box_area = args.get('min_box_area', 10)
        self.mot20 = args.get('mot20', False)
    else:
        self.track_thresh = args.track_thresh
        self.track_buffer = args.track_buffer
        self.match_thresh = args.match_thresh
        self.min_box_area = args.min_box_area
        self.mot20 = args.mot20

    self.buffer_size = int(frame_rate / 30.0 * self.track_buffer)
    self.max_time_lost = self.buffer_size
    self.kalman_filter = KalmanFilter()

    # Initialize shared Kalman filter
    STrack.shared_kalman = self.kalman_filter

def update(self, output_results, img_info=None, img_size=None):
    """
```

```
Update tracker with new detections

Args:
    output_results: Detection results (N, 5) [x1, y1, x2, y2, score]
    img_info: Image info
    img_size: Image size

Returns:
    List of active tracks
"""
self.frame_id += 1
activated_starcks = []
refind_stracks = []
lost_stracks = []
removed_stracks = []

if output_results.shape[1] == 5:
    scores = output_results[:, 4]
    bboxes = output_results[:, :4]
else:
    output_results = output_results.cpu().numpy()
    scores = output_results[:, 4] * output_results[:, 5]
    bboxes = output_results[:, :4]

# Filter by score
remain_inds = scores > self.track_thresh
inds_low = scores > 0.1
inds_high = scores < self.track_thresh

inds_second = np.logical_and(inds_low, inds_high)
dets_second = bboxes[inds_second]
dets = bboxes[remain_inds]
scores_keep = scores[remain_inds]
scores_second = scores[inds_second]

# Convert to tlwh format
if len(dets) > 0:
```

```
    detections = [STrack(STrack.tlbr_to_tlwh(tlbr), s)
                  for (tlbr, s) in zip(dets, scores_keep)]
else:
    detections = []

# Add newly detected tracklets
unconfirmed = []
tracked_stracks = []
for track in self.tracked_stracks:
    if not track.is_activated:
        unconfirmed.append(track)
    else:
        tracked_stracks.append(track)

# Step 2: First association, with high score detection boxes
strack_pool = joint_stracks(tracked_stracks, self.lost_stracks)

# Predict current location with KF
STrack.multi_predict(strack_pool)

dists = matching.iou_distance(strack_pool, detections)

if not self.mot20:
    dists = matching.fuse_score(dists, detections)

matches, u_track, u_detection = matching.linear_assignment(
    dists, thresh=self.match_thresh
)

for itracked, idet in matches:
    track = strack_pool[itracked]
    det = detections[idet]
    if track.state == 'tracked':
        track.update(detections[idet], self.frame_id)
        activated_starcks.append(track)
    else:
        track.re_activate(det, self.frame_id, new_id=False)
```

```
        refind_stracks.append(track)

# Step 3: Second association, with low score detection boxes
if len(dets_second) > 0:
    detections_second = [STrack(STrack.tlbr_to_tlwh(tlbr), s)
                         for (tlbr, s) in zip(dets_second, scores_second)]
else:
    detections_second = []

r_tracked_stracks = [strack_pool[i] for i in u_track
                     if strack_pool[i].state == 'tracked']

dists = matching.iou_distance(r_tracked_stracks, detections_second)
matches, u_track, u_detection_second = matching.linear_assignment(
    dists, thresh=0.5
)

for itracked, idet in matches:
    track = r_tracked_stracks[itracked]
    det = detections_second[idet]
    if track.state == 'tracked':
        track.update(det, self.frame_id)
        activated_starcks.append(track)
    else:
        track.re_activate(det, self.frame_id, new_id=False)
        refind_stracks.append(track)

for it in u_track:
    track = r_tracked_stracks[it]
    if not track.state == 'lost':
        track.mark_lost()
        lost_stracks.append(track)

# Deal with unconfirmed tracks
detections = [detections[i] for i in u_detection]
dists = matching.iou_distance(unconfirmed, detections)
```

```
        if not self.mot20:
            dists = matching.fuse_score(dists, detections)

        matches, u_unconfirmed, u_detection = matching.linear_assignment(
            dists, thresh=0.7
        )

        for itracked, idet in matches:
            unconfirmed[itracked].update(detections[idet], self.frame_id)
            activated_starcks.append(unconfirmed[itracked])

        for it in u_unconfirmed:
            track = unconfirmed[it]
            track.mark_removed()
            removed_stracks.append(track)

        # Step 4: Init new stracks
        for inew in u_detection:
            track = detections[inew]
            if track.score < self.track_thresh:
                continue

            track.activate(self.kalman_filter, self.frame_id)
            activated_starcks.append(track)

        # Step 5: Update state
        for track in self.lost_stracks:
            if self.frame_id - track.frame_id > self.max_time_lost:
                track.mark_removed()
                removed_stracks.append(track)

        self.tracked_stracks = [t for t in self.tracked_stracks
                                if t.state == 'tracked']
        self.tracked_stracks = joint_stracks(self.tracked_stracks,
activated_starcks)
        self.tracked_stracks = joint_stracks(self.tracked_stracks, refind_stracks)
        self.lost_stracks = sub_stracks(self.lost_stracks, self.tracked_stracks)
```

```
        self.lost_stracks.extend(lost_stracks)
        self.lost_stracks = sub_stracks(self.lost_stracks, self.removed_stracks)
        self.removed_stracks.extend(removed_stracks)
        self.tracked_stracks, self.lost_stracks = remove_duplicate_stracks(
            self.tracked_stracks, self.lost_stracks
        )

        # Get current output
        output_stracks = [track for track in self.tracked_stracks if
track.is_activated]

        return output_stracks


class KalmanFilter:
    """
    Kalman Filter for object tracking
    """

    def __init__(self):
        """Initialize Kalman Filter"""
        if not SCIPY_AVAILABLE:
            raise ImportError("scipy is required for KalmanFilter. Install with: pip
install scipy")

        ndim, dt = 4, 1.

        # Create Kalman filter model matrices
        self._motion_mat = np.eye(2 * ndim, 2 * ndim)
        for i in range(ndim):
            self._motion_mat[i, ndim + i] = dt

        self._update_mat = np.eye(ndim, 2 * ndim)

        # Motion and observation uncertainty
        self._std_weight_position = 1. / 20
        self._std_weight_velocity = 1. / 160
```

```
def initiate(self, measurement):
    """
    Initialize track from unassociated measurement

    Args:
        measurement: Bounding box in format (x, y, a, h)

    Returns:
        mean: Mean vector
        covariance: Covariance matrix
    """
    mean_pos = measurement
    mean_vel = np.zeros_like(mean_pos)
    mean = np.r_[mean_pos, mean_vel]

    std = [
        2 * self._std_weight_position * measurement[3],
        2 * self._std_weight_position * measurement[3],
        1e-2,
        2 * self._std_weight_position * measurement[3],
        10 * self._std_weight_velocity * measurement[3],
        10 * self._std_weight_velocity * measurement[3],
        1e-5,
        10 * self._std_weight_velocity * measurement[3]
    ]

    covariance = np.diag(np.square(std))
    return mean, covariance

def predict(self, mean, covariance):
    """
    Run Kalman filter prediction step

    Args:
        mean: Mean vector
        covariance: Covariance matrix
```

```
    Returns:
        mean: Predicted mean
        covariance: Predicted covariance
    """
    std_pos = [
        self._std_weight_position * mean[3],
        self._std_weight_position * mean[3],
        1e-2,
        self._std_weight_position * mean[3]
    ]

    std_vel = [
        self._std_weight_velocity * mean[3],
        self._std_weight_velocity * mean[3],
        1e-5,
        self._std_weight_velocity * mean[3]
    ]

    motion_cov = np.diag(np.square(np.r_[std_pos, std_vel]))

    mean = np.dot(self._motion_mat, mean)
    covariance = np.linalg.multi_dot((
        self._motion_mat, covariance, self._motion_mat.T
    )) + motion_cov

    return mean, covariance

def project(self, mean, covariance):
    """
    Project state distribution to measurement space

    Args:
        mean: Mean vector
        covariance: Covariance matrix

    Returns:
```

```
        mean: Projected mean
        covariance: Projected covariance
    """
    std = [
        self._std_weight_position * mean[3],
        self._std_weight_position * mean[3],
        1e-1,
        self._std_weight_position * mean[3]
    ]

    innovation_cov = np.diag(np.square(std))

    mean = np.dot(self._update_mat, mean)
    covariance = np.linalg.multi_dot((
        self._update_mat, covariance, self._update_mat.T
    ))

    return mean, covariance + innovation_cov

# ==========================================
# ✅ FIXED: Complete multi_predict method
# ==========================================
def multi_predict(self, mean, covariance):
    """
    Run Kalman filter prediction step for multiple tracks

    Args:
        mean: Mean vectors (N, 8)
        covariance: Covariance matrices (N, 8, 8)

    Returns:
        mean: Predicted means
        covariance: Predicted covariances
    """
    std_pos = [
        self._std_weight_position * mean[:, 3],
        self._std_weight_position * mean[:, 3],
```

```
        1e-2 * np.ones_like(mean[:, 3]),
        self._std_weight_position * mean[:, 3]
    ]

    std_vel = [
        self._std_weight_velocity * mean[:, 3],
        self._std_weight_velocity * mean[:, 3],
        1e-5 * np.ones_like(mean[:, 3]),
        self._std_weight_velocity * mean[:, 3]
    ]

    sqr = np.square(np.r_[std_pos, std_vel]).T

    motion_cov = []
    for i in range(len(mean)):
        motion_cov.append(np.diag(sqr[i]))
    motion_cov = np.asarray(motion_cov)

    mean = np.dot(mean, self._motion_mat.T)
    left = np.dot(self._motion_mat, covariance).transpose((1, 0, 2))
    covariance = np.dot(left, self._motion_mat.T) + motion_cov

    return mean, covariance

def update(self, mean, covariance, measurement):
    """
    Run Kalman filter correction step

    Args:
        mean: Predicted mean
        covariance: Predicted covariance
        measurement: Measurement vector

    Returns:
        mean: Updated mean
        covariance: Updated covariance
    """
```

```
    projected_mean, projected_cov = self.project(mean, covariance)

    chol_factor, lower = scipy.linalg.cho_factor(
        projected_cov, lower=True, check_finite=False
    )
    kalman_gain = scipy.linalg.cho_solve(
        (chol_factor, lower),
        np.dot(covariance, self._update_mat.T).T,
        check_finite=False
    ).T

    innovation = measurement - projected_mean

    new_mean = mean + np.dot(innovation, kalman_gain.T)
    new_covariance = covariance - np.linalg.multi_dot((
        kalman_gain, projected_cov, kalman_gain.T
    ))

    return new_mean, new_covariance

def gating_distance(self, mean, covariance, measurements,
                    only_position=False, metric='maha'):
    """
    Compute gating distance between state distribution and measurements

    Args:
        mean: Mean vector
        covariance: Covariance matrix
        measurements: Measurement vectors (N, 4)
        only_position: Use only position for distance
        metric: Distance metric ('maha' or 'gaussian')

    Returns:
        distances: Gating distances
    """
    mean, covariance = self.project(mean, covariance)
```

```
        if only_position:
            mean, covariance = mean[:2], covariance[:2, :2]
            measurements = measurements[:, :2]

        d = measurements - mean

        if metric == 'gaussian':
            return np.sum(d * d, axis=1)
        elif metric == 'maha':
            cholesky_factor = np.linalg.cholesky(covariance)
            z = scipy.linalg.solve_triangular(
                cholesky_factor, d.T, lower=True,
                check_finite=False, overwrite_b=True
            )
            return np.sum(z * z, axis=0)
        else:
            raise ValueError('Invalid distance metric')


class matching:
    """Matching utilities for tracking"""

    @staticmethod
    def iou_distance(atracks, btracks):
        """
        Compute IOU distance between tracks

        Args:
            atracks: Track list A
            btracks: Track list B

        Returns:
            cost_matrix: IOU distance matrix
        """
        if (len(atracks) > 0 and isinstance(atracks[0], np.ndarray)) or \
           (len(btracks) > 0 and isinstance(btracks[0], np.ndarray)):
            atlbrs = atracks
```

```
        btlbrs = btracks
    else:
        atlbrs = [track.tlbr for track in atracks]
        btlbrs = [track.tlbr for track in btracks]

    ious = np.zeros((len(atlbrs), len(btlbrs)), dtype=np.float32)
    if ious.size == 0:
        return ious

    ious = bbox_ious(
        np.ascontiguousarray(atlbrs, dtype=np.float32),
        np.ascontiguousarray(btlbrs, dtype=np.float32)
    )

    cost_matrix = 1 - ious

    return cost_matrix

@staticmethod
def fuse_score(cost_matrix, detections):
    """
    Fuse cost matrix with detection scores

    Args:
        cost_matrix: Cost matrix
        detections: Detection list

    Returns:
        fused_cost: Fused cost matrix
    """
    if cost_matrix.size == 0:
        return cost_matrix

    iou_sim = 1 - cost_matrix
    det_scores = np.array([det.score for det in detections])
    det_scores = np.expand_dims(det_scores, axis=0).repeat(
        cost_matrix.shape[0], axis=0
```

```
        )

        fuse_sim = iou_sim * det_scores
        fuse_cost = 1 - fuse_sim

        return fuse_cost

    @staticmethod
    def linear_assignment(cost_matrix, thresh):
        """
        Linear assignment with Hungarian algorithm

        Args:
            cost_matrix: Cost matrix
            thresh: Threshold for matching

        Returns:
            matches: Matched pairs
            unmatched_a: Unmatched from A
            unmatched_b: Unmatched from B
        """
        if cost_matrix.size == 0:
            return (
                np.empty((0, 2), dtype=int),
                tuple(range(cost_matrix.shape[0])),
                tuple(range(cost_matrix.shape[1]))
            )

        matches, unmatched_a, unmatched_b = [], [], []

        # ==========================================
        # ✅ FIXED: Better linear assignment implementation
        # ==========================================
        if LAP_AVAILABLE:
            # Use lap library (faster)
            cost, x, y = lap.lapjv(cost_matrix, extend_cost=True,
cost_limit=thresh)
```

```
        for ix, mx in enumerate(x):
            if mx >= 0:
                matches.append([ix, mx])

        unmatched_a = np.where(x < 0)[0]
        unmatched_b = np.where(y < 0)[0]

    elif SCIPY_AVAILABLE:
        # Fallback to scipy
        cost_matrix_masked = cost_matrix.copy()
        cost_matrix_masked[cost_matrix_masked > thresh] = thresh + 1e5

        row_ind, col_ind = linear_sum_assignment(cost_matrix_masked)

        # Filter matches by threshold
        for i, j in zip(row_ind, col_ind):
            if cost_matrix[i, j] <= thresh:
                matches.append([i, j])
            else:
                unmatched_a.append(i)
                unmatched_b.append(j)

        # Find all unmatched
        matched_rows = set([m[0] for m in matches])
        matched_cols = set([m[1] for m in matches])

        unmatched_a = list(set(range(cost_matrix.shape[0])) - matched_rows)
        unmatched_b = list(set(range(cost_matrix.shape[1])) - matched_cols)

    else:
        raise ImportError("Either 'lap' or 'scipy' is required for linear
assignment")

    matches = np.asarray(matches) if matches else np.empty((0, 2),
dtype=int)
```

```
    return matches, unmatched_a, unmatched_b


def bbox_ious(atlbrs, btlbrs):
    """
    Compute IOU between two sets of boxes

    Args:
        atlbrs: Boxes A in tlbr format (N, 4)
        btlbrs: Boxes B in tlbr format (M, 4)

    Returns:
        ious: IOU matrix (N, M)
    """
    ious = np.zeros((atlbrs.shape[0], btlbrs.shape[0]), dtype=np.float32)
    if ious.size == 0:
        return ious

    # Compute IOU
    for i, atlbr in enumerate(atlbrs):
        for j, btlbr in enumerate(btlbrs):
            # Intersection
            x1 = max(atlbr[0], btlbr[0])
            y1 = max(atlbr[1], btlbr[1])
            x2 = min(atlbr[2], btlbr[2])
            y2 = min(atlbr[3], btlbr[3])

            inter_area = max(0, x2 - x1) * max(0, y2 - y1)

            # Union
            box1_area = (atlbr[2] - atlbr[0]) * (atlbr[3] - atlbr[1])
            box2_area = (btlbr[2] - btlbr[0]) * (btlbr[3] - btlbr[1])
            union_area = box1_area + box2_area - inter_area

            # IOU
            ious[i, j] = inter_area / union_area if union_area > 0 else 0
```

```
    return ious


def joint_stracks(tlista, tlistb):
    """
    Join two track lists

    Args:
        tlista: Track list A
        tlistb: Track list B

    Returns:
        Combined track list
    """
    exists = {}
    res = []
    for t in tlista:
        exists[t.track_id] = 1
        res.append(t)
    for t in tlistb:
        tid = t.track_id
        if not exists.get(tid, 0):
            exists[tid] = 1
            res.append(t)
    return res


def sub_stracks(tlista, tlistb):
    """
    Subtract track list B from A

    Args:
        tlista: Track list A
        tlistb: Track list B

    Returns:
        Subtracted track list
```

```
    """
    stracks = {}
    for t in tlista:
        stracks[t.track_id] = t
    for t in tlistb:
        tid = t.track_id
        if stracks.get(tid, 0):
            del stracks[tid]
    return list(stracks.values())


def remove_duplicate_stracks(stracksa, stracksb):
    """
    Remove duplicate tracks

    Args:
        stracksa: Track list A
        stracksb: Track list B

    Returns:
        Deduplicated track lists
    """
    pdist = matching.iou_distance(stracksa, stracksb)
    pairs = np.where(pdist < 0.15)
    dupa, dupb = list(), list()

    for p, q in zip(*pairs):
        timep = stracksa[p].frame_id - stracksa[p].start_frame
        timeq = stracksb[q].frame_id - stracksb[q].start_frame
        if timep > timeq:
            dupb.append(q)
        else:
            dupa.append(p)

    resa = [t for i, t in enumerate(stracksa) if i not in dupa]
    resb = [t for i, t in enumerate(stracksb) if i not in dupb]
```

```
    return resa, resb


if __name__ == "__main__":
    print("🧪 ByteTrack module loaded successfully!")

    # Test ByteTrack initialization
    config = {
        'track_thresh': 0.5,
        'track_buffer': 30,
        'match_thresh': 0.8,
        'min_box_area': 10,
        'mot20': False
    }

    try:
        tracker = BYTETracker(config, frame_rate=30)
        print(f"✅ ByteTrack initialized successfully!")
        print(f"📊 Track threshold: {tracker.track_thresh}")
        print(f"📊 Track buffer: {tracker.track_buffer}")
        print(f"📊 Match threshold: {tracker.match_thresh}")
    except Exception as e:
        print(f"❌ Error: {e}")
```

```
auto_scale_lr = dict(base_batch_size=16, enable=False)
backend_args = None
base_lr = 0.004
custom_hooks = [
    dict(
        ema_type='ExpMomentumEMA',
        momentum=0.0002,
        priority=49,
        type='EMAHook',
        update_buffers=True),
    dict(
        switch_epoch=280,
```

```
        switch_pipeline=[
            dict(backend_args=None, type='LoadImageFromFile'),
            dict(type='LoadAnnotations', with_bbox=True),
            dict(
                keep_ratio=True,
                ratio_range=(
                    0.1,
                    2.0,
                ),
                scale=(
                    640,
                    640,
                ),
                type='RandomResize'),
            dict(crop_size=(
                640,
                640,
            ), type='RandomCrop'),
            dict(type='YOLOXHSVRandomAug'),
            dict(prob=0.5, type='RandomFlip'),
            dict(
                pad_val=dict(img=(
                    114,
                    114,
                    114,
                )),
                size=(
                    640,
                    640,
                ),
                type='Pad'),
            dict(type='PackDetInputs'),
        ],
        type='PipelineSwitchHook'),
]
data_root = 'data/coco/'
dataset_type = 'CocoDataset'
```

```
default_hooks = dict(
    checkpoint=dict(interval=10, max_keep_ckpts=3, type='CheckpointHook'),
    logger=dict(interval=50, type='LoggerHook'),
    param_scheduler=dict(type='ParamSchedulerHook'),
    sampler_seed=dict(type='DistSamplerSeedHook'),
    timer=dict(type='IterTimerHook'),
    visualization=dict(type='DetVisualizationHook'))
default_scope = 'mmdet'
env_cfg = dict(
    cudnn_benchmark=False,
    dist_cfg=dict(backend='nccl'),
    mp_cfg=dict(mp_start_method='fork', opencv_num_threads=0))
img_scales = [
    (
        640,
        640,
    ),
    (
        320,
        320,
    ),
    (
        960,
        960,
    ),
]
interval = 10
load_from = None
log_level = 'INFO'
log_processor = dict(by_epoch=True, type='LogProcessor', window_size=50)
max_epochs = 300
model = dict(
    backbone=dict(
        act_cfg=dict(inplace=True, type='SiLU'),
        arch='P5',
        channel_attention=True,
        deepen_factor=0.67,
```

```
    expand_ratio=0.5,
    norm_cfg=dict(type='SyncBN'),
    type='CSPNeXt',
    widen_factor=0.75),
bbox_head=dict(
    act_cfg=dict(inplace=True, type='SiLU'),
    anchor_generator=dict(
        offset=0, strides=[
            8,
            16,
            32,
        ], type='MlvlPointGenerator'),
    bbox_coder=dict(type='DistancePointBBoxCoder'),
    exp_on_reg=True,
    feat_channels=192,
    in_channels=192,
    loss_bbox=dict(loss_weight=2.0, type='GIoULoss'),
    loss_cls=dict(
        beta=2.0,
        loss_weight=1.0,
        type='QualityFocalLoss',
        use_sigmoid=True),
    norm_cfg=dict(type='SyncBN'),
    num_classes=80,
    pred_kernel_size=1,
    share_conv=True,
    stacked_convs=2,
    type='RTMDetSepBNHead',
    with_objectness=False),
data_preprocessor=dict(
    batch_augments=None,
    bgr_to_rgb=False,
    mean=[
        103.53,
        116.28,
        123.675,
    ],
```

```
        std=[
            57.375,
            57.12,
            58.395,
        ],
        type='DetDataPreprocessor'),
    neck=dict(
        act_cfg=dict(inplace=True, type='SiLU'),
        expand_ratio=0.5,
        in_channels=[
            192,
            384,
            768,
        ],
        norm_cfg=dict(type='SyncBN'),
        num_csp_blocks=2,
        out_channels=192,
        type='CSPNeXtPAFPN'),
    test_cfg=dict(
        max_per_img=300,
        min_bbox_size=0,
        nms=dict(iou_threshold=0.65, type='nms'),
        nms_pre=30000,
        score_thr=0.001),
    train_cfg=dict(
        allowed_border=-1,
        assigner=dict(topk=13, type='DynamicSoftLabelAssigner'),
        debug=False,
        pos_weight=-1),
    type='RTMDet')
optim_wrapper = dict(
    optimizer=dict(lr=0.004, type='AdamW', weight_decay=0.05),
    paramwise_cfg=dict(
        bias_decay_mult=0, bypass_duplicate=True, norm_decay_mult=0),
    type='OptimWrapper')
param_scheduler = [
    dict(
```

```
        begin=0, by_epoch=False, end=1000, start_factor=1e-05,
        type='LinearLR'),
    dict(
        T_max=150,
        begin=150,
        by_epoch=True,
        convert_to_iter_based=True,
        end=300,
        eta_min=0.0002,
        type='CosineAnnealingLR'),
]
resume = False
stage2_num_epochs = 20
test_cfg = dict(type='TestLoop')
test_dataloader = dict(
    batch_size=5,
    dataset=dict(
        ann_file='annotations/instances_val2017.json',
        backend_args=None,
        data_prefix=dict(img='val2017/'),
        data_root='data/coco/',
        pipeline=[
            dict(backend_args=None, type='LoadImageFromFile'),
            dict(keep_ratio=True, scale=(
                640,
                640,
            ), type='Resize'),
            dict(
                pad_val=dict(img=(
                    114,
                    114,
                    114,
                )),
                size=(
                    640,
                    640,
                ),
```

```
            type='Pad'),
        dict(type='LoadAnnotations', with_bbox=True),
        dict(
            meta_keys=(
                'img_id',
                'img_path',
                'ori_shape',
                'img_shape',
                'scale_factor',
            ),
            type='PackDetInputs'),
    ],
    test_mode=True,
    type='CocoDataset'),
  drop_last=False,
  num_workers=10,
  persistent_workers=True,
  sampler=dict(shuffle=False, type='DefaultSampler'))
test_evaluator = dict(
  ann_file='data/coco/annotations/instances_val2017.json',
  backend_args=None,
  format_only=False,
  metric='bbox',
  proposal_nums=(
    100,
    1,
    10,
  ),
  type='CocoMetric')
test_pipeline = [
  dict(backend_args=None, type='LoadImageFromFile'),
  dict(keep_ratio=True, scale=(
    640,
    640,
  ), type='Resize'),
  dict(pad_val=dict(img=(
    114,
```

```
        114,
        114,
    )), size=(
        640,
        640,
    ), type='Pad'),
    dict(type='LoadAnnotations', with_bbox=True),
    dict(
        meta_keys=(
            'img_id',
            'img_path',
            'ori_shape',
            'img_shape',
            'scale_factor',
        ),
        type='PackDetInputs'),
]
train_cfg = dict(
    dynamic_intervals=[
        (
            280,
            1,
        ),
    ],
    max_epochs=300,
    type='EpochBasedTrainLoop',
    val_interval=10)
train_dataloader = dict(
    batch_sampler=None,
    batch_size=32,
    dataset=dict(
        ann_file='annotations/instances_train2017.json',
        backend_args=None,
        data_prefix=dict(img='train2017/'),
        data_root='data/coco/',
        filter_cfg=dict(filter_empty_gt=True, min_size=32),
        pipeline=[
```

```
dict(backend_args=None, type='LoadImageFromFile'),
dict(type='LoadAnnotations', with_bbox=True),
dict(img_scale=(
    640,
    640,
), pad_val=114.0, type='CachedMosaic'),
dict(
    keep_ratio=True,
    ratio_range=(
        0.1,
        2.0,
    ),
    scale=(
        1280,
        1280,
    ),
    type='RandomResize'),
dict(crop_size=(
    640,
    640,
), type='RandomCrop'),
dict(type='YOLOXHSVRandomAug'),
dict(prob=0.5, type='RandomFlip'),
dict(
    pad_val=dict(img=(
        114,
        114,
        114,
    )),
    size=(
        640,
        640,
    ),
    type='Pad'),
dict(
    img_scale=(
        640,
```

```
            640,
        ),
        max_cached_images=20,
        pad_val=(
            114,
            114,
            114,
        ),
        ratio_range=(
            1.0,
            1.0,
        ),
        type='CachedMixUp'),
    dict(type='PackDetInputs'),
],
type='CocoDataset'),
num_workers=10,
persistent_workers=True,
pin_memory=True,
sampler=dict(shuffle=True, type='DefaultSampler'))
train_pipeline = [
dict(backend_args=None, type='LoadImageFromFile'),
dict(type='LoadAnnotations', with_bbox=True),
dict(img_scale=(
    640,
    640,
), pad_val=114.0, type='CachedMosaic'),
dict(
    keep_ratio=True,
    ratio_range=(
        0.1,
        2.0,
    ),
    scale=(
        1280,
        1280,
    ),
```

```
        type='RandomResize'),
    dict(crop_size=(
        640,
        640,
    ), type='RandomCrop'),
    dict(type='YOLOXHSVRandomAug'),
    dict(prob=0.5, type='RandomFlip'),
    dict(pad_val=dict(img=(
        114,
        114,
        114,
    )), size=(
        640,
        640,
    ), type='Pad'),
    dict(
        img_scale=(
            640,
            640,
        ),
        max_cached_images=20,
        pad_val=(
            114,
            114,
            114,
        ),
        ratio_range=(
            1.0,
            1.0,
        ),
        type='CachedMixUp'),
    dict(type='PackDetInputs'),
]
train_pipeline_stage2 = [
    dict(backend_args=None, type='LoadImageFromFile'),
    dict(type='LoadAnnotations', with_bbox=True),
    dict(
```

```
        keep_ratio=True,
        ratio_range=(
            0.1,
            2.0,
        ),
        scale=(
            640,
            640,
        ),
        type='RandomResize'),
    dict(crop_size=(
        640,
        640,
    ), type='RandomCrop'),
    dict(type='YOLOXHSVRandomAug'),
    dict(prob=0.5, type='RandomFlip'),
    dict(pad_val=dict(img=(
        114,
        114,
        114,
    )), size=(
        640,
        640,
    ), type='Pad'),
    dict(type='PackDetInputs'),
]
tta_model = dict(
    tta_cfg=dict(max_per_img=100, nms=dict(iou_threshold=0.6, type='nms')),
    type='DetTTAModel')
tta_pipeline = [
    dict(backend_args=None, type='LoadImageFromFile'),
    dict(
        transforms=[
            [
                dict(keep_ratio=True, scale=(
                    640,
                    640,
```

```
    ), type='Resize'),
    dict(keep_ratio=True, scale=(
        320,
        320,
    ), type='Resize'),
    dict(keep_ratio=True, scale=(
        960,
        960,
    ), type='Resize'),
],
[
    dict(prob=1.0, type='RandomFlip'),
    dict(prob=0.0, type='RandomFlip'),
],
[
    dict(
        pad_val=dict(img=(
            114,
            114,
            114,
        )),
        size=(
            960,
            960,
        ),
        type='Pad'),
],
[
    dict(type='LoadAnnotations', with_bbox=True),
],
[
    dict(
        meta_keys=(
            'img_id',
            'img_path',
            'ori_shape',
            'img_shape',
```

```
                'scale_factor',
                'flip',
                'flip_direction',
            ),
            type='PackDetInputs'),
        ],
    ],
    type='TestTimeAug'),
]
val_cfg = dict(type='ValLoop')
val_dataloader = dict(
    batch_size=5,
    dataset=dict(
        ann_file='annotations/instances_val2017.json',
        backend_args=None,
        data_prefix=dict(img='val2017/'),
        data_root='data/coco/',
        pipeline=[
            dict(backend_args=None, type='LoadImageFromFile'),
            dict(keep_ratio=True, scale=(
                640,
                640,
            ), type='Resize'),
            dict(
                pad_val=dict(img=(
                    114,
                    114,
                    114,
                )),
                size=(
                    640,
                    640,
                ),
                type='Pad'),
            dict(type='LoadAnnotations', with_bbox=True),
            dict(
                meta_keys=(
```

```
                'img_id',
                'img_path',
                'ori_shape',
                'img_shape',
                'scale_factor',
            ),
            type='PackDetInputs'),
    ],
    test_mode=True,
    type='CocoDataset'),
  drop_last=False,
  num_workers=10,
  persistent_workers=True,
  sampler=dict(shuffle=False, type='DefaultSampler'))
val_evaluator = dict(
  ann_file='data/coco/annotations/instances_val2017.json',
  backend_args=None,
  format_only=False,
  metric='bbox',
  proposal_nums=(
    100,
    1,
    10,
  ),
  type='CocoMetric')
vis_backends = [
  dict(type='LocalVisBackend'),
]
visualizer = dict(
  name='visualizer',
  type='DetLocalVisualizer',
  vis_backends=[
    dict(type='LocalVisBackend'),
  ])
```

```
auto_scale_lr = dict(base_batch_size=1024)
backend_args = dict(backend='local')
base_lr = 0.004
codec = dict(
    input_size=(
        192,
        256,
    ),
    normalize=False,
    sigma=(
        4.9,
        5.66,
    ),
    simcc_split_ratio=2.0,
    type='SimCCLabel',
    use_dark=False)
custom_hooks = [
    dict(
        ema_type='ExpMomentumEMA',
        momentum=0.0002,
        priority=49,
        type='EMAHook',
        update_buffers=True),
    dict(
        switch_epoch=390,
        switch_pipeline=[
            dict(backend_args=dict(backend='local'), type='LoadImage'),
            dict(type='GetBBoxCenterScale'),
            dict(direction='horizontal', type='RandomFlip'),
            dict(type='RandomHalfBody'),
            dict(
                rotate_factor=60,
                scale_factor=[
                    0.75,
                    1.25,
                ],
                shift_factor=0.0,
```

```
    type='RandomBBoxTransform'),
dict(input_size=(
    192,
    256,
), type='TopdownAffine'),
dict(type='mmdet.YOLOXHSVRandomAug'),
dict(
    transforms=[
        dict(p=0.1, type='Blur'),
        dict(p=0.1, type='MedianBlur'),
        dict(
            max_height=0.4,
            max_holes=1,
            max_width=0.4,
            min_height=0.2,
            min_holes=1,
            min_width=0.2,
            p=0.5,
            type='CoarseDropout'),
    ],
    type='Albumentation'),
dict(
    encoder=dict(
        input_size=(
            192,
            256,
        ),
        normalize=False,
        sigma=(
            4.9,
            5.66,
        ),
        simcc_split_ratio=2.0,
        type='SimCCLabel',
        use_dark=False),
    type='GenerateTarget'),
dict(type='PackPoseInputs'),
```

```
        ],
        type='mmdet.PipelineSwitchHook'),
]
data_mode = 'topdown'
data_root = 'data/coco/'
dataset_type = 'CocoDataset'
default_hooks = dict(
    checkpoint=dict(
        interval=10,
        max_keep_ckpts=1,
        rule='greater',
        save_best='coco/AP',
        type='CheckpointHook'),
    logger=dict(interval=50, type='LoggerHook'),
    param_scheduler=dict(type='ParamSchedulerHook'),
    sampler_seed=dict(type='DistSamplerSeedHook'),
    timer=dict(type='IterTimerHook'),
    visualization=dict(enable=False, type='PoseVisualizationHook'))
default_scope = 'mmpose'
env_cfg = dict(
    cudnn_benchmark=False,
    dist_cfg=dict(backend='nccl'),
    mp_cfg=dict(mp_start_method='fork', opencv_num_threads=0))
load_from = None
log_level = 'INFO'
log_processor = dict(
    by_epoch=True, num_digits=6, type='LogProcessor', window_size=50)
max_epochs = 420
model = dict(
    backbone=dict(
        _scope_='mmdet',
        act_cfg=dict(type='SiLU'),
        arch='P5',
        channel_attention=True,
        deepen_factor=0.67,
        expand_ratio=0.5,
        init_cfg=dict(
```

```
            checkpoint=

'https://download.openmmlab.com/mmpose/v1/projects/rtmposev1/cspnext-m
_udp-aic-coco_210e-256x192-f2f7d6f6_20230130.pth',
            prefix='backbone.',
            type='Pretrained'),
        norm_cfg=dict(type='SyncBN'),
        out_indices=(4, ),
        type='CSPNeXt',
        widen_factor=0.75),
    data_preprocessor=dict(
        bgr_to_rgb=True,
        mean=[
            123.675,
            116.28,
            103.53,
        ],
        std=[
            58.395,
            57.12,
            57.375,
        ],
        type='PoseDataPreprocessor'),
    head=dict(
        decoder=dict(
            input_size=(
                192,
                256,
            ),
            normalize=False,
            sigma=(
                4.9,
                5.66,
            ),
            simcc_split_ratio=2.0,
            type='SimCCLabel',
            use_dark=False),
```

```
        final_layer_kernel_size=7,
        gau_cfg=dict(
            act_fn='SiLU',
            drop_path=0.0,
            dropout_rate=0.0,
            expansion_factor=2,
            hidden_dims=256,
            pos_enc=False,
            s=128,
            use_rel_bias=False),
        in_channels=768,
        in_featuremap_size=(
            6,
            8,
        ),
        input_size=(
            192,
            256,
        ),
        loss=dict(
            beta=10.0,
            label_softmax=True,
            type='KLDiscretLoss',
            use_target_weight=True),
        out_channels=17,
        simcc_split_ratio=2.0,
        type='RTMCCHead'),
    test_cfg=dict(flip_test=True),
    type='TopdownPoseEstimator')
optim_wrapper = dict(
    optimizer=dict(lr=0.004, type='AdamW', weight_decay=0.05),
    paramwise_cfg=dict(
        bias_decay_mult=0, bypass_duplicate=True, norm_decay_mult=0),
    type='OptimWrapper')
param_scheduler = [
    dict(
        begin=0, by_epoch=False, end=1000, start_factor=1e-05,
```

```
        type='LinearLR'),
    dict(
        T_max=210,
        begin=210,
        by_epoch=True,
        convert_to_iter_based=True,
        end=420,
        eta_min=0.0002,
        type='CosineAnnealingLR'),
]
randomness = dict(seed=21)
resume = False
stage2_num_epochs = 30
test_cfg = dict()
test_dataloader = dict(
    batch_size=64,
    dataset=dict(
        ann_file='annotations/person_keypoints_val2017.json',
        data_mode='topdown',
        data_prefix=dict(img='val2017/'),
        data_root='data/coco/',
        pipeline=[
            dict(backend_args=dict(backend='local'), type='LoadImage'),
            dict(type='GetBBoxCenterScale'),
            dict(input_size=(
                192,
                256,
            ), type='TopdownAffine'),
            dict(type='PackPoseInputs'),
        ],
        test_mode=True,
        type='CocoDataset'),
    drop_last=False,
    num_workers=10,
    persistent_workers=True,
    sampler=dict(round_up=False, shuffle=False, type='DefaultSampler'))
test_evaluator = dict(
```

```
    ann_file='data/coco/annotations/person_keypoints_val2017.json',
    type='CocoMetric')
train_cfg = dict(by_epoch=True, max_epochs=420, val_interval=10)
train_dataloader = dict(
    batch_size=256,
    dataset=dict(
        ann_file='annotations/person_keypoints_train2017.json',
        data_mode='topdown',
        data_prefix=dict(img='train2017/'),
        data_root='data/coco/',
        pipeline=[
            dict(backend_args=dict(backend='local'), type='LoadImage'),
            dict(type='GetBBoxCenterScale'),
            dict(direction='horizontal', type='RandomFlip'),
            dict(type='RandomHalfBody'),
            dict(
                rotate_factor=80,
                scale_factor=[
                    0.6,
                    1.4,
                ],
                type='RandomBBoxTransform'),
            dict(input_size=(
                192,
                256,
            ), type='TopdownAffine'),
            dict(type='mmdet.YOLOXHSVRandomAug'),
            dict(
                transforms=[
                    dict(p=0.1, type='Blur'),
                    dict(p=0.1, type='MedianBlur'),
                    dict(
                        max_height=0.4,
                        max_holes=1,
                        max_width=0.4,
                        min_height=0.2,
                        min_holes=1,
```

```
                min_width=0.2,
                p=1.0,
                type='CoarseDropout'),
            ],
            type='Albumentation'),
        dict(
            encoder=dict(
                input_size=(
                    192,
                    256,
                ),
                normalize=False,
                sigma=(
                    4.9,
                    5.66,
                ),
                simcc_split_ratio=2.0,
                type='SimCCLabel',
                use_dark=False),
            type='GenerateTarget'),
        dict(type='PackPoseInputs'),
    ],
    type='CocoDataset'),
num_workers=10,
persistent_workers=True,
sampler=dict(shuffle=True, type='DefaultSampler'))
train_pipeline = [
    dict(backend_args=dict(backend='local'), type='LoadImage'),
    dict(type='GetBBoxCenterScale'),
    dict(direction='horizontal', type='RandomFlip'),
    dict(type='RandomHalfBody'),
    dict(
        rotate_factor=80,
        scale_factor=[
            0.6,
            1.4,
        ],
```

```
    type='RandomBBoxTransform'),
dict(input_size=(
    192,
    256,
), type='TopdownAffine'),
dict(type='mmdet.YOLOXHSVRandomAug'),
dict(
    transforms=[
        dict(p=0.1, type='Blur'),
        dict(p=0.1, type='MedianBlur'),
        dict(
            max_height=0.4,
            max_holes=1,
            max_width=0.4,
            min_height=0.2,
            min_holes=1,
            min_width=0.2,
            p=1.0,
            type='CoarseDropout'),
    ],
    type='Albumentation'),
dict(
    encoder=dict(
        input_size=(
            192,
            256,
        ),
        normalize=False,
        sigma=(
            4.9,
            5.66,
        ),
        simcc_split_ratio=2.0,
        type='SimCCLabel',
        use_dark=False),
    type='GenerateTarget'),
dict(type='PackPoseInputs'),
```

```
]
train_pipeline_stage2 = [
    dict(backend_args=dict(backend='local'), type='LoadImage'),
    dict(type='GetBBoxCenterScale'),
    dict(direction='horizontal', type='RandomFlip'),
    dict(type='RandomHalfBody'),
    dict(
        rotate_factor=60,
        scale_factor=[
            0.75,
            1.25,
        ],
        shift_factor=0.0,
        type='RandomBBoxTransform'),
    dict(input_size=(
        192,
        256,
    ), type='TopdownAffine'),
    dict(type='mmdet.YOLOXHSVRandomAug'),
    dict(
        transforms=[
            dict(p=0.1, type='Blur'),
            dict(p=0.1, type='MedianBlur'),
            dict(
                max_height=0.4,
                max_holes=1,
                max_width=0.4,
                min_height=0.2,
                min_holes=1,
                min_width=0.2,
                p=0.5,
                type='CoarseDropout'),
        ],
        type='Albumentation'),
    dict(
        encoder=dict(
            input_size=(
```

```
            192,
            256,
        ),
        normalize=False,
        sigma=(
            4.9,
            5.66,
        ),
        simcc_split_ratio=2.0,
        type='SimCCLabel',
        use_dark=False),
    type='GenerateTarget'),
  dict(type='PackPoseInputs'),
]
val_cfg = dict()
val_dataloader = dict(
  batch_size=64,
  dataset=dict(
    ann_file='annotations/person_keypoints_val2017.json',
    data_mode='topdown',
    data_prefix=dict(img='val2017/'),
    data_root='data/coco/',
    pipeline=[
      dict(backend_args=dict(backend='local'), type='LoadImage'),
      dict(type='GetBBoxCenterScale'),
      dict(input_size=(
        192,
        256,
      ), type='TopdownAffine'),
      dict(type='PackPoseInputs'),
    ],
    test_mode=True,
    type='CocoDataset'),
  drop_last=False,
  num_workers=10,
  persistent_workers=True,
  sampler=dict(round_up=False, shuffle=False, type='DefaultSampler'))
```

```
val_evaluator = dict(
    ann_file='data/coco/annotations/person_keypoints_val2017.json',
    type='CocoMetric')
val_pipeline = [
    dict(backend_args=dict(backend='local'), type='LoadImage'),
    dict(type='GetBBoxCenterScale'),
    dict(input_size=(
        192,
        256,
    ), type='TopdownAffine'),
    dict(type='PackPoseInputs'),
]
vis_backends = [
    dict(type='LocalVisBackend'),
]
visualizer = dict(
    name='visualizer',
    type='PoseLocalVisualizer',
    vis_backends=[
        dict(type='LocalVisBackend'),
    ])
```

```
"""
Utility functions for RTMPose + ByteTrack Pose Estimator
"""

import cv2
import numpy as np
from pathlib import Path
from typing import List, Union, Tuple, Dict, Optional
import json
import time
from datetime import datetime

# ============================================
```

```
# ✅ FIXED: Import pandas at module level with error handling
# ============================================
try:
    import pandas as pd
    PANDAS_AVAILABLE = True
except ImportError:
    PANDAS_AVAILABLE = False
    print("⚠️ pandas not installed, CSV export will be disabled")


class VideoInfo:
    """Video information class"""

    @staticmethod
    def get_video_info(video_path: Union[str, Path]) -> Optional[dict]:
        """
        Get video information

        Args:
            video_path: Path to video file

        Returns:
            Dictionary containing video information
        """
        cap = cv2.VideoCapture(str(video_path))

        if not cap.isOpened():
            return None

        info = {
            'width': int(cap.get(cv2.CAP_PROP_FRAME_WIDTH)),
            'height': int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT)),
            'fps': int(cap.get(cv2.CAP_PROP_FPS)),
            'frame_count': int(cap.get(cv2.CAP_PROP_FRAME_COUNT)),
            'duration': int(cap.get(cv2.CAP_PROP_FRAME_COUNT)) /
max(int(cap.get(cv2.CAP_PROP_FPS)), 1),
            'codec': int(cap.get(cv2.CAP_PROP_FOURCC))
```

```
        }

        cap.release()
        return info

    @staticmethod
    def print_video_info(video_path: Union[str, Path]):
        """
        Print video information

        Args:
            video_path: Path to video file
        """
        info = VideoInfo.get_video_info(video_path)

        if info:
            print("\n📹 Video Information:")
            print("="*50)
            print(f"📐 Dimensions: {info['width']}x{info['height']}")
            print(f"🎬 FPS: {info['fps']}")
            print(f"📊 Frame Count: {info['frame_count']}")
            print(f"⏱️  Duration: {info['duration']:.2f} seconds")
            print("="*50)
        else:
            print("❌ Could not read video information")


class ImageInfo:
    """Image information class"""

    @staticmethod
    def get_image_info(image_path: Union[str, Path]) -> Optional[dict]:
        """
        Get image information

        Args:
```

```
        image_path: Path to image file

    Returns:
        Dictionary containing image information
    """
    img = cv2.imread(str(image_path))

    if img is None:
        return None

    info = {
        'width': img.shape[1],
        'height': img.shape[0],
        'channels': img.shape[2] if len(img.shape) > 2 else 1,
        'size': Path(image_path).stat().st_size / 1024,  # KB
        'dtype': str(img.dtype)
    }

    return info

@staticmethod
def print_image_info(image_path: Union[str, Path]):
    """
    Print image information

    Args:
        image_path: Path to image file
    """
    info = ImageInfo.get_image_info(image_path)

    if info:
        print("\n📸 Image Information:")
        print("="*50)
        print(f"📐 Dimensions: {info['width']}x{info['height']}")
        print(f"🎨 Channels: {info['channels']}")
        print(f"💾 Size: {info['size']:.2f} KB")
        print(f"📊 Data Type: {info['dtype']}")
```

```
            print("="*50)
        else:
            print("❌ Could not read image information")


class FileManager:
    """File management class"""

    @staticmethod
    def get_all_images(directory: Union[str, Path]) -> List[Path]:
        """
        Get all images in directory

        Args:
            directory: Directory path to search

        Returns:
            List of image file paths
        """
        directory = Path(directory)
        if not directory.exists():
            return []

        extensions = ['.jpg', '.jpeg', '.png', '.bmp', '.webp', '.tiff']

        images = []
        for ext in extensions:
            images.extend(directory.glob(f'*{ext}'))
            images.extend(directory.glob(f'*{ext.upper()}'))

        return sorted(images)

    @staticmethod
    def get_all_videos(directory: Union[str, Path]) -> List[Path]:
        """
        Get all videos in directory
```

```
    Args:
        directory: Directory path to search

    Returns:
        List of video file paths
    """
    directory = Path(directory)
    if not directory.exists():
        return []

    extensions = ['.mp4', '.avi', '.mov', '.mkv', '.flv', '.wmv', '.webm']

    videos = []
    for ext in extensions:
        videos.extend(directory.glob(f'*{ext}'))
        videos.extend(directory.glob(f'*{ext.upper()}'))

    return sorted(videos)

@staticmethod
def create_output_path(input_path: Union[str, Path],
                       output_dir: Union[str, Path],
                       prefix: str = 'output_',
                       suffix: str = '') -> Path:
    """
    Create output path for processed file

    Args:
        input_path: Input file path
        output_dir: Output directory
        prefix: Prefix for output filename
        suffix: Suffix for output filename

    Returns:
        Output file path
    """
    input_path = Path(input_path)
```

```
        output_dir = Path(output_dir)
        output_dir.mkdir(parents=True, exist_ok=True)

        stem = input_path.stem
        ext = input_path.suffix

        return output_dir / f"{prefix}{stem}{suffix}{ext}"

    @staticmethod
    def get_timestamp() -> str:
        """
        Get current timestamp string

        Returns:
            Timestamp string in format YYYYMMDD_HHMMSS
        """
        return datetime.now().strftime("%Y%m%d_%H%M%S")


class ResultSaver:
    """Results saving class"""

    @staticmethod
    def save_results_json(results: dict, output_path: Union[str, Path]):
        """
        Save results to JSON file

        Args:
            results: Results dictionary to save
            output_path: Output file path
        """
        output_path = Path(output_path)
        output_path.parent.mkdir(parents=True, exist_ok=True)

        with open(output_path, 'w', encoding='utf-8') as f:
            json.dump(results, f, ensure_ascii=False, indent=2)
```

```
        print(f"💾 JSON results saved to: {output_path}")

    @staticmethod
    def save_keypoints_csv(keypoints_data: List[dict], output_path: Union[str, Path]):
        """
        Save keypoints to CSV file

        Args:
            keypoints_data: List of keypoints dictionaries
            output_path: Output CSV file path
        """
        # ============================================
        # ✅ FIXED: Better pandas handling
        # ============================================
        if not PANDAS_AVAILABLE:
            print("⚠️  pandas not installed, skipping CSV export")
            print("💡 Install with: pip install pandas")
            return

        try:
            output_path = Path(output_path)
            output_path.parent.mkdir(parents=True, exist_ok=True)

            df = pd.DataFrame(keypoints_data)
            df.to_csv(output_path, index=False)

            print(f"💾 Keypoints CSV saved to: {output_path}")
        except Exception as e:
            print(f"❌ Error saving CSV: {e}")

    @staticmethod
    def save_tracking_results(tracking_data: List[dict], output_path: Union[str, Path]):
        """
        Save tracking results to JSON file
```

```
        Args:
            tracking_data: List of tracking dictionaries
            output_path: Output file path
        """
        output_path = Path(output_path)
        output_path.parent.mkdir(parents=True, exist_ok=True)

        results = {
            'timestamp': datetime.now().isoformat(),
            'total_frames': len(tracking_data),
            'tracking_data': tracking_data
        }

        with open(output_path, 'w', encoding='utf-8') as f:
            json.dump(results, f, ensure_ascii=False, indent=2)

        print(f"💾 Tracking results saved to: {output_path}")


class VisualizationHelper:
    """Visualization helper class"""

    @staticmethod
    def draw_bbox(frame: np.ndarray,
                  bbox: Tuple[int, int, int, int],
                  track_id: Optional[int] = None,
                  color: Tuple[int, int, int] = (255, 0, 0),
                  thickness: int = 2) -> np.ndarray:
        """
        Draw bounding box on frame

        Args:
            frame: Input frame
            bbox: Bounding box (x1, y1, x2, y2)
            track_id: Track ID (optional)
            color: Box color
            thickness: Line thickness
```

```
    Returns:
        Frame with drawn bbox
    """
    x1, y1, x2, y2 = map(int, bbox)
    cv2.rectangle(frame, (x1, y1), (x2, y2), color, thickness)

    if track_id is not None:
        label = f"ID: {track_id}"
        font = cv2.FONT_HERSHEY_SIMPLEX
        font_scale = 0.6
        font_thickness = 2

        # Get text size
        (text_width, text_height), baseline = cv2.getTextSize(
            label, font, font_scale, font_thickness
        )

        # Draw background rectangle for text
        cv2.rectangle(frame,
                      (x1, y1 - text_height - 10),
                      (x1 + text_width + 10, y1),
                      color, -1)

        # Draw text
        cv2.putText(frame, label,
                    (x1 + 5, y1 - 5),
                    font, font_scale, (255, 255, 255), font_thickness)

    return frame

@staticmethod
def draw_keypoints(frame: np.ndarray,
                   keypoints: np.ndarray,
                   confidences: np.ndarray,
                   keypoint_colors: List[Tuple[int, int, int]],
                   radius: int = 5,
```

```
                conf_threshold: float = 0.5) -> np.ndarray:
    """
    Draw keypoints on frame

    Args:
        frame: Input frame
        keypoints: Keypoint coordinates (N, 2)
        confidences: Keypoint confidences (N,)
        keypoint_colors: Colors for each keypoint
        radius: Circle radius
        conf_threshold: Confidence threshold

    Returns:
        Frame with drawn keypoints
    """
    for idx, (kpt, conf) in enumerate(zip(keypoints, confidences)):
        if conf > conf_threshold:
            x, y = int(kpt[0]), int(kpt[1])
            color = keypoint_colors[idx % len(keypoint_colors)]

            # Draw filled circle
            cv2.circle(frame, (x, y), radius, color, -1)
            # Draw border
            cv2.circle(frame, (x, y), radius, (255, 255, 255), 2)

    return frame

@staticmethod
def draw_skeleton(frame: np.ndarray,
                  keypoints: np.ndarray,
                  confidences: np.ndarray,
                  skeleton_links: List[List[int]],
                  color: Tuple[int, int, int] = (0, 255, 0),
                  thickness: int = 2,
                  conf_threshold: float = 0.5) -> np.ndarray:
    """
    Draw skeleton connections on frame
```

```
    Args:
        frame: Input frame
        keypoints: Keypoint coordinates (N, 2)
        confidences: Keypoint confidences (N,)
        skeleton_links: List of keypoint pairs to connect
        color: Line color
        thickness: Line thickness
        conf_threshold: Confidence threshold

    Returns:
        Frame with drawn skeleton
    """
    for link in skeleton_links:
        # ============================================
        # ✅ FIXED: Better index handling for skeleton links
        # ============================================
        start_idx = link[0]
        end_idx = link[1]

        if (start_idx < len(keypoints) and end_idx < len(keypoints) and
            confidences[start_idx] > conf_threshold and
            confidences[end_idx] > conf_threshold):

            start_point = tuple(map(int, keypoints[start_idx]))
            end_point = tuple(map(int, keypoints[end_idx]))

            cv2.line(frame, start_point, end_point, color, thickness)

    return frame

@staticmethod
def put_fps_text(frame: np.ndarray,
                 fps: float,
                 position: Tuple[int, int] = (10, 30),
                 color: Tuple[int, int, int] = (0, 255, 0)) -> np.ndarray:
    """
```

```
    Put FPS text on frame

    Args:
        frame: Input frame
        fps: FPS value
        position: Text position
        color: Text color

    Returns:
        Frame with FPS text
    """
    text = f'FPS: {fps:.2f}'
    cv2.putText(frame, text, position,
                cv2.FONT_HERSHEY_SIMPLEX, 1, color, 2)
    return frame

@staticmethod
def put_info_text(frame: np.ndarray,
                  text: str,
                  position: Tuple[int, int] = (10, 70),
                  color: Tuple[int, int, int] = (0, 255, 0)) -> np.ndarray:
    """
    Put info text on frame

    Args:
        frame: Input frame
        text: Text to display
        position: Text position
        color: Text color

    Returns:
        Frame with info text
    """
    cv2.putText(frame, text, position,
                cv2.FONT_HERSHEY_SIMPLEX, 0.8, color, 2)
    return frame
```

```
class PerformanceMonitor:
    """Performance monitoring class"""

    def __init__(self):
        """Initialize performance monitor"""
        self.start_time = None
        self.frame_times = []
        self.frame_count = 0

    def start(self):
        """Start timing"""
        self.start_time = time.time()
        self.frame_times = []
        self.frame_count = 0

    def add_frame(self):
        """Add frame timing"""
        if self.start_time:
            self.frame_times.append(time.time())
            self.frame_count += 1

    def get_current_fps(self) -> float:
        """
        Get current FPS

        Returns:
            Current FPS value
        """
        if len(self.frame_times) < 2:
            return 0.0

        # Calculate FPS from last N frames
        n_frames = min(30, len(self.frame_times))
        time_diff = self.frame_times[-1] - self.frame_times[-n_frames]

        if time_diff > 0:
```

```
        return n_frames / time_diff
    return 0.0

def get_stats(self) -> Optional[dict]:
    """
    Get performance statistics

    Returns:
        Dictionary containing performance metrics
    """
    if not self.start_time or not self.frame_times:
        return None

    total_time = time.time() - self.start_time
    avg_fps = self.frame_count / total_time if total_time > 0 else 0

    return {
        'total_time': total_time,
        'frame_count': self.frame_count,
        'avg_fps': avg_fps,
        'current_fps': self.get_current_fps()
    }

def print_stats(self):
    """Print performance statistics"""
    stats = self.get_stats()
    if stats:
        print("\n📊 Performance Statistics:")
        print("="*50)
        print(f"⏱️  Total Time: {stats['total_time']:.2f} seconds")
        print(f"📊 Frame Count: {stats['frame_count']}")
        print(f"📈 Average FPS: {stats['avg_fps']:.2f}")
        print(f"⚡ Current FPS: {stats['current_fps']:.2f}")
        print("="*50)
```

```
class ModelDownloader:
    """Model download helper class"""

    @staticmethod
    def download_file(url: str, output_path: Union[str, Path]) -> bool:
        """
        Download file from URL

        Args:
            url: Download URL
            output_path: Output file path

        Returns:
            True if successful, False otherwise
        """
        try:
            import urllib.request

            # ============================================
            # ✅ FIXED: Better tqdm handling
            # ============================================
            try:
                from tqdm import tqdm
                TQDM_AVAILABLE = True
            except ImportError:
                TQDM_AVAILABLE = False
                print("💡 Tip: Install tqdm for progress bar: pip install tqdm")

            output_path = Path(output_path)
            output_path.parent.mkdir(parents=True, exist_ok=True)

            print(f"📥 Downloading from: {url}")
            print(f"📁 Saving to: {output_path}")

            if TQDM_AVAILABLE:
                class DownloadProgressBar(tqdm):
                    def update_to(self, b=1, bsize=1, tsize=None):
```

```
                if tsize is not None:
                    self.total = tsize
                self.update(b * bsize - self.n)

            with DownloadProgressBar(unit='B', unit_scale=True,
                                     miniters=1, desc=output_path.name) as t:
                urllib.request.urlretrieve(url, filename=output_path,
                                           reporthook=t.update_to)
        else:
            # Download without progress bar
            urllib.request.urlretrieve(url, filename=output_path)

        print(f"✅ Download completed!")
        return True

    except Exception as e:
        print(f"❌ Download failed: {e}")
        return False


def test_utils():
    """Test utility functions"""
    print("🧪 Testing utility functions...")

    # Test FileManager
    print("\n📁 Testing FileManager:")
    images = FileManager.get_all_images('input/images')
    print(f"Number of images found: {len(images)}")

    videos = FileManager.get_all_videos('input/videos')
    print(f"Number of videos found: {len(videos)}")

    # Test timestamp
    print(f"\n⏰ Current timestamp: {FileManager.get_timestamp()}")

    # Test PerformanceMonitor
```

```
    print("\n⏱️  Testing PerformanceMonitor:")
    monitor = PerformanceMonitor()
    monitor.start()

    for i in range(10):
        time.sleep(0.1)
        monitor.add_frame()

    monitor.print_stats()

    print("\n✅ All tests completed successfully!")


if __name__ == "__main__":
    test_utils()
```

```
auto_scale_lr = dict(base_batch_size=16, enable=False)
backend_args = None
base_lr = 0.004
custom_hooks = [
    dict(
        ema_type='ExpMomentumEMA',
        momentum=0.0002,
        priority=49,
        type='EMAHook',
        update_buffers=True),
    dict(
        switch_epoch=280,
        switch_pipeline=[
            dict(backend_args=None, type='LoadImageFromFile'),
            dict(type='LoadAnnotations', with_bbox=True),
            dict(
                keep_ratio=True,
                ratio_range=(
                    0.1,
```

```
                2.0,
            ),
            scale=(
                640,
                640,
            ),
            type='RandomResize'),
        dict(crop_size=(
            640,
            640,
        ), type='RandomCrop'),
        dict(type='YOLOXHSVRandomAug'),
        dict(prob=0.5, type='RandomFlip'),
        dict(
            pad_val=dict(img=(
                114,
                114,
                114,
            )),
            size=(
                640,
                640,
            ),
            type='Pad'),
        dict(type='PackDetInputs'),
    ],
    type='PipelineSwitchHook'),
]
data_root = 'data/coco/'
dataset_type = 'CocoDataset'
default_hooks = dict(
    checkpoint=dict(interval=10, max_keep_ckpts=3, type='CheckpointHook'),
    logger=dict(interval=50, type='LoggerHook'),
    param_scheduler=dict(type='ParamSchedulerHook'),
    sampler_seed=dict(type='DistSamplerSeedHook'),
    timer=dict(type='IterTimerHook'),
    visualization=dict(type='DetVisualizationHook'))
```

```
default_scope = 'mmdet'
env_cfg = dict(
    cudnn_benchmark=False,
    dist_cfg=dict(backend='nccl'),
    mp_cfg=dict(mp_start_method='fork', opencv_num_threads=0))
img_scales = [
    (
        640,
        640,
    ),
    (
        320,
        320,
    ),
    (
        960,
        960,
    ),
]
interval = 10
load_from = None
log_level = 'INFO'
log_processor = dict(by_epoch=True, type='LogProcessor', window_size=50)
max_epochs = 300
model = dict(
    backbone=dict(
        act_cfg=dict(inplace=True, type='SiLU'),
        arch='P5',
        channel_attention=True,
        deepen_factor=0.67,
        expand_ratio=0.5,
        norm_cfg=dict(type='SyncBN'),
        type='CSPNeXt',
        widen_factor=0.75),
    bbox_head=dict(
        act_cfg=dict(inplace=True, type='SiLU'),
        anchor_generator=dict(
```

```
        offset=0, strides=[
            8,
            16,
            32,
        ], type='MlvlPointGenerator'),
    bbox_coder=dict(type='DistancePointBBoxCoder'),
    exp_on_reg=True,
    feat_channels=192,
    in_channels=192,
    loss_bbox=dict(loss_weight=2.0, type='GIoULoss'),
    loss_cls=dict(
        beta=2.0,
        loss_weight=1.0,
        type='QualityFocalLoss',
        use_sigmoid=True),
    norm_cfg=dict(type='SyncBN'),
    num_classes=80,
    pred_kernel_size=1,
    share_conv=True,
    stacked_convs=2,
    type='RTMDetSepBNHead',
    with_objectness=False),
data_preprocessor=dict(
    batch_augments=None,
    bgr_to_rgb=False,
    mean=[
        103.53,
        116.28,
        123.675,
    ],
    std=[
        57.375,
        57.12,
        58.395,
    ],
    type='DetDataPreprocessor'),
neck=dict(
```

```
        act_cfg=dict(inplace=True, type='SiLU'),
        expand_ratio=0.5,
        in_channels=[
            192,
            384,
            768,
        ],
        norm_cfg=dict(type='SyncBN'),
        num_csp_blocks=2,
        out_channels=192,
        type='CSPNeXtPAFPN'),
    test_cfg=dict(
        max_per_img=300,
        min_bbox_size=0,
        nms=dict(iou_threshold=0.65, type='nms'),
        nms_pre=30000,
        score_thr=0.001),
    train_cfg=dict(
        allowed_border=-1,
        assigner=dict(topk=13, type='DynamicSoftLabelAssigner'),
        debug=False,
        pos_weight=-1),
    type='RTMDet')
optim_wrapper = dict(
    optimizer=dict(lr=0.004, type='AdamW', weight_decay=0.05),
    paramwise_cfg=dict(
        bias_decay_mult=0, bypass_duplicate=True, norm_decay_mult=0),
    type='OptimWrapper')
param_scheduler = [
    dict(
        begin=0, by_epoch=False, end=1000, start_factor=1e-05,
        type='LinearLR'),
    dict(
        T_max=150,
        begin=150,
        by_epoch=True,
        convert_to_iter_based=True,
```

```
        end=300,
        eta_min=0.0002,
        type='CosineAnnealingLR'),
]
resume = False
stage2_num_epochs = 20
test_cfg = dict(type='TestLoop')
test_dataloader = dict(
    batch_size=5,
    dataset=dict(
        ann_file='annotations/instances_val2017.json',
        backend_args=None,
        data_prefix=dict(img='val2017/'),
        data_root='data/coco/',
        pipeline=[
            dict(backend_args=None, type='LoadImageFromFile'),
            dict(keep_ratio=True, scale=(
                640,
                640,
            ), type='Resize'),
            dict(
                pad_val=dict(img=(
                    114,
                    114,
                    114,
                )),
                size=(
                    640,
                    640,
                ),
                type='Pad'),
            dict(type='LoadAnnotations', with_bbox=True),
            dict(
                meta_keys=(
                    'img_id',
                    'img_path',
                    'ori_shape',
```

```
                'img_shape',
                'scale_factor',
            ),
            type='PackDetInputs'),
        ],
        test_mode=True,
        type='CocoDataset'),
    drop_last=False,
    num_workers=10,
    persistent_workers=True,
    sampler=dict(shuffle=False, type='DefaultSampler'))
test_evaluator = dict(
    ann_file='data/coco/annotations/instances_val2017.json',
    backend_args=None,
    format_only=False,
    metric='bbox',
    proposal_nums=(
        100,
        1,
        10,
    ),
    type='CocoMetric')
test_pipeline = [
    dict(backend_args=None, type='LoadImageFromFile'),
    dict(keep_ratio=True, scale=(
        640,
        640,
    ), type='Resize'),
    dict(pad_val=dict(img=(
        114,
        114,
        114,
    )), size=(
        640,
        640,
    ), type='Pad'),
    dict(type='LoadAnnotations', with_bbox=True),
```

```
    dict(
        meta_keys=(
            'img_id',
            'img_path',
            'ori_shape',
            'img_shape',
            'scale_factor',
        ),
        type='PackDetInputs'),
]
train_cfg = dict(
    dynamic_intervals=[
        (
            280,
            1,
        ),
    ],
    max_epochs=300,
    type='EpochBasedTrainLoop',
    val_interval=10)
train_dataloader = dict(
    batch_sampler=None,
    batch_size=32,
    dataset=dict(
        ann_file='annotations/instances_train2017.json',
        backend_args=None,
        data_prefix=dict(img='train2017/'),
        data_root='data/coco/',
        filter_cfg=dict(filter_empty_gt=True, min_size=32),
        pipeline=[
            dict(backend_args=None, type='LoadImageFromFile'),
            dict(type='LoadAnnotations', with_bbox=True),
            dict(img_scale=(
                640,
                640,
            ), pad_val=114.0, type='CachedMosaic'),
            dict(
```

```
    keep_ratio=True,
    ratio_range=(
        0.1,
        2.0,
    ),
    scale=(
        1280,
        1280,
    ),
    type='RandomResize'),
dict(crop_size=(
    640,
    640,
), type='RandomCrop'),
dict(type='YOLOXHSVRandomAug'),
dict(prob=0.5, type='RandomFlip'),
dict(
    pad_val=dict(img=(
        114,
        114,
        114,
    )),
    size=(
        640,
        640,
    ),
    type='Pad'),
dict(
    img_scale=(
        640,
        640,
    ),
    max_cached_images=20,
    pad_val=(
        114,
        114,
        114,
```

```
            ),
            ratio_range=(
                1.0,
                1.0,
            ),
            type='CachedMixUp'),
        dict(type='PackDetInputs'),
    ],
    type='CocoDataset'),
  num_workers=10,
  persistent_workers=True,
  pin_memory=True,
  sampler=dict(shuffle=True, type='DefaultSampler'))
train_pipeline = [
  dict(backend_args=None, type='LoadImageFromFile'),
  dict(type='LoadAnnotations', with_bbox=True),
  dict(img_scale=(
    640,
    640,
  ), pad_val=114.0, type='CachedMosaic'),
  dict(
    keep_ratio=True,
    ratio_range=(
      0.1,
      2.0,
    ),
    scale=(
      1280,
      1280,
    ),
    type='RandomResize'),
  dict(crop_size=(
    640,
    640,
  ), type='RandomCrop'),
  dict(type='YOLOXHSVRandomAug'),
  dict(prob=0.5, type='RandomFlip'),
```

```
    dict(pad_val=dict(img=(
        114,
        114,
        114,
    )), size=(
        640,
        640,
    ), type='Pad'),
    dict(
        img_scale=(
            640,
            640,
        ),
        max_cached_images=20,
        pad_val=(
            114,
            114,
            114,
        ),
        ratio_range=(
            1.0,
            1.0,
        ),
        type='CachedMixUp'),
    dict(type='PackDetInputs'),
]
train_pipeline_stage2 = [
    dict(backend_args=None, type='LoadImageFromFile'),
    dict(type='LoadAnnotations', with_bbox=True),
    dict(
        keep_ratio=True,
        ratio_range=(
            0.1,
            2.0,
        ),
        scale=(
            640,
```

```
            640,
        ),
        type='RandomResize'),
    dict(crop_size=(
        640,
        640,
    ), type='RandomCrop'),
    dict(type='YOLOXHSVRandomAug'),
    dict(prob=0.5, type='RandomFlip'),
    dict(pad_val=dict(img=(
        114,
        114,
        114,
    )), size=(
        640,
        640,
    ), type='Pad'),
    dict(type='PackDetInputs'),
]
tta_model = dict(
    tta_cfg=dict(max_per_img=100, nms=dict(iou_threshold=0.6, type='nms')),
    type='DetTTAModel')
tta_pipeline = [
    dict(backend_args=None, type='LoadImageFromFile'),
    dict(
        transforms=[
            [
                dict(keep_ratio=True, scale=(
                    640,
                    640,
                ), type='Resize'),
                dict(keep_ratio=True, scale=(
                    320,
                    320,
                ), type='Resize'),
                dict(keep_ratio=True, scale=(
                    960,
```

```
            960,
        ), type='Resize'),
    ],
    [
        dict(prob=1.0, type='RandomFlip'),
        dict(prob=0.0, type='RandomFlip'),
    ],
    [
        dict(
            pad_val=dict(img=(
                114,
                114,
                114,
            )),
            size=(
                960,
                960,
            ),
            type='Pad'),
    ],
    [
        dict(type='LoadAnnotations', with_bbox=True),
    ],
    [
        dict(
            meta_keys=(
                'img_id',
                'img_path',
                'ori_shape',
                'img_shape',
                'scale_factor',
                'flip',
                'flip_direction',
            ),
            type='PackDetInputs'),
    ],
],
```

```
        type='TestTimeAug'),
]
val_cfg = dict(type='ValLoop')
val_dataloader = dict(
    batch_size=5,
    dataset=dict(
        ann_file='annotations/instances_val2017.json',
        backend_args=None,
        data_prefix=dict(img='val2017/'),
        data_root='data/coco/',
        pipeline=[
            dict(backend_args=None, type='LoadImageFromFile'),
            dict(keep_ratio=True, scale=(
                640,
                640,
            ), type='Resize'),
            dict(
                pad_val=dict(img=(
                    114,
                    114,
                    114,
                )),
                size=(
                    640,
                    640,
                ),
                type='Pad'),
            dict(type='LoadAnnotations', with_bbox=True),
            dict(
                meta_keys=(
                    'img_id',
                    'img_path',
                    'ori_shape',
                    'img_shape',
                    'scale_factor',
                ),
                type='PackDetInputs'),
```

```
        ],
        test_mode=True,
        type='CocoDataset'),
    drop_last=False,
    num_workers=10,
    persistent_workers=True,
    sampler=dict(shuffle=False, type='DefaultSampler'))
val_evaluator = dict(
    ann_file='data/coco/annotations/instances_val2017.json',
    backend_args=None,
    format_only=False,
    metric='bbox',
    proposal_nums=(
        100,
        1,
        10,
    ),
    type='CocoMetric')
vis_backends = [
    dict(type='LocalVisBackend'),
]
visualizer = dict(
    name='visualizer',
    type='DetLocalVisualizer',
    vis_backends=[
        dict(type='LocalVisBackend'),
    ])
```

```
auto_scale_lr = dict(base_batch_size=1024)
backend_args = dict(backend='local')
base_lr = 0.004
codec = dict(
    input_size=(
        192,
        256,
```

```
    ),
    normalize=False,
    sigma=(
        4.9,
        5.66,
    ),
    simcc_split_ratio=2.0,
    type='SimCCLabel',
    use_dark=False)
custom_hooks = [
    dict(
        ema_type='ExpMomentumEMA',
        momentum=0.0002,
        priority=49,
        type='EMAHook',
        update_buffers=True),
    dict(
        switch_epoch=390,
        switch_pipeline=[
            dict(backend_args=dict(backend='local'), type='LoadImage'),
            dict(type='GetBBoxCenterScale'),
            dict(direction='horizontal', type='RandomFlip'),
            dict(type='RandomHalfBody'),
            dict(
                rotate_factor=60,
                scale_factor=[
                    0.75,
                    1.25,
                ],
                shift_factor=0.0,
                type='RandomBBoxTransform'),
            dict(input_size=(
                192,
                256,
            ), type='TopdownAffine'),
            dict(type='mmdet.YOLOXHSVRandomAug'),
            dict(
```

```
            transforms=[
                dict(p=0.1, type='Blur'),
                dict(p=0.1, type='MedianBlur'),
                dict(
                    max_height=0.4,
                    max_holes=1,
                    max_width=0.4,
                    min_height=0.2,
                    min_holes=1,
                    min_width=0.2,
                    p=0.5,
                    type='CoarseDropout'),
            ],
            type='Albumentation'),
        dict(
            encoder=dict(
                input_size=(
                    192,
                    256,
                ),
                normalize=False,
                sigma=(
                    4.9,
                    5.66,
                ),
                simcc_split_ratio=2.0,
                type='SimCCLabel',
                use_dark=False),
            type='GenerateTarget'),
        dict(type='PackPoseInputs'),
    ],
    type='mmdet.PipelineSwitchHook'),
]
data_mode = 'topdown'
data_root = 'data/coco/'
dataset_type = 'CocoDataset'
default_hooks = dict(
```

```
    checkpoint=dict(
        interval=10,
        max_keep_ckpts=1,
        rule='greater',
        save_best='coco/AP',
        type='CheckpointHook'),
    logger=dict(interval=50, type='LoggerHook'),
    param_scheduler=dict(type='ParamSchedulerHook'),
    sampler_seed=dict(type='DistSamplerSeedHook'),
    timer=dict(type='IterTimerHook'),
    visualization=dict(enable=False, type='PoseVisualizationHook'))
default_scope = 'mmpose'
env_cfg = dict(
    cudnn_benchmark=False,
    dist_cfg=dict(backend='nccl'),
    mp_cfg=dict(mp_start_method='fork', opencv_num_threads=0))
load_from = None
log_level = 'INFO'
log_processor = dict(
    by_epoch=True, num_digits=6, type='LogProcessor', window_size=50)
max_epochs = 420
model = dict(
    backbone=dict(
        _scope_='mmdet',
        act_cfg=dict(type='SiLU'),
        arch='P5',
        channel_attention=True,
        deepen_factor=0.67,
        expand_ratio=0.5,
        init_cfg=dict(
            checkpoint=

'https://download.openmmlab.com/mmpose/v1/projects/rtmposev1/cspnext-m
_udp-aic-coco_210e-256x192-f2f7d6f6_20230130.pth',
            prefix='backbone.',
            type='Pretrained'),
        norm_cfg=dict(type='SyncBN'),
```

```
    out_indices=(4, ),
    type='CSPNeXt',
    widen_factor=0.75),
data_preprocessor=dict(
    bgr_to_rgb=True,
    mean=[
        123.675,
        116.28,
        103.53,
    ],
    std=[
        58.395,
        57.12,
        57.375,
    ],
    type='PoseDataPreprocessor'),
head=dict(
    decoder=dict(
        input_size=(
            192,
            256,
        ),
        normalize=False,
        sigma=(
            4.9,
            5.66,
        ),
        simcc_split_ratio=2.0,
        type='SimCCLabel',
        use_dark=False),
    final_layer_kernel_size=7,
    gau_cfg=dict(
        act_fn='SiLU',
        drop_path=0.0,
        dropout_rate=0.0,
        expansion_factor=2,
        hidden_dims=256,
```

```
            pos_enc=False,
            s=128,
            use_rel_bias=False),
        in_channels=768,
        in_featuremap_size=(
            6,
            8,
        ),
        input_size=(
            192,
            256,
        ),
        loss=dict(
            beta=10.0,
            label_softmax=True,
            type='KLDiscretLoss',
            use_target_weight=True),
        out_channels=17,
        simcc_split_ratio=2.0,
        type='RTMCCHead'),
    test_cfg=dict(flip_test=True),
    type='TopdownPoseEstimator')
optim_wrapper = dict(
    optimizer=dict(lr=0.004, type='AdamW', weight_decay=0.05),
    paramwise_cfg=dict(
        bias_decay_mult=0, bypass_duplicate=True, norm_decay_mult=0),
    type='OptimWrapper')
param_scheduler = [
    dict(
        begin=0, by_epoch=False, end=1000, start_factor=1e-05,
        type='LinearLR'),
    dict(
        T_max=210,
        begin=210,
        by_epoch=True,
        convert_to_iter_based=True,
        end=420,
```

```
        eta_min=0.0002,
        type='CosineAnnealingLR'),
]
randomness = dict(seed=21)
resume = False
stage2_num_epochs = 30
test_cfg = dict()
test_dataloader = dict(
    batch_size=64,
    dataset=dict(
        ann_file='annotations/person_keypoints_val2017.json',
        data_mode='topdown',
        data_prefix=dict(img='val2017/'),
        data_root='data/coco/',
        pipeline=[
            dict(backend_args=dict(backend='local'), type='LoadImage'),
            dict(type='GetBBoxCenterScale'),
            dict(input_size=(
                192,
                256,
            ), type='TopdownAffine'),
            dict(type='PackPoseInputs'),
        ],
        test_mode=True,
        type='CocoDataset'),
    drop_last=False,
    num_workers=10,
    persistent_workers=True,
    sampler=dict(round_up=False, shuffle=False, type='DefaultSampler'))
test_evaluator = dict(
    ann_file='data/coco/annotations/person_keypoints_val2017.json',
    type='CocoMetric')
train_cfg = dict(by_epoch=True, max_epochs=420, val_interval=10)
train_dataloader = dict(
    batch_size=256,
    dataset=dict(
        ann_file='annotations/person_keypoints_train2017.json',
```

```
data_mode='topdown',
data_prefix=dict(img='train2017/'),
data_root='data/coco/',
pipeline=[
    dict(backend_args=dict(backend='local'), type='LoadImage'),
    dict(type='GetBBoxCenterScale'),
    dict(direction='horizontal', type='RandomFlip'),
    dict(type='RandomHalfBody'),
    dict(
        rotate_factor=80,
        scale_factor=[
            0.6,
            1.4,
        ],
        type='RandomBBoxTransform'),
    dict(input_size=(
        192,
        256,
    ), type='TopdownAffine'),
    dict(type='mmdet.YOLOXHSVRandomAug'),
    dict(
        transforms=[
            dict(p=0.1, type='Blur'),
            dict(p=0.1, type='MedianBlur'),
            dict(
                max_height=0.4,
                max_holes=1,
                max_width=0.4,
                min_height=0.2,
                min_holes=1,
                min_width=0.2,
                p=1.0,
                type='CoarseDropout'),
        ],
        type='Albumentation'),
    dict(
        encoder=dict(
```

```
            input_size=(
                192,
                256,
            ),
            normalize=False,
            sigma=(
                4.9,
                5.66,
            ),
            simcc_split_ratio=2.0,
            type='SimCCLabel',
            use_dark=False),
        type='GenerateTarget'),
    dict(type='PackPoseInputs'),
],
type='CocoDataset'),
num_workers=10,
persistent_workers=True,
sampler=dict(shuffle=True, type='DefaultSampler'))
train_pipeline = [
dict(backend_args=dict(backend='local'), type='LoadImage'),
dict(type='GetBBoxCenterScale'),
dict(direction='horizontal', type='RandomFlip'),
dict(type='RandomHalfBody'),
dict(
    rotate_factor=80,
    scale_factor=[
        0.6,
        1.4,
    ],
    type='RandomBBoxTransform'),
dict(input_size=(
    192,
    256,
), type='TopdownAffine'),
dict(type='mmdet.YOLOXHSVRandomAug'),
dict(
```

```
        transforms=[
            dict(p=0.1, type='Blur'),
            dict(p=0.1, type='MedianBlur'),
            dict(
                max_height=0.4,
                max_holes=1,
                max_width=0.4,
                min_height=0.2,
                min_holes=1,
                min_width=0.2,
                p=1.0,
                type='CoarseDropout'),
        ],
        type='Albumentation'),
    dict(
        encoder=dict(
            input_size=(
                192,
                256,
            ),
            normalize=False,
            sigma=(
                4.9,
                5.66,
            ),
            simcc_split_ratio=2.0,
            type='SimCCLabel',
            use_dark=False),
        type='GenerateTarget'),
    dict(type='PackPoseInputs'),
]
train_pipeline_stage2 = [
    dict(backend_args=dict(backend='local'), type='LoadImage'),
    dict(type='GetBBoxCenterScale'),
    dict(direction='horizontal', type='RandomFlip'),
    dict(type='RandomHalfBody'),
    dict(
```

```
    rotate_factor=60,
    scale_factor=[
        0.75,
        1.25,
    ],
    shift_factor=0.0,
    type='RandomBBoxTransform'),
dict(input_size=(
    192,
    256,
), type='TopdownAffine'),
dict(type='mmdet.YOLOXHSVRandomAug'),
dict(
    transforms=[
        dict(p=0.1, type='Blur'),
        dict(p=0.1, type='MedianBlur'),
        dict(
            max_height=0.4,
            max_holes=1,
            max_width=0.4,
            min_height=0.2,
            min_holes=1,
            min_width=0.2,
            p=0.5,
            type='CoarseDropout'),
    ],
    type='Albumentation'),
dict(
    encoder=dict(
        input_size=(
            192,
            256,
        ),
        normalize=False,
        sigma=(
            4.9,
            5.66,
```

```
            ),
            simcc_split_ratio=2.0,
            type='SimCCLabel',
            use_dark=False),
        type='GenerateTarget'),
    dict(type='PackPoseInputs'),
]
val_cfg = dict()
val_dataloader = dict(
    batch_size=64,
    dataset=dict(
        ann_file='annotations/person_keypoints_val2017.json',
        data_mode='topdown',
        data_prefix=dict(img='val2017/'),
        data_root='data/coco/',
        pipeline=[
            dict(backend_args=dict(backend='local'), type='LoadImage'),
            dict(type='GetBBoxCenterScale'),
            dict(input_size=(
                192,
                256,
            ), type='TopdownAffine'),
            dict(type='PackPoseInputs'),
        ],
        test_mode=True,
        type='CocoDataset'),
    drop_last=False,
    num_workers=10,
    persistent_workers=True,
    sampler=dict(round_up=False, shuffle=False, type='DefaultSampler'))
val_evaluator = dict(
    ann_file='data/coco/annotations/person_keypoints_val2017.json',
    type='CocoMetric')
val_pipeline = [
    dict(backend_args=dict(backend='local'), type='LoadImage'),
    dict(type='GetBBoxCenterScale'),
    dict(input_size=(
```

```
        192,
        256,
    ), type='TopdownAffine'),
    dict(type='PackPoseInputs'),
]
vis_backends = [
    dict(type='LocalVisBackend'),
]
visualizer = dict(
    name='visualizer',
    type='PoseLocalVisualizer',
    vis_backends=[
        dict(type='LocalVisBackend'),
    ])
```

```
auto_scale_lr = dict(base_batch_size=16, enable=False)
backend_args = None
base_lr = 0.004
custom_hooks = [
    dict(
        ema_type='ExpMomentumEMA',
        momentum=0.0002,
        priority=49,
        type='EMAHook',
        update_buffers=True),
    dict(
        switch_epoch=280,
        switch_pipeline=[
            dict(backend_args=None, type='LoadImageFromFile'),
            dict(type='LoadAnnotations', with_bbox=True),
            dict(
                keep_ratio=True,
                ratio_range=(
                    0.1,
                    2.0,
```

```
            ),
            scale=(
                640,
                640,
            ),
            type='RandomResize'),
        dict(crop_size=(
            640,
            640,
        ), type='RandomCrop'),
        dict(type='YOLOXHSVRandomAug'),
        dict(prob=0.5, type='RandomFlip'),
        dict(
            pad_val=dict(img=(
                114,
                114,
                114,
            )),
            size=(
                640,
                640,
            ),
            type='Pad'),
        dict(type='PackDetInputs'),
    ],
    type='PipelineSwitchHook'),
]
data_root = 'data/coco/'
dataset_type = 'CocoDataset'
default_hooks = dict(
    checkpoint=dict(interval=10, max_keep_ckpts=3, type='CheckpointHook'),
    logger=dict(interval=50, type='LoggerHook'),
    param_scheduler=dict(type='ParamSchedulerHook'),
    sampler_seed=dict(type='DistSamplerSeedHook'),
    timer=dict(type='IterTimerHook'),
    visualization=dict(type='DetVisualizationHook'))
default_scope = 'mmdet'
```

```
env_cfg = dict(
    cudnn_benchmark=False,
    dist_cfg=dict(backend='nccl'),
    mp_cfg=dict(mp_start_method='fork', opencv_num_threads=0))
img_scales = [
    (
        640,
        640,
    ),
    (
        320,
        320,
    ),
    (
        960,
        960,
    ),
]
interval = 10
load_from = None
log_level = 'INFO'
log_processor = dict(by_epoch=True, type='LogProcessor', window_size=50)
max_epochs = 300
model = dict(
    backbone=dict(
        act_cfg=dict(inplace=True, type='SiLU'),
        arch='P5',
        channel_attention=True,
        deepen_factor=0.67,
        expand_ratio=0.5,
        norm_cfg=dict(type='SyncBN'),
        type='CSPNeXt',
        widen_factor=0.75),
    bbox_head=dict(
        act_cfg=dict(inplace=True, type='SiLU'),
        anchor_generator=dict(
            offset=0, strides=[
```

```
            8,
            16,
            32,
        ], type='MlvlPointGenerator'),
    bbox_coder=dict(type='DistancePointBBoxCoder'),
    exp_on_reg=True,
    feat_channels=192,
    in_channels=192,
    loss_bbox=dict(loss_weight=2.0, type='GIoULoss'),
    loss_cls=dict(
        beta=2.0,
        loss_weight=1.0,
        type='QualityFocalLoss',
        use_sigmoid=True),
    norm_cfg=dict(type='SyncBN'),
    num_classes=80,
    pred_kernel_size=1,
    share_conv=True,
    stacked_convs=2,
    type='RTMDetSepBNHead',
    with_objectness=False),
data_preprocessor=dict(
    batch_augments=None,
    bgr_to_rgb=False,
    mean=[
        103.53,
        116.28,
        123.675,
    ],
    std=[
        57.375,
        57.12,
        58.395,
    ],
    type='DetDataPreprocessor'),
neck=dict(
    act_cfg=dict(inplace=True, type='SiLU'),
```

```
        expand_ratio=0.5,
        in_channels=[
            192,
            384,
            768,
        ],
        norm_cfg=dict(type='SyncBN'),
        num_csp_blocks=2,
        out_channels=192,
        type='CSPNeXtPAFPN'),
    test_cfg=dict(
        max_per_img=300,
        min_bbox_size=0,
        nms=dict(iou_threshold=0.65, type='nms'),
        nms_pre=30000,
        score_thr=0.001),
    train_cfg=dict(
        allowed_border=-1,
        assigner=dict(topk=13, type='DynamicSoftLabelAssigner'),
        debug=False,
        pos_weight=-1),
    type='RTMDet')
optim_wrapper = dict(
    optimizer=dict(lr=0.004, type='AdamW', weight_decay=0.05),
    paramwise_cfg=dict(
        bias_decay_mult=0, bypass_duplicate=True, norm_decay_mult=0),
    type='OptimWrapper')
param_scheduler = [
    dict(
        begin=0, by_epoch=False, end=1000, start_factor=1e-05,
        type='LinearLR'),
    dict(
        T_max=150,
        begin=150,
        by_epoch=True,
        convert_to_iter_based=True,
        end=300,
```

```
        eta_min=0.0002,
        type='CosineAnnealingLR'),
]
resume = False
stage2_num_epochs = 20
test_cfg = dict(type='TestLoop')
test_dataloader = dict(
    batch_size=5,
    dataset=dict(
        ann_file='annotations/instances_val2017.json',
        backend_args=None,
        data_prefix=dict(img='val2017/'),
        data_root='data/coco/',
        pipeline=[
            dict(backend_args=None, type='LoadImageFromFile'),
            dict(keep_ratio=True, scale=(
                640,
                640,
            ), type='Resize'),
            dict(
                pad_val=dict(img=(
                    114,
                    114,
                    114,
                )),
                size=(
                    640,
                    640,
                ),
                type='Pad'),
            dict(type='LoadAnnotations', with_bbox=True),
            dict(
                meta_keys=(
                    'img_id',
                    'img_path',
                    'ori_shape',
                    'img_shape',
```

```
                'scale_factor',
            ),
            type='PackDetInputs'),
        ],
        test_mode=True,
        type='CocoDataset'),
    drop_last=False,
    num_workers=10,
    persistent_workers=True,
    sampler=dict(shuffle=False, type='DefaultSampler'))
test_evaluator = dict(
    ann_file='data/coco/annotations/instances_val2017.json',
    backend_args=None,
    format_only=False,
    metric='bbox',
    proposal_nums=(
        100,
        1,
        10,
    ),
    type='CocoMetric')
test_pipeline = [
    dict(backend_args=None, type='LoadImageFromFile'),
    dict(keep_ratio=True, scale=(
        640,
        640,
    ), type='Resize'),
    dict(pad_val=dict(img=(
        114,
        114,
        114,
    )), size=(
        640,
        640,
    ), type='Pad'),
    dict(type='LoadAnnotations', with_bbox=True),
    dict(
```

```
        meta_keys=(
            'img_id',
            'img_path',
            'ori_shape',
            'img_shape',
            'scale_factor',
        ),
        type='PackDetInputs'),
]
train_cfg = dict(
    dynamic_intervals=[
        (
            280,
            1,
        ),
    ],
    max_epochs=300,
    type='EpochBasedTrainLoop',
    val_interval=10)
train_dataloader = dict(
    batch_sampler=None,
    batch_size=32,
    dataset=dict(
        ann_file='annotations/instances_train2017.json',
        backend_args=None,
        data_prefix=dict(img='train2017/'),
        data_root='data/coco/',
        filter_cfg=dict(filter_empty_gt=True, min_size=32),
        pipeline=[
            dict(backend_args=None, type='LoadImageFromFile'),
            dict(type='LoadAnnotations', with_bbox=True),
            dict(img_scale=(
                640,
                640,
            ), pad_val=114.0, type='CachedMosaic'),
            dict(
                keep_ratio=True,
```

```
    ratio_range=(
        0.1,
        2.0,
    ),
    scale=(
        1280,
        1280,
    ),
    type='RandomResize'),
dict(crop_size=(
    640,
    640,
), type='RandomCrop'),
dict(type='YOLOXHSVRandomAug'),
dict(prob=0.5, type='RandomFlip'),
dict(
    pad_val=dict(img=(
        114,
        114,
        114,
    )),
    size=(
        640,
        640,
    ),
    type='Pad'),
dict(
    img_scale=(
        640,
        640,
    ),
    max_cached_images=20,
    pad_val=(
        114,
        114,
        114,
    ),
```

```
            ratio_range=(
                1.0,
                1.0,
            ),
            type='CachedMixUp'),
        dict(type='PackDetInputs'),
    ],
    type='CocoDataset'),
  num_workers=10,
  persistent_workers=True,
  pin_memory=True,
  sampler=dict(shuffle=True, type='DefaultSampler'))
train_pipeline = [
  dict(backend_args=None, type='LoadImageFromFile'),
  dict(type='LoadAnnotations', with_bbox=True),
  dict(img_scale=(
    640,
    640,
  ), pad_val=114.0, type='CachedMosaic'),
  dict(
    keep_ratio=True,
    ratio_range=(
      0.1,
      2.0,
    ),
    scale=(
      1280,
      1280,
    ),
    type='RandomResize'),
  dict(crop_size=(
    640,
    640,
  ), type='RandomCrop'),
  dict(type='YOLOXHSVRandomAug'),
  dict(prob=0.5, type='RandomFlip'),
  dict(pad_val=dict(img=(
```

```
            114,
            114,
            114,
        )), size=(
            640,
            640,
        ), type='Pad'),
        dict(
            img_scale=(
                640,
                640,
            ),
            max_cached_images=20,
            pad_val=(
                114,
                114,
                114,
            ),
            ratio_range=(
                1.0,
                1.0,
            ),
            type='CachedMixUp'),
        dict(type='PackDetInputs'),
]
train_pipeline_stage2 = [
        dict(backend_args=None, type='LoadImageFromFile'),
        dict(type='LoadAnnotations', with_bbox=True),
        dict(
            keep_ratio=True,
            ratio_range=(
                0.1,
                2.0,
            ),
            scale=(
                640,
                640,
```

```
        ),
        type='RandomResize'),
    dict(crop_size=(
        640,
        640,
    ), type='RandomCrop'),
    dict(type='YOLOXHSVRandomAug'),
    dict(prob=0.5, type='RandomFlip'),
    dict(pad_val=dict(img=(
        114,
        114,
        114,
    )), size=(
        640,
        640,
    ), type='Pad'),
    dict(type='PackDetInputs'),
]
tta_model = dict(
    tta_cfg=dict(max_per_img=100, nms=dict(iou_threshold=0.6, type='nms')),
    type='DetTTAModel')
tta_pipeline = [
    dict(backend_args=None, type='LoadImageFromFile'),
    dict(
        transforms=[
            [
                dict(keep_ratio=True, scale=(
                    640,
                    640,
                ), type='Resize'),
                dict(keep_ratio=True, scale=(
                    320,
                    320,
                ), type='Resize'),
                dict(keep_ratio=True, scale=(
                    960,
                    960,
```

```
        ), type='Resize'),
    ],
    [
        dict(prob=1.0, type='RandomFlip'),
        dict(prob=0.0, type='RandomFlip'),
    ],
    [
        dict(
            pad_val=dict(img=(
                114,
                114,
                114,
            )),
            size=(
                960,
                960,
            ),
            type='Pad'),
    ],
    [
        dict(type='LoadAnnotations', with_bbox=True),
    ],
    [
        dict(
            meta_keys=(
                'img_id',
                'img_path',
                'ori_shape',
                'img_shape',
                'scale_factor',
                'flip',
                'flip_direction',
            ),
            type='PackDetInputs'),
    ],
],
type='TestTimeAug'),
```

```
]
val_cfg = dict(type='ValLoop')
val_dataloader = dict(
    batch_size=5,
    dataset=dict(
        ann_file='annotations/instances_val2017.json',
        backend_args=None,
        data_prefix=dict(img='val2017/'),
        data_root='data/coco/',
        pipeline=[
            dict(backend_args=None, type='LoadImageFromFile'),
            dict(keep_ratio=True, scale=(
                640,
                640,
            ), type='Resize'),
            dict(
                pad_val=dict(img=(
                    114,
                    114,
                    114,
                )),
                size=(
                    640,
                    640,
                ),
                type='Pad'),
            dict(type='LoadAnnotations', with_bbox=True),
            dict(
                meta_keys=(
                    'img_id',
                    'img_path',
                    'ori_shape',
                    'img_shape',
                    'scale_factor',
                ),
                type='PackDetInputs'),
        ],
```

```
        test_mode=True,
        type='CocoDataset'),
    drop_last=False,
    num_workers=10,
    persistent_workers=True,
    sampler=dict(shuffle=False, type='DefaultSampler'))
val_evaluator = dict(
    ann_file='data/coco/annotations/instances_val2017.json',
    backend_args=None,
    format_only=False,
    metric='bbox',
    proposal_nums=(
        100,
        1,
        10,
    ),
    type='CocoMetric')
vis_backends = [
    dict(type='LocalVisBackend'),
]
visualizer = dict(
    name='visualizer',
    type='DetLocalVisualizer',
    vis_backends=[
        dict(type='LocalVisBackend'),
    ])
```

```
auto_scale_lr = dict(base_batch_size=1024)
backend_args = dict(backend='local')
base_lr = 0.004
codec = dict(
    input_size=(
        192,
        256,
    ),
```

```
    normalize=False,
    sigma=(
        4.9,
        5.66,
    ),
    simcc_split_ratio=2.0,
    type='SimCCLabel',
    use_dark=False)
custom_hooks = [
    dict(
        ema_type='ExpMomentumEMA',
        momentum=0.0002,
        priority=49,
        type='EMAHook',
        update_buffers=True),
    dict(
        switch_epoch=390,
        switch_pipeline=[
            dict(backend_args=dict(backend='local'), type='LoadImage'),
            dict(type='GetBBoxCenterScale'),
            dict(direction='horizontal', type='RandomFlip'),
            dict(type='RandomHalfBody'),
            dict(
                rotate_factor=60,
                scale_factor=[
                    0.75,
                    1.25,
                ],
                shift_factor=0.0,
                type='RandomBBoxTransform'),
            dict(input_size=(
                192,
                256,
            ), type='TopdownAffine'),
            dict(type='mmdet.YOLOXHSVRandomAug'),
            dict(
                transforms=[
```

```
                dict(p=0.1, type='Blur'),
                dict(p=0.1, type='MedianBlur'),
                dict(
                    max_height=0.4,
                    max_holes=1,
                    max_width=0.4,
                    min_height=0.2,
                    min_holes=1,
                    min_width=0.2,
                    p=0.5,
                    type='CoarseDropout'),
            ],
            type='Albumentation'),
        dict(
            encoder=dict(
                input_size=(
                    192,
                    256,
                ),
                normalize=False,
                sigma=(
                    4.9,
                    5.66,
                ),
                simcc_split_ratio=2.0,
                type='SimCCLabel',
                use_dark=False),
            type='GenerateTarget'),
        dict(type='PackPoseInputs'),
    ],
    type='mmdet.PipelineSwitchHook'),
]
data_mode = 'topdown'
data_root = 'data/coco/'
dataset_type = 'CocoDataset'
default_hooks = dict(
    checkpoint=dict(
```

```
        interval=10,
        max_keep_ckpts=1,
        rule='greater',
        save_best='coco/AP',
        type='CheckpointHook'),
    logger=dict(interval=50, type='LoggerHook'),
    param_scheduler=dict(type='ParamSchedulerHook'),
    sampler_seed=dict(type='DistSamplerSeedHook'),
    timer=dict(type='IterTimerHook'),
    visualization=dict(enable=False, type='PoseVisualizationHook'))
default_scope = 'mmpose'
env_cfg = dict(
    cudnn_benchmark=False,
    dist_cfg=dict(backend='nccl'),
    mp_cfg=dict(mp_start_method='fork', opencv_num_threads=0))
load_from = None
log_level = 'INFO'
log_processor = dict(
    by_epoch=True, num_digits=6, type='LogProcessor', window_size=50)
max_epochs = 420
model = dict(
    backbone=dict(
        _scope_='mmdet',
        act_cfg=dict(type='SiLU'),
        arch='P5',
        channel_attention=True,
        deepen_factor=0.67,
        expand_ratio=0.5,
        init_cfg=dict(
            checkpoint=

'https://download.openmmlab.com/mmpose/v1/projects/rtmposev1/cspnext-m
_udp-aic-coco_210e-256x192-f2f7d6f6_20230130.pth',
            prefix='backbone.',
            type='Pretrained'),
        norm_cfg=dict(type='SyncBN'),
        out_indices=(4, ),
```

```
    type='CSPNeXt',
    widen_factor=0.75),
data_preprocessor=dict(
    bgr_to_rgb=True,
    mean=[
        123.675,
        116.28,
        103.53,
    ],
    std=[
        58.395,
        57.12,
        57.375,
    ],
    type='PoseDataPreprocessor'),
head=dict(
    decoder=dict(
        input_size=(
            192,
            256,
        ),
        normalize=False,
        sigma=(
            4.9,
            5.66,
        ),
        simcc_split_ratio=2.0,
        type='SimCCLabel',
        use_dark=False),
    final_layer_kernel_size=7,
    gau_cfg=dict(
        act_fn='SiLU',
        drop_path=0.0,
        dropout_rate=0.0,
        expansion_factor=2,
        hidden_dims=256,
        pos_enc=False,
```

```
        s=128,
        use_rel_bias=False),
    in_channels=768,
    in_featuremap_size=(
        6,
        8,
    ),
    input_size=(
        192,
        256,
    ),
    loss=dict(
        beta=10.0,
        label_softmax=True,
        type='KLDiscretLoss',
        use_target_weight=True),
    out_channels=17,
    simcc_split_ratio=2.0,
    type='RTMCCHead'),
test_cfg=dict(flip_test=True),
type='TopdownPoseEstimator')
optim_wrapper = dict(
optimizer=dict(lr=0.004, type='AdamW', weight_decay=0.05),
paramwise_cfg=dict(
    bias_decay_mult=0, bypass_duplicate=True, norm_decay_mult=0),
type='OptimWrapper')
param_scheduler = [
dict(
    begin=0, by_epoch=False, end=1000, start_factor=1e-05,
    type='LinearLR'),
dict(
    T_max=210,
    begin=210,
    by_epoch=True,
    convert_to_iter_based=True,
    end=420,
    eta_min=0.0002,
```

```
        type='CosineAnnealingLR'),
]
randomness = dict(seed=21)
resume = False
stage2_num_epochs = 30
test_cfg = dict()
test_dataloader = dict(
    batch_size=64,
    dataset=dict(
        ann_file='annotations/person_keypoints_val2017.json',
        data_mode='topdown',
        data_prefix=dict(img='val2017/'),
        data_root='data/coco/',
        pipeline=[
            dict(backend_args=dict(backend='local'), type='LoadImage'),
            dict(type='GetBBoxCenterScale'),
            dict(input_size=(
                192,
                256,
            ), type='TopdownAffine'),
            dict(type='PackPoseInputs'),
        ],
        test_mode=True,
        type='CocoDataset'),
    drop_last=False,
    num_workers=10,
    persistent_workers=True,
    sampler=dict(round_up=False, shuffle=False, type='DefaultSampler'))
test_evaluator = dict(
    ann_file='data/coco/annotations/person_keypoints_val2017.json',
    type='CocoMetric')
train_cfg = dict(by_epoch=True, max_epochs=420, val_interval=10)
train_dataloader = dict(
    batch_size=256,
    dataset=dict(
        ann_file='annotations/person_keypoints_train2017.json',
        data_mode='topdown',
```

```
data_prefix=dict(img='train2017/'),
data_root='data/coco/',
pipeline=[
    dict(backend_args=dict(backend='local'), type='LoadImage'),
    dict(type='GetBBoxCenterScale'),
    dict(direction='horizontal', type='RandomFlip'),
    dict(type='RandomHalfBody'),
    dict(
        rotate_factor=80,
        scale_factor=[
            0.6,
            1.4,
        ],
        type='RandomBBoxTransform'),
    dict(input_size=(
        192,
        256,
    ), type='TopdownAffine'),
    dict(type='mmdet.YOLOXHSVRandomAug'),
    dict(
        transforms=[
            dict(p=0.1, type='Blur'),
            dict(p=0.1, type='MedianBlur'),
            dict(
                max_height=0.4,
                max_holes=1,
                max_width=0.4,
                min_height=0.2,
                min_holes=1,
                min_width=0.2,
                p=1.0,
                type='CoarseDropout'),
        ],
        type='Albumentation'),
    dict(
        encoder=dict(
            input_size=(
```

```
                192,
                256,
            ),
            normalize=False,
            sigma=(
                4.9,
                5.66,
            ),
            simcc_split_ratio=2.0,
            type='SimCCLabel',
            use_dark=False),
        type='GenerateTarget'),
    dict(type='PackPoseInputs'),
],
type='CocoDataset'),
num_workers=10,
persistent_workers=True,
sampler=dict(shuffle=True, type='DefaultSampler'))
train_pipeline = [
dict(backend_args=dict(backend='local'), type='LoadImage'),
dict(type='GetBBoxCenterScale'),
dict(direction='horizontal', type='RandomFlip'),
dict(type='RandomHalfBody'),
dict(
    rotate_factor=80,
    scale_factor=[
        0.6,
        1.4,
    ],
    type='RandomBBoxTransform'),
dict(input_size=(
    192,
    256,
), type='TopdownAffine'),
dict(type='mmdet.YOLOXHSVRandomAug'),
dict(
    transforms=[
```

```
            dict(p=0.1, type='Blur'),
            dict(p=0.1, type='MedianBlur'),
            dict(
                max_height=0.4,
                max_holes=1,
                max_width=0.4,
                min_height=0.2,
                min_holes=1,
                min_width=0.2,
                p=1.0,
                type='CoarseDropout'),
        ],
        type='Albumentation'),
    dict(
        encoder=dict(
            input_size=(
                192,
                256,
            ),
            normalize=False,
            sigma=(
                4.9,
                5.66,
            ),
            simcc_split_ratio=2.0,
            type='SimCCLabel',
            use_dark=False),
        type='GenerateTarget'),
    dict(type='PackPoseInputs'),
]
train_pipeline_stage2 = [
    dict(backend_args=dict(backend='local'), type='LoadImage'),
    dict(type='GetBBoxCenterScale'),
    dict(direction='horizontal', type='RandomFlip'),
    dict(type='RandomHalfBody'),
    dict(
        rotate_factor=60,
```

```
    scale_factor=[
        0.75,
        1.25,
    ],
    shift_factor=0.0,
    type='RandomBBoxTransform'),
dict(input_size=(
    192,
    256,
), type='TopdownAffine'),
dict(type='mmdet.YOLOXHSVRandomAug'),
dict(
    transforms=[
        dict(p=0.1, type='Blur'),
        dict(p=0.1, type='MedianBlur'),
        dict(
            max_height=0.4,
            max_holes=1,
            max_width=0.4,
            min_height=0.2,
            min_holes=1,
            min_width=0.2,
            p=0.5,
            type='CoarseDropout'),
    ],
    type='Albumentation'),
dict(
    encoder=dict(
        input_size=(
            192,
            256,
        ),
        normalize=False,
        sigma=(
            4.9,
            5.66,
        ),
```

```
        simcc_split_ratio=2.0,
        type='SimCCLabel',
        use_dark=False),
    type='GenerateTarget'),
  dict(type='PackPoseInputs'),
]
val_cfg = dict()
val_dataloader = dict(
  batch_size=64,
  dataset=dict(
    ann_file='annotations/person_keypoints_val2017.json',
    data_mode='topdown',
    data_prefix=dict(img='val2017/'),
    data_root='data/coco/',
    pipeline=[
      dict(backend_args=dict(backend='local'), type='LoadImage'),
      dict(type='GetBBoxCenterScale'),
      dict(input_size=(
        192,
        256,
      ), type='TopdownAffine'),
      dict(type='PackPoseInputs'),
    ],
    test_mode=True,
    type='CocoDataset'),
  drop_last=False,
  num_workers=10,
  persistent_workers=True,
  sampler=dict(round_up=False, shuffle=False, type='DefaultSampler'))
val_evaluator = dict(
  ann_file='data/coco/annotations/person_keypoints_val2017.json',
  type='CocoMetric')
val_pipeline = [
  dict(backend_args=dict(backend='local'), type='LoadImage'),
  dict(type='GetBBoxCenterScale'),
  dict(input_size=(
    192,
```

```
        256,
    ), type='TopdownAffine'),
    dict(type='PackPoseInputs'),
]
vis_backends = [
    dict(type='LocalVisBackend'),
]
visualizer = dict(
    name='visualizer',
    type='PoseLocalVisualizer',
    vis_backends=[
        dict(type='LocalVisBackend'),
    ])
```

```
"""
Application Interface for RTMPose + ByteTrack Pose Estimator
"""
import os
os.environ['KMP_DUPLICATE_LIB_OK'] = 'TRUE'

from pose_estimator import RTMPoseEstimator
from config import Config
from pathlib import Path
import sys


class PoseEstimatorApp:
    """Class for managing the application"""

    def __init__(self):
        """Initialize the application"""
        print("="*60)
        print("🤸 RTMPose + ByteTrack Pose Estimation Application")
        print("="*60)
```

```
    # Create Estimator object
    self.estimator = None
    self.det_conf = Config.DET_CONFIDENCE
    self.pose_conf = Config.POSE_CONFIDENCE
    self.device = Config.DEVICE
    self.tracking_enabled = True

    self._init_estimator()

def _init_estimator(self):
    """Initialize the estimator"""
    try:
        print(f"\n🔄 Initializing estimator...")
        self.estimator = RTMPoseEstimator(
            det_conf=self.det_conf,
            pose_conf=self.pose_conf,
            device=self.device
        )

        if not self.tracking_enabled:
            self.estimator.tracker = None
            print("⚠️  Tracking disabled")

    except Exception as e:
        print(f"❌ Error initializing estimator: {e}")
        print(f"\n💡 Make sure you have installed:")
        print(f"   pip install openmim")
        print(f"   mim install mmdet mmpose")
        sys.exit(1)

def process_image(self, image_path: str):
    """Process a single image"""
    try:
        self.estimator.predict_image(
            image_path=image_path,
            save_result=Config.SAVE_RESULTS
        )
```

```
        except Exception as e:
            print(f"❌ Error processing image: {e}")

    def process_video(self, video_path: str):
        """Process a video file"""
        try:
            self.estimator.predict_video(
                video_path=video_path,
                save_result=Config.SAVE_RESULTS,
                show_live=Config.SHOW_LIVE
            )
        except Exception as e:
            print(f"❌ Error processing video: {e}")

    def process_camera(self):
        """Process camera stream"""
        try:
            self.estimator.predict_video(
                video_path=0,
                save_result=Config.SAVE_RESULTS,
                show_live=True
            )
        except Exception as e:
            print(f"❌ Error opening camera: {e}")

    def change_detection_confidence(self):
        """Change detection confidence threshold"""
        print(f"\n📊 Current detection confidence: {self.det_conf}")
        try:
            new_conf = float(input("Enter new confidence (0.0 - 1.0): ").strip())
            if 0 <= new_conf <= 1:
                self.det_conf = new_conf
                self.estimator.det_conf = new_conf
                print(f"✅ Detection confidence changed to: {new_conf}")
            else:
                print("❌ Value must be between 0 and 1")
        except ValueError:
```

```
        print("❌ Invalid value!")

    def change_pose_confidence(self):
        """Change pose confidence threshold"""
        print(f"\n📊 Current pose confidence: {self.pose_conf}")
        try:
            new_conf = float(input("Enter new confidence (0.0 - 1.0): ").strip())
            if 0 <= new_conf <= 1:
                self.pose_conf = new_conf
                self.estimator.pose_conf = new_conf
                print(f"✅ Pose confidence changed to: {new_conf}")
            else:
                print("❌ Value must be between 0 and 1")
        except ValueError:
            print("❌ Invalid value!")

    def toggle_tracking(self):
        """Toggle tracking on/off"""
        self.tracking_enabled = not self.tracking_enabled

        if self.tracking_enabled:
            if self.estimator.tracker is None:
                self.estimator._init_tracker()
            print("✅ Tracking enabled")
        else:
            self.estimator.tracker = None
            print("⚠️  Tracking disabled")

    def change_device(self):
        """Change computing device"""
        print(f"\n💻 Current device: {self.device}")
        print("Available devices:")
        print("1. cuda:0 (GPU)")
        print("2. cpu")

        choice = input("Choose device: ").strip()
```

```
        if choice == '1':
            import torch
            if torch.cuda.is_available():
                self.device = 'cuda:0'
                print("✅ Switched to GPU")
            else:
                print("❌ CUDA not available!")
                return
        elif choice == '2':
            self.device = 'cpu'
            print("✅ Switched to CPU")
        else:
            print("❌ Invalid choice!")
            return

        # Reinitialize estimator with new device
        print("🔄 Reinitializing estimator...")
        self._init_estimator()

    def show_settings(self):
        """Show current settings"""
        print("\n" + "="*60)
        print("⚙️  Current Settings:")
        print("="*60)
        print(f"Detection Model: {self.estimator.det_model_name}")
        print(f"Pose Model: {self.estimator.pose_model_name}")
        print(f"Detection Confidence: {self.det_conf}")
        print(f"Pose Confidence: {self.pose_conf}")
        print(f"Device: {self.device}")
        print(f"Tracking: {'Enabled' if self.tracking_enabled else 'Disabled'}")
        print(f"Save Results: {Config.SAVE_RESULTS}")
        print(f"Show Live: {Config.SHOW_LIVE}")
        print("="*60)

    def batch_process_images(self):
        """Batch process all images in input folder"""
        from utils import FileManager
```

```
    images = FileManager.get_all_images(Config.INPUT_IMAGES_DIR)

    if len(images) == 0:
        print(f"❌ No images found in: {Config.INPUT_IMAGES_DIR}")
        return

    print(f"\n📁 Found {len(images)} image(s)")
    confirm = input("Process all images? (y/n): ").strip().lower()

    if confirm != 'y':
        print("❌ Cancelled")
        return

    print("\n🔄 Processing images...")
    for idx, img_path in enumerate(images, 1):
        print(f"\n[{idx}/{len(images)}] Processing: {img_path.name}")
        try:
            self.process_image(str(img_path))
        except Exception as e:
            print(f"❌ Error: {e}")
            continue

    print(f"\n✅ Batch processing completed!")

def batch_process_videos(self):
    """Batch process all videos in input folder"""
    from utils import FileManager

    videos = FileManager.get_all_videos(Config.INPUT_VIDEOS_DIR)

    if len(videos) == 0:
        print(f"❌ No videos found in: {Config.INPUT_VIDEOS_DIR}")
        return

    print(f"\n📁 Found {len(videos)} video(s)")
    confirm = input("Process all videos? (y/n): ").strip().lower()
```

```
    if confirm != 'y':
        print("❌ Cancelled")
        return

    print("\n🔄 Processing videos...")
    for idx, vid_path in enumerate(videos, 1):
        print(f"\n[{idx}/{len(videos)}] Processing: {vid_path.name}")
        try:
            self.process_video(str(vid_path))
        except Exception as e:
            print(f"❌ Error: {e}")
            continue

    print(f"\n✅ Batch processing completed!")

def run_interactive(self):
    """Run the application in interactive mode"""
    while True:
        print("\n" + "="*60)
        print("📋 Main Menu:")
        print("="*60)
        print("1.  Process Image")
        print("2.  Process Video")
        print("3.  Use Camera")
        print("4.  Batch Process Images")
        print("5.  Batch Process Videos")
        print("6.  Change Detection Confidence")
        print("7.  Change Pose Confidence")
        print("8.  Toggle Tracking (On/Off)")
        print("9.  Change Device (GPU/CPU)")
        print("10. Show Current Settings")
        print("11. Show Paths")
        print("0.  Exit")
        print("="*60)

        choice = input("\n👉 Choose an option: ").strip()
```

```
        if choice == '1':
            path = input("📁 Enter image path: ").strip()
            if Path(path).exists():
                self.process_image(path)
            else:
                print("❌ File not found!")

        elif choice == '2':
            path = input("📁 Enter video path: ").strip()
            if Path(path).exists():
                self.process_video(path)
            else:
                print("❌ File not found!")

        elif choice == '3':
            print("📹 Opening camera... (Press Q to exit)")
            self.process_camera()

        elif choice == '4':
            self.batch_process_images()

        elif choice == '5':
            self.batch_process_videos()

        elif choice == '6':
            self.change_detection_confidence()

        elif choice == '7':
            self.change_pose_confidence()

        elif choice == '8':
            self.toggle_tracking()

        elif choice == '9':
            self.change_device()
```

```
        elif choice == '10':
            self.show_settings()

        elif choice == '11':
            Config.print_paths()
            Config.print_model_info()

        elif choice == '0':
            print("\n👋 Thank you for using the application!")
            break

        else:
            print("❌ Invalid choice!")

        input("\n⏸️  Press Enter to continue...")


if __name__ == "__main__":
    app = PoseEstimatorApp()
    app.run_interactive()
```

```
name: rtmpose_tracker

channels:
  - pytorch
  - nvidia
  - conda-forge
  - defaults

dependencies:
  - python=3.10

  # PyTorch 2.0 (Stable with CUDA 11.8 for GTX 1660 Ti)
  - pytorch=2.0.1
  - torchvision=0.15.2
```

```
- torchaudio=2.0.2
- pytorch-cuda=11.8

# Core Libraries
- numpy=1.24.3       # Pinned to avoid version 2.0 conflicts
- scipy
- pandas
- opencv
- pillow
- tqdm
- matplotlib

# IMPORTANT FIXES
# We removed 'lap' and added 'lapx' in pip section (Works on Windows)

- pip
- pip:
    - lapx>=0.5.5          # Fixes 'lap' build error on Windows
    - chumpy
    - onnxruntime-gpu      # Essential for inference speed
    - openmim
    - mmengine
    - "mmcv>=2.0.1"
    - "mmdet>=3.1.0"
    - "mmpose>=1.1.0"
    - shapely
    - colorama
    - terminaltables
    - json_tricks
    - xtcocotools
```
